



单片机语言 C51

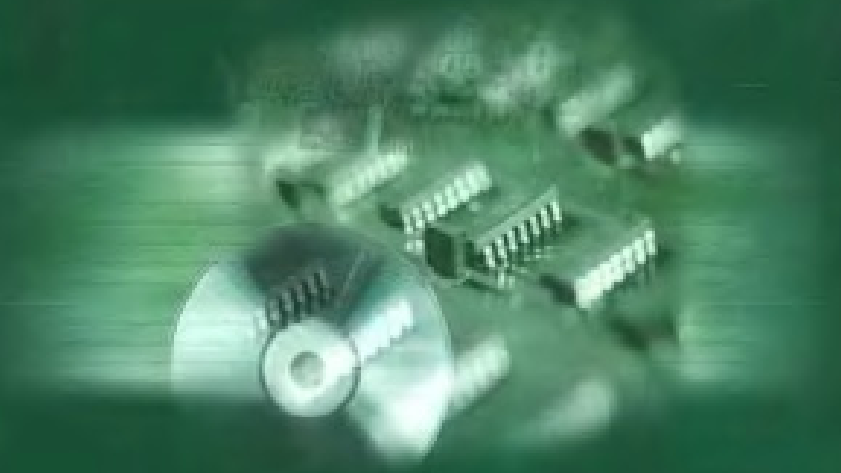
应用实战集锦

范风强 兰婵丽 编著



电子工业出版社
PUBLISHING HOUSE OF ELECTRONICS INDUSTRY

[http:// www.phei.com.cn](http://www.phei.com.cn)



本书贴有激光防伪标志。凡没有防伪标志者，属盗版图书。



责任编辑：竺南直
封面设计：欧美尼设计室



ISBN 7-5053-8590-9/TN·1773

定价：36.00 元



单片机语言 C51 应用实战集锦

范风强 兰婵丽 编著

電子工業出版社

Publishing House of Electronics Industry

北京 • BEIJING

内 容 简 介

使用 C 语言开发速度快, 代码可重复使用, 程序结构清晰、易懂、易维护, 易开发一些比较大型的项目。目前, 许多编译器都已经支持了 C51, 而且是 Windows 视窗界面。keilc51 是目前单片机开发最为流行的软件。

本书收集并整理了许多实用的采用 C51 单片机开发的程序, 这些程序既可以给读者以开拓思路, 参考的用途, 又是实际的开发程序, 可以直接作为程序应用在相同的开发系统上。通过本书的学习, 读者可以进一步了解和掌握 C51 编程的思路和方法。

本书适用于从事单片机项目开发与应用的工程技术人员阅读。

未经许可, 不得以任何方式复制或抄袭本书之部分或全部内容。

版权所有, 侵权必究。

图书在版编目 (CIP) 数据

单片机语言 C51 应用实战集锦/范风强等编著. —北京: 电子工业出版社, 2003.3

ISBN 7-5053-8590-9

I. 单… II. 范… III. 单片微型计算机—程序设计 IV. TP368.1

中国版本图书馆 CIP 数据核字 (2003) 第 017889 号

责任编辑: 竺南直

印 刷: 北京李史山胶印厂

出版发行: 电子工业出版社 <http://www.phei.com.cn>

北京市海淀区万寿路 173 信箱 邮编 100036

经 销: 各地新华书店

开 本: 787×1092 1/16 印张: 24.5 字数: 548 千字

版 次: 2003 年 3 月第 1 版 2003 年 3 月第 1 次印刷

印 数: 6000 册 定价: 36.00 元

凡购买电子工业出版社的图书, 如有缺损问题, 请向购买书店调换。若书店售缺, 请与本社发行部联系。

联系电话: (010) 68279077

前 言

采用单片机 C51 语言编程具有很多优越性。如果你不懂得单片机的指令集,也能够编写出完美的单片机程序;无须懂得单片机的具体硬件,也能够编出符合硬件实际的专业水平的程序;不同函数的数据实行覆盖,能有效利用片上有限的 RAM 空间。程序具有坚固性,数据被破坏是导致程序运行异常的重要因素。C 语言对数据进行了许多专业性的处理,避免了运行中间非异步的破坏;C 语言提供复杂的数据类型(数组、结构、联合、枚举、指针等),极大地增强了程序处理能力和灵活性;提供了 auto、static、const 等存储类型和专门针对 8051 单片机的 data、idata、pdata、xdata、code 等存储类型,自动为变量合理地分配地址;提供 small、compact、large 等编译模式,以适应片上存储器的大小。中断服务程序的现场保护和恢复,中断向量表的填写,是直接和单片机相关的,都由 C 编译器代办。它还提供常用的标准函数库,以供用户直接使用;在头文件中定义宏、说明复杂数据类型和函数原型,这有利于程序的移植和支持单片机的系列化产品的开发;有严格的句法检查,错误很少,可容易地在高级语言的水平上迅速地被排掉。可方便地接受多种实用程序的服务:如片上资源的初始化有专门的实用程序自动生成;有实时多任务操作系统可调度多道任务,简化用户编程,可提高运行的安全性等。

正是因为它有如此之多的优越性,使之成为了 51 系列单片机流行的开发语言。本书收集并整理了 40 个实用开发子程序,这些程序对学习开发 51 系列单片机的初学者有相当重要的入门指导作用,对于有经验的开发人员来说,也极具参考价值。

在单片机开发的过程中,最常见的是键盘、显示等接口部分用于人机交互,A/D、D/A 以及 I²C 总线用于数据采集和传输,抗干扰、通信等用于把现场实时信号完整地发送到控制中心等。对于这些项目的开发,不同的开发者有不同的思路以及不同的技巧。这些都反映在开发的程序上的不同。对于我们提供的这些程序,不是说它们写得很好,而是它们经过了多项目的检验,稳定而且实用。利用它们可以减少开发时间,同时也能给从事开发的技术人员启发思路,这也是本书的目的所在。

在本书的创作过程中,聂家财、钟开智、兰德昌、刘文涛等给予了许多帮助,在此表示感谢。

由于作者水平有限,书中难免有不足和缺陷之处,希望广大读者给予指正,并提出宝贵意见。作者的电子信箱是 fangfq2000@etang.com。

编著者
2002 年秋

目 录

程序一	实时时钟芯片 DS1302 的 C51 程序例子	(1)
程序二	C430 与 C51 的一点区别	(6)
程序三	一个菜单的例子	(7)
程序四	DS1820 单芯片温度测量	(8)
程序五	keilc 6.20c 版直接嵌入汇编的方法	(10)
程序六	用计算机并口模拟 SPI 通信的 C 源程序	(11)
程序七	CRC16-STANDARD 的快速算法	(14)
程序八	在 PC 上用并行口模拟 I ² C 总线的 C 源代码	(16)
程序九	一种在 C51 中写二进制的方法	(19)
程序十	CRC 算法原理及 C 语言实现	(19)
程序十一	软件陷阱	(26)
程序十二	一个简单的 VB 串口发送程序	(27)
程序十三	12864 汉字液晶显示驱动程序	(29)
程序十四	12232 点阵液晶基本驱动程序	(33)
程序十五	串口中断服务函数集	(37)
程序十六	93C46 读写程序	(46)
程序十七	20045 读写程序	(50)
程序十八	一组小程序集锦	(53)
程序十九	AVR asm 源程序	(78)
程序二十	AVR 单片机一个简单的通信程序	(81)
程序二十一	TG19264A 接口程序	(82)
程序二十二	TG19264A 接口程序 (AVR 模拟方式)	(85)
程序二十三	常用的几种码制转换 BCD, HEX, BIN	(97)
程序二十四	16x2 字符液晶屏驱动演示程序一	(98)
程序二十五	16x2 字符液晶屏驱动演示程序二	(103)
程序二十六	PS7219 代码	(107)
程序二十七	2051 的 AD 代码	(109)
程序二十八	ARV19264 型液晶显示字库	(115)
程序二十九	液晶 CKW19264A 型接口程序 (模拟方式)	(124)
程序三十	I ² C 总线驱动程序	(135)
程序三十一	240128 型液晶代码	(141)
程序三十二	飞机游戏	(158)
程序三十三	PC 键代码	(174)
程序三十四	拼音输入法模块	(182)
程序三十五	串行口代码	(203)

程序三十六	蛇游戏代码	(209)
程序三十七	与液晶模块 T6963C 连接代码	(226)
程序三十八	键盘输入法设计草案	(237)
程序三十九	16*4 液晶汉字代码	(285)
程序四十	智能化家电控制	(291)
附录 A	MCS-51 单片机定点运算子程序库	(310)
附录 B	MCS-51 单片机浮点运算子程序库	(334)
附录 C	单片机 C51 编程几个有用的模块	(369)
附录 D	头文件 W77E58.h	(379)

程序一 实时时钟芯片 DS1302 的 C51 程序例子

```
/******  
/* 实时时钟模块 时钟芯片型号: DS1302 */  
/*/  
/******  
sbit T_CLK = P2^7; /*实时时钟时钟线引脚 */  
sbit T_IO = P1^4; /*实时时钟数据线引脚 */  
sbit T_RST = P1^5; /*实时时钟复位线引脚 */  
/******  
*  
* 名称: v_RTInputByte  
* 说明:  
* 功能: 往 DS1302 写入 1Byte 数据  
* 调用:  
* 输入: ucDa 写入的数据  
* 返回值: 无  
*****  
void v_RTInputByte(uchar ucDa)  
{  
    uchar i;  
    ACC = ucDa;  
    for(i=8; i>0; i--)  
    {  
        T_IO = ACC0; /*相当于汇编中的 RRC */  
        T_CLK = 1;  
        T_CLK = 0;  
        ACC = ACC >> 1;  
    }  
}  
/******  
*  
* 名称: uchar uc_RTOutputByte  
* 说明:  
* 功能: 从 DS1302 读取 1Byte 数据  
* 调用:  
* 输入:  
* 返回值: ACC  
*****  
uchar uc_RTOutputByte(void)  
{
```



```

uchar i;
for(i=8; i>0; i--)
{
ACC = ACC >> 1; /*相当于汇编中的 RRC */
ACC7 = T_IO;
T_CLK = 1;
T_CLK = 0;
}
return(ACC);
}
/*****
*
* 名称: v_W1302
* 说明: 先写地址, 后写命令/数据
* 功能: 往 DS1302 写入数据
* 调用: v_RTInputByte()
* 输入: ucAddr: DS1302 地址, ucDa: 要写的数据
* 返回值: 无
*****/
void v_W1302(uchar ucAddr, uchar ucDa)
{
T_RST = 0;
T_CLK = 0;
T_RST = 1;
v_RTInputByte(ucAddr); /* 地址, 命令 */
v_RTInputByte(ucDa); /* 写 1Byte 数据*/
T_CLK = 1;
T_RST = 0;
}
/*****
*
* 名称: uc_R1302
* 说明: 先写地址, 后读命令/数据
* 功能: 读取 DS1302 某地址的数据
* 调用: v_RTInputByte(), uc_RTOutputByte()
* 输入: ucAddr: DS1302 地址
* 返回值: ucDa :读取的数据
*****/
uchar uc_R1302(uchar ucAddr)
{
uchar ucDa;
T_RST = 0;
T_CLK = 0;
T_RST = 1;
v_RTInputByte(ucAddr); /* 地址, 命令 */

```

```

ucDa = uc_RTOutputByte(); /* 读 1Byte 数据 */
T_CLK = 1;
T_RST = 0;
return(ucDa);
}

/*****
*
* 名称: v_BurstW1302T
* 说明: 先写地址, 后写数据(时钟多字节方式)
* 功能: 往 DS1302 写入时钟数据(多字节方式)
* 调用: v_RTInputByte()
* 输入: pSecDa: 时钟数据地址 格式为: 秒 分 时 日 月 星期 年 控制
* 8Byte (BCD 码) 1B 1B 1B 1B 1B 1B 1B 1B
* 返回值: 无
*****/

void v_BurstW1302T(uchar *pSecDa)
{
uchar i;
v_W1302(0x8e,0x00); /* 控制命令,WP=0,写操作?*/
T_RST = 0;
T_CLK = 0;
T_RST = 1;
v_RTInputByte(0xbe); /* 0xbe:时钟多字节写命令 */
for (i=8;i>0;i--) /*8Byte = 7Byte 时钟数据 + 1Byte 控制*/
{
v_RTInputByte(*pSecDa);/* 写 1Byte 数据*/
pSecDa++;
}
T_CLK = 1;
T_RST = 0;
}

/*****
*
* 名称: v_BurstR1302T
* 说明: 先写地址, 后读命令/数据(时钟多字节方式)
* 功能: 读取 DS1302 时钟数据
* 调用: v_RTInputByte() , uc_RTOutputByte()
* 输入: pSecDa: 时钟数据地址 格式为: 秒 分 时 日 月 星期 年
* 7Byte (BCD 码) 1B 1B 1B 1B 1B 1B 1B
* 返回值: ucDa :读取的数据
*****/

void v_BurstR1302T(uchar *pSecDa)
{
uchar i;
T_RST = 0;

```

```

T_CLK = 0;
T_RST = 1;
v_RTInputByte(0xbf); /* 0xbf:时钟多字节读命令 */
for (i=8; i>0; i--)
{
    *pSecDa = uc_RTOutputByte(); /* 读 1Byte 数据 */
    pSecDa++;
}
T_CLK = 1;
T_RST = 0;
}

/*****
*
* 名称: v_BurstW1302R
* 说明: 先写地址, 后写数据(寄存器多字节方式)
* 功能: 往 DS1302 寄存器数写入数据(多字节方式)
* 调用: v_RTInputByte()
* 输入: pReDa: 寄存器数据地址
* 返回值: 无
*****/

void v_BurstW1302R(uchar *pReDa)
{
    uchar i;
    v_W1302(0x8e, 0x00); /* 控制命令, WP=0, 写操作? */
    T_RST = 0;
    T_CLK = 0;
    T_RST = 1;
    v_RTInputByte(0xfe); /* 0xfe:时钟多字节写命令 */
    for (i=31; i>0; i--) /* 31Byte 寄存器数据 */
    {
        v_RTInputByte(*pReDa); /* 写 1Byte 数据 */
        pReDa++;
    }
    T_CLK = 1;
    T_RST = 0;
}

/*****
*
* 名称: uc_BurstR1302R
* 说明: 先写地址, 后读命令/数据(寄存器多字节方式)
* 功能: 读取 DS1302 寄存器数据
* 调用: v_RTInputByte(), uc_RTOutputByte()
* 输入: pReDa: 寄存器数据地址
* 返回值: 无
*****/

```

```

void v_BurstR1302R(uchar *pReDa)
{
    uchar i;
    T_RST = 0;
    T_CLK = 0;
    T_RST = 1;
    v_RTInputByte(0xff); /* 0xbf:时钟多字节读命令 */
    for (i=31; i>0; i--) /*31Byte 寄存器数据 */
    {
        *pReDa = uc_RTOutputByte(); /* 读 1Byte 数据 */
        pReDa++;
    }
    T_CLK = 1;
    T_RST = 0;
}

/*****
*
* 名称: v_Set1302
* 说明:
* 功能: 设置初始时间
* 调用: v_W1302()
* 输入: pSecDa: 初始时间地址。初始时间格式为: 秒 分 时 日 月 星期 年
* 7Byte (BCD 码) 1B 1B 1B 1B 1B 1B 1B
* 返回值: 无
*****/

void v_Set1302(uchar *pSecDa)
{
    uchar i;
    uchar ucAddr = 0x80;
    v_W1302(0x8e, 0x00); /* 控制命令, WP=0, 写操作? */
    for (i = 7; i > 0; i--)
    {
        v_W1302(ucAddr, *pSecDa); /* 秒 分 时 日 月 星期 年 */

        pSecDa++;
        ucAddr += 2;
    }
    v_W1302(0x8e, 0x80); /* 控制命令, WP=1, 写保护? */
}

/*****
*
* 名称: v_Get1302
* 说明:
* 功能: 读取 DS1302 当前时间
* 调用: uc_R1302()
*****/

```

* 输入: ucCurtime: 保存当前时间地址。当前时间格式为: 秒 分 时 日 月 星期 年
 * 7Byte (BCD 码) 1B 1B 1B 1B 1B 1B 1B
 * 返回值: 无

*****/

```
void v_Get1302(uchar ucCurtime[])
{
  uchar i;
  uchar ucAddr = 0x81;
  for (i=0;i<7;i++)
  {
    ucCurtime[i] = uc_R1302(ucAddr);/*格式为: 秒 分 时 日 月
    星期 年 */
    ucAddr += 2;
  }
}
```

程序二 C430 与 C51 的一点区别

C430 与 C51 语法上基本一样, 但是编程时要注意以下几点。

(1) 如果要判断 P2.0 是否为 1, C51 可以写为: if(P2&BIT0==BIT0); 但是在 C430 这样写会得不到结果, 而要写为: if((P2&BIT0) == BIT0) 才对。

(2) 在 C51 中如果要想程序等待可以直接用 while(1), 但是写 C430 程序时我曾经遇到 while(1)无效, 后来发现是我没设置 WDT, 加入 WDTCTL = WDTPW+WDTHOLD, 一切正常。

(3) C51 有 bit flag 等指令来定义位, 而 CP430 没有相关指令, 但是可以这样实现: 先定义一个变量 uchar flag, 这样就有 8 个位变量可以使用。假设 C51 有这样的程序:

```
bit flag;
flag = 0;
while(flag==0); //等待

在 C430 里可以写成:

uchar flag;
flag &= ~BIT1;
while( (flag&BIT1) != BIT1 );
效果一样
```

程序三 一个菜单的例子

```
/* Module :menu.c

#include
#include
#define SIZE_OF_KEYBD_MENU 20 //菜单长度

uchar KeyFuncIndex=0;
//uchar KeyFuncIndexNew=0;
void (*KeyFuncPtr)(); //按键功能指针
typedef struct
{
    uchar KeyStateIndex; //当前状态索引号
    uchar KeyDnState; //按下"向下"键时转向的状态索引号
    uchar KeyUpState; //按下"向上"键时转向的状态索引号
    uchar KeyCrState; //按下"回车"键时转向的状态索引号
    void (*CurrentOperate)(); //当前状态应该执行的功能操作
} KbdTabStruct;

KbdTabStruct code KeyTab[SIZE_OF_KEYBD_MENU]=
{
    { 0, 0, 0, 1, (*DnmmyJob) }, //顶层

    { 1, 2, 0, 3, (*DspUserInfo) }, //第二层
    { 2, 1, 1, 9, (*DspServiceInfo) }, //第二层

    { 3, 0, 0, 1, (*DspVoltInfo) }, //第三层>>DspUserInfo 的展开
    { 4, 0, 0, 1, (*DspCurrInfo) }, //第三层>>DspUserInfo 的展开
    { 5, 0, 0, 1, (*DspFreqInfo) }, //第三层>>DspUserInfo 的展开
    { 6, 0, 0, 1, (*DspCableInfo) }, //第三层>>DspUserInfo 的展开
    .....
    { 9, 0, 0, 1, (*DspSetVoltLevel) }, //第三层>>DspServiceInfo 的展开
    .....
};

void GetKeyInput(void)
{
    uchar KeyValue;
    KeyValue=P1&0x07; //去掉高 5bit
    delay(50000);
```

```

switch(KeyValue)
{
    case 1: //回车键,找出新的菜单状态编号
    {
        KeyFuncIndex=KeyTab[KeyFuncIndex].KeyCrState;
        break;
    }
    case 2: //向上键,找出新的菜单状态编号
    {
        KeyFuncIndex=KeyTab[KeyFuncIndex].KeyUpState;
        break;
    }
    case 4: //向下键,找出新的菜单状态编号
    {
        KeyFuncIndex=KeyTab[KeyFuncIndex].KeyDnState;
        break;
    }
    default: //按键错误的处理
    .....
    break;
}
KeyFuncPtr=KeyTab[KeyFuncIndex].CurrentOperate;
(*KeyFuncPtr)();//执行当前按键的操作
}
//其中 KeyTab 的设计值得读者仔细研究

```

程序四 DS1820 单芯片温度测量

```

//这里以 11.0592MHz 晶体为例
//sbit DQ =P2^1;//根据实际情况定义端口, DQ 代表端口

typedef unsigned char byte;
typedef unsigned int word;

//延时
void delay(word useconds)
{
    for(;useconds>0;useconds--);
}

//复位
byte ow_reset(void)

```

```

{
    byte presence;
    DQ = 0; //设 DQ 低电平
    delay(29); //延迟 480μs
    DQ = 1; //高电平
    delay(3); //等待
    presence = DQ; //取允许信号
    delay(25); //延时
    return(presence); //返回允许信号
} // 0=presence, 1 = no part

```

//从 1-wire 总线上读取一个字节

byte read_byte(void)

```

{
    byte i;
    byte value = 0;
    for (i=8;i>0;i--)
    {
        value>>=1;
        DQ = 0; //DQ 低
        DQ = 1; //返回高电平
        delay(1); //for (i=0; i<3; i++);
        if(DQ)value|=0x80;
        delay(6); //延时
    }
    return(value);
}

```

//向 1-WIRE 总线上写一个字节

void write_byte(char val)

```

{
    byte i;
    for (i=8; i>0; i--) //一次写一字节
    {
        DQ = 0; //DQ 低
        DQ = val&0x01;
        delay(5); //端口悬挂
        DQ = 1;
        val=val/2;
    }
    delay(5);
}

```

//读取温度

char Read_Temperature(void)


```

{
    union{
        byte c[2];
        int x;
    }temp;

    ow_reset();
    write_byte(0xCC); //跳过 ROM
    write_byte(0xBE); //读可擦写芯片
    temp.c[1]=read_byte();
    temp.c[0]=read_byte();
    ow_reset();
    write_byte(0xCC); //跳过 ROM
    write_byte(0x44); //开始翻转
    return temp.x/2;
}

```

程序五 keilc 6.20c 版直接嵌入汇编的方法

```

//<asm.h>
#ifdef ASM
    unsigned long shiftR1(register unsigned long);
#else
    extern unsigned long shiftR1(register unsigned long);
#endif
//end of asm.h

```

```

//<asm.c>
#define ASM
#include <asm.h>
#include <reg52.h>
#pragma OT(4,speed)
unsigned long shiftR1(register unsigned long x)
{
    #pragma asm
    clr c
    mov a,r4
    rrc a
    mov r4,a

    mov a,r5
    rrc a

```

```

mov r5,a

mov a,r6
rrc a
mov r6,a

mov a,r7
rrc a
mov r7,a

#pragma endasm
return(x);
}
//end of asm.c

```

将此源文件加入要编译的工程文件。

将光标指向此文件，选择右键菜单“option for file 'asm.c'”，

将属性单“properties”中的“Generate Assembler SRC File”“Assemble SRC File”

两项设置成黑体的“√”将“Link Public Only”的“√”去掉，再编译即可。

用此方法可以在 c 源代码的任意位置用#pragma asm 和#pragma endasm 嵌入汇编语句。

但要注意的是在直接使用形参时要小心，在不同的优化级别下产生的汇编代码有所不同，可以察看对应的.lst 文件，得到正确的优化级别后，#pragma OT(x,speed)锁定优化级别（这里的值是 0~9）。

程序六 用计算机并口模拟 SPI 通信的 C 源程序

```

#include
#include
#include
#include
#include

#define LPT_PORT 0x378
#define CLR_WCK(X) {X=X&~(1<<0); outportb(LPT_PORT,X); } // data.0
#define SET_WCK(X) {X=X|(1<<0); outportb(LPT_PORT,X); }
#define CLR_BCK(X) {X=X&~(1<<2); outportb(LPT_PORT,X); } // data.2
#define SET_BCK(X) {X=X|(1<<2); outportb(LPT_PORT,X); }
#define CLR_DATA(X) {X=X&~(1<<3); outportb(LPT_PORT,X); } // data.3
#define SET_DATA(X) {X=X|(1<<3); outportb(LPT_PORT,X); }
#define FALSE 0
#define TRUE 1
void test_comm()
{
unsigned char data ;

```

```

data = 0;
printf("Please press enter to begin send data\n");
getch();
printf("Pull down WCK data.0\n");
CLR_WCK(data);
getch();
printf("Pull up WCK data.0\n");
SET_WCK(data);
getch();

printf("Pull down BCK data.2\n");
CLR_BCK(data);
getch();
printf("Pull up BCK data.2\n");
SET_BCK(data);
getch();

printf("Pull down DATA data.3\n");
CLR_DATA(data);
getch();
printf("Pull up DATA data.3\n");
SET_DATA(data);
getch();
}
// 注：缓冲区大小的发送必须用 dword
void short_delay(int n)
{
int i;
for(i=0;i<N*1000;i++)
{int temp =0;}
}
int send_spi_data(unsigned char *buffer, unsigned long size)
{
unsigned char buff[1024];
unsigned char *buf=buff;
unsigned char data;
int i,j,k;
data =0;
if((size%4)!=0) return FALSE;
memcpy(buff,buffer,size);

do{
SET_WCK(data);
for(k=0;k<2;k++){
for(j=0;j<2;j++){
printf(".");

```

```

    for(i=0;i<8;i++){
        if((*buf)&0x80){
            SET_DATA(data);
        }else{
            CLR_DATA(data);
        }
        short_delay(1);
        // delay(1);
        SET_BCK(data);
        short_delay(1);
        // delay(1);
        CLR_BCK(data);
        short_delay(1);
        // delay(1);
        *buf<<=1;
    }
    buf++;
    size--;
}
// buf++;
// size--;
CLR_WCK(data);
}
SET_WCK(data);
}while(size>0);
return TRUE;
}
/*
void main()
{
    int i;
    unsigned char tmpdata[4];
    tmpdata[0] = 0x34;
    tmpdata[1] = 0x12;
    tmpdata[2] = 0x56;
    tmpdata[3] = 0x78;
    // for(i=0;i<500;i++)
    for(i=0;i<50;i++)
    {
        send_spi_data(tmpdata,4);
    }

    // test_comm();
}
*/

```

程序七 CRC16-STANDARD 的快速算法

;入口: R0-原 CRC 结果高位, R1-原 CRC 结果低位, R2-下个参与计算的字节值

;(最开始可设置 R0=0, R1=0, R2=第一字节值)

;出口: R0-CRC 结果高位, R1-CRC 结果低位

影响: A, DPH, DPL

CRC16:

PUSH DPH

PUSH DPL

PUSH ACC

MOV A,R1

XRL A,R2

MOV DPH,#20h

MOV DPL,A

CLR A

MOVC A,@A+DPTR

XRL A,R0

MOV R1,A

INC DPH

CLR A

MOVC A,@A+DPTR

MOV R0,A

POP ACC

POP DPL

POP DPh

RET

NOP

NOP

LJMP MAIN

ORG 2000H ;(根据实际情况而定)

CRCTABLE:

DB

000H,0C1H,081H,040H,001H,0C0H,080H,041H,001H,0C0H,080H,041H,000H,0C1H,081H,040H

DB

001H,0C0H,080H,041H,000H,0C1H,081H,040H,000H,0C1H,081H,040H,001H,0C0H,080H,041H

DB

001H,0C0H,080H,041H,000H,0C1H,081H,040H,000H,0C1H,081H,040H,001H,0C0H,080H,041H

DB

000H,0C1H,081H,040H,001H,0C0H,080H,041H,001H,0C0H,080H,041H,000H,0C1H,081H,040H

DB

001H,0C0H,080H,041H,000H,0C1H,081H,040H,000H,0C1H,081H,040H,001H,0C0H,080H,041H
 DB
 000H,0C1H,081H,040H,001H,0C0H,080H,041H,001H,0C0H,080H,041H,000H,0C1H,081H,040H
 DB
 000H,0C1H,081H,040H,001H,0C0H,080H,041H,001H,0C0H,080H,041H,000H,0C1H,081H,040H
 DB
 001H,0C0H,080H,041H,000H,0C1H,081H,040H,000H,0C1H,081H,040H,001H,0C0H,080H,041H
 DB
 001H,0C0H,080H,041H,000H,0C1H,081H,040H,000H,0C1H,081H,040H,001H,0C0H,080H,041H
 DB
 000H,0C1H,081H,040H,001H,0C0H,080H,041H,001H,0C0H,080H,041H,000H,0C1H,081H,040H
 DB
 000H,0C1H,081H,040H,001H,0C0H,080H,041H,001H,0C0H,080H,041H,000H,0C1H,081H,040H
 DB
 001H,0C0H,080H,041H,000H,0C1H,081H,040H,000H,0C1H,081H,040H,001H,0C0H,080H,041H
 DB
 000H,0C1H,081H,040H,001H,0C0H,080H,041H,001H,0C0H,080H,041H,000H,0C1H,081H,040H
 DB
 001H,0C0H,080H,041H,000H,0C1H,081H,040H,000H,0C1H,081H,040H,001H,0C0H,080H,041H
 DB
 000H,0C1H,081H,040H,001H,0C0H,080H,041H,001H,0C0H,080H,041H,000H,0C1H,081H,040H
 DB
 001H,0C0H,080H,041H,000H,0C1H,081H,040H,000H,0C1H,081H,040H,001H,0C0H,080H,041H
 DB
 000H,0C1H,081H,040H,001H,0C0H,080H,041H,001H,0C0H,080H,041H,000H,0C1H,081H,040H
 DB
 000H,0C0H,0C1H,001H,0C3H,003H,002H,0C2H,0C6H,006H,007H,0C7H,005H,0C5H,0C4H,004H
 DB
 0CCH,00CH,00DH,0CDH,00FH,0CFH,0CEH,00EH,00AH,0CAH,0CBH,00BH,0C9H,009H,008H,0C8H
 DB
 0D8H,018H,019H,0D9H,01BH,0DBH,0DAH,01AH,01EH,0DEH,0DFH,01FH,0DDH,01DH,01CH,0DCH
 DB
 014H,0D4H,0D5H,015H,0D7H,017H,016H,0D6H,0D2H,012H,013H,0D3H,011H,0D1H,0D0H,010H
 DB
 0F0H,030H,031H,0F1H,033H,0F3H,0F2H,032H,036H,0F6H,0F7H,037H,0F5H,035H,034H,0F4H
 DB
 03CH,0FCH,0FDH,03DH,0FFH,03FH,03EH,0FEH,0FAH,03AH,03BH,0FBH,039H,0F9H,0F8H,038H
 DB
 028H,0E8H,0E9H,029H,0EBH,02BH,02AH,0EAH,0EEH,02EH,02FH,0EFH,02DH,0EDH,0ECH,02CH
 DB
 0E4H,024H,025H,0E5H,027H,0E7H,0E6H,026H,022H,0E2H,0E3H,023H,0E1H,021H,020H,0E0H
 DB
 0A0H,060H,061H,0A1H,063H,0A3H,0A2H,062H,066H,0A6H,0A7H,067H,0A5H,065H,064H,0A4H
 DB
 06CH,0ACH,0ADH,06DH,0AFH,06FH,06EH,0AEH,0AAH,06AH,06BH,0ABH,069H,0A9H,0A8H,068H
 DB

```

078H,0B8H,0B9H,079H,0BBH,07BH,07AH,0BAH,0BEH,07EH,07FH,03FH,07DH,0BDH,0BCH,07CH
DB
0B4H,074H,075H,0B5H,077H,0B7H,0B6H,076H,072H,0B2H,0B3H,073H,0B1H,071H,070H,0B0H
DB
050H,090H,091H,051H,093H,053H,052H,092H,096H,056H,057H,097H,055H,095H,094H,054H
DB
09CH,05CH,05DH,09DH,05FH,09FH,09EH,05EH,05AH,09AH,09BH,05BH,099H,059H,058H,098H
DB
088H,048H,049H,089H,04BH,08BH,08AH,04AH,04EH,08EH,08FH,04FH,08DH,04DH,04CH,08CH
DB
044H,084H,085H,045H,087H,047H,046H,086H,082H,042H,043H,083H,041H,081H,080H,040H
NOP
NOP
LJMP MAIN

```

程序八 在 PC 上用并行口模拟 I²C 总线的 C 源代码

在微机上模拟 I²C 总线的设计中，用并行口的 D0 (PIN2) 模拟 SCL 信号，用 D1 (PIN3) 模拟 SDA 信号。根据 IIC 总线的电平规范，程序如下：

```

#include <dos.h>
#include <stdio.h>
#include <conio.h>
#include <bios.h>

#define DELAY_TIME 10
#define FALSE 0
#define TRUE 1
const unsigned char SCL_PIN=0x01;
const unsigned char SDA_OIN=0x02;

static void SET_SCL(void)
{
    asm{
        mov dx,0x378
        in ax,dx
        or ax,SCL_PIN    //将 SCL 置 1
        out dx,ax
    }
}

static void CLR_SCL(void)

```

```

{
    asm{
        mov dx,0x378
        in ax,dx
        or ax,0xff-SCL_PIN    //将 SCL 置 0
        out dx,ax
    }
}

static void SET_SDA(void)
{
    asm{
        mov dx,0x378
        in ax,dx
        or ax,SDA_PIN        //将 SDA 置 1
        out dx,ax
    }
}

static void CLR_SDA(void)
{
    asm{
        mov dx,0x378
        in ax,dx
        or ax,0xff-SDA_PIN    //将 SDA 置 0
        out dx,ax
    }
}

void DELAY(int t=DELAY_TIME)    //延时子程序
{
    for(int i=0;i<t;i++);
}

void IIC_Start(void)            //启动 I2C 总线
{
    //在 SCL 为高电平时，使 SDA 出现一个负跳变
    SET_SDA();
    SET_SCL();
    DELAY();
    CLR_SDA();
    DELAY();
    CLR_SCL();
    DELAY();
}

```



```

void IIC_Stop(void)          //终止 I2C 总线
{
    //在 SCL 为高电平时, 使 SDA 出现一个正跳变
    CLR_SDA();
    SET_SCL();
    DELAY();
    SET_SDA();
    DELAY();
    CLR_SCL();
    DELAY();
}

```

```

void SEND_0(void)           //发送 BIT0
{
    //在 SCL 为高电平时, 使 SDA 保持低电平
    CLR_SDA();
    SET_SCL();
    DELAY();
    CLR_SCL();
    DELAY();
}

```

```

void SEND_1(void)           //发送 BIT1
{
    //在 SCL 为高电平时, 使 SDA 保持高电平
    SET_SDA();
    SET_SCL();
    DELAY();
    CLR_SCL();
    DELAY();
}

```

```

int Check_Acknowledge(void)
{
    //发送完每个字节检查 SLAVE 的应答信号
    SET_SDA();          //主器件释放 SDA 线
    SET_SCL();
    DELAY(DELAY_TIME/2);
    unsigned char b=inportb(0x378); //采样信号线
    DELAY(DELAY_TIME/2);
    CLR_SCL();
    DELAY();
    if(b&0xSDA_PIN)      //SALVE 返回 1
        return FLASE;
}

```

```

    return TRUE;
}

```

使用上述程序，可以在 PC 上用并行口模拟 I²C，进行实验、监控和调试。

程序九 一种在 C51 中写二进制的方法

```

#define LongToBin(n) \
( \
((n >> 21) & 0x80) | \
((n >> 18) & 0x40) | \
((n >> 15) & 0x20) | \
((n >> 12) & 0x10) | \
((n >> 9) & 0x08) | \
((n >> 6) & 0x04) | \
((n >> 3) & 0x02) | \
((n) & 0x01) \
)

#define Bin(n) LongToBin(0x##n##1)

void main(void)
{
    unsigned char c;

    c = Bin(10101001); // then c = 0xA9
}

```

程序十 CRC 算法原理及 C 语言实现

1. CRC 原理

CRC 的全称是 “Cyclic Redundancy Check”，中文名称是 “循环冗余码”，“CRC 校验” 就是 “循环冗余校验”。

CRC 的应用范围很广泛，最常见的就是在网络传输中进行信息的校对。它的计算是非常严格的。程序只要被改动了一个字节（甚至只是大小写的改动），它的值就会跟原来的不同。所以只要给程序计算好 CRC 值，储存在某个地方，然后在程序中随机地再对文件进行 CRC 校验，接着跟第一次生成并保存好的 CRC 值进行比较，如果相等就说明你的程序没有被修改/破解过，如果不等，那么很可能是程序遭到了病毒的感染。下面介绍 CRC 的原理，

首先看两个式子：

$$\text{式一： } 9 / 3 = 3 \quad (\text{余数} = 0)$$

$$\text{式二： } (9 + 2) / 3 = 3 \quad (\text{余数} = 2)$$

大家知道，除法运算就是将被减数重复地减去除数 X 次，然后留下余数。所以上面的两个式子可以用二进制计算如下。

对式一：

$$1001 \quad \rightarrow 9$$

$$0011 \quad - \rightarrow 3$$

$$0110 \quad \rightarrow 6$$

$$0011 \quad - \rightarrow 3$$

$$0011 \quad \rightarrow 3$$

$$0011 \quad - \rightarrow 3$$

$$0000 \quad \rightarrow 0, \text{ 余数}$$

一共减了 3 次，所以商是 3，而最后一次减出来的结果是 0，所以余数为 0。

对式二：

$$1011 \quad \rightarrow 11$$

$$0011 \quad - \rightarrow 3$$

$$1000 \quad \rightarrow 8$$

$$0011 \quad - \rightarrow 3$$

$$0101 \quad \rightarrow 5$$

$$0011 \quad - \rightarrow 3$$

$$0010 \quad \rightarrow 2, \text{ 余数}$$

一共减了 3 次，所以商是 3，而最后一次减出来的结果是 2，所以余数为 2。

二进制减法运算的规则是，如果遇到 0-1 的情况，那么要从高位借 1，就变成了 $(10+0)-1=1$

注意最后的余数：

$$11110 \quad \rightarrow 30$$

$$1001 \quad - \rightarrow 9$$

$$1100 \quad \rightarrow 12 \quad (\text{很奇怪吧？为什么不是 21 呢？})$$

$$1001 \quad - \rightarrow 9$$

$$101 \quad \rightarrow 3, \text{ 余数} \rightarrow \text{the CRC!}$$

这个式子的计算过程不是直接减的，而是用 XOR 的方式来运算，最后得到一个余数。

这就是 CRC 的运算方法，CRC 的本质是进行 XOR 运算，运算的过程不用管它，因为运算过程对最后的结果没有意义；我们真正感兴趣的只是最终得到的余数，这个余数就是 CRC 值。

进行一个 CRC 运算需要选择一个除数，这个除数叫做“poly”，宽度 W 就是最高位的位置，所以刚才的例子中的除数 9，这个 poly 1001 的 W 是 3，而不是 4，注意最高位总是 1。

如果计算一个位串的 CRC 码，想确定每一个位都被处理过，要在目标位串后面加上 W 个 0 位。现在根据 CRC 的规范来改写一下上面的例子。

Poly = 1001, 宽度 $W=3$

位串 Bitstring = 11110

$$\text{Bitstring} + \text{W zeroes} = 11110 + 000 = 11110000$$

```

11110000
10011111 -
-----
11001111
10011111 -
-----
10101111
10011111 -
-----
01101111
00001111 -
-----
11001111
10011111 -
-----
10101111 --> 3, 余数 --> the CRC!

```

注意:

① 只有当 **Bitstring** 的最高位为 1，我们才将它与 **poly** 进行 XOR 运算，否则我们只是将 **Bitstring** 左移一位。

② XOR 运算的结果就是被操作位串 Bitstring 与 poly 的低 W 位进行 XOR 运算, 因为最高位总为 0。

2. CRC 具体编程

由于速度的关系，CRC 的实现主要是通过查表法，对于 CRC-16 和 CRC-32，各自有一个现成的表，可以直接引入到程序中使用。如果没有这个表就自己编程来生成这个表。这个表的 C 语言描述如下。

```
for (i = 0; i < 256; i++)
{
    crc = i;
```

```

for (j = 0; j < 8; j++)
{
    if (crc & 1)
        crc = (crc >> 1) ^ 0xEDB88320;
    else
        crc >>= 1;
}
crc32tbl[i] = crc;
}

```

生成表之后，就可以进行运算了。

算法如下：

(1) 将寄存器向右边移动一个字节。

(2) 将刚移出的那个字节与字符串中的新字节进行 XOR 运算，得出一个指向值表 table[0..255]的索引。

(3) 将索引所指的表值与寄存器做 XOR 运算。

(4) 如果数据没有全部处理完，则跳到步骤 (1)。

这个算法的 C 语言描述如下：

```

temp = (oldcrc ^ abyte) & 0x000000FF;
crc = (( oldcrc >> 8) & 0x00FFFFFF) ^ crc32tbl[temp];
return crc;

```

3. CRC32 原理的实现

```

include windows.inc
include kernel32.inc
include user32.inc
includelib kernel32.lib
includelib user32.lib

WndProc      proto :DWORD, :DWORD, :DWORD, :DWORD
init_crc32table proto
arraycrc32   proto

.const
IDC_BUTTON_OPEN    equ 3000
IDC_EDIT_INPUT     equ 3001

.data
szDlgName      db "lc_dialog", 0
szTitle        db "CRC demo by LC", 0
szTemplate     db "字符串 ""%s"" 的 CRC32 值是: %X", 0
crc32tbl       dd 256 dup(0) CRC-32 table
szBuffer       db 255 dup(0)

```

```

.data?
szText          db  300 dup(?)

.code
main:
    invoke GetModuleHandle, NULL
    invoke DialogBoxParam, eax, offset szDlgName, 0, WndProc, 0
    invoke ExitProcess, eax

WndProc proc uses ebx hWnd:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM

    .if uMsg == WM_CLOSE
        invoke EndDialog, hWnd, 0

    .elseif uMsg == WM_COMMAND
        mov eax, wParam
        mov edx, eax
        shr edx, 16
        movzx eax, ax
        .if edx == BN_CLICKED
            .IF eax == IDCANCEL
                invoke EndDialog, hWnd, NULL
            .ELSEIF eax == IDC_BUTTON_OPEN || eax == IDOK
                *****
                关键代码开始
                *****
                取得用户输入的字符串:
                invoke GetDlgItemText, hWnd, IDC_EDIT_INPUT, addr szBuffer, 255

                初始化 crc32table:
                invoke init_crc32table

                下面赋值给寄存器 ebx, 以便进行 crc32 转换:
                EBX 是待转换的字符串的首地址:
                lea ebx, szBuffer

                进行 crc32 转换:
                invoke arraycrc32

                格式化输出:
                invoke wsprintf, addr szText, addr szTemplate, addr szBuffer, eax

                显示结果:
                invoke MessageBox, hWnd, addr szText, addr szTitle, MB_OK
            .ENDIF

```

```

        .endif
    .ELSE
        mov eax,FALSE
        ret
    .ENDIF
    mov eax,TRUE
    ret
WndProc endp
;*****
;函数功能: 生成 CRC-32 表
;*****
init_crc32table proc

```

如果用 C 语言来表示, 应该如下:

```

    for (i = 0; i < 256; i++)
    {
        crc = i;
        for (j = 0; j < 8; j++)
        {
            if (crc & 1)
                crc = (crc >> 1) ^ 0xEDB88320;
            else
                crc >>= 1;
        }
        crc32tbl[i] = crc;
    }

```

把上面的语句改成 assembly 形式为:

```

        mov     ecx, 256      repeat for every DWORD in table
        mov     edx, 0EDB88320h
$BigLoop:
        lea     eax, [ecx-1]
        push    ecx
        mov     ecx, 8
$SmallLoop:
        shr     eax, 1
        jnc     @F
        xor     eax, edx
@@:
        dec     ecx
        jne     $SmallLoop
        pop     ecx
        mov     [crc32tbl+ecx*4-4], eax
        dec     ecx
        jne     $BigLoop

```

```

        ret
init_crc32table endp

;*****
;函数功能: 计算 CRC-32
;*****
arraycrc32 proc

```

计算 CRC-32, 我采用的是把整个字符串当做一个数组, 然后把这个数组的首地址赋值给 EBX, 把数组的长度赋值给 ECX, 然后循环计算, 返回值 (计算出来的 CRC-32 值) 储存在 EAX 中:

```

    参数:
        EBX = address of first byte
    返回值:
        EAX = CRC-32 of the entire array
        EBX = ?
        ECX = 0
        EDX = ?
    mov  eax, -1 ; 先初始化 eax
    or   ebx, ebx
    jz   $Done ; 避免出现空指针
@@:
    mov  dl, [ebx]
    or   dl, dl
    je   $Done 判断是否对字符串扫描完毕

```

这里用查表法来计算 CRC-32, 因此非常快速。

因为这是 assembly 代码, 所以不需要给这个过程传递参数, 只需要把 oldcrc 赋值给 EAX, 以及把 byte 赋值给 DL。

在 C 语言中的形式:

```

temp = (oldcrc ^ abyte) & 0x000000FF;
crc = (( oldcrc >> 8) & 0x0FFFFFFF) ^ crc32tbl[temp];

```

```

    参数:
        EAX = old CRC-32
        DL = a byte
    返回值:
        EAX = new CRC-32
        EDX = ?

```

```

xor    dl, al
movzx  edx, dl

```



```

        shr     eax, 8
        xor     eax, [crc32tbl+edx*4]
        inc     ebx
        jmp     @B

$Done:
        not     eax
        ret
arraycrc32     endp

end main
;***** over *****
;by LC

```

下面是它的.rc 文件:

```

#include "resource.h"

#define IDC_BUTTON_OPEN 3000
#define IDC_EDIT_INPUT 3001
#define IDC_STATIC -1

LC_DIALOG DIALOGEX 10, 10, 195, 60
STYLE DS_SETFONT | DS_CENTER | WS_MINIMIZEBOX | WS_VISIBLE | WS_CAPTION |
WS_SYSMENU
CAPTION "lc's assembly framework"
FONT 9, "宋体", 0, 0, 0x0
BEGIN
    LTEXT        "请输入一个字符串 (区分大小写): ", IDC_STATIC, 11, 7, 130, 10
    EDITTEXT     IDC_EDIT_INPUT, 11, 20, 173, 12, ES_AUTOHSCROLL
    DEFPUSHBUTTON "Ca&lc", IDC_BUTTON_OPEN, 71, 39, 52, 15
END

```

程序十一 软件陷阱

CPU 受到干扰后, 往往将一些操作数当做指令码来执行, 造成程序执行混乱。在编程时, 主要有以下几种处理办法。

(1) 中断向量区

```

        ORG 0000H
START:  LJMP MAIN
        LJMP INT0
        NOP
        NOP
        LJMP ERR    ;陷阱

```

```

    LJMP TOINT
    NOP
    NOP
    LJMP ERR    ;陷阱

```

```

    ORG 0040H
ERR:

```

(2) 在表格区

在表格区的最后安排 5 个字节的陷阱

```

TABEL1:
DB -----
DB -----
NOP
NOP
LJMP ERR

```

(3) 在未使用的 ROM 空间

未使用的 ROM 空间一般全是 0FFH，对于 51 来说是"MOV R7,A"的单字节指令，程序一旦弹飞到这个区域，将会飞流直下。一般在一些固定的地址加入软件陷阱，捕获弹飞的程序。

```

ORG 6000H
NOP
NOP
LJMP ERR
ORG 7FFBH
NOP
NOP
LJMP ERR

```

(4) 在子程序后面

```

XXXX:
.....
.....
RET
NOP
NOP
LJMP ERR

```

以及在一些长跳转的断裂点...

注: ERR 子程序，应当重新设定堆栈等一些初始化的参数，但对于 RAM 区的部分数据可以判断保留。

程序十二 一个简单的 VB 串口发送程序

'-----发送按钮 Click 事件子程序-----

```

Private Sub Fasong_Click()
Dim JIHAO(0) As Byte      '机号
Dim head_data(4) As Byte  '5 Byte 控制字
Dim end_data(0) As Byte   '1 Byte 结束字
    JIHAO(0) = Val(Text3.Text)
    head_data(0) = Val(Text4.Text)
    head_data(2) = &HEE      'TIMH
    head_data(3) = &HEE      'TIML
    head_data(4) = Val(Combo1.Text) 'INMOD
    end_data(0) = &HFF
    If Combo2.Text = "增加" Then head_data(1) = &H99
    If Combo2.Text = "清空" Then head_data(1) = &H33
    If Combo2.Text = "删除" Then head_data(1) = &H32
    Ready = 0: ErrCount = 0
    On Error GoTo ERRORCOM      '打开错误处理
'-----
    If com1.Value Then MSComm1.CommPort = 1 'Use com1
    If com2.Value Then MSComm1.CommPort = 2 'Use com2

    MSComm1.Settings = FORM1.Combo3.Text + ",M,8,2" '设定波特率和置校验和位为 1
    MSComm1.InputLen = 0
    MSComm1.PortOpen = -1      'Open the port
    MSComm1.OutBufferCount = 0
    MSComm1.Output = JIHAO      '发送机号
    MSComm1.PortOpen = False    '关闭串口
    MSComm1.Settings = FORM1.Combo3.Text + ",S,8,2" '设定波特率和置校验和位为空
    MSComm1.OutBufferCount = 0
    MSComm1.PortOpen = True
    MSComm1.Output = head_data
    MSComm1.Output = Text2.Text
    MSComm1.Output = end_data
    MSComm1.PortOpen = False
    Text1.Text = "发送成功!" + Chr(13) & Chr(10) + "发送至" + Text3.Text + "屏体," + "信息编号:" + Text4.Text
+ Chr(13) & Chr(10) + Chr(13) & Chr(10) + Text1.Text
    GoTo comend
ERRORCOM:
    Text1.Text = "ERROR!请重新选择 COM 口!" + Chr(13) & Chr(10) + Chr(13) & Chr(10) + Text1.Text
comend:
    On Error GoTo 0
End Sub

```

程序十三 12864 汉字液晶显示驱动程序

```
sbit p_csa=P2^6;
sbit p_csb=P2^7;
sbit p_gnda=P2^5;
sbit p_gndb=P2^4;
sbit p_di=P2^3;
sbit p_rw=P2^2;
sbit p_e=P2^1;

/*忙判别*/
void lcd_busy(void) {
    p_di=0;p_rw=1;P0=0xff;
    while (1) {
        p_e=1;
        if (P0<0x80) break;
        p_e=0;
    }
    p_e=0;
}

/*设置 xy*/
void set_xy(unsigned char x,unsigned char y) {
    if (x>=64) {p_csa=0;p_csb=1;} else {p_csb=0;p_csa=1;}
    lcd_busy();
    p_di=p_rw=0;P0=0x40x;p_e=1;p_e=0;
    lcd_busy();
    p_di=p_rw=0;P0=0xb8ly;p_e=1;p_e=0;
    P0=0xff;
}

void lw(unsigned char x,unsigned char y,unsigned char dd) {
    set_xy(x,y);
    lcd_busy();p_di=1;p_rw=0;P0=dd;p_e=1;p_e=0;
    P0=0xff;
}

/*显示初始化*/
#pragma disable
void lcd_init(void) {
    unsigned char x,y;
    p_gnda=p_gndb=0;
    /*开显示*/
    p_e=p_di=p_rw=0;
    p_csa=p_csb=0;
```

```

p_csa=1;P0=0x3f;p_e=1;p_e=0;p_csa=0;
p_csb=1;P0=0x3f;p_e=1;p_e=0;p_csb=0;
/*0 行开始显示*/
p_csa=1;lcd_busy();p_di=p_rw=0;P0=0xc0;p_e=1;p_e=0;p_csa=0;
p_csb=1;lcd_busy();p_di=p_rw=0;P0=0xc0;p_e=1;p_e=0;p_csb=0;
for (y=0;y<8;y++) {
    for (x=0;x<128;x++) lw(x,y,0);
}
}

```

```

unsigned char code asc[]={
0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,
0x0,0x0,0x38,0xfc,0xfc,0x38,0x0,0x0,0x0,0x0,0xd,0xd,0x0,0x0,0x0,
0x0,0xe,0x1e,0x0,0x0,0x1e,0xe,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,
0x20,0xf8,0xf8,0x20,0xf8,0xf8,0x20,0x2,0xf,0xf,0x2,0xf,0xf,0x2,0x0,
0x38,0x7c,0x44,0x47,0x47,0xcc,0x98,0x0,0x6,0xc,0x8,0x38,0x38,0xf,0x7,0x0,
0x30,0x30,0x0,0x80,0xc0,0x60,0x30,0x0,0xc,0x6,0x3,0x1,0x0,0xc,0xc,0x0,
0x80,0xd8,0x7c,0xe4,0xbc,0xd8,0x40,0x0,0x7,0xf,0x8,0x8,0x7,0xf,0x8,0x0,
0x0,0x10,0x1e,0xe,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,
0x0,0x0,0xf0,0xf8,0xc,0x4,0x0,0x0,0x0,0x0,0x3,0x7,0xc,0x8,0x0,0x0,
0x0,0x0,0x4,0xc,0xf8,0xf0,0x0,0x0,0x0,0x8,0xc,0x7,0x3,0x0,0x0,
0x80,0xa0,0xe0,0xc0,0xc0,0xe0,0xa0,0x80,0x0,0x2,0x3,0x1,0x1,0x3,0x2,0x0,
0x0,0x80,0x80,0xe0,0xe0,0x80,0x80,0x0,0x0,0x0,0x0,0x3,0x3,0x0,0x0,0x0,
0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x10,0x1e,0xe,0x0,0x0,0x0,
0x80,0x80,0x80,0x80,0x80,0x80,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,
0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0xc,0xc,0x0,0x0,0x0,
0x0,0x0,0x0,0x80,0xc0,0x60,0x30,0x0,0xc,0x6,0x3,0x1,0x0,0x0,0x0,0x0,
0xf0,0xf8,0xc,0xc4,0xc,0xf8,0xf0,0x3,0x7,0xc,0x8,0xc,0x7,0x3,0x0,
0x0,0x10,0x18,0xfc,0xfc,0x0,0x0,0x0,0x8,0x8,0xf,0xf,0x8,0x8,0x0,
0x8,0xc,0x84,0xc4,0x64,0x3c,0x18,0x0,0xe,0xf,0x9,0x8,0x8,0xc,0xc,0x0,
0x8,0xc,0x44,0x44,0x44,0xfc,0xb8,0x0,0x4,0xc,0x8,0x8,0x8,0xf,0x7,0x0,
0xc0,0xe0,0xb0,0x98,0xfc,0xfc,0x80,0x0,0x0,0x0,0x8,0xf,0xf,0x8,0x0,
0x7c,0x7c,0x44,0x44,0x44,0xc4,0x84,0x0,0x4,0xc,0x8,0x8,0x8,0xf,0x7,0x0,
0xf0,0xf8,0x4c,0x44,0x44,0xc0,0x80,0x0,0x7,0xf,0x8,0x8,0x8,0xf,0x7,0x0,
0xc,0xc,0x4,0x84,0xc4,0x7c,0x3c,0x0,0x0,0x0,0xf,0xf,0x0,0x0,0x0,0x0,
0xb8,0xfc,0x44,0x44,0x44,0xfc,0xb8,0x0,0x7,0xf,0x8,0x8,0x8,0xf,0x7,0x0,
0x38,0x7c,0x44,0x44,0x44,0xfc,0xf8,0x0,0x0,0x8,0x8,0x8,0xc,0x7,0x3,0x0,
0x0,0x0,0x0,0x30,0x30,0x0,0x0,0x0,0x0,0x0,0x6,0x6,0x0,0x0,0x0,
0x0,0x0,0x0,0x30,0x30,0x0,0x0,0x0,0x0,0x8,0xe,0x6,0x0,0x0,0x0,
0x0,0x80,0xc0,0x60,0x30,0x18,0x8,0x0,0x0,0x0,0x1,0x3,0x6,0xc,0x8,0x0,
0x0,0x20,0x20,0x20,0x20,0x20,0x20,0x0,0x0,0x1,0x1,0x1,0x1,0x1,0x0,
0x0,0x8,0x18,0x30,0x60,0xc0,0x80,0x0,0x0,0x8,0xc,0x6,0x3,0x1,0x0,0x0,
0x18,0x1c,0x4,0xc4,0xe4,0x3c,0x18,0x0,0x0,0x0,0x0,0xd,0xd,0x0,0x0,0x0,
0xf0,0xf8,0x8,0xc8,0xc8,0xf8,0xf0,0x0,0x7,0xf,0x8,0xb,0xb,0xb,0x1,0x0,
0xe0,0xf0,0x98,0x8c,0x98,0xf0,0xe0,0x0,0xf,0xf,0x0,0x0,0x0,0xf,0xf,0x0,
0x4,0xfc,0xfc,0x44,0x44,0xfc,0xb8,0x0,0x8,0xf,0xf,0x8,0x8,0xf,0x7,0x0,

```

0xf0,0xf8,0xc,0x4,0x4,0xc,0x18,0x0,0x3,0x7,0xc,0x8,0x8,0xc,0x6,0x0,
 0x4,0xfc,0xfc,0x4,0xc,0xf8,0xf0,0x0,0x8,0xf,0xf,0x8,0xc,0x7,0x3,0x0,
 0x4,0xfc,0xfc,0x44,0xe4,0xc,0x1c,0x0,0x8,0xf,0xf,0x8,0xc,0xe,0x0,
 0x4,0xfc,0xfc,0x44,0xe4,0xc,0x1c,0x0,0x8,0xf,0xf,0x8,0x0,0x0,0x0,0x0,
 0xf0,0xf8,0xc,0x84,0x84,0x8c,0x98,0x0,0x3,0x7,0xc,0x8,0x8,0x7,0xf,0x0,
 0xfc,0xfc,0x40,0x40,0x40,0xfc,0xfc,0x0,0xf,0xf,0x0,0x0,0x0,0xf,0xf,0x0,
 0x0,0x0,0x4,0xfc,0xfc,0x4,0x0,0x0,0x0,0x0,0x8,0xf,0xf,0x8,0x0,0x0,
 0x0,0x0,0x0,0x4,0xfc,0xfc,0x4,0x0,0x7,0xf,0x8,0x8,0xf,0x7,0x0,0x0,
 0x4,0xfc,0xfc,0xc0,0xe0,0x3c,0x1c,0x0,0x8,0xf,0xf,0x0,0x1,0xf,0xe,0x0,
 0x4,0xfc,0xfc,0x4,0x0,0x0,0x0,0x0,0x8,0xf,0xf,0x8,0x8,0xc,0xe,0x0,
 0xfc,0xfc,0x38,0x70,0x38,0xfc,0xfc,0x0,0xf,0xf,0x0,0x0,0x0,0xf,0xf,0x0,
 0xfc,0xfc,0x38,0x70,0xe0,0xfc,0xfc,0x0,0xf,0xf,0x0,0x0,0x0,0xf,0xf,0x0,
 0xf8,0xfc,0x4,0x4,0x4,0xfc,0xf8,0x0,0x7,0xf,0x8,0x8,0x8,0xf,0x7,0x0,
 0x4,0xfc,0xfc,0x44,0x44,0x7c,0x38,0x0,0x8,0xf,0xf,0x8,0x0,0x0,0x0,0x0,
 0xf8,0xfc,0x4,0x4,0x4,0xfc,0xf8,0x0,0x7,0xf,0x8,0xe,0x3c,0x3f,0x27,0x0,
 0x4,0xfc,0xfc,0x44,0xc4,0xfc,0x38,0x0,0x8,0xf,0xf,0x0,0x0,0xf,0xf,0x0,
 0x18,0x3c,0x64,0x44,0xc4,0x9c,0x18,0x0,0x6,0xe,0x8,0x8,0xf,0x7,0x0,
 0x0,0x1c,0xc,0xfc,0xfc,0xc,0x1c,0x0,0x0,0x0,0x8,0xf,0xf,0x8,0x0,0x0,
 0xfc,0xfc,0x0,0x0,0x0,0xfc,0xfc,0x0,0x7,0xf,0x8,0x8,0xf,0x7,0x0,
 0xfc,0xfc,0x0,0x0,0x0,0xfc,0xfc,0x0,0x1,0x3,0x6,0xc,0x6,0x3,0x1,0x0,
 0xfc,0xfc,0x0,0xc0,0x0,0xfc,0xfc,0x0,0x7,0xf,0xe,0x3,0xe,0xf,0x7,0x0,
 0xc,0x3c,0xf0,0xe0,0xf0,0x3c,0xc,0x0,0xc,0xf,0x3,0x1,0x3,0xf,0xc,0x0,
 0x0,0x3c,0x7c,0xc0,0xc0,0x7c,0x3c,0x0,0x0,0x0,0x8,0xf,0xf,0x8,0x0,0x0,
 0x1c,0xc,0x84,0xc4,0x64,0x3c,0x1c,0x0,0xe,0xf,0x9,0x8,0xc,0xe,0x0,
 0x0,0x0,0xfc,0xfc,0x4,0x4,0x0,0x0,0x0,0x0,0xf,0xf,0x8,0x8,0x0,0x0,
 0x38,0x70,0xe0,0xc0,0x80,0x0,0x0,0x0,0x0,0x0,0x1,0x3,0x7,0xe,0x0,
 0x0,0x0,0x4,0x4,0xfc,0xfc,0x0,0x0,0x0,0x0,0x8,0xf,0xf,0x0,0x0,
 0x8,0xc,0x6,0x3,0x6,0xc,0x8,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,
 0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x20,0x20,0x20,0x20,0x20,0x20,0x20,
 0x0,0x0,0x3,0x7,0x4,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,
 0x0,0xa0,0xa0,0xa0,0xe0,0xc0,0x0,0x0,0x7,0xf,0x8,0x8,0x7,0xf,0x8,0x0,
 0x4,0xfc,0xfc,0x20,0x60,0xc0,0x80,0x0,0x0,0xf,0xf,0x8,0x8,0xf,0x7,0x0,
 0xc0,0xe0,0x20,0x20,0x20,0x60,0x40,0x0,0x7,0xf,0x8,0x8,0xc,0x4,0x0,
 0x80,0xc0,0x60,0x24,0xfc,0xfc,0x0,0x0,0x7,0xf,0x8,0x8,0x7,0xf,0x8,0x0,
 0xc0,0xe0,0xa0,0xa0,0xa0,0xe0,0xc0,0x0,0x7,0xf,0x8,0x8,0x8,0xc,0x4,0x0,
 0x40,0xf8,0xfc,0x44,0xc,0x18,0x0,0x0,0x8,0xf,0xf,0x8,0x0,0x0,0x0,0x0,
 0xc0,0xe0,0x20,0x20,0xc0,0xe0,0x20,0x0,0x27,0x6f,0x48,0x7f,0x3f,0x0,0x0,
 0x4,0xfc,0xfc,0x40,0x20,0xe0,0xc0,0x0,0x8,0xf,0xf,0x0,0x0,0xf,0xf,0x0,
 0x0,0x0,0x20,0xec,0xec,0x0,0x0,0x0,0x0,0x0,0x8,0xf,0xf,0x8,0x0,0x0,
 0x0,0x0,0x0,0x0,0x20,0xec,0xec,0x0,0x0,0x30,0x70,0x40,0x40,0x7f,0x3f,0x0,
 0x4,0xfc,0xfc,0x80,0xc0,0x60,0x20,0x0,0x8,0xf,0xf,0x1,0x3,0xe,0xc,0x0,
 0x0,0x0,0x4,0xfc,0xfc,0x0,0x0,0x0,0x0,0x0,0x8,0xf,0xf,0x8,0x0,0x0,
 0xe0,0xe0,0x60,0xc0,0x60,0xe0,0xc0,0x0,0xf,0xf,0x0,0x7,0x0,0xf,0xf,0x0,
 0x20,0xe0,0xc0,0x20,0x20,0xe0,0xc0,0x0,0x0,0xf,0xf,0x0,0x0,0xf,0xf,0x0,
 0xc0,0xe0,0x20,0x20,0x20,0xe0,0xc0,0x0,0x7,0xf,0x8,0x8,0xf,0x7,0x0,
 0x20,0xe0,0xc0,0x20,0x20,0xe0,0xc0,0x0,0x40,0x7f,0x7f,0x48,0x8,0xf,0x7,0x0,

```

0xc0,0xe0,0x20,0x20,0xc0,0xe0,0x20,0x0,0x7,0xf,0x8,0x48,0x7f,0x7f,0x40,0x0,
0x20,0xe0,0xc0,0x60,0x20,0xe0,0xc0,0x0,0x8,0xf,0xf,0x8,0x0,0x0,0x0,0x0,
0x40,0xe0,0xa0,0x20,0x20,0x60,0x40,0x0,0x4,0xc,0x9,0x9,0xb,0xe,0x4,0x0,
0x20,0x20,0xf8,0xfc,0x20,0x20,0x0,0x0,0x0,0x7,0xf,0x8,0xc,0x4,0x0,
0xe0,0xe0,0x0,0x0,0xe0,0xe0,0x0,0x0,0x7,0xf,0x8,0x8,0x7,0xf,0x8,0x0,
0x0,0xe0,0xe0,0x0,0x0,0xe0,0xe0,0x0,0x3,0x7,0xc,0xc,0x7,0x3,0x0,
0xe0,0xe0,0x0,0x80,0x0,0xe0,0xe0,0x0,0x7,0xf,0xc,0x7,0xc,0xf,0x7,0x0,
0x20,0x60,0xc0,0x80,0xc0,0x60,0x20,0x0,0x8,0xc,0x7,0x3,0x7,0xc,0x8,0x0,
0xe0,0xe0,0x0,0x0,0x0,0xe0,0xe0,0x0,0x47,0x4f,0x48,0x48,0x68,0x3f,0x1f,0x0,
0x60,0x60,0x20,0xa0,0xe0,0x60,0x20,0x0,0xc,0xe,0xb,0x9,0x8,0xc,0xc,0x0,
0x0,0x40,0x40,0xf8,0xbc,0x4,0x4,0x0,0x0,0x0,0x7,0xf,0x8,0x8,0x0,
0x0,0x0,0x0,0xbc,0xbc,0x0,0x0,0x0,0x0,0xf,0xf,0x0,0x0,0x0,
0x0,0x4,0x4,0xbc,0xf8,0x40,0x40,0x0,0x0,0x8,0x8,0xf,0x7,0x0,0x0,0x0,
0x8,0xc,0x4,0xc,0x8,0xc,0x4,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,
0x80,0xc0,0x60,0x30,0x60,0xc0,0x80,0x0,0x7,0x7,0x4,0x4,0x7,0x7,0x0
};

```

```

unsigned char code hz[]={
/*解: 0*/
0x20,0x10,0xec,0x27,0xf4,0x2c,0xe4,0x40,
0x22,0x9e,0x2,0xd2,0x22,0x1f,0x2,0x0,
0x80,0x40,0x3f,0x9,0x3f,0x89,0xff,0x8,
0xa,0x9,0x9,0xff,0x9,0x9,0x8,0x0
};

```

//显示汉字, fb==1:反白显示

```

void dh(unsigned char x,unsigned char y,unsigned char n,unsigned char fb) {
    unsigned char i,dd;
    for (i=0;i<16;i++) {
        dd=hz[i+n*32];
        if (fb) if (n!=kongbai) dd=255-dd;
        lw(x*8+i,y,dd);
        dd=hz[i+n*32+16];
        if (fb) if (n!=kongbai) dd=255-dd;
        lw(x*8+i,y+1,dd);
    }
}

```

//显示字符, fb==1:反白显示

```

void da(unsigned char x,unsigned char y,unsigned char n,unsigned char fb) {
    unsigned char i,dd;
    n-=32;
    if (n>128) n=0;

```

```

        if (fb) dd=255; else dd=0;
        lw(x*8,y,dd);
        lw(x*8,y+1,dd);
        for (i=0;i<7;i++) {
            dd=asc[i+n*16];
            if (fb) dd=255-dd;
            lw(x*8+i+1,y,dd);
            dd=asc[i+n*16+8];
            if (fb) dd=255-dd;
            lw(x*8+i+1,y+1,dd);
        }
    }
}

```

程序十四 12232 点阵液晶基本驱动程序

说明:

该液晶左右分 MASTER 和 SLAVE, 上下共分 4 页, 左边列地址从 0-61, 右边列地址从 0-61, 由于芯片内部线路的落后, 所以使用较麻烦。

```
#define E2 P3_5
```

```
#define E1 P3_6
```

```
#define A0 P3_7
```

```
#define DATA P2
```

```
/*-----
```

调用方式: void OutMI(uchar i)

函数说明: 发指令 i 到主窗口。(内函数, 私有, 用户不直接调用)

```
-----*/
```

```
void OutMI(uchar i)
```

```
{
```

```
E1=1;_nop();_nop();
```

```
A0=0;_nop();_nop();
```

```
DATA=i;_nop();_nop();
```

```
E1=0;_nop();_nop();
```

```
}
```

```
/*-----
```

```
--
```

调用方式: void OutMD(uchar i)

函数说明: 发数据 i 到主窗口。(内函数, 私有, 用户不直接调用)

```
-----
```

```
*/
```

```
void OutMD(uchar i)
```

```
{
```



```

E1=1;_nop();_nop();
A0=1;_nop();_nop();
DATA=i;_nop();_nop();
E1=0;_nop();_nop();
}
/*-----*/
调用方式: void OutSI(uchar i)
函数说明: 发指令 i 到从窗口。(内函数, 私有, 用户不直接调用)
/*-----*/

void OutSI(uchar i)
{
A0=0;_nop();_nop();
E2=1;_nop();_nop();
DATA=i;_nop();_nop();
E2=0;_nop();_nop();
}

/*-----*/
调用方式: void OutSD(uchar i)
函数说明: 发数据 i 到从窗口。(内函数, 私有, 用户不直接调用)
/*-----*/

void OutSD(uchar i)
{
A0=1;_nop();_nop();
E2=1;_nop();_nop();
DATA=i;_nop();_nop();
E2=0;_nop();_nop();
}

/*-----*/
调用方式: void LcdIni(void)
函数说明: 12232 点阵液晶初始化, 开机后仅调用一次。
/*-----*/

void LcdIni(void)
{
OutMI(0XE2);OutSI(0XE2);//复位
OutMI(0XAE);OutSI(0XAE);//POWER SAVE
OutMI(0XA4);OutSI(0XA4);//动态驱动
OutMI(0XA9);OutSI(0XA9);//1/32 占空比
OutMI(0XA0);OutSI(0XA0);//时钟线输出
OutMI(0XEE);OutSI(0XEE);//写模式

OutMI(0X00);OutMI(0XC0);
OutSI(0X00);OutSI(0XC0);

```

```
OutMI(0XAF);OutSI(0XAF);
}
```

```
/*-----*/
```

调用方式: void SetPage(uchar page0,uchar page1)

函数说明: 同时设置主从显示页为 0—3 页。(内函数, 私有, 用户不直接调用)

```
-----*/
```

```
void SetPage(uchar page0,uchar page1)
{
OutMI(0xB8|page1);OutSI(0xB8|page0);
}
```

```
/*-----*/
```

调用方式: void SetAddress(uchar address0,uchar address1)

函数说明: 同时设置主从列地址为 0—121。(内函数, 私有, 用户不直接调用)

```
-----*/
```

```
void SetAddress(uchar address0,uchar address1)
{
OutMI(address1&0x7F);OutSI(address0&0x7F);
}
```

```
/*-----*/
```

调用方式: void PutChar0(uchar ch)

函数说明: 在左页当前地址画一个字节 8 个点。(内函数, 私有, 用户不直接调用)

```
-----*/
```

```
void PutChar0(uchar ch)
{
OutSD(ch);
}
```

```
/*-----*/
```

调用方式: void PutChar1(uchar ch)

函数说明: 在右页当前地址画一个字节 8 个点。(内函数, 私有, 用户不直接调用)

```
-----*/
```

```
void PutChar1(uchar ch)
{
OutMD(ch);
}
```

```
/*-----*/
```

调用方式: void DrawBmp(uchar x,bit layer,uchar width,uchar *bmp)

函数说明: 画一个图, 横坐标是 x, layer 表示上下层, width 是图形的宽, 高都是 16, bmp 是图形指针, 使用 zimo21 软件, 采用纵向取模得到 bmp 数据。

```
-----*/
```

```
void DrawBmp(uchar x0,bit layer,uchar width,uchar *bmp)
```

```

{
uchar x,address,i=0; //address 表示显存的物理地址
uchar page=0;
bit window=0; //page 表示上下两页, window 表示左右两页
if (layer) page=2;

for (x=x0;x<x0+width;x++)
{
if (x>60) {window=1;address=x%61;}
else address=x;

SetPage(page,page);
SetAddress(address,address);

if (window) PutChar1 bmp[i];
else PutChar0 bmp[i]; //画上层

SetPage(page+1,page+1);
SetAddress(address,address);

if (window) PutChar1 bmp[i+width];
else PutChar0 bmp[i+width]; //画下层
i++;
}
}

/*-----
调用方式: void clrscr(void)
函数说明: 清屏
-----*/

void clrscr(void)
{
uchar i;
uchar page;
for (page=0;page<4;page++)
{
SetPage(page,page);
SetAddress(0,0);
for (i=0;i<61;i++){PutChar0(0);PutChar1(0);}
}
}

```

程序十五 串口中断服务函数集

```
//串口中断服务程序, 仅需做简单调用即可完成串口输入输出的处理
//出入均设有缓冲区, 大小可任意设置。
//可供使用的函数名:
//char getbyte(void);从接收缓冲区取一个 byte,如不想等待则在调用前检测 inbufsign 是否为 1。
//getline(char idata *line, unsigned char n); 获取一行数据回车结束, 必须定义最大输入字符数
//putbyte(char c);放入一个字节到发送缓冲区
//putbytes(unsigned char *outplace,j);放一串数据到发送缓冲区, 自定义长度
//putstring(unsigned char code *puts);发送一个定义在程序存储区的字符串到串口
//puthex(unsigned char c);发送一个字节的 hex 码, 分成两个字节发。
//putchar(uchar c,uchar j);输出一个无符号字符数的十进制表示, 必须标示小数点的位置,自动删除前面
无用的零
//putint(uint ui,uchar j);输出一个无符号整型数的十进制表示, 必须标示小数点的位置,自动删除前面无
用的零
//delay(unsigned char d); 延时 n x 100ns
//putinbuf(uchar c);人工输入一个字符到输入缓冲区
//CR;发送一个回车换行
//*****

#include
#define uchar unsigned char
#define uint unsigned int
#define OLEN 32 /* 发串行缓冲 */
idata unsigned char outbuf[OLEN]; /* 存储缓冲 */
unsigned char idata *outlast=outbuf; //最后由中断传输出去的字节位置
unsigned char idata *putlast=outbuf; //最后放入发送缓冲区的字节位置
#define ILEN 12 /* 收串行缓冲 */
idata unsigned char inbuf[ILEN];
unsigned char idata *inlast=inbuf; //最后由中断进入接收缓冲区的字节位置
unsigned char idata *getlast=inbuf; //最后取走的字节位置
bit outbufsign0; //最后一个数据帧 BUF 发完标志 发完=0
bit outbufsign; //输出缓冲区非空标志 有=1
bit inbufsign; //接收缓冲区非空标志 有=1
bit inbufful; //输入缓冲区满标志 满=1
#define CR putstring("\r\n") //CR=回车换行

//*****
//延时 n x 100ns
void delay(unsigned char d) //在源程序开头定义是否用 w77e58 或 22. 1184M 晶振
{unsigned char j;
do{ d--;
```

```

//110592 & 89c52
#ifndef cpuw77e58
    #ifndef xtal221184
        j=21;           //k=38 cpu80320 100us k=21 cpu 8052
        #else
        j=42;
        #endif
    #else
        #ifndef xtal221184
        j=38;
        #else
        j=76;
        #endif
    #endif

do {j--;} while(j!=0);
}while(d!=0);
}
//*****
//放入一个字节到发送缓冲区

putbyte(char c)
{uchar i,j;
    ES=0;           /*暂停串行中断，以免数据比较时出错? */
    //if (outlast==putlast)
    while ( (((outlast-putlast)==2) && (outlast > putlast )) || ((outlast < putlast) && (OLEN-(putlast-
outlast)==2)))
    { ES=1; c++;c--;ES=0;
    //    i=(0-TH1);
    //    do {i--;j=39; do {j--;}while(j!=0); }while(i!=0);    //i=39
    }
    *putlast=c;           //放字节进入缓冲区
    putlast++;           //发送缓冲区指针加一
    if (putlast==outbuf+OLEN) putlast=outbuf; //指针到了顶部换到底部
    outbufsign=1;
    if (!outbufsign0) {outbufsign0=1;TI=1; } //缓冲区开始为空置为有，启动发送
    ES=1;
}
//*****
//放一串数据到发送缓冲区
putbytes(unsigned char *outplace,unsigned char j)
{
    int i;
    for(i=0;i>4)&0x0f;
    putbyte(hex_[ch]);
}

```

```

ch=c&0x0f;
putbyte(hex_[ch]);
}
//*****
//从接收缓冲区取一个 byte,如不想等待则在调用前检测 inbufsign 是否为 1。
uchar getbyte (void)
{ char idata c;
  while (!inbufsign);    //缓冲区空等待
  ES=0;
  c= *getlast;           //取数据
  getlast++;             //最后取走的数据位置加一
  inbufful=0;            //输入缓冲区的满标志清零
  if (getlast==inbuf+ILEN) getlast=inbuf; //地址到顶部回到底部
  if (getlast==inlast) inbufsign=0;      //地址相等置接收缓冲区空空标志,再取数前要检该标志
  ES=1;
  return (c);            //取回数据
}
//*****
//接收一行数据,必须定义放数据串的指针位置和大小    del=0x7f,backspace=0x08,cr=0x0d,lf=0x0a
void getline (uchar idata *line, unsigned char n)
{ unsigned char cnt = 0; //定义已接收的长度
  char c;
  do {
    if ((c = getbyte ()) == 0x0d)  c = 0x00;    //读一个字节,如果是回车换成结束符
    if (c == 0x08 || c == 0x7f)      //BACKSPACE 和 DEL 的处理
    { if (cnt != 0)                  //已经输入退掉一个字符
      { cnt--;                      //总数目减一
        line--;                    //指针减一
        putbyte (0x08);            //屏幕回显的处理
        putbyte (' ');
        putbyte (0x08);
      }
    }
    else
    { putbyte (*line = c);           //其他字符取入,回显
      line++;                      //指针加一
      cnt++;                       //总数目加一
    }
  } while (cnt < n - 1 && c != 0x00 && c != 0x1b); //数目到了,回车或 ESC 停止
  *line = 0;                      //再加上停止符 0
}
//*****
//人工输入一个字符到输入缓冲区
putinbuf(uchar c)
{ES=0; if(!inbufful)

```

```

    { *inlast= c;          //放入数据
      inlast++;            //最后放入的位置加一
      if (inlast==inbuf+ILEN) inlast=inbuf; //地址到顶部回到底部
      if (inlast==getlast) inbufful=1; //接收缓冲区满置满标志
      inbufsign=1;
    }
  ES=1;
}

//*****
//串口中断处理
serial () interrupt 4
{
  if (TI)
  {
    TI = 0;
    if (outbufsign)
      //if (putlast==outlast) outbufsign=0;
    //else
    { SBUF=*outlast; //未发送完继续发送
      outlast++;      //最后传出去的字节位置加一
      if (outlast==outbuf+OLEN) outlast=outbuf; //地址到顶部回到底部
      if (putlast==outlast) outbufsign=0; //数据发送完置发送缓冲区空标志
    }
    else outbufsign=0;
  }
  if (RI)
  {
    RI = 0;
    if (!inbufful)
    {
      *inlast= SBUF;      //放入数据
      inlast++;          //最后放入的位置加一
      inbufsign=1;
      if (inlast==inbuf+ILEN) inlast=inbuf; //地址到顶部回到底部
      if (inlast==getlast) inbufful=1; //接收缓冲区满置满标志
    }
  }
}

//*****
//串口初始化
0xfd=19200,0xfa=9600,0xf4=4800,0xe8=2400,0xd0=1200
serial_init () {
  SCON = 0x50;          /* mode 1: 8-bit UART, enable receiver */
  TMOD |= 0x20;         /* timer 1 mode 2: 8-Bit reload */
  PCON |= 0x80; TH1 = 0xfa; //fa,      //baud*2 /* reload value 19200 baud */
  TR1 = 1;              /* timer 1 run */
  ES = 1; REN=1; EA=1; SM2=1; //SM2=1 时收到的第 9 位为 1 才置位 RI 标志
  TMOD |= 0x01; //th1 auto load 2X8,th0 1X16
}

```

```

    TH0=31; TL0=0; //X 32 =1S
    TR0=1; //ET0=1;

}
//*****
//测试用主函数

void main(void)
{char c;
idata unsigned char free[16];
unsigned char idata *freep=free;
serial_init();

putstring("jdiopuejls;j;klj");
delay(10);

    while(1)
    { putstring("com is ready! ");}
    c=getbyte();
    putbyte(0x20);
    puthex(c);
        switch(c)
        {case 'r':
            putbytes(inbuf,ILEN);
            break;
        case 'g':
            getline(freep,10);
            putbyte(0x20);
            putstring(freep);
            break;
        default:
            putbyte(c);
        }
    }

//W77E58 头文件;
/*--BYTE Registers-----*/
sfr P0    = 0x80;
sfr P1    = 0x90;
sfr P2    = 0xA0;
sfr P3    = 0xB0;
#define p0 P0
#define p1 P1
#define p2 P2

```



```
#define p3 P3
```

```
sfr PSW    = 0xD0;
sfr ACC    = 0xE0;
sfr B      = 0xF0;
sfr SP     = 0x81;
sfr DPL    = 0x82;
sfr DPH    = 0x83;
sfr PCON   = 0x87; //PCON.7(SMOD)波特率加倍, PCON.1(PD)掉电方式, PCON.0(IDL)冻结方式
                //PCON.6(SMOD0)帧错检测允许, PCON.3(GF1)PCON.2(GF0)
sfr TCON   = 0x88; //定时控制寄存器
sfr TMOD   = 0x89; // "gate,c/t,m1,m0"x2 定时器方式 GATE=1 时只有 intx=1 时才可以开放定时器 x;
                //c/t =1 时计数方式, =0 时定时方式。m1m0=00 时 13 位计数, =01 时 16 位=10 时
                自装入 8 位,
```

```
                //11 时定时器 0 分两个, 定时器 1 停止。
```

```
sfr TL0    = 0x8A;
sfr TL1    = 0x8B;
sfr TH0    = 0x8C;
sfr TH1    = 0x8D;
sfr IE     = 0xA8;
sfr IP     = 0xB8; //中断优先级
sfr SCON   = 0x98; //串口控制与状态
sfr SBUF   = 0x99;
```

```
/*--W77E58 Extensions-----*/
```

```
sfr T2CON  = 0xC8;
sfr T2MOD  = 0xC9; //HC5,HC4,HC3,HC2,T2CR,-,T2OE,DCEN
                //hc5-hc2=1 时外中断 5-2 标志硬件自动清除。
                //t2cr=1 捕获完成时自动复位
                //t2oe=1 定时器 2 输出允许
                //dcen=1 减计数允许, 结合外部输入 t2ex 使用, 16 位自装入模式
```

```
sfr RCAP2L = 0xCA; //重装的预置数, 捕获的输出
sfr RCAP2H = 0xCB;
sfr TL2    = 0xCC;
sfr TH2    = 0xCD;
```

```
sfr DPL1   = 0x84;
sfr DPH1   = 0x85;
sfr DPS    = 0x86;
```

```
sfr CKCON  = 0x8E; //wd1,wd0,t2m,t1m,t0m,md2,md1,md0
                //wd1wd0:看门狗计数 00=217,01=220,10=223,11=226
                //t2m,t1m,t0m 等于 1 时时钟 4 分频
                //md2,md1,md0 MOVX 执行机器周期 000=2, 001=3。。。111=9
```

```
sfr EXIF   = 0x91; //ie5,ie4,ie3,ie2,XI/RG,RGMD,RGSL,-
```

```

//ie5,ie3=1 外中断下跳变中断标志
//ie4,ie3=2 外中断上跳变中断标志

sfr P4      = 0xa5;//低四位有效
sfr SADDR   = 0xa9;//从机串口 0 地址
sfr SADDR1  = 0xaa;//从机串口 1 地址
sfr SADEN   = 0xb9;//串口 0 地址屏蔽, 等于 0 时所有地址都会引起中断
sfr SADEN1  = 0xba;//串口 0 地址屏蔽, 等于 0 时所有地址都会引起中断
sfr SCON1   = 0xc0;
sfr SBUF1   = 0xc1;
sfr ROMMAP  = 0xc2;//ROMMAP.7 为等待信号使能, 用 movx 指令时, wait 脚为 p4.0
sfr PMR     = 0xc4;//CD1,CD0,SWB,-,XTOFF,ALE-OFF,-DME0
//cd0cd1=0 时钟不变, =1-1/4, =2-1/64, =3=1/1024
//swb=1 强制 4 分频, 外中断或串口中断唤醒
//aleoff=1 时 ale 信号终止, 外部内存访问时自动唤醒
//dme0=1 时内部 1kram 使能

sfr STATUS= 0xc5;//-,HIP,LIP,XTUP,SPTA1,SPRA1,SPTA0,SPRA0
//hip=1 正在处理高优先级中断
//lip=1 正在处理低优先级中断
//spta1 串口 0 正在发送数据
//spra1 串口 0 正在接收数据
//spta0 串口 0 正在发送数据
//spra0 串口 0 正在接收数据

sfr TA      = 0xc7;//保护位
sfr WDCON   = 0xd8;//SMOD_1,POR,-,WDIF,WTRF,EWT,RWT
//smod1 加倍串口 1 的波特率,pro 电源复位标志
//por 电源复位标志
//wdif 看门狗定时中断标志, 软清
//wtrf 看门狗定时器复位标志, 看门狗引起复位后, 该位置 1
//ewt 允许看门狗定时器自动复位
//rwt 复位看门狗定时器, 如果看门狗溢出还没有被复位计数器, 将会引起中断,

```

再过 512 周期将复位

```

sfr EIE     = 0xe8;//-, -, -, EWDI,EX5,EX4,EX3,EX2
//ewdi=1, 允许看门狗中断?
//ex5-ex2: 外部中断允许

sfr EIP     = 0xf8;//-, -, -, pwdi, px5,px4,px3,px2
//pwdi=1 看门狗中断优先
//px5-px2=1 外部中断优先

```

/*--BIT Registers-----*/

/* PSW */

```

sbit CY     = 0xd7;
sbit AC     = 0xd6;
sbit F0     = 0xd5;
sbit RS1    = 0xd4;
sbit RS0    = 0xd3;

```

```
sbit OV    = 0xD2;
sbit P     = 0xD0;
```

```
/* TCON */
```

```
sbit TF1    = 0x8F; // 定时器 1 溢出标志, 自动清零
sbit TR1    = 0x8E; // timer1 运行控制位
sbit TF0    = 0x8D; // 定时器 0 溢出标志, 自动清零
sbit TR0    = 0x8C; // timer0 运行控制位
sbit IE1    = 0x8B; // 外中断 1 跳变中断请求标志, 自动清零
sbit IT1    = 0x8A; // 中断 1 跳变检测使能
sbit IE0    = 0x89; // 外中断 0 跳变中断请求标志, 自动清零
sbit IT0    = 0x88; // 中断 0 跳变检测使能
```

```
/* IE      */
```

```
sbit EA     = 0xAF; //
sbit ES     = 0xAC;
sbit ET1    = 0xAB;
sbit EX1    = 0xAA;
sbit ET0    = 0xA9;
sbit EX0    = 0xA8;
```

```
/* IP      */
```

```
sbit PS     = 0xBC; // 串口 0 中断优先设定
sbit PT1    = 0xBB; // 定时器 1 中断优先
sbit PX1    = 0xBA; // 外中断 1
sbit PT0    = 0xB9; // 定时器 0 中断优先
sbit PX0    = 0xB8; // 外中断 0
```

```
/* P3      */
```

```
sbit RD     = 0xB7;
sbit WR     = 0xB6;
sbit T1     = 0xB5;
sbit T0     = 0xB4;
sbit INT1   = 0xB3;
sbit INT0   = 0xB2;
sbit TXD    = 0xB1;
sbit RXD    = 0xB0;
```

```
/* SCON */
```

```
sbit SM0    = 0x9F; // SM0/FE 串口 0 方式 0 选择或帧错标志, 人工清零
```

sbit SM1 = 0x9E; // 串口 1 工作方式选择 0-4/12clk 同步, 1-10 位可设 bps 异步, 2-64/32clk 异步, 3-11 位可变 bps 异步

sbit SM2 = 0x9D; // 允许方式 2 和 3 的多机通讯控制位。方式 2、3 中=1 时接收到的第 9 位为 0 时不启动接收中断 ri 标志

// 方式 1 中, =1 时只有收到有效停止时才启动 ri, 方式 0 中=1 将波特率提高 3 倍。

```

sbit REN  = 0x9C;//允许接收标志
sbit TB8  = 0x9B;//方式 23 中要发送的第 9 位。
sbit RB8  = 0x9A;//方式 23 中接收到的第 9 位。
sbit TI   = 0x99;//发送中断标志
sbit RI   = 0x98;//接收中断标志

/*--W77E58 Extensions-----*/
/* PSW      */
sbit F1    = 0xd1;

/* IE      */
sbit ET2   = 0xAD;//定时器 2 允许
sbit ES1   = 0xae;//串口 1 优先

/* IP      */
sbit PT2   = 0xBD;//定时器 2 优先
sbit PS1   = 0xbe;//串口 1 优先

/* P1      */
sbit T2    = 0x90;//计数器 2 输入端口
sbit T2EX  = 0x91;//计数器捕获触发
sbit RXD1  = 0x92;//串口 1 入
sbit TXD1  = 0x93;//串口 2 出
sbit INT2  = 0x94;//外中断 2, 上跳变触发
sbit INT3  = 0x95;//外中断 3, 下跳变触发
sbit INT4  = 0x96;//外中断 4, 上跳变触发
sbit INT5  = 0x97;//外中断 5, 下跳变触发

/* T2CON */
sbit TF2   = 0xCF;//定时器 2 溢出标志, 软清除
sbit EXP2  = 0xCE;//定时器 2 外部标志, 当 exen2=1 且 t2ex 引脚负跳变引起捕获或重载时置位, 软清。
sbit RCLK  = 0xCD;//接收时钟标志, =1 时串口 0 用定时器 2 溢出做时钟。
sbit TCLK  = 0xCC;//发送时钟标志, =1 时串口 0 用定时器 2 溢出做时钟。
sbit EXEN2 = 0xCB;//定时器 2 外部允许标志, =1 时若未用作波特率发生器, t2ex 脚的负跳变引起捕获
sbit TR2   = 0xCA;//定时器 2 运行控制位
sbit C_T2  = 0xC9;//计数/定时选择, =1 时计数
sbit CP_RL = 0xC8;//捕获/重载标志, =1 时,外部允许时, vt2ex 负跳变发生捕获
//=0 时 溢出或外部允许时, vt2ex 负跳变发生重载

/* SCON1   */ //注释参考 SCON
sbit SM0_1 = 0xC7;
sbit SM1_1 = 0xC6;
sbit SM2_1 = 0xC5;
sbit REN_1 = 0xC4;
sbit TB8_1 = 0xC3;

```

```

sbit RB8_1 = 0xC2;
sbit TI_1  = 0xC1;
sbit RI_1  = 0xC0;

/* WDCON */
sbit SMOD_1= 0xDF; //加倍串口 1 的波特率,pro 电源复位标志
sbit POR    = 0xDE; //电源复位标志
sbit WDIF   = 0xDB; //看门狗定时中断标志, 软清
sbit WTRF   = 0xDA; //看门狗定时器复位标志, 看门狗引起复位后, 该位置 1
sbit EWT    = 0xD9; // 允许看门狗定时器自动复位
sbit RWT    = 0xD8; //复位看门狗定时器, 如果看门狗溢出还没有被复位计数器, 将会引起中断, 再
过 512 周期将复位

```

```

/* EIE */
sbit EWDI   = 0xec; //允许看门狗中断
sbit EX5    = 0xeb; //ex5-ex2; 外部中断允许
sbit EX4    = 0xea;
sbit EX3    = 0xe9;
sbit EX2    = 0xe8;

/* EIP */
sbit PWDI   = 0xfc; //pmdi=1 看门狗中断优先
sbit PX5    = 0xfb; //px5-px2=1 外部中断优先
sbit PX4    = 0xfa;
sbit PX3    = 0xf9;
sbit PX2    = 0xf8;

```

程序十六 93C46 读写程序

```

#include
sbit CS=P2^7;
sbit SK=P2^6;
sbit DI=P2^5;
sbit DO=P2^4;

/*
extern unsigned char ReadChar(unsigned char address);
extern void WriteChar(unsigned char address,unsigned char InData);
extern void ReadString(unsigned char data *RamAddress,unsigned char RomAddress,unsigned char Number);
extern void WriteString(unsigned char data *RamAddress,unsigned char RomAddress,unsigned char Number);
*/

```

```

void Ewen(void) {
    unsigned char temp,InData;
    CS=0;
    SK=0;
    CS=1;
    InData=0x98;    // 10011XXXX
    for(temp=9;temp!=0;temp--) {    // 9
        DI=InData&0x80;
        SK=1;    SK=0;
        InData<<=1;
    }
    CS=0;
}

// 关所有程序指令
void Ewds(void) {
    unsigned char temp,InData;
    CS=0;
    SK=0;
    CS=1;
    InData=0x80;    // 10000XXXX
    for(temp=9;temp!=0;temp--) {    // 9
        DI=InData&0x80;
        SK=1;    SK=0;
        InData<<=1;
    }
    CS=0;
}

// 读 data
unsigned int Read(unsigned char address) {
    unsigned char temp;
    unsigned int result;
    Ewen();
    SK=0;    DI=1;    // 110 A5-A0
    CS=0;    CS=1;
    SK=1;    SK=0;    // 1
    address=address&0x3f0x80;
    for(temp=8;temp!=0;temp--) {    // 8
        DI=address&0x80;
        SK=1;    SK=0;
        address<<=1;
    }
    DO=1;
    for(temp=16;temp!=0;temp--) {    // 16

```

```

        SK=1;
        result=(result<<=1;
    }
    for(temp=16;temp!=0;temp--) {    // 16
        DI=InData&0x8000;
        SK=1;    SK=0;
        InData<<=1;
    }
    CS=0;    DO=1;
    CS=1;    SK=1;
    while(DO==0) {                // busy test
        SK=0;    SK=1;
    }
    SK=0;    CS=0;
    Ewds();
}

```

/*

// Erase 记忆区 An - A0.

```

void Erase(unsigned char address) {
    unsigned char temp;
    Ewen();
    SK=0;    DI=1;    // 111 A5-A0
    CS=0;    CS=1;
    SK=1;    SK=0;                // 1
    address|=0xc0;
    for(temp=8;temp!=0;temp--) {    // 8
        DI=address&0x80;
        SK=1;    SK=0;
        address<<=1;
    }
    CS=0;    DO=1;
    CS=1;    SK=1;
    while(DO==0) {
        SK=0;    SK=1;
    }
    SK=0;    CS=0;
    Ewds();
}

```

// 擦除存储，有效电压为 4.5V~5.5V.

```

void Eral(void) {
    unsigned char temp,InData;
    Ewen();
    CS=0;

```

```

    SK=0;
    CS=1;
    InData=0x90;    // 10010XXXX
    for(temp=9;temp!=0;temp--) {    // 9
        DI=InData&0x80;
        SK=1;    SK=0;
        InData<<=1;
    }
    CS=0;    DO=1;
    CS=1;    SK=1;
    while(DO==0) {
        SK=0;    SK=1;
    }
    SK=0;    CS=0;
    Ewds();
}

```

// Writes all memory locations. Valid only at VCC = 4.5V to 5.5V.

```

void Wral(unsigned int InData) {
    unsigned char temp,address;
    Ewen();
    CS=0;
    SK=0;
    CS=1;
    address=0x88;    // 10001XXXX
    for(temp=9;temp!=0;temp--) {    // 9
        DI=address&0x80;
        SK=1;    SK=0;
        address<<=1;
    }
    for(temp=16;temp!=0;temp--) {    // 16
        DI=InData&0x8000;
        SK=1;    SK=0;
        InData<<=1;
    }
    CS=0;    DO=1;
    CS=1;    SK=1;
    while(DO==0) {
        SK=0;    SK=1;
    }
    SK=0;    CS=0;
    Ewds();
}

*/

```



```

unsigned char ReadChar(unsigned char address) {
    unsigned char temp=address>>1;
    if(address&0x01) return((unsigned char)(Read(temp)>>8));
    else return((unsigned char)(Read(temp)));
}

void WriteChar(unsigned char address,unsigned char InData) {
    unsigned char temp=address>>1;
    if(address&0x01) Write(temp,(unsigned int)(Read(temp)&0x00ff|(InData<<1) {
        temp=*RamAddress;
        RamAddress++;
        temp=temp|(*RamAddress)<<1,temp);
        RomAddress++;
        RomAddress++;
        Number--;
        Number--;
    }
    if(Number) WriteChar(RomAddress,*RamAddress);
}

```

程序十七 20045 读写程序

```

/*****
 * X25043/45 application ; Procedures
 * absolute one address access
 *****/

//使用函数: write_status(status) 写状态, 一般写 0
//clr_wchdog(void) 清除看门狗
//unsigned char read_byte(address) //读一个字节
//void write_byte(address,Data) //写一个字节

#define ALONE_COMPILE
#ifdef ALONE_COMPILE
#include "INTRINS.H"
#endif

#ifdef nop2()
#define nop2() _nop();_nop();_nop();
#endif

#define WREN 0x06

```

```

#define WRDI      0x04
#define RDSR      0x05
#define WRSR      0x01
#define READ      0x03
#define WRITE     0x02

//define      PORT    P1
sfr  PORT=0x90; //25045 的 4 根 io 脚接在同一端口, 本例为 p1
//请根据实际电路更改引脚定义
#define  _SI      0x80 //si 接在 p1.3, 0x80=00001000b
#define  _SCK     0x40 //sck 接在 p1.2, 0x80=00000100b
#define  _SO      0x20 //so 接在 p1.1
#define  _CS      0x10 //cs 接在 p1.0

//-----
#ifndef  dword
#define  dword    unsigned long
#define  word     unsigned int
#define  byte     unsigned char
typedef  union{
        word      w;
        byte      bh;
        byte      bl;
    }WordType;

    typedef  union{
        dword      dw;
        word        w[2];
        byte        b[4];
    }DwordType;

#endif

//-----
void  _w_byte(Data)
char Data;
{
    char i;

    PORT &= (_SCK^0xff);
    for(i=0;i<255 ? (0x08/READ): READ));
    _w_byte((char)(address & 0x00ff));
    result=_r_byte();
    nop2();nop2();
    PORT |=_CS;

```

// 悬挂

```
        return(result);
    }
//-----
void    write_byte(address,Data)
        unsigned int address;
        char Data;
{
    write_status(WREN);
    nop2();nop2();
    PORT &= (_CS^0xff);
    nop2();nop2();
    _w_byte((unsigned char)(address>255 ? (0x08|WRITE): WRITE));
    _w_byte((unsigned char)(address & 0x00ff));
    _w_byte(Data);
    nop2();nop2();
    PORT |= _CS;
    wait_free();
    return;
}
/*
//-----
unsigned long    read_data(format,address)
        unsigned char    format;
        unsigned int    address;
{
    DwordType    result;

    switch(format&0xdf)
    {
    case 'L':
        result.b[0]=read_byte(address);
        result.b[1]=read_byte(address+1);
        result.b[2]=read_byte(address+2);
        result.b[3]=read_byte(address+3);
        break;
    case 'D':
        result.b[2]=read_byte(address);
        result.b[3]=read_byte(address+1);
        break;
    case 'C':
        result.b[3]=read_byte(address);
        break;
    }
    return(result.dw);
}
```

```

}
//-----
void write_data(format,address,Data)
    unsigned char format;
    unsigned int address;
    DwordType data* Data;
{
    switch(format&0xdf)
    {
        case 'L':
            write_byte(address , Data->b[0]);
            write_byte(address+1, Data->b[1]);
            write_byte(address+2, Data->b[2]);
            write_byte(address+3, Data->b[3]);
            break;
        case 'D':
            write_byte(address , Data->b[2]);
            write_byte(address+1, Data->b[3]);
            break;
        case 'C':
            write_byte(address , Data->b[3]);
            break;
    }
}
//-----
*/

```

程序十八 一组小程序集锦

1. 十六进制与十进制互换程序

```

unsigned char d[10]; //用于显示的 10 位显示缓存

//=====
//十六进制 to 十进制输出子程序:显示数据, 起始位, 结束位, 有无小数点
//=====
void output(unsigned long dd,unsigned char s,unsigned char e,unsigned char dip) {
    unsigned long div;
    unsigned char tm[8],i,j;
    div=10000000;
    for (i=0;i<8;i++) {

```

```

        tm[i]=dd/div;
        dd%=div;
        div/=10;
    }
    for (i=0;i<6;i++) {
        if (tm[i]!=0) break;
        tm[i]=nul;
    }
    tm[5]=dip;          //小数点控制, 请看“串行 LED 数码管显示驱动程序”
    j=7;
    for (i=s;i<e;i++) {
        d[i]=tm[j];
        j--;
    }
}

```

//把显示位 5~9 位的十进制数转换成为十六进制数

```

unsigned int input(void) {
    unsigned int dd,dat;
    dd=10000;dat=0;
    for (i=5;i<10;i++) {
        dat+=dd*temp;
        dd/=10;
    }
    return(dat);
}

```

2. 24c01-24c16 读写驱动程序

```

sbit a0=ACC^0;          //定义 ACC 的位, 利用 ACC 操作速度最快
sbit a1=ACC^1;
sbit a2=ACC^2;
sbit a3=ACC^3;
sbit a4=ACC^4;
sbit a5=ACC^5;
sbit a6=ACC^6;
sbit a7=ACC^7;

```

```

void s24(void) {
    _nop_();scl=0;sda=1;scl=1;_nop_();sda=0;_nop_();scl=0;
}
void s240(void) {
    _nop_();scl=0;sda=1;scl=1;_nop_();sda=0;_nop_();scl=0;
}
void p24(void) {

```

```

        sda=0;scl=1;_nop_();sda=1;
    }
    void p240(void) {
        sda0=0;scl0=1;_nop_();sda0=1;
    }
    unsigned char rd24(void) {
        sda=1;
        scl=1;a7=sda;scl=0;
        scl=1;a6=sda;scl=0;
        scl=1;a5=sda;scl=0;
        scl=1;a4=sda;scl=0;
        scl=1;a3=sda;scl=0;
        scl=1;a2=sda;scl=0;
        scl=1;a1=sda;scl=0;
        scl=1;a0=sda;scl=0;
        sda=1;scl=1;scl=0;
        return(ACC);
    }

    void wd24(unsigned char dd) {
        ACC=dd;
        sda=a7;scl=1;scl=0;
        sda=a6;scl=1;scl=0;
        sda=a5;scl=1;scl=0;
        sda=a4;scl=1;scl=0;
        sda=a3;scl=1;scl=0;
        sda=a2;scl=1;scl=0;
        sda=a1;scl=1;scl=0;
        sda=a0;scl=1;scl=0;
        sda=1;scl=1;
    }

    unsigned char read(unsigned int address){
        unsigned char dd;
        dd=((address&0x7ff)/256)<<1;
        s24();wd24(0xa0ldd);scl=0;wd24(address);scl=0;
        s24();wd24(0xa1ldd);scl=0;dd=rd24();p24();return(dd);
    }

    void write(unsigned int address,unsigned char dd){
        unsigned char ddd;
        ddd=((address&0x7ff)/256)<<1;
        s24();wd24(0xa0lddd);scl=0;wd24(address);scl=0;wd24(dd);scl=0;p24();
        time=0; //time 为定时器时间参考，time 增加 1 代表 1ms,如果没有用定时
器，取消该行

```

```

while (1) {
    s24();
    wd24(0xa0ddd);
    sda=1;
    if (sda==0) break;
    if (time>10) break;    //此行防止由于 eeprom 器件损坏后的死循环
    scl=0;
}
}

```

3. 通用 93c06-93c86 系列

```

/*普通封装*/
//93c01=93c46
//93c56=93c66
//93c76=93c86
//dipx 可以自行定义
#define di_93 dip3
#define sk_93 dip2
#define cs_93 dip1
#define do_93 dip4
#define gnd_93 dip5
#define org_93 dip6

void high46(void){
    di_93=1;
    sk_93=1;nop;sk_93=0;
    nop;
}
void low46(void){
    di_93=0;
    sk_93=1;nop;sk_93=0;
    nop;
}
void wd46(unsigned char dd){
    unsigned char i;
    for (i=0;i<8;i++) {
        if (dd>=0x80) high46(); else low46();
        dd=dd<<1;
    }
}
unsigned char rd46(void){
    unsigned char i,dd;
    do_93=1;
    for (i=0;i<8;i++) {

```

```

        dd<<=1;
        sk_93=1;nop;sk_93=0;
        nop;
        if (do_93) ddl=1;
    }
    return(dd);
}

```

```

void ewen46(void){
    nop;
    cs_93=1;
    high46();
    wd46(0x30);
    cs_93=0;
}

```

//特殊封装

```

#define di_93a dip5
#define sk_93a dip4
#define cs_93a dip3
#define do_93a dip6
#define gnd_93a dip7
#define vcc_93a out_vcc(2)

```

```

void high46a(void){
    di_93a=1;
    sk_93a=1;nop;sk_93a=0;
    nop;
}

```

```

void low46a(void){
    di_93a=0;
    sk_93a=1;nop;sk_93a=0;
    nop;
}

```

```

void wd46a(unsigned char dd){
    unsigned char i;
    for (i=0;i<8;i++) {
        if (dd>=0x80) high46a(); else low46a();
        dd=dd<<1;
    }
}

```

```

unsigned char rd46a(void){
    unsigned char i,dd;
    do_93a=1;
    for (i=0;i<8;i++) {

```



```

        dd<=1;
        sk_93a=1;nop;sk_93a=0;nop;
        if (do_93a) ddl=1;
    }
    return(dd);
}

unsigned int read93c46_word(unsigned char address) {
    unsigned int dat;
    unsigned char dat0,dat1;
    gnd_93a=0;
    gnd_93=0;
    cs_93=sk_93=0;org_93=1;cs_93=1;
    nop;
    address=address>>1;
    address=address&0x80;
    high46();
    wd46(address);dat1=rd46();dat0=rd46();
    cs_93=0;dat=dat1*256+dat0;
    return(dat);
}

bit write93c46_word(unsigned char address,unsigned int dat){
    unsigned char e,temp=address;
    e=0;
    while (e<3) {
        gnd_93a=0;
        gnd_93=0;
        cs_93=sk_93=0;org_93=1;cs_93=1;
        ewen46();
        nop;
        cs_93=1;
        nop;
        high46();
        address=0x80;
        address>>=1;
        wd46(address);wd46(dat/256);wd46(dat%256);
        cs_93=0;nop;cs_93=1;
        time=0;do_93=1;
        while (1) {
            if (do_93==1) break;
            if (time>20) break;
        }
        cs_93=0;
        if (read93c46_word(temp)==dat) {

```

```

        return(0);
    }
    e++;
}
return(1);
}

//=====
//93c57
void ewen57(void){
    nop;
    cs_93=1;
    dip7=0;
    high46();
    low46();
    wd46(0x60);
    cs_93=0;
}

unsigned int read93c57_word(unsigned int address) {
    unsigned int dat;
    unsigned char dat0,dat1;
    gnd_93=0;
    cs_93=sk_93=0;org_93=1;cs_93=1;
    address=address>>1;
    high46();
    high46();
    wd46(address);dat1=rd46();dat0=rd46();
    cs_93=0;dat=dat1*256+dat0;
    return(dat);
}

bit write93c57_word(unsigned int address,unsigned int dat){
    unsigned char e;
    unsigned int temp=address;
    e=0;
    while (e<3) {
        gnd_93=0;
        cs_93=sk_93=0;org_93=1;cs_93=1;
        ewen57();
        cs_93=1;
        nop;
        high46();
        low46();
        address>>=1;

```

```

        address|=0x80;
        wd46(address);wd46(dat/256);wd46(dat%256);
        cs_93=0;nop;cs_93=1;
        time=0;do_93=1;
        while (1) {
            if (do_93==1) break;
            if (time>20) break;
        }
        cs_93=0;
        if (read93c57_word(temp)==dat) {
            return(0);
        }
        e++;
    }
    return(1);
}

```

```

void ewen56(void){
    nop;
    cs_93=1;
    high46();
    low46();
    low46();
    wd46(0xc0);
    cs_93=0;
}

unsigned int read93c56_word(unsigned int address) {
    unsigned int dat;
    unsigned char dat0,dat1;
    gnd_93=0;
    cs_93=sk_93=0;org_93=1;cs_93=1;
    address=address>>1;
    high46();
    high46();
    low46();
    wd46(address);dat1=rd46();dat0=rd46();
    cs_93=0;dat=dat1*256+dat0;
    return(dat);
}

bit write93c56_word(unsigned int address,unsigned int dat){
    unsigned char e;
    unsigned int temp=address;
    e=0;
    while (e<3) {

```

```

        gnd_93=0;
        cs_93=sk_93=0;org_93=1;cs_93=1;
        ewen56();
        nop;
        cs_93=1;
        nop;
        high46();
        low46();
        high46();
        address>>=1;
        wd46(address);wd46(dat/256);wd46(dat%256);
        cs_93=0;nop;cs_93=1;
        TH0=0;
        time=0;do_93=1;
        while (1) {
            if (do_93==1) break;
            if (time) break;
        }
        cs_93=0;
        if (read93c56_word(temp)==dat) {
            return(0);
        }
        e++;
    }
    return(1);
}

//=====
//93c76   93c86
//=====
void ewen76(void){
    nop;
    cs_93=1;
    dip7=1;
    high46();
    low46();
    low46();
    high46();
    high46();
    wd46(0xff);
    cs_93=0;
}

unsigned int read93c76_word(unsigned int address) {
    unsigned char dat0,dat1;

```

```

    gnd_93=0;
    cs_93=sk_93=0;org_93=1;cs_93=1;
    address>>=1;
    high46();
    high46();
    low46();
    if((address&0x200)==0x200) high46(); else low46();
    if ((address&0x100)==0x100) high46(); else low46();
    wd46(address);dat1=rd46();dat0=rd46();
    cs_93=0;
    return(dat1*256|dat0);
)

bit write93c76_word(unsigned int address,unsigned int dat){
    unsigned char e;
    unsigned int temp=address;
    e=0;
    address>>=1;
    while (e<3) {
        gnd_93=0;
        cs_93=sk_93=0;org_93=1;cs_93=1;
        ewen76();
        nop;
        cs_93=1;
        high46();
        low46();
        high46();
        if((address&0x200)==0x200) high46(); else low46();
        if ((address&0x100)==0x100) high46(); else low46();
        wd46(address);wd46(dat/256);wd46(dat%256);
        cs_93=0;nop;cs_93=1;
        time=0;do_93=1;
        while (1) {
            if (do_93==1) break;
            if (time>10) break;
        }
        cs_93=0;
        if (read93c76_word(temp)==dat) {
            return(0);
        }
        e++;
    }
    return(1);
}

```

4. 1330 液晶驱动

//E-1330 点阵液晶屏驱动程序

/*

线路接口

89C51	-----	E-1330
P1.0-1.7	-----	D0-7
P3.0	-----	A0
P3.1	-----	R/W
P3.2	-----	E
RESET	-----	/RES

*/

#include <reg51.h>

sbit p_a0=P3^0;

sbit p_rw=P3^1;

sbit p_e=P3^2;

//指令写入函数

void ctrl(unsigned char c) {

 p_a0=1; //a0 为 1 代表写入指令

 p_rw=0;

 p_e=1;P1=c;p_e=0;

}

//数据和指令参数写入函数

void write(unsigned char d) {

 p_a0=0; //a0 为 0 代表写入数据或指令参数

 p_rw=0;

 p_e=1;P1=d;p_e=0;

}

//数据和光标地址读出函数

unsigned char read(void) {

 unsigned char rd;

 p_a0=1; //a0 为 1 代表读数据和光标地址,a0 为 0 代表读状态标志, 由于 E-1330 功能很强, 一般不用读状态标志

 p_rw=1;

 P=0xff; //把 P1 置为高电平, 只有置为高电平才能正确读入数据

 p_e=1;rd=P1;p_e=0;

 return(rd);

}

/*

其他函数可以根据资料自行组合,

如设置 CGROM 相对地址为 0000H, 用以下语句即可:

```

    ctrl(0x5c);    //写入 5C 指令
    write(0);
    write(0);      //写入 5C 指令的参数 0000
*/

```

5. 单个汉字库字模提取程序

(tc2.0 编译)

```

#include "stdio.h"
#include "dos.h"
#include "process.h"
#include "string.h"

```

```

void main(void) {
    long int num_bytes,qm,wm;
    unsigned char d,i,j,k,a[132],b[132];
    unsigned char * data;
    unsigned char * hz;
    static unsigned char dd[103];
    FILE *fp;

    if ((fp=fopen("hzk16f","rb"))==NULL) {
        printf("can't open hzk16\n");
        exit(1);
    }
    clrscr();
    while (1) {

        data=(unsigned char *) malloc(33);
        printf("please input:\n");
        scanf("%s",dd); /*输入一个汉字*/

        qm=* dd;      /*通过区位码计算其在 hzk16f 文件中的偏移地址*/
        qm=qm-161;
        if (qm>87) exit(0);
        wm=(dd+1);
        wm=wm-161;
        if (wm>94) exit(0);
        num_bytes=((long)qm*94+wm)*32;
        fseek(fp,num_bytes,SEEK_SET);
        fgets(data,33,fp);
        for (i=0;i<32;i++) b[i]=* data++;
        for (i=0;i<32;i+=2) a[i/2]=b[i];
        for (i=0;i<32;i+=2) a[i/2+16]=b[i+1];
    }
}

```

```

    for (i=8;i<16;i++) b[i]=a[i];
    for (i=8;i<16;i++) a[i]=a[i+8];
    for (i=8;i<16;i++) a[i+8]=b[i];

    /*转换, hzf16f 在电脑的储存格式是以行为字节计算的, 一般的 lcd 都采用以列为字节计算*/
    for (k=0;k<32;k+=8) {
        for (j=0;j<8;j++) {
            d=0;
            for (i=0;i<8;i++) {
                if (a[i+k]>=0x80) {
                    switch (i) {
                        case 0:d+=1;break;
                        case 1:d+=2;break;
                        case 2:d+=4;break;
                        case 3:d+=8;break;
                        case 4:d+=0x10;break;
                        case 5:d+=0x20;break;
                        case 6:d+=0x40;break;
                        case 7:d+=0x80;break;
                    }
                }
            }
            for (i=0;i<8;i++) a[i+k]<=<=1;
            b[j+k]=d;
        }
    }
    clrscr();
    printf("/*%s:*/\n",dd);    /*输出 0x00 格式的十六进制数*/
    for (k=0;k<32;k+=8) {
        for (j=0;j<8;j++) printf("0x%x,",b[j+k]);
        printf("\n");
    }
    getch();
}
}

```

6. 按键扫描驱动程序

```

unsigned char key,key_h,kpush;
unsigned int key_l;

```

//按键连接到 p1.0、p1.1、p1.2

```

void int_r0(void) interrupt 1 {
    unsigned char dd,i;

```



```

    TL0=TL0+30;TH0=0xfb;    //800
    /* 按键判别 */
    if ((P1&0x7)==0x7) {
        if ((key_l>30)&&(key_l<800)&&(key_h>30)) {           //释放按键，如果之前按键时间少于 1
秒，读入键值
            key=kpush;
        }
        if ((++key_h)>200) key_h=200;
        key_l=0;
        if (key>=0x80) key=0;                                //如果之前的按键为长按 1 秒，清除键值
    } else {
        kpush=P1&0x7;
        key_l++;
        if ((key_l>800)&&(key_h>30)) {                         //如果按键超过 1 秒，键值加 0x80 标志长按键
            key=kpush|0x80;
            key_h=0;
            key_l=0;
        }
    }
}
void main(void) {
    TMOD=0x1;TR0=1;ET0=1;EA=1;
    while (1) {
        while (!key) {}
        switch (key) {
            case 1:break;
            case 2:break;
        }
    }
}

```

7. 串行驱动 led 显示

//一个 74hc595 位移寄存器驱动三极管驱动 led 位。
 //两个 74hc595 驱动 led 段，方式位 5 位 x8 段 x2=10 个数码管
 //5 分频，每次扫描时间位 1.25ms
 //定义特殊符号

```

#define nul 0xf
#define qc 0xc
#define qb 0xb
#define q_ 0xa
#define q__ 0xd
#define q___ 0xe
#define qp 0x10

```

```

#define qe 0x11
#define qj 0x12
#define qn 0x13
#define qf 0x14
#define qa 0x15
#define qr 0x16
#define qd 0x17
#define qu 0x18
#define ql 0x19
#define qh 0x1a
#define qwen 0x1b
#define qt 0x1c
#define qia 0x1d
#define qlb 0x1e
#define qlc 0x1f
#define qld 0x20
#define qle 0x21
#define qlf 0x22
#define qlg 0x23
#define qldp 0x24

```

//显示段信息，不同 led 排列组合的段信息只需更改 8 个数值即可。
//因此，该定义具有通用性。

```

// 显示
// -d 20
// lc 40 le 10
// -g 80
// lb 2 lf 4
// _a1 .dp 8
#define pa 1
#define pb 2
#define pc 0x40
#define pd 0x20
#define pe 0x10
#define pf 4
#define pg 0x80
#define pdp 8
#define l0 pdp+pg
#define l1 255-pf-pe
#define l2 pdp+pc+pf
#define l3 pdp+pc+pb
#define l4 pdp+pa+pb+pd
#define l5 pdp+pb+pe
#define l6 pdp+pe

```

```

#define 17 pdp+pc+pg+pb+pa
#define 18 pdp
#define 19 pdp+pb
#define 1a pdp+pa
#define 1b pdp+pd+pe
#define 1c pdp+pg+pe+pf
#define 1d pdp+pc+pd
#define 1e pdp+pe+pf
#define 1f pdp+pe+pf+pa
#define 1_ 255-pg
#define 1n1 255
#define 1l pdp+pg+pd+pf+pe
#define 1p pdp+pa+pf
#define 1t pdp+pd+pe+pf
#define 1r pdp+pe+pf+pg+pa
#define 1n pdp+pg+pa
#define 1h pdp+pd+pe+pa
#define 1y pdp+pb+pd
#define 1u pdp+pg+pd
#define 1__ pdp+pg+pb+pc+pe+pf
#define 1___ 1__-pg
#define 1_1 255-pa
#define 1_2 255-pa-pg
#define 1j 255-(pe+pf+pa)
#define 1wen 255-(pd+pe+pg+pb)
#define 1all 0

```

```

#define 1la 255-pa
#define 1lb 255-pb
#define 1lc 255-pc
#define 1ld 255-pd
#define 1le 255-pe
#define 1lf 255-pf
#define 1lg 255-pg
#define 1ldp 255-pdp

```

//串行送出的位信息，目前是 10 位 led 显示。

```
unsigned char code un_dig[]={0x7f,0xbf,0xdf,0xef,0xf7,0xfb};
```

//串行送出的短信息。

```
unsigned char code un_disp[]={10,11,12,13,14,15,16,17,18,19,1_,1b,1c,1__,1___,1n1,1p,1e,1j,1n,1f,1a,1r,1d,1u,
1l,1h,1wen,1t,1la,1lb,1lc,1ld,1le,1lf,1lg,1ldp,1n1};
```

```

sbit d_clk=P0^0;           //移位时钟
sbit d_dat=P0^1;           //移位数据
sbit d_st=P0^2;            //移位锁定

```

```

unsigned char dig;          //位扫描计数器
unsigned char d[10];       //显示缓冲

//送出 8 位串行数据
void out_disp(unsigned char dd) {
    unsigned char i;
    for (i=0;i<8;i++) {
        if (dd&1) d_dat=1; else d_dat=0;
        d_clk=0;
        dd>>=1;
        d_clk=1;
    }
}

//控制小数点和闪烁, 显示数据0x040 表示有小数点; 显示数据0x80 表示闪烁。
void out_displ(unsigned char dd) {
    if (dd>=0x80) {
        if (s001>flash_time) {out_disp(0xff);return;}
    }
    dd&=0x7f;
    if (dd>=0x40) {
        dd=un_disp[dd&0x3f]^pdp;
    } else dd=un_disp[dd];
    out_disp(dd);
}

unsigned int s001;         //闪烁时间参考
void int_t0(void) interrupt 1 {
    unsigned char dd;
    TL0=TL0+30;TH0=0xfb;   //800
    time++;
    if ((++s001)>=800) s001=0;
    // 显示
    if ((++dig)>4) dig=0;
    d_st=0;
    dd=d[dig+5];
    out_displ(dd);
    dd=d[dig];
    out_displ(dd);
    out_disp(un_dig[dig]);
    d_st=1;
}

void main(void) {
    unsigned char i;
    TMOD=0x1;
    TR0=ET0=1;

```

```

    EA=1;
    for (i=0;i<10;i++) d[i]=i;    //display test
    while (1) {}
}

```

8. 软件狗

;汇编

```
ERRORP SEGMENT CODE
```

```
PUBLIC error
```

```
RSEG ERRORP
```

```
error:
```

```
    CLR EA
```

```
    MOVDPTR,#ERR1
```

```
    PUSH    DPL
```

```
    PUSH    DPH
```

```
    RETI
```

```
ERR1:
```

```
    CLR A
```

```
    PUSH    ACC
```

```
    PUSH    ACC
```

```
    RETI
```

```
    END
```

//以下程序只是一个范例

```
void error(void);
```

//定时器 0，清除定时器 1 的计时

```
void int_t0(void) interrupt 1 {
```

```
    TL0=TL0+68;TH0=0xfd;    //700
```

```
    TH1=0xfb;
```

```
}
```

//定时器 1，中断作为看门狗

```
void int_t1(void) interrupt 3 {
```

```
    error();    //复位
```

```
}
```

```
unsigned char adds;
```

```
unsigned char b_job0[5][3];
```

个，其中一个有效，其余 2 个备用

```
unsigned char b_job1[5][3];
```

```
unsigned char b_job2[5][3];
```

//job0 用到的数据，如 A/D 采集的数据，一共 5 组，每组 3

```

void job0(void) {
    unsigned char i;
    adds=0;
    for (i=0;i<5;i++) {                //数据采集部分，此处简化过程
        b_job0[i][0]=b_job0[i][1]=b_job0[i][2]=123;
    }
    while (1) {}
}

void job1(void) {
    unsigned char i;
    adds=1;
    for (i=0;i<5;i++) {                //数据采集部分，此处简化过程
        b_job0[i][0]=b_job0[i][1]=b_job0[i][2]=23;
    }
    while (1) {}
}

void job2(void) {
    unsigned char i;
    adds=2;
    for (i=0;i<5;i++) {                //数据采集部分，此处简化过程
        b_job0[i][0]=b_job0[i][1]=b_job0[i][2]=12;
    }
    while (1) {}
}

```

//为了在复位时不把 b_power 清零，连接时必须和 nostart.obj 连接

```

void main(void) {
    unsigned int b_power;
    unsigned char b_test_ram,i,j;
    TMOD=0x11;
    TH0=0xfd;TH1=0xfb;
    ET0=TR0=1;
    ET1=TR1=1;
    EA=1;
    if (b_power!=0x1234) {                //b_power 不等于 0x1234 表示刚开机
        b_power=0x1234;
        adds=0;                          //第一次执行 job0
    } else {                              //软件复位处理程序，主要是根据产生复位的地址来继续执行
        //RAM 数据错误检测和恢复，3 中取 2 相等法
        for (i=0;i<5;i++) {
            for (j=0;j<2;j++) {
                b_test_ram=job0[i][j];
                if (b_test_ram==job0[i][j+1]) break;
                b_test_ram=job0[i][j+1];
            }
        }
    }
}

```

```

        }
        if (j==2) break;
    }
    if (i!=5) {}//处理 job0 数据出错

    for (i=0;i<5;i++) {
        for (j=0;j<2;j++) {
            b_test_ram=job1[i][j];
            if (b_test_ram==job1[i][j+1]) break;
            b_test_ram=job1[i][j+1];
        }
        if (j==2) break;
    }
    if (i!=5) {}//处理 job1 数据出错

    for (i=0;i<5;i++) {
        for (j=0;j<2;j++) {
            b_test_ram=job2[i][j];
            if (b_test_ram==job2[i][j+1]) break;
            b_test_ram=job2[i][j+1];
        }
        if (j==2) break;
    }
    if (i!=5) {}//处理 job2 数据出错

    switch (adds) {
        case 0:job0();break;
        case 1:job1();break;
        case 2:job2();break;
    }
}
while (1) {}
}

```

9. P89CXX 编程器控制 CPU 接收和控制程序

```

#include <reg52.h>
#include <intrins.h>

unsigned char time,b_break,b_break_3;
//35.555ms
void int_t0(void) interrupt 1 {
    TH0=0;
    if ((++b_break_3)>2) b_break=1;
    time++;
}

```

```

    }

void wait20us(void) {
    TL0=210;TH0=0xff;
    time=0;
    while (!time) {}
}

void wait35ms(void) {
    TH0=0;time=0;
    while (!time) {}
}

void wait1s(void) {
    time=0;
    while (time<30) {}
}

}

unsigned char rec(void) {
    TH0=0;b_break_3=0;b_break=0;
    while (RI==0) {
        if (b_break) return(1);
    }
    RI=0;
    SBUF=SBUF;
    return(SBUF);
}

/*返回 1 表示失败*/
bit sen(unsigned char d) {
    SBUF=d;
    TH0=0;b_break_3=0;b_break=0;
    while (RI==0) {
        if (b_break) return(1);
    }
    RI=0;
    if (SBUF!=d) return(1);
    return(0);
}

#define b_init 0

#define b_erase 1
#define b_erase_block 2 //P89C51RC+、P89C51RD+特有
#define b_erase_sbyte_bvec 3 //P89C51RC+、P89C51RD+特有

```



```

#define b_read 4

#define b_program_byte 5
#define b_program_sbyte 6 //P89C51RC+、P89C51RD+特有
#define b_program_boot_vector 7 //P89C51RC+、P89C51RD+特有

#define b_lock1 8
#define b_lock12 9
#define b_lock123 10

#define b_cpu 12 //定义需要编程的 cpu 类型,0:P89C51 1:P89C52 2:P89C54
3:P89C58 4:P89C51RC+ 5:P89C51RD+

sbit p_oe=P3^2;
sbit p_we=P3^3;

//输入控制模式
void mode(unsigned char m) {
    P3=(m<<4)*0xf;
}
void out_adds(unsigned int adds) {
    P0=adds;
    P2=adds/256;
}
unsigned char in_data(void) {
    P1=0xff;
    p_oe=0;
    return(P1);
    p_oe=1;
}

//判别 cpu 类型
unsigned char code un_cpu[]={0xf4,0xff,0xfb,0xf9,0x89,0x80};
unsigned char read_id(void){
    unsigned char d;
    mode(0xf);
    //读 Mft ID
    out_adds(0);d=in_data();
    if (d!=0x15) return(0);
    //读 Mft ID1
    out_adds(1);d=in_data();
    if (d!=0xc2) return(0);
    //读 Mft ID2
    out_adds(2);d=in_data();
    return(d); //返回 Mft ID2
}

```

```

}

//芯片初始化
bit init(void) {
    unsigned char b_error=0,time_calc;
    mode(0xe);
    wait35ms();           //防止电脑处理不过来

    p_we=0;
    time=0;               //等待 0.5 秒
    time_calc=time;
    while (1) {
        while (time==time_calc) {}
        time_calc=time;
        if (sen(0xf) return(1); //发送 “正在初始化” 标志: 0xf
        if (time>15) break;
    }
    wait35ms();//test
    p_we=1;
    //抽样检查擦除是否成功
    mode(0x4);
    out_adds(0);if (in_data()!=0xff) b_error=1;
    out_adds(1);if (in_data()!=0xff) b_error=1;
    out_adds(0xff);if (in_data()!=0xff) b_error=1;
    out_adds(0xffff);if (in_data()!=0xff) b_error=1;
    out_adds(0x1fff);if (in_data()!=0xff) b_error=1;
    sen(b_error);          //根据 b_error 发送
    return(b_error);
}

bit program_byte(void) {
    unsigned char d,n;
    unsigned int adds;
    do {
        d=rec();
        if (!b_break) {
            out_adds(adds);
            P1=d;
            n=0;
            while (1) {
                mode(0x3);
                p_we=0;
                wait20us();
                p_we=1;
                mode(0x5);
            }
        }
    } while (1);
}

```

```

        if (in_data()!=d) n++; else break;
        if (n>2) return(1); //编程校验 3 次失败
    }
    adds++;
} else return(0); //pc 通信超时或编程完成。在此不会区分，但无所谓
} while (adds);
return(1); //地址超过 64k
}

```

```

bit lock123(void) {
    unsigned char i,b_error=0;
    unsigned int adds;
    wait20us();
    for (i=0;i<3;i++)
        mode(0x6);
        out_adds(i);
        p_we=0;
        wait20us();
        p_we=1;
    }
    //检查前 4kb 是否加密成功(为了节省时间，只检查前 4kb)
    mode(0x0);
    for (adds=0;adds<4096;adds++) {
        out_adds(adds);
        if (in_data()!=0xff) {b_error=1;break;}
    }
    sen(b_error);
    return(b_error);
}

```

```

void main(void) {
    unsigned char cpu;
    unsigned char i,start;
    EA=1;
    SCON=0xd8;PCON=0x80;
    TMOD=0x21;
    TL1=TH1=0xff;TR1=1;
    TH0=0;ET0=TR0=1;
    while (1) {
        REN=1;
        start=rec();
        if ((!b_break)&&(start==0x0)) {
            i=rec();
            if (!b_break) {
                switch (i) {

```

```

        case b_cpu:
            i=rec();
            if (!b_break) cpu=i;
            wait35ms();           //防止电脑处理不过来
            if (un_cpu[i]!=read_id()) sen(1); else sen(0);
            break;
        case b_init:
            if (init()) wait1s();   //初始化错误或通信错误等待 1 秒, 让 pc
程序超时并报告错误

            break;
        case b_program_byte:
            if (program_byte()) wait1s(); //编程错误或通信错误等待 1 秒, 让 pc 程
序超时并报告错误

            break;
        case b_lock123:
            if (lock123()) wait1s(); //加密错误或通信错误等待 1 秒, 让 pc 程
序超时并报告错误

            break;
    }
}
}
}
}
}

```

10. 软件 A/D

测量电压范围是 2~15V, 而且速度低 (约 1kHz), 但仅用一个电容和一个电阻, 在某些场合是有用的。

原理是利用 470K 电阻对 1μF 电容充电, 利用 P0.0 口作为检测电压, 当电压低于 $1/3V_{CC}$ 时, P0.0 读入的 I/O 电平为 0, 当充电电压超过 $1/3V_{CC}$ 时, P0.0 读入的 I/O 电平为 1。通过测量此过程所用的时间, 就能判断输入电压 (需要换算)。

程序的编写用定时器 0 实现

*/

//设计时需要计算过 2V 充电时测量的电压时间小于 250ms, 否则 time 溢出。

//如需要高精度, time,vol 换成 int, 测量时间会长一点

sbit v_input=P0^0;

unsigned char time,vol,n;

unsigned int total;

void int_t0(void) interrupt 1 {

TL0+=24;TH0=0xfb; //1000 个机器周期

time++;

if (v_input) {

```

v_input=0; //把电容电压放调
total+=time;
time=0;
if ((++n)>10) { //统计 10 次测量的时间
    n=0;
    vol=total/10;    //vol 的值为测量的电压（还没有转换）
    total=0;
}
v_input=1;
}
}

```

程序十九 AVR asm 源程序

1. 16 位整型数开方

```
.include "8515def.inc"
```

```

.def      numlo      = r16
.def      numhi      = r17
.def      sqrt       = r18
.def      suber      = r24
.def      suberh     = r25
; enter with the 16 bit Number in r16,r17
.org      0x00
rjmp     reset

.org      0x20

reset:    ldi         r16,0x02           ; Stack Pointer Setup
          out         SPH,r16           ; Stack Pointer High Byte
          ldi         r16,0x5f         ; Stack Pointer Setup
          out         SPL,r16         ; Stack Pointer Low Byte

loopm:    ldi         r16,0x64
          ldi         r17,0x01
          rcall      sqrt
          nop
          rjmp       loopm

Sqrt:

```

```

        clr      sqrt
        ldi      suber,1           ; initialize the seed to be subtracted
        clr      suberh           ; for each iteration
loop:    sub      numlo,suber
        sbc      numhi,suberh
        brlo     exit
        inc      sqrt
        adiw     suber,2           ; keep the number to subtract ODD.
        rjmp     loop
exit:
        ret                          ; the sqrt(num) on exit is stored in r1

```

2. 32 位整型数开方

;input r15:r14:r13:r12 (it make a copy from r3::r0)
 ;output r25:r24
 ;use r20,r21,r22,r26,r28
 ;输入返回数据: $\text{lsqrt}(0x5300164) = 0x2471$
 ;对应于 8MHz 晶振时开方所用时间为 $48.38\mu\text{s}$

```

.include "8515def.inc"
        .org 0x00
        rjmp     reset
        .org 0x20
reset:
        ldi      r16,0x02           ; Stack Pointer Setup
        out      SPH,r16           ; Stack Pointer High Byte
        ldi      r16,0x5F           ; Stack Pointer Setup
        out      SPL,r16           ; Stack Pointer Low Byte
; 验证
;test r25:r24 = lsqrt(0x5300164)
loopm:
        ldi      r16,0x64
        mov      r12,r16
        ldi      r16,0x01
        mov      r13,r16
        ldi      r16,0x30
        mov      r14,r16
        ldi      r16,0x05
        mov      r15,r16
        rcall     lsqrt
        nop
        rjmp     loopm

```

lsqrt:		
	clr	r20
	clr	r21
	clr	r22
	clr	r24
	clr	r25
	clr	r26
	ldi	r28,16
m03:		
	rol	r15
	rol	r20
	rol	r21
	rol	r22
	rol	r15
rol	r20	
	rol	r21
	rol	r22
	lsl	r24
	rol	r25
	rol	r26
	cp	r24,r20
	cpc	r25,r21
	cpc	r26,r22
	brsh	m01
	sbc	r20,r24
	sbc	r21,r25
	sbc	r22,r26
	subi	r24,-2
m05:		
m01:		
	dec	r28
	sbrs	r28,0
	sbrc	r28,1
	rjmp	m03
	breq	m08
	mov	r15,r14
	mov	r14,r13
	mov	r13,r12
	rjmp	m03
m08:		
	lsr	r26
	ror	r25
	ror	r24

```

        ret

    rjmp L2
    ret
prog1:
    ldi R24,2
    sts keyone,R24
    ret
prog2:
    ldi R24,3
    sts keyone,R24
    ret
prog3:
    ldi R24,4
    sts keyone,R24
    ret
prog4:
    ldi R24,5
    sts keyone,R24
    ret
prog5:
    ldi R24,1
    sts keyone,R24
    ret

```

程序二十 AVR 单片机一个简单的通信程序

/*以下这段程序演示标准 UART 的使用，本例使用 4MHz 晶振，通信波特率为 19200，一般程序处理可以只定义接收中断，发送就直接往 UDR 中传送数据就可以了。波特率的设置换算可以查芯片资料，有详细的计算公式以及速查对照表。C 语言编译器使用 ICCAVR6.22A。

在 AVR 单片机的中断向量表经常容易搞错，表中左边那一列是向量地址而不是向量编号，两者只差 1，请在调试程序的时候记得核查一下编号。

```

*/
#include "io2313.h"

#pragma interrupt_handler uart_rec:8 //定义接收中断向量
unsigned char i;

/*****串口接收中断*****/
void uart_rec()
{
    i=UDR;
}

```



```

/***** 主程序 *****/
void main()
{
    UBRR=12;           //对应于 4M,19200 波特率
    UCR=0x98;          //允许接收中断, 允许发送
    SREG|=0x80;         //开中断

    while(1){
        if (i)          //有字符收到
        {
            USR&=~0x40;   //清发送完标志
            UDR=i;         //发送数据(回传接收数据)
            while(!(USR&0x40)); //等待发送结束
            USR&=~0x40;   //清发送完标志
            i=0;           //清变量, 以备下一次接收
        }
    }
}

```

程序二十一 TG19264A 接口程序

;连线方式:

LCM---89C52	*LCM---89C52*	*LCM-----89C52*	*LCM-----89C52* *
DB0---P0.0	*DB4---P0.4*	*D/I-----P2.6*	*CS1-----P2.4* *
DB1---P0.1	*DB5---P0.5*	*R/W-----P2.7*	*CS2-----P2.5* *
DB2---P0.2	*DB6---P0.6*	*RST-----VCC*	*CS3-----P3.2* *
DB3---P0.3	*DB7---P0.7*	*E-----P2.3*	*

89C52 的晶振频率为 12MHz

;接口引脚定义:

```

ELCM    EQU    P2.3
CS1     EQU    P2.4
CS2     EQU    P2.5
CS3     EQU    P3.2
DI      EQU    P2.6
RW      EQU    P2.7
DATA_LCM EQU    P0

```

;内存 RAM 分配

```

XPOS    EQU    30H    ;X 坐标
YPOS    EQU    31H    ;Y 坐标

```

;液晶片分区参数

PD1 EQU 40H ;每个分区宽 64 点

;主程序开始

```
ORG    0000H
JMP     START
org     0023h
LJMP    RS232
```

```
START: MOV     SP,#60H      ; void main() {
MAIN:  LCALL    DELAY400MS  ;      void delay(void);
      CALL     LCDRESET ;   void lcdreset(void);
      MOV      A,#55H
      CALL     LCDFILL      ;   void lcdfill(uchar a );
      MOV      DPTR,#STRING1 ;   uchar *p = *string1;
      CALL     PUTSTR       ;   void putstr(uchar *p);
      MOV      YPOS,#2
      MOV      XPOS,#0
      CALL     PUTSTR       ;   void putatr(uchar *p);
M1:    AJMP     M1          ;   while(1);
      ;   }
```

;长延时程序，主要用于初始化之前，CPU 等待 LCM 准备好(400MS)

DELAY400MS:

```
      MOV      R0,#20
DL4_PA: MOV      R1,#100
DL4_PB: MOV      R2,#100
      DJNZ     R2,$
      DJNZ     R1,DL4_PB
      DJNZ     R0,DL4_PA
      RET
```

;短延时程序，主要用于显示演示速度

DELAY:

```
      MOVR5,#4
DLY_PA: MOVR4,#0
      DJNZ     R4,$
      DJNZ     R5,DLY_PA
      RET
```

;获取字符串内字符编码，C=0 显示结束，字符串以 0FFH 结尾作为结束标志

;以两字节组成一个字符：前一字节表示是全角(>=80H)还是半角(< --

DB 000H,000H,080H,040H,020H,010H,008H,000H

```

DB 000H,001H,002H,004H,008H,010H,020H,000H

;-- 文字: > --
DB 000H,008H,010H,020H,040H,080H,000H,000H
DB 000H,020H,010H,008H,004H,002H,001H,000H

;-- 文字: ? --
DB 000H,070H,048H,008H,008H,008H,0F0H,000H
DB 000H,000H,000H,030H,036H,001H,000H,000H

;-- 文字: / --
DB 000H,000H,000H,000H,080H,060H,018H,004H
DB 000H,060H,018H,006H,001H,000H,000H,000H

;-- 文字: { --
DB 000H,000H,000H,000H,080H,07CH,002H,002H
DB 000H,000H,000H,000H,000H,03FH,040H,040H

;-- 文字: } --
DB 000H,002H,002H,07CH,080H,000H,000H,000H
DB 000H,040H,040H,03FH,000H,000H,000H,000H

;-- 文字: [ --
DB 000H,000H,000H,0FEH,002H,002H,002H,000H
DB 000H,000H,000H,07FH,040H,040H,040H,000H

;-- 文字: ] --
DB 000H,002H,002H,002H,0FEH,000H,000H,000H
DB 000H,040H,040H,040H,07FH,000H,000H,000H

;-- 文字: \ --
DB 000H,00CH,030H,0C0H,000H,000H,000H,000H
DB 000H,000H,000H,001H,006H,038H,0C0H,000H

;-- 文字: | --
DB 000H,000H,000H,000H,0FFH,000H,000H,000H
DB 000H,000H,000H,000H,0FFH,000H,000H,000H
STRING1:DB 80H, 00H, 80H, 01H, 80H, 02H, 80H, 03H, 80H, 04H, 80H, 05H
DB 80H, 06H, 80H, 07H, 80H, 08H, 80H, 09H, 80H, 0aH, 80H, 0BH,0fH
STRING2:DB 00H, 00H, 00H, 01H, 00H, 02H, 00H, 03H, 00H, 04H, 00H, 05H
DB 00H, 06H, 00H, 07H, 00H, 08H, 00H, 09H, 00H, 0AH, 00H, 0BH
DB 00H, 0CH, 00H, 0dH, 00H, 0eH, 00H, 0FH, 00H, 10H, 00H, 11H
DB 00H, 12H, 00H, 13H, 00H, 14H, 00H, 15H, 00H, 16H, 00H, 17H
DB 00H, 18H, 00H, 19H, 00H, 1AH, 00H, 1BH, 00H, 1CH, 00H, 1DH
DB 00H, 1EH, 00H, 1FH, 00H, 20H, 00H, 21H, 00H, 22H, 00H, 23H

```

```

DB 00H, 24H, 00H, 25H, 00H, 26H, 00H, 27H, 00H, 28H, 00H, 29H
DB 00H, 2AH, 00H, 2BH, 00H, 2CH, 00H, 2DH, 00H, 2EH, 00H, 2FH
DB 00H, 30H, 00H, 31H, 00H, 32H, 00H, 33H, 00H, 34H, 00H, 35H
DB 00H, 36H, 00H, 37H, 00H, 38H, 00H, 39H, 00H, 3AH, 00H, 3BH
DB 00H, 3CH, 00H, 3DH, 00H, 3EH, 00H, 3FH, 00H, 40H, 00H, 41H
DB 00H, 42H, 00H, 43H, 00H, 44H, 00H, 45H, 00H, 46H, 00H, 47H, 0FFH

```

```
RS232: RETI
```

```
END
```

程序二十二 TG19264A 接口程序 (AVR 模拟方式)

```

;
;*****
;连线:
;
;*LCM-----S8515*      *LCM---S8515*      *LCM-----S8515*      *LCM-----S8515*      *
;*DB0-----PA0*        *DB4-----PA4*        *D/I-----PC6*        *CS1-----PC5*        *
;*DB1-----PA1*        *DB5-----PA5*        *R/W-----PC7*        *CS2-----PC4*        *
;*DB2-----PA2*        *DB6-----PA6*        */RST-----VCC*        *CS3-----PD2*        *
;*DB3-----PA3*        *DB7-----PA7*        *E-----PC3*        *
;注:AT90S8515 的晶振频率为 8MHz
;*****
;include"8515def.inc"      ;器件配置文件
;接口引脚定义:
.equ      DI      =PC6      ;数据/命令
.equ      RW      =PC7      ;读/写
.equ      ELCM     =PC3      ;操作允许, 高电平有效 (允许)
.equ      CS1      =PC4      ;左区选中, 低电平有效
.equ      CS2      =PC5      ;中区选中, 低电平有效
.equ      CS3      =PD2      ;右区选中, 低电平有效
.equ      Colend    = 0x40    ;每一个分区宽 64 点

.def      status    = r1
.def      eeaddr1    = r5      ;eeprom 地址低 8 位
.def      eeaddrh    = r6      ;eeprom 地址高 8 位
.def      temp       = r3
.def      templ      = r4
.def      rxpos      = r9      ;X 坐标备份
.def      rypos      = r10     ;X 坐标备份
.def      mode       = r15
.def      cbyte      = r16     ;通用寄存器, 主要在液晶显示数据
.def      count      = r17
.def      ioreg      = r18

```

```

.def      baud      = r20
.def      maxcol     = r21          ;字符点阵宽度（8 或 16）
.def      col        = r22          ;X 坐标
.def      row        = r23          ;Y 坐标
.def      lbyte      = r24
.def      hbyte      = r25

.macro     Dptrinc                                ;AVR 没有带进位立即数的加法，没办法
    subi   r30,low(-0x0001)                    ;才出此下策
    sbci   r31,high(-0x0001)
.endmacro

;主程序开始
.cseg
    .org    $000
    rjmp    RESET
    .org    URXCaddr

;*****
; * Program starts here after power up. *
;*****
.org    10                                ;保留空间给应用程序（中断向量区）
RESET:  cli                                  ;关全局中断允许位(SREG.7)=I
        ldi    cbyte,low(RAMEND)            ;堆栈顶部设定
        out    SPL,cbyte                    ;
        ldi    cbyte,high(RAMEND)           ;
        out    SPH,cbyte                    ;
        clr    eeaddrl                      ;内部 EEPROM 指针清零
        clr    eeaddrh                      ;指向起始点
        clr    mode                         ;显示模式清零
        clr    cbyte                        ;显示内容清零
        out    GIMSK,cbyte                  ;通用屏蔽寄存器清零
        out    timsk,cbyte                  ;定时/计数器中断屏蔽寄存器清零
        out    wdtcr,cbyte                  ;看门狗定时控制寄存器清零

        clr    count                        ;程序软件计数器（R17）清零
        out    MCUCR,count                  ;MCU 通用控制寄存器清零
        ser    ioreg                        ;置 FF. (R18=0xFF)
        out    DDRD,ioreg                  ;定义端口 D 为输出
        out    DDRC,ioreg                  ;定义端口 C 为输出
        out    portb,ioreg                  ;端口 B 置 FF
        out    portd,ioreg                  ;端口 D 置 FF
        rjmp    Main                        ;初始化结束，进入主程序

```

```

;*****
;

```

; 主程序

```

;*****
;

```

```

Main:    ldi        cbyte, low(ramend)           ;堆栈顶部设定
        out        SPL, cbyte                   ;keep our stack      healthy!!!
        ldi        cbyte, high(ramend)          ;确认堆栈指针正确
        out        SPH, cbyte
        rcall      delay                        ;延时等待 LCM 复位好
        rcall      delay
        rcall      Initlcd                     ;液晶模块初始化
        ldi        col, 0x00                    ; X = 0
        ldi        row, 0x00                   ; Y = 0 (第一行)
        ldi        r30, low(String1*2)         ; Z 指针指向第一个字符串
        ldi        r31, high(String1*2)
        rcall      Putstr                      ;显示输出这个字符串
        ldi        col, 0x00                    ; X = 0
        ldi        row, 0x02                   ; Y = 2 (第二行)
        ldi        r30, low(String2*2)         ; Z 指针指向第二个字符串
        ldi        r31, high(String2*2)
        rcall      Putstr                      ;显示输出这个字符串

Main1:   wdr
        rjmp       Main1                      ;程序死循环，到此为止。

```

```

;*****
;

```

; LCM 系统复位

```

;*****
;

```

```

InitLcd:
        ldi        cbyte, 0x3e                 ;关显示
        rcall      WrCmd1
        rcall      WrCmd2
        rcall      WrCmd3
        ldi        cbyte, 0x3F                 ;开显示
        rcall      WrCmd1
        rcall      WrCmd2
        rcall      WrCmd3
        ldi        cbyte, 0xC0                 ;设定起始地址
        rcall      WrCmd1
        rcall      WrCmd2
        rcall      WrCmd3
        rcall      Cls                         ;清屏
        ret

```

```

;*****
;

```

; 延时程序

```

;*****
;

```

```

Delay:    push    temp
          push    temp1
          clr     temp
dt11:    clr     temp1
dt21:    nop
          dec temp1
          brne dt21
          dec temp
          brne dt11
          pop temp1
          pop temp
          ret

```

;获取字符串内字符编码，C=0 显示结束，字符串以 0FFH 结尾作为结束标志

;以两字节组成一个字符：前一字节表示是全角(>=80H)还是半角(=Colend 是中区或右区

```

          rcall    Wrdatal          ;输出该（数据）字节
          rjmp     LW3              ;返回
LW1:      cpi      col,Colend*2      ;在中与右间区分
          brsh     LW2              ;col>=2Colend 则为右区
          rcall    Wrdatal          ;中区数据输出
          rjmp     LW3              ;返回
LW2:      rcall    Wrdatal          ;右区数据输出
LW3:      ret                      ;返回

```

;清屏，全屏幕清零

```

Cls:      ldi      row,0x00          ;Y 坐标起点
          ldi      cbyte,0x00        ;填充数据 0x00
lflpb:    ldi      col,0x00          ;X 坐标起点
lflpa:    rcall    WrData            ;数据写输出(分区自动判别)
          rcall    Cusornext         ;计算下一个该填充位置
          cpi      col,0x00          ;有否产生换行?
          brne     lflpa             ;还未完成本行处理，继续
          cpi      row,0x00          ;是否已结束?
          brne     lflpb             ;没有结束，继续下一行处理
          ret

```

;连续输出时的坐标指针换算，自动指向下一个可写入地址

```

Cusornext:
          andi     row,0x07          ;防止意外，确认只保留低 3 位
          inc      col              ;X=X+1

```

```

        cpi        col,Colend*3          ; 是否出界 (右边界=3Colend)
        brlo       Cusret                ; 未出界(col= 则转移
        andi       cbyte,0x3f           ; 保留区内坐标
        ori        cbyte,0x40           ; X 方向坐标指令
        rcall      Wrcmd1
        mov        cbyte,rypos          ; Y 方向定位指令发出
        rcall      Wrcmd1
        rjmp       Locatex
locaterm:                                ; 中或右
        cpi        cbyte,Colend*2       ; 中与右的分界线判别
        brsh       Locater              ; >=则为右区
        andi       cbyte,0x3f           ; 保留区内坐标
        ori        cbyte,0x40           ; X 方向坐标指令
        rcall      Wrcmd2
        mov        cbyte,rypos          ; Y 方向定位指令发出
        rcall      Wrcmd2
        rjmp       Locatex
Locater:
        andi       cbyte,0x3f           ; 保留区内坐标
        ori        cbyte,0x40           ; X 方向坐标指令
        rcall      Wrcmd3
        mov        cbyte,rypos          ; Y 方向定位指令发出
        rcall      Wrcmd3
locatex:
        pop        cbyte
        ret

```

HZKDOT:

```

; C3515  0
        .DB 0x04,0x04,0xC4,0x44,0x5F,0x44,0x44,0xF4
        .DB 0x44,0x4F,0x54,0x64,0x44,0x46,0x04,0x00
        .DB 0x80,0x40,0x3F,0x00,0x40,0x40,0x20,0x20
        .DB 0x13,0x0C,0x18,0x24,0x43,0x80,0xE0,0x00

; C4843  1
        .DB 0x00,0xFE,0x4A,0x4A,0x00,0xFE,0xEA,0xAA
        .DB 0xAA,0xFE,0x00,0x4A,0x4A,0xFE,0x00,0x00
        .DB 0x02,0x83,0x42,0x22,0x12,0x1B,0x02,0x02
        .DB 0x02,0x0B,0x12,0x22,0x62,0xC3,0x02,0x00

; C2590  2
        .DB 0x00,0xFE,0x02,0xD2,0x52,0x52,0xD2,0x3E
        .DB 0xD2,0x16,0x1A,0x12,0xFF,0x02,0x00,0x00
        .DB 0x00,0xFF,0x50,0x53,0x52,0x4A,0x6B,0x50
        .DB 0x4F,0x54,0x7B,0x40,0xFF,0x00,0x00,0x00

; C2842  3

```



```

.DB 0x00,0xFE,0x22,0xD2,0x0E,0x20,0xB8,0x4F
.DB 0xB2,0x9E,0x80,0x9F,0x72,0x8A,0x06,0x00
.DB 0x00,0xFF,0x04,0x08,0x07,0x21,0x12,0x0A
.DB 0x46,0x82,0x7E,0x06,0x0A,0x12,0x31,0x00
; C0308 4
.DB 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00
.DB 0x00,0xC0,0x30,0x08,0x04,0x02,0x00,0x00
.DB 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00
.DB 0x00,0x03,0x0C,0x10,0x20,0x40,0x00,0x00
; C2567 5
.DB 0x00,0x00,0xFC,0x44,0x54,0x54,0x7C,0x55
.DB 0xD6,0x54,0x7C,0x54,0x54,0x44,0x44,0x00
.DB 0x80,0x60,0x1F,0x80,0x9F,0x55,0x35,0x15
.DB 0x1F,0x15,0x15,0x35,0x5F,0x80,0x00,0x00
; C2211 6
.DB 0x00,0x08,0xE8,0xA8,0xA8,0xA8,0xA8,0xFF
.DB 0xA8,0xA8,0xA8,0xA8,0xE8,0x0C,0x08,0x00
.DB 0x00,0x40,0x23,0x12,0x0A,0x06,0x02,0xFF
.DB 0x02,0x06,0x0A,0x12,0x23,0x60,0x20,0x00
; C0309 7
.DB 0x00,0x00,0x02,0x04,0x08,0x30,0xC0,0x00
.DB 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00
.DB 0x00,0x00,0x40,0x20,0x10,0x0C,0x03,0x00
.DB 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00
; C5148 8
.DB 0x04,0x04,0x04,0x84,0xE4,0x3C,0x27,0x24
.DB 0x24,0x24,0x24,0xF4,0x24,0x06,0x04,0x00
.DB 0x04,0x02,0x01,0x00,0xFF,0x09,0x09,0x09
.DB 0x09,0x49,0x89,0x7F,0x00,0x00,0x00,0x00
; C4762 9
.DB 0x00,0xFE,0x02,0x22,0xDA,0x06,0x00,0xFE
.DB 0x92,0x92,0x92,0x92,0xFF,0x02,0x00,0x00
.DB 0x00,0xFF,0x08,0x10,0x08,0x07,0x00,0xFF
.DB 0x42,0x24,0x08,0x14,0x22,0x61,0x20,0x00
; C2511 10
.DB 0x00,0x00,0x80,0x40,0x30,0x0C,0x00,0xC0
.DB 0x07,0x1A,0x20,0x40,0x80,0x80,0x80,0x00
.DB 0x01,0x01,0x20,0x70,0x28,0x24,0x23,0x20
.DB 0x20,0x28,0x30,0x60,0x00,0x01,0x00,0x00
; C4330 11
.DB 0x10,0x10,0x92,0x92,0x92,0x92,0x92,0x92
.DB 0xD2,0x9A,0x12,0x02,0xFF,0x02,0x00,0x00
.DB 0x00,0x00,0x3F,0x10,0x10,0x10,0x10,0x10
.DB 0x3F,0x00,0x40,0x80,0x7F,0x00,0x00,0x00

```

EZKDOT:

```

;--文字:A--1
    .DB 0x00,0x00,0xC0,0x38,0xE0,0x00,0x00,0x00
    .DB 0x20,0x3C,0x23,0x02,0x02,0x27,0x38,0x20
;--文字:B--2
    .DB 0x08,0xF8,0x88,0x88,0x88,0x70,0x00,0x00
    .DB 0x20,0x3F,0x20,0x20,0x20,0x11,0x0E,0x00
;--文字:C--3
    .DB 0xC0,0x30,0x08,0x08,0x08,0x08,0x38,0x00
    .DB 0x07,0x18,0x20,0x20,0x20,0x10,0x08,0x00
;--文字:D--4
    .DB 0x08,0xF8,0x08,0x08,0x08,0x10,0xE0,0x00
    .DB 0x20,0x3F,0x20,0x20,0x20,0x10,0x0F,0x00
;--文字:E--5
    .DB 0x08,0xF8,0x88,0x88,0xE8,0x08,0x10,0x00
    .DB 0x20,0x3F,0x20,0x20,0x23,0x20,0x18,0x00
;--文字:F--6
    .DB 0x08,0xF8,0x88,0x88,0xE8,0x08,0x10,0x00
    .DB 0x20,0x3F,0x20,0x00,0x03,0x00,0x00,0x00
;--文字:G--7
    .DB 0xC0,0x30,0x08,0x08,0x08,0x38,0x00,0x00
    .DB 0x07,0x18,0x20,0x20,0x22,0x1E,0x02,0x00
;--文字:H--8
    .DB 0x08,0xF8,0x08,0x00,0x00,0x08,0xF8,0x08
    .DB 0x20,0x3F,0x21,0x01,0x01,0x21,0x3F,0x20
;--文字:I--9
    .DB 0x00,0x08,0x08,0xF8,0x08,0x08,0x00,0x00
    .DB 0x00,0x20,0x20,0x3F,0x20,0x20,0x00,0x00
;--文字:J--A
    .DB 0x00,0x00,0x08,0x08,0xF8,0x08,0x08,0x00
    .DB 0xC0,0x80,0x80,0x80,0x7F,0x00,0x00,0x00
;--文字:K--B
    .DB 0x08,0xF8,0x88,0xC0,0x28,0x18,0x08,0x00
    .DB 0x20,0x3F,0x20,0x01,0x26,0x38,0x20,0x00
;--文字:L--C
    .DB 0x08,0xF8,0x08,0x00,0x00,0x00,0x00,0x00
    .DB 0x20,0x3F,0x20,0x20,0x20,0x20,0x30,0x00
;--文字:M--D
    .DB 0x08,0xF8,0xF8,0x00,0xF8,0xF8,0x08,0x00
    .DB 0x20,0x3F,0x00,0x3F,0x00,0x3F,0x20,0x00
;--文字:N--E
    .DB 0x08,0xF8,0x30,0xC0,0x00,0x08,0xF8,0x08
    .DB 0x20,0x3F,0x20,0x00,0x07,0x18,0x3F,0x00
;--文字:O--F
    .DB 0xE0,0x10,0x08,0x08,0x08,0x10,0xE0,0x00
    .DB 0x0F,0x10,0x20,0x20,0x20,0x10,0x0F,0x00

```

```

;--文字:P--0x10
    .DB 0x08,0xF8,0x08,0x08,0x08,0x08,0xF0,0x00
    .DB 0x20,0x3F,0x21,0x01,0x01,0x01,0x00,0x00
;--文字:Q--0x11
    .DB 0xE0,0x10,0x08,0x08,0x08,0x10,0xE0,0x00
    .DB 0x0F,0x18,0x24,0x24,0x38,0x50,0x4F,0x00
;--文字:R--0x12
    .DB 0x08,0xF8,0x88,0x88,0x88,0x88,0x70,0x00
    .DB 0x20,0x3F,0x20,0x00,0x03,0x0C,0x30,0x20
;--文字:S--0x13
    .DB 0x00,0x70,0x88,0x08,0x08,0x08,0x38,0x00
    .DB 0x00,0x38,0x20,0x21,0x21,0x22,0x1C,0x00
;--文字:T--0x14
    .DB 0x18,0x08,0x08,0xF8,0x08,0x08,0x18,0x00
    .DB 0x00,0x00,0x20,0x3F,0x20,0x00,0x00,0x00
;--文字:U--0x15
    .DB 0x08,0xF8,0x08,0x00,0x00,0x08,0xF8,0x08
    .DB 0x00,0x1F,0x20,0x20,0x20,0x20,0x1F,0x00
;--文字:V--0x16
    .DB 0x08,0x78,0x88,0x00,0x00,0xC8,0x38,0x08
    .DB 0x00,0x00,0x07,0x38,0x0E,0x01,0x00,0x00
;--文字:W--0x17
    .DB 0xF8,0x08,0x00,0xF8,0x00,0x08,0xF8,0x00
    .DB 0x03,0x3C,0x07,0x00,0x07,0x3C,0x03,0x00
;--文字:X--0x18
    .DB 0x08,0x18,0x68,0x80,0x80,0x68,0x18,0x08
    .DB 0x20,0x30,0x2C,0x03,0x03,0x2C,0x30,0x20
;--文字:Y--0x19
    .DB 0x08,0x38,0xC8,0x00,0xC8,0x38,0x08,0x00
    .DB 0x00,0x00,0x20,0x3F,0x20,0x00,0x00,0x00
;--文字:Z--0x1a
    .DB 0x10,0x08,0x08,0x08,0xC8,0x38,0x08,0x00
    .DB 0x20,0x38,0x26,0x21,0x20,0x20,0x18,0x00
;--文字:a--0x1b
    .DB 0x00,0x00,0x80,0x80,0x80,0x80,0x00,0x00
    .DB 0x00,0x19,0x24,0x22,0x22,0x22,0x3F,0x20
;--文字:b--0x1c
    .DB 0x08,0xF8,0x00,0x80,0x80,0x00,0x00,0x00
    .DB 0x00,0x3F,0x11,0x20,0x20,0x11,0x0E,0x00
;--文字:c--0x1d
    .DB 0x00,0x00,0x00,0x80,0x80,0x80,0x00,0x00
    .DB 0x00,0x0E,0x11,0x20,0x20,0x20,0x11,0x00
;--文字:d--0x1e
    .DB 0x00,0x00,0x00,0x80,0x80,0x88,0xF8,0x00
    .DB 0x00,0x0E,0x11,0x20,0x20,0x10,0x3F,0x20

```

```

;--文字:e--0x1f
.DB 0x00,0x00,0x80,0x80,0x80,0x80,0x00,0x00
.DB 0x00,0x1F,0x22,0x22,0x22,0x22,0x13,0x00
;--文字:f--0x20
.DB 0x00,0x80,0x80,0xF0,0x88,0x88,0x88,0x18
.DB 0x00,0x20,0x20,0x3F,0x20,0x20,0x00,0x00
;--文字:g--0x21
.DB 0x00,0x00,0x80,0x80,0x80,0x80,0x80,0x00
.DB 0x00,0x6B,0x94,0x94,0x94,0x93,0x60,0x00
;--文字:h--0x22
.DB 0x08,0xF8,0x00,0x80,0x80,0x80,0x00,0x00
.DB 0x20,0x3F,0x21,0x00,0x00,0x20,0x3F,0x20
;--文字:i--0x23
.DB 0x00,0x80,0x98,0x98,0x00,0x00,0x00,0x00
.DB 0x00,0x20,0x20,0x3F,0x20,0x20,0x00,0x00
;--文字:j--0x24
.DB 0x00,0x00,0x00,0x80,0x98,0x98,0x00,0x00
.DB 0x00,0xC0,0x80,0x80,0x80,0x7F,0x00,0x00
;--文字:k--0x25
.DB 0x08,0xF8,0x00,0x00,0x80,0x80,0x80,0x00
.DB 0x20,0x3F,0x24,0x02,0x2D,0x30,0x20,0x00
;--文字:l--0x26
.DB 0x00,0x08,0x08,0xF8,0x00,0x00,0x00,0x00
.DB 0x00,0x20,0x20,0x3F,0x20,0x20,0x00,0x00
;--文字:m--0x27
.DB 0x80,0x80,0x80,0x80,0x80,0x80,0x80,0x00
.DB 0x20,0x3F,0x20,0x00,0x3F,0x20,0x00,0x3F
;--文字:n--0x28
.DB 0x80,0x80,0x00,0x80,0x80,0x80,0x00,0x00
.DB 0x20,0x3F,0x21,0x00,0x00,0x20,0x3F,0x20
;--文字:o--0x29
.DB 0x00,0x00,0x80,0x80,0x80,0x80,0x00,0x00
.DB 0x00,0x1F,0x20,0x20,0x20,0x20,0x1F,0x00
;--文字:p--0x2a
.DB 0x80,0x80,0x00,0x80,0x80,0x00,0x00,0x00
.DB 0x80,0xFF,0xA1,0x20,0x20,0x11,0x0E,0x00
;--文字:q--0x2b
.DB 0x00,0x00,0x00,0x80,0x80,0x80,0x80,0x00
.DB 0x00,0x0E,0x11,0x20,0x20,0xA0,0xFF,0x80
;--文字:r--0x2c
.DB 0x80,0x80,0x80,0x00,0x80,0x80,0x80,0x00
.DB 0x20,0x20,0x3F,0x21,0x20,0x00,0x01,0x00
;--文字:s--0x2d
.DB 0x00,0x00,0x80,0x80,0x80,0x80,0x80,0x00
.DB 0x00,0x33,0x24,0x24,0x24,0x24,0x19,0x00

```

```

;--文字:t--0x2e
.DB 0x00,0x80,0x80,0xE0,0x80,0x80,0x00,0x00
.DB 0x00,0x00,0x00,0x1F,0x20,0x20,0x00,0x00
;--文字:u--0x2f
.DB 0x80,0x80,0x00,0x00,0x00,0x80,0x80,0x00
.DB 0x00,0x1F,0x20,0x20,0x20,0x10,0x3F,0x20
;--文字:v--0x30
.DB 0x80,0x80,0x80,0x00,0x00,0x80,0x80,0x80
.DB 0x00,0x01,0x0E,0x30,0x08,0x06,0x01,0x00
;--文字:w--0x31
.DB 0x80,0x80,0x00,0x80,0x00,0x80,0x80,0x80
.DB 0x0F,0x30,0x0C,0x03,0x0C,0x30,0x0F,0x00
;--文字:x--0x32
.DB 0x00,0x80,0x80,0x00,0x80,0x80,0x80,0x00
.DB 0x00,0x20,0x31,0x2E,0x0E,0x31,0x20,0x00
;--文字:y--0x33
.DB 0x80,0x80,0x80,0x00,0x00,0x80,0x80,0x80
.DB 0x80,0x81,0x8E,0x70,0x18,0x06,0x01,0x00
;--文字:z--0x34
.DB 0x00,0x80,0x80,0x80,0x80,0x80,0x80,0x00
.DB 0x00,0x21,0x30,0x2C,0x22,0x21,0x30,0x00
;--文字:0--0x35
.DB 0x00,0xE0,0x10,0x08,0x08,0x10,0xE0,0x00
.DB 0x00,0x0F,0x10,0x20,0x20,0x10,0x0F,0x00
;--文字:1--0x36
.DB 0x00,0x10,0x10,0xF8,0x00,0x00,0x00,0x00
.DB 0x00,0x20,0x20,0x3F,0x20,0x20,0x00,0x00
;--文字:2--0x37
.DB 0x00,0x70,0x08,0x08,0x08,0x88,0x70,0x00
.DB 0x00,0x30,0x28,0x24,0x22,0x21,0x30,0x00
;--文字:3--0x38
.DB 0x00,0x30,0x08,0x88,0x88,0x48,0x30,0x00
.DB 0x00,0x18,0x20,0x20,0x20,0x11,0x0E,0x00
;--文字:4--0x39
.DB 0x00,0x00,0xC0,0x20,0x10,0xF8,0x00,0x00
.DB 0x00,0x07,0x04,0x24,0x24,0x3F,0x24,0x00
;--文字:5--0x3a
.DB 0x00,0xF8,0x08,0x88,0x88,0x08,0x08,0x00
.DB 0x00,0x19,0x21,0x20,0x20,0x11,0x0E,0x00
;--文字:6--0x3b
.DB 0x00,0xE0,0x10,0x88,0x88,0x18,0x00,0x00
.DB 0x00,0x0F,0x11,0x20,0x20,0x11,0x0E,0x00
;--文字:7--0x3c
.DB 0x00,0x38,0x08,0x08,0xC8,0x38,0x08,0x00
.DB 0x00,0x00,0x00,0x3F,0x00,0x00,0x00,0x00

```

```

;--文字:8--0x3d
.DB 0x00,0x70,0x88,0x08,0x08,0x88,0x70,0x00
.DB 0x00,0x1C,0x22,0x21,0x21,0x22,0x1C,0x00
;--文字:9--0x3e
.DB 0x00,0xF0,0x10,0x08,0x08,0x10,0xE0,0x00
.DB 0x00,0x00,0x31,0x22,0x22,0x11,0x0F,0x00
;--文字:!--0x3f
.DB 0x00,0x00,0x00,0xF8,0x00,0x00,0x00,0x00
.DB 0x00,0x00,0x00,0x33,0x30,0x00,0x00,0x00
;--文字:@--0x40
.DB 0xC0,0x30,0xC8,0x28,0xE8,0x10,0xE0,0x00
.DB 0x07,0x18,0x27,0x24,0x23,0x14,0x0B,0x00
;--文字:#--0x41
.DB 0x40,0xC0,0x78,0x40,0xC0,0x78,0x40,0x00
.DB 0x04,0x3F,0x04,0x04,0x3F,0x04,0x04,0x00
;--文字:$--0x42
.DB 0x00,0x70,0x88,0xFC,0x08,0x30,0x00,0x00
.DB 0x00,0x18,0x20,0xFF,0x21,0x1E,0x00,0x00
;--文字:%--0x43
.DB 0xF0,0x08,0xF0,0x00,0xE0,0x18,0x00,0x00
.DB 0x00,0x21,0x1C,0x03,0x1E,0x21,0x1E,0x00
;--文字:^^--0x44
.DB 0x00,0x00,0x04,0x02,0x02,0x02,0x04,0x00
.DB 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00
;--文字:&--0x45
.DB 0x00,0xF0,0x08,0x88,0x70,0x00,0x00,0x00
.DB 0x1E,0x21,0x23,0x24,0x19,0x27,0x21,0x10
;--文字:*--0x46
.DB 0x40,0x40,0x80,0xF0,0x80,0x40,0x40,0x00
.DB 0x02,0x02,0x01,0x0F,0x01,0x02,0x02,0x00
;--文字:(--0x47
.DB 0x00,0x00,0x00,0xE0,0x18,0x04,0x02,0x00
.DB 0x00,0x00,0x00,0x07,0x18,0x20,0x40,0x00
;--文字:)--0x48
.DB 0x00,0x02,0x04,0x18,0xE0,0x00,0x00,0x00
.DB 0x00,0x40,0x20,0x18,0x07,0x00,0x00,0x00
;--文字:_--0x49
.DB 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00
.DB 0x80,0x80,0x80,0x80,0x80,0x80,0x80,0x80
;--文字:++--0x4a
.DB 0x00,0x00,0x00,0xF0,0x00,0x00,0x00,0x00
.DB 0x01,0x01,0x01,0x1F,0x01,0x01,0x01,0x00
;--文字:---0x4b
.DB 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00
.DB 0x00,0x01,0x01,0x01,0x01,0x01,0x01,0x01

```

```

;--文字:=-0x4c
.DB 0x40,0x40,0x40,0x40,0x40,0x40,0x40,0x00
.DB 0x04,0x04,0x04,0x04,0x04,0x04,0x04,0x00
;--文字:<--0x4d
.DB 0x00,0x00,0x80,0x40,0x20,0x10,0x08,0x00
.DB 0x00,0x01,0x02,0x04,0x08,0x10,0x20,0x00
;--文字:>--0x4e
.DB 0x00,0x08,0x10,0x20,0x40,0x80,0x00,0x00
.DB 0x00,0x20,0x10,0x08,0x04,0x02,0x01,0x00
;--文字:~--0x4f
.DB 0x00,0x70,0x48,0x08,0x08,0x08,0xF0,0x00
.DB 0x00,0x00,0x00,0x30,0x36,0x01,0x00,0x00
;--文字:/--0x50
.DB 0x00,0x00,0x00,0x00,0x80,0x60,0x18,0x04
.DB 0x00,0x60,0x18,0x06,0x01,0x00,0x00,0x00
;--文字:{--0x51
.DB 0x00,0x00,0x00,0x00,0x80,0x7C,0x02,0x02
.DB 0x00,0x00,0x00,0x00,0x00,0x3F,0x40,0x40
;--文字:)--0x52
.DB 0x00,0x02,0x02,0x7C,0x80,0x00,0x00,0x00
.DB 0x00,0x40,0x40,0x3F,0x00,0x00,0x00,0x00
;--文字:[--0x53
.DB 0x00,0x00,0x00,0xFE,0x02,0x02,0x02,0x00
.DB 0x00,0x00,0x00,0x7F,0x40,0x40,0x40,0x00
;--文字:]--0x54
.DB 0x00,0x02,0x02,0x02,0xFE,0x00,0x00,0x00
.DB 0x00,0x40,0x40,0x40,0x7F,0x00,0x00,0x00
;--文字:\--0x55
.DB 0x00,0x0C,0x30,0xC0,0x00,0x00,0x00,0x00
.DB 0x00,0x00,0x00,0x01,0x06,0x38,0xC0,0x00
;--文字:|--0x56
.DB 0x00,0x00,0x00,0x00,0xFF,0x00,0x00,0x00
.DB 0x00,0x00,0x00,0x00,0xFF,0x00,0x00,0x00

```

STRING1:

```

.DB 0x80,0x00,0x80,0x01,0x80,0x02,0x80,0x03
.DB 0x80,0x04,0x80,0x05,0x80,0x06,0x80,0x07
.DB 0x80,0x08,0x80,0x09,0x80,0x0a,0x80,0x0B
.DB 0xff,0x00

```

STRING2:

```

.DB 0x00,0x00,0x00,0x01,0x00,0x02,0x00,0x03,0x00,0x04,0x00,0x05
.DB 0x00,0x06,0x00,0x07,0x00,0x08,0x00,0x09,0x00,0x0A,0x00,0x0B
.DB 0x00,0x0C,0x00,0x0d,0x00,0x0e,0x00,0x0F,0x00,0x10,0x00,0x11
.DB 0x00,0x12,0x00,0x13,0x00,0x14,0x00,0x15,0x00,0x16,0x00,0x17
.DB 0x00,0x18,0x00,0x19,0x00,0x1A,0x00,0x1B,0x00,0x1C,0x00,0x1D
.DB 0x00,0x1E,0x00,0x1F,0x00,0x20,0x00,0x21,0x00,0x22,0x00,0x23

```

```
.DB 0x00,0x24,0x00,0x25,0x00,0x26,0x00,0x27,0x00,0x28,0x00,0x29
.DB 0x00,0x2A,0x00,0x2B,0x00,0x2C,0x00,0x2D,0x00,0x2E,0x00,0x2F
.DB 0x00,0x30,0x00,0x31,0x00,0x32,0x00,0x33,0x00,0x34,0x00,0x35
.DB 0x00,0x36,0x00,0x37,0x00,0x38,0x00,0x39,0x00,0x3A,0x00,0x3B
.DB 0x00,0x3C,0x00,0x3D,0x00,0x3E,0x00,0x3F,0x00,0x40,0x00,0x41
.DB 0x00,0x42,0x00,0x43,0x00,0x44,0x00,0x45,0x00,0x46,0x00,0x47
.DB 0xFF,0x00
```

程序二十三 常用的几种码制转换 BCD, HEX, BIN

```
#include <reg51.h>
#include <intrins.h>
#include <stdio.h>
#include <ctype.h>
#define LongToBin(n) \

((n >> 21) & 0x80) | \
((n >> 18) & 0x40) | \
((n >> 15) & 0x20) | \
((n >> 12) & 0x10) | \
((n >> 9) & 0x08) | \
((n >> 6) & 0x04) | \
((n >> 3) & 0x02) | \
((n) & 0x01) \
)

#define Bin(n) LongToBin(0x##n##1)

/***** HEX 转 BCD*****/
/**bcd_data(<0x255,>0)****/
unsigned char BCD2HEX(unsigned int bcd_data)
{
    unsigned char temp;
    temp=((bcd_data>>8)*100)+((bcd_data>>4)*10)+(bcd_data&0x0f);
    return temp;
}
/***** HEX 转 BCD*****/
/**hex_data(<0xff,>0)****/
unsigned int HEX2BCD(unsigned char hex_data)
{
    unsigned int bcd_data;
    unsigned char temp;
```



```

temp=hex_data%100;
bcd_data=((unsigned int)hex_data)/100<<8;
bcd_data=bcd_data*temp/10<<4;
bcd_data=bcd_data*temp%10;
return bcd_data;
}

void main(void)
{
    unsigned int c;

    c= Bin(10101001); // then c = 0xA9
    c=BCD2HEX(0x255); //255 转成 HEX 为 0xff
    c=HEX2BCD(0xff);   //0xff 转成 BCD 码为 255
}

```

程序二十四 16x2 字符液晶屏驱动演示程序一

连接线图: LCM-----51 LCM-----51 LCM-----51
 DB0----P0.0 DB4----P0.4 RS-----P2.0
 DB1----P0.1 DB5----P0.5 RW-----P2.1
 DB2----P0.2 DB6----P0.6 E-----P2.7
 DB3----P0.3 DB7----P0.7 VLCD 接 1K2 电阻到 GND

[注]:AT89C51 的晶振频率为 12MHz

=====*/

```

#include <reg51.h>
#include<intrins.h>

//变量类型标识的宏定义，大家都喜欢这么做
#define  Uchar unsigned char
#define  Uint unsigned int

// 控制引脚定义，不同的连接必须修改的部分
sbit  RS  = P2^0;
sbit  RW  = P2^1;
sbit  Elcm = P2^7;
#define DataPort P0                // 数据端口

#define Busy    0x80

```

```

code char exampl[]="For an example.      - By xiaoqi\n";

void Delay400Ms(void);
void Delay5Ms(void);
void WaitForEnable( void );
void LcdWriteData( char dataW );
void LcdWriteCommand( Uchar CMD,Uchar AttrbC );
void LcdReset( void );
void Display( Uchar dd );
void DispOneChar(Uchar x,Uchar y,Uchar Wdata);
void ePutstr(Uchar x,Uchar y, Uchar code *ptr);

//测试主程序
void main(void)
{
    Uchar temp;

    Delay400Ms();

    LcdReset();
    temp = 32;
    ePutstr(0,0,exampl);      // 上面一行显示一个预定字符串

    Delay400Ms();
    Delay400Ms();
    Delay400Ms();
    Delay400Ms();
    Delay400Ms();
    Delay400Ms();
    Delay400Ms();
    Delay400Ms();

    while(1)
    {
        temp &= 0x7f;      // 只显示 ASCII 字符
        if (temp<32)temp=32; // 屏蔽控制字符，不予显示
        Display( temp++ );
        Delay400Ms();
    }
}

/*=====
显示字符串
=====*/

```

```

void ePutstr(Uchar x,Uchar y, Uchar code *ptr) {
    Uchar i,l=0;
    while (ptr[l]>31){l++;};
    for (i=0;i<l;i++) {
        DispOneChar(x++,y,ptr[i]);
        if ( x == 16 ){
            x = 0; y ^= 1;
        }
    }
}

/*=====
演示一行连续字符串，配合上位程序演示移动字符串
=====*/

void Display( Uchar dd ) {

    Uchar i;

    for (i=0;i<16;i++) {
        DispOneChar(i,1,dd++);
        dd &= 0x7f;
        if (dd<32) dd=32;
    }
}

/*=====
显示光标定位
=====*/

void LocateXY( char posx,char posy) {

    Uchar temp;

    temp = posx & 0xf;
    posy &= 0x1;
    if ( posy )temp |= 0x40;
    temp |= 0x80;
    LcdWriteCommand(temp,0);
}

/*=====
按指定位置显示数出一个字符
=====*/

void DispOneChar(Uchar x,Uchar y,Uchar Wdata) {

    LocateXY( x, y);           // 定位显示地址

```

```

    LcdWriteData( Wdata );          // 写字符
}

/*=====
初始化程序, 必须按照产品资料介绍的初始化过程进行
=====*/
void LcdReset( void ) {

    LcdWriteCommand( 0x38, 0);      // 显示模式设置(不检测忙信号)
    Delay5Ms();
    LcdWriteCommand( 0x38, 0);      // 共三次
    Delay5Ms();
    LcdWriteCommand( 0x38, 0);
    Delay5Ms();

    LcdWriteCommand( 0x38, 1);      // 显示模式设置(以后均检测忙信号)
    LcdWriteCommand( 0x08, 1);      // 显示关闭
    LcdWriteCommand( 0x01, 1);      // 显示清屏
    LcdWriteCommand( 0x06, 1);      // 显示光标移动设置
    LcdWriteCommand( 0x0c, 1);      // 显示开及光标设置
}

/*=====
写控制字符子程序: E=1 RS=0 RW=0
=====*/
void LcdWriteCommand( Uchar CMD,Uchar AttrbC ) {

    if (AttrbC) WaitForEnable();    // 检测忙信号?

    RS = 0;   RW = 0; _nop_();

    DataPort = CMD; _nop_();        // 送控制字子程序

    Elcm = 1;_nop_();_nop_();Elcm = 0;    // 操作允许脉冲信号
}

/*=====
当前位置写字符子程序: E=1 RS=1 RW=0
=====*/
void LcdWriteData( char dataW ) {

    WaitForEnable();                // 检测忙信号

    RS = 1; RW = 0; _nop_();

```

```

DataPort = dataW: _nop_();

Elcm = 1; _nop_(); _nop_(); Elcm = 0;      // 操作允许脉冲信号

}

/*=====
正常读写操作之前必须检测 LCD 控制器状态:  CS=1 RS=0 RW=1
DB7:    0  LCD 控制器空闲; 1  LCD 控制器忙
=====*/
void WaitForEnable( void ) {

    DataPort = 0xff;

    RS =0; RW = 1; _nop_();  Elcm = 1; _nop_(); _nop_();

    while( DataPort & Busy );

    Elcm = 0;
}

// 短延时
void Delay5Ms(void)
{
    Uint i = 5552;
    while(i--);
}

//长延时
void Delay400Ms(void)
{
    Uchar i = 5;
    Uint j;
    while(i--)
    {
        j=7269;
        while(j--);
    };
}

```

程序二十五 16x2 字符液晶屏驱动演示程序二

连接方式: LCM-----51 LCM-----51 LCM-----51
 DB0----P0.0 DB4----P0.4 RW-----P2.0
 DB1----P0.1 DB5----P0.5 RC-----P2.1
 DB2----P0.2 DB6----P0.6 E-----P2.7 => 74ls00+wr+rd
 DB3----P0.3 DB7----P0.7 VLCD 接 1K2 电阻到 GND

[注]:AT89C51 的晶振频率为 12MHz

```

=====*/
//产生汇编文件
#pragma src
#include <reg51.h>
#include<intrins.h>

//===== 变量类型标识的宏定义, 大家都喜欢这么做 =====
#define Uchar unsigned char
#define Uint unsigned int

//===== LCM1602A 端口地址定义 =====
char xdata Lcd1602CmdPort _at_ 0x7cff; //E=1 RS=0 RW=0
char xdata Lcd1602WdataPort _at_ 0x7eff; //E=1 RS=1 RW=0
char xdata Lcd1602StatusPort _at_ 0x7dff; //CS=1 RS=0 RW=1

#define Busy 0x80 // 忙判别位

code char exampl[]="For an example. - By xiaoqi\n";

void Delay400Ms(void);
void Delay5Ms(void);
void LcdWriteData( char dataW );
void LcdWriteCommand( Uchar CMD,Uchar AttribC );
void LcdReset( void );
void Display( Uchar dd );
void DispOneChar(Uchar x,Uchar y,Uchar Wdata);
void cPutstr(Uchar x,Uchar y, Uchar code *ptr);

//=====测试主程序 =====
void main(void)
{
    Uchar temp;

```

```

Delay400Ms();                // 等待 lcm 进入工作状态

LcdReset();                  // 初始化
temp = 32;
ePutstr(0,0,exempl);        // 上面一行显示一个预定字符串

Delay400Ms();                // 保留显示内容
Delay400Ms();
Delay400Ms();
Delay400Ms();
Delay400Ms();
Delay400Ms();
Delay400Ms();
Delay400Ms();

while(1)
{
    temp &= 0x7f;            // 只显示 ASCII 字符

    if (temp<32)temp=32; // 屏蔽控制字符，不予显示
    Display( temp++ );

    Delay400Ms();
}
}

/*=====
显示字符串
=====*/

void ePutstr(Uchar x,Uchar y, Uchar code *ptr) {
Uchar i,l=0;
    while (ptr[l] >31){l++;};
    for (i=0;i<l;i++) {
        DispOneChar(x++,y,ptr[i]);
        if ( x == 16 ){
            x = 0; y ^= 1;
        }
    }
}

/*=====
演示一行连续字符串，配合上位程序演示移动字符串
=====*/

void Display( Uchar dd ) {

```

```

Uchar i;

    for (i=0;i<16;i++) {

        DispOneChar(i,1,dd++);

        dd &= 0x7f;
        if (dd<32) dd=32;
    }
}

/*=====
显示光标定位
=====*/
void LocateXY( char posx,char posy) {

Uchar temp;

    temp = posx & 0xf;
    posy &= 0x1;
    if ( posy )temp |= 0x40;
    temp |= 0x80;
    LcdWriteCommand(temp,0);
}

/*=====
按指定位置显示数出一个字符
=====*/
void DispOneChar(Uchar x,Uchar y,Uchar Wdata) {

    LocateXY( x, y );           // 定位显示地址
    LcdWriteData( Wdata );       // 写字符
}

/*=====
初始化程序, 必须按照产品资料介绍的初始化过程进行
=====*/
void LcdReset( void ) {

    LcdWriteCommand( 0x38, 0);   // 显示模式设置(不检测忙信号)
    Delay5Ms();
    LcdWriteCommand( 0x38, 0);   // 共三次
    Delay5Ms();
    LcdWriteCommand( 0x38, 0);

```



```

        Delay5Ms();

        LcdWriteCommand( 0x38, 1);           // 显示模式设置(以后均检测忙信号)
        LcdWriteCommand( 0x08, 1);           // 显示关闭
        LcdWriteCommand( 0x01, 1);           // 显示清屏
        LcdWriteCommand( 0x06, 1);           // 显示光标移动设置
        LcdWriteCommand( 0x0c, 1);           // 显示开及光标设置
    }

    /*=====
    写控制字符子程序: E=1 RS=0 RW=0
    =====*/
    void LcdWriteCommand( Uchar CMD,Uchar AttribC ) {

        if (AttribC) while( Lcd1602StatusPort & Busy );      // 检测忙信号?
        Lcd1602CmdPort = CMD;
    }

    /*=====
    当前位置写字符子程序: E =1 RS=1 RW=0
    =====*/
    void LcdWriteData( char dataW ) {

        while( Lcd1602StatusPort & Busy );      // 检测忙信号
        Lcd1602WdataPort = dataW;
    }

    // 短延时
    void Delay5Ms(void)
    {
        Uint i = 5552;
        while(i--);
    }

    //长延时
    void Delay400Ms(void)
    {
        Uchar i = 5;
        Uint j;
        while(i--)
        {
            j=7269;
            while(j--);
        };
    }
}

```

程序二十六 PS7219 代码

```
#include "reg51.h"
#include "ps7219.h"

void ps7219_reset()
{
    unsigned char i;
    ps7219_pin_RST=0;
    for(i=0;i<125;i++)
        ps7219_delay();
    ps7219_pin_RST=1;
    for(i=0;i<255;i++)
        ps7219_delay();
    ps7219_pin_RST=0;
    for(i=0;i<125;i++)
        ps7219_delay();
}

void ps7219_init()
{
    ps7219_reset();
    ps7219_send_data(addr_scan_count,0x04);
    ps7219_send_data(addr_light_con,0x0f);
    ps7219_send_data(addr_trans_mode,0xff);
    ps7219_send_data(addr_close,0x01);
}

void ps7219_echo(unsigned char da_1,da_2,da_3,da_4)
{
    ps7219_send_data(0x01,da_1);
    ps7219_send_data(0x02,da_2);
    ps7219_send_data(0x03,da_3);
    ps7219_send_data(0x04,da_4);
}

void ps7219_send_data(unsigned char addr,da)
{
    unsigned char i,byte_out;
    byte_out=addr;
```

```

ps7219_pin_DIN=1;
ps7219_pin_CLK=1;
ps7219_pin_LOAD=0;

for(i=0;i<8;i++)
{
ps7219_pin_CLK=1;
ps7219_pin_DIN=(bit)(byte_out&0x80);
byte_out=byte_out<<1;
ps7219_pin_CLK=0;
ps7219_delay();
}

ps7219_pin_CLK=1;
byte_out=da;
for(i=0;i<7;i++)
{
ps7219_pin_CLK=1;
ps7219_pin_DIN=(bit)(byte_out&0x80);
byte_out=byte_out<<1;
ps7219_pin_CLK=0;
ps7219_delay();
}
ps7219_pin_CLK=1;
ps7219_pin_LOAD=1;
ps7219_pin_DIN=(bit)(byte_out&0x80);
ps7219_pin_CLK=0;
ps7219_delay();
ps7219_pin_CLK=1;
}

void ps7219_delay(void)
{
unsigned char i;
for(i=0;i<125;i++)
{}
}

```

PS7219.H 代碼

```

#define      addr_bit_1      0x01
#define      addr_bit_2      0x02
#define      addr_bit_3      0x03
#define      addr_bit_4      0x04
#define      addr_bit_5      0x05

```

```

#define      addr_bit_6      0x06
#define      addr_bit_7      0x07
#define      addr_bit_8      0x08
#define      addr_trans_mode  0x09
#define      addr_light_con   0x0a
#define      addr_scan_count  0x0b
#define      addr_close       0x0c
#define      addr_echo_test   0x0d

```

```

sbit      ps7219_pin_RST      =   P1^4;
sbit      ps7219_pin_LOAD     =   P1^5;
sbit      ps7219_pin_DIN      =   P1^6;
sbit      ps7219_pin_CLK      =   P1^7;

```

```

void  ps7219_init();
void  ps7219_reset();
void  ps7219_echo(unsigned char da_1,da_2,da_3,da_4);
void  ps7219_send_data(unsigned char addr,da);
void  ps7219_delay(void);

```

程序二十七 2051 的 AD 代码

```

#ifndef AT89CX051_HEADER_FILE
#define AT89CX051_HEADER_FILE 1

/*-----
Byte Registers
-----*/

sfr SP      = 0x81;
sfr DPL     = 0x82;
sfr DPH     = 0x83;
sfr PCON    = 0x87;
sfr TCON    = 0x88;
sfr TMOD    = 0x89;
sfr TL0     = 0x8A;
sfr TL1     = 0x8B;
sfr TH0     = 0x8C;
sfr TH1     = 0x8D;
sfr P1      = 0x90;
sfr SCON    = 0x98;
sfr SBUF    = 0x99;

```

```

sfr IE      = 0xA8;
sfr P3      = 0xB0;
sfr IP      = 0xB8;
sfr PSW     = 0xD0;
sfr ACC     = 0xE0;
sfr B       = 0xF0;

```

```

/*-----
PCON Bit Values
-----*/

```

```

#define IDL_    0x01
#define STOP_   0x02
#define EWT_    0x04
#define EPFW_   0x08
#define WTR_    0x10
#define PFW_    0x20
#define POR_    0x40
#define SMOD_   0x80

```

```

/*-----
TCON Bit Registers
-----*/

```

```

sbit IT0  = 0x88;
sbit IE0  = 0x89;
sbit IT1  = 0x8A;
sbit IE1  = 0x8B;
sbit TR0  = 0x8C;
sbit TF0  = 0x8D;
sbit TR1  = 0x8E;
sbit TF1  = 0x8F;

```

```

/*-----
TMOD Bit Values
-----*/

```

```

#define T0_M0_  0x01
#define T0_M1_  0x02
#define T0_CT_  0x04
#define T0_GATE_ 0x08
#define T1_M0_  0x10
#define T1_M1_  0x20
#define T1_CT_  0x40
#define T1_GATE_ 0x80

```

```

#define T1_MASK_ 0xF0
#define T0_MASK_ 0x0F

```

```

/*-----
P1 Bit Registers
-----*/

```

```

sbit P1_0 = 0x90;
sbit P1_1 = 0x91;
sbit P1_2 = 0x92;
sbit P1_3 = 0x93;
sbit P1_4 = 0x94;
sbit P1_5 = 0x95;
sbit P1_6 = 0x96;
sbit P1_7 = 0x97;

```

```

sbit AIN0 = 0x90;      /* + 输入 */
sbit AIN1 = 0x91;      /* - 输入 */

```

```

/*-----
SCON Bit Registers
-----*/

```

```

sbit RI    = 0x98;
sbit TI    = 0x99;
sbit RB8   = 0x9A;
sbit TB8   = 0x9B;
sbit REN   = 0x9C;
sbit SM2   = 0x9D;
sbit SM1   = 0x9E;
sbit SM0   = 0x9F;

```

```

/*-----
IE Bit Registers
-----*/

```

```

sbit EX0   = 0xA8;      /* 1=Enable External interrupt 0 */
sbit ET0   = 0xA9;      /* 1=Enable Timer 0 interrupt */
sbit EX1   = 0xAA;      /* 1=Enable External interrupt 1 */
sbit ET1   = 0xAB;      /* 1=Enable Timer 1 interrupt */
sbit ES    = 0xAC;      /* 1=Enable Serial port interrupt */
sbit ET2   = 0xAD;      /* 1=Enable Timer 2 interrupt */

```

```

sbit EA    = 0xAF;      /* 0=Disable all interrupts */

```

```

/*-----
P3 Bit Registers (Mnemonics & Ports)
-----*/

```

```

sbit P3_0 = 0xB0;
sbit P3_1 = 0xB1;

```

```

sbit P3_2 = 0xB2;
sbit P3_3 = 0xB3;
sbit P3_4 = 0xB4;
sbit P3_5 = 0xB5;
/* P3_6 Hardwired as AOUT */
sbit P3_7 = 0xB7;

sbit RXD  = 0xB0;      /* Serial data input */
sbit TXD  = 0xB1;      /* Serial data output */
sbit INT0 = 0xB2;      /* External interrupt 0 */
sbit INT1 = 0xB3;      /* External interrupt 1 */
sbit T0   = 0xB4;      /* Timer 0 external input */
sbit T1   = 0xB5;      /* Timer 1 external input */
sbit AOUT = 0xB6;      /* Analog comparator output */

```

```

/*-----

```

IP Bit Registers

```

-----*/

```

```

sbit PX0 = 0xB8;
sbit PT0 = 0xB9;
sbit PX1 = 0xBA;
sbit PT1 = 0xBB;
sbit PS  = 0xBC;

```

```

/*-----

```

PSW Bit Registers

```

-----*/

```

```

sbit P    = 0xD0;
sbit FL   = 0xD1;
sbit OV   = 0xD2;
sbit RS0  = 0xD3;
sbit RS1  = 0xD4;
sbit F0   = 0xD5;
sbit AC   = 0xD6;
sbit CY   = 0xD7;

```

```

/*-----

```

Interrupt Vectors:

Interrupt Address = (Number * 8) + 3

```

-----*/

```

```

#define IE0_VECTOR 0 /* 0x03 External interrupt 0 */
#define TF0_VECTOR 1 /* 0x0B Timer 0 */
#define IE1_VECTOR 2 /* 0x13 External interrupt 1 */
#define TF1_VECTOR 3 /* 0x1B Timer 1 */
#define SIO_VECTOR 4 /* 0x23 Serial port */

```

```

/*-----
-----*/
#endif

/* io 分配: */
/* OUTPUT: */
/* P1.0 ..... 模拟量输入 */
/* P1.1 ..... DA 输入比较基准电压 */
/* P1.2~7..... R-2R DA 电阻网络 */
/* P3.7 ..... LED 模拟亮度输出 */
/* CPU CLOCK EQU 6M */

//#pragma src
#include "AT89x051.h"
#include <stdlib.h>
#include<math.h>
#include<intrins.h>

//变量类型标识的宏定义
#define Uchar unsigned char
#define Uint unsigned int

#define Ledlight() (P3 &= 0x7f)
#define Leddark() (P3 |= 0x80)

sbit P36 = P3^6; // 比较器内部判断脚
sbit LED = P3^7; // 一个发光二极管观察亮度变化

// 内部标志位定义
bit less; // 比较是否大于, 1.小于, 0.大于

// 全局变量定义
Uchar timer1, // 通用延时计数器
timer2, // 按键蜂鸣器反应定时器
adcddata, // ad 转换变量
pwm1; // PWM 输出比例

// 函数列表
void DelayMs(unsigned int number); // 毫秒延时
void timers0(); // 在定时器中断中做数码管的扫描显示(ct0)
void Initall(void); // 系统初始化
void timers1(void); // TCl 定时器中断用于扫描显示与键盘
Uchar adcread(void); // adc 转换程序

```



```

void main(void) using 0
{
    DelayMs(120);
    Initall();
    pwm1 = adcread();
    LED=1;
    while(1)
    {
        pwm1 = adcread();
        timer2=10;
        while (timer2);
    }
}

// 毫秒延时
void DelayMs(unsigned int number)
{
    unsigned char temp;

    for(;number!=0;number--)
        for(temp=112;temp!=0;temp--);
}

/*****
    在定时器中断中做 LED 的 PWM 输出
*****/
void timers0() interrupt 1 using 1
{
    TH0 = 0xff;
    TL0 = 0xd0;
    timer1--;
    if (timer1==pwm1)LED=0;
    if (timer1==0){
        LED=1;
        timer1=0x40;
        timer2--;
    };
}

/*****
; * 6 位 ADC 转换
; *****/
Uchar adcread(void)
{
    Uchar i=0x3f,temp=0;

```

```

    P36=1;
    P1 = 3; _nop();_nop();          // 从零开始
    while ((i--)&& (P36))
    {
        temp += 4;
        P1 = temp/3;
        _nop();
    }
    temp >>= 2;
    return temp;
}

/*****
;* 系统初始化
;*****/
void Initall(void)
{
    TMOD = 0x11;          // 0001 0001 16 进制计数器
    IP = 0x8;             // 0000 1000 t1 优先
    IE = 0x8A;            // 1000 1010 t0,t1 中断允许
    TCON = 5;             // 0000 0101 外部中断低电平触发
    TR0 = 1;              // 打开定时器中断, IE 中已经打开, 在明示一下
    TR1 = 0;
    ET0 = 1;
    ET1 = 0;
    P1 = 0xff;
}

/*****
TC1 定时器中断用于扫描显示与键盘(ct1)
;*****/
void timers1(void) interrupt 3 using 2
{
    _nop();              // 实验中没有启用
}

```

程序二十八 ARV19264 型液晶显示字库

```

/*****
/* 定义中文字库 */
;*****/

```

```

unsigned char const Hzk[]={
/* C3515 0 */
    0x04,0x04,0xC4,0x44,0x5F,0x44,0x44,0xF4,
    0x44,0x4F,0x54,0x64,0x44,0x46,0x04,0x00,
    0x80,0x40,0x3F,0x00,0x40,0x40,0x20,0x20,
    0x13,0x0C,0x18,0x24,0x43,0x80,0xE0,0x00,
/* C4843 1 */
    0x00,0xFE,0x4A,0x4A,0x00,0xFE,0xEA,0xAA,
    0xAA,0xFE,0x00,0x4A,0x4A,0xFE,0x00,0x00,
    0x02,0x83,0x42,0x22,0x12,0x1B,0x02,0x02,
    0x02,0x0B,0x12,0x22,0x62,0xC3,0x02,0x00,
/* C2590 2 */
    0x00,0xFE,0x02,0xD2,0x52,0x52,0xD2,0x3E,
    0xD2,0x16,0x1A,0x12,0xFF,0x02,0x00,0x00,
    0x00,0xFF,0x50,0x53,0x52,0x4A,0x6B,0x50,
    0x4F,0x54,0x7B,0x40,0xFF,0x00,0x00,0x00,
/* C2842 3 */
    0x00,0xFE,0x22,0xD2,0x0E,0x20,0xB8,0x4F,
    0xB2,0x9E,0x80,0x9F,0x72,0x8A,0x06,0x00,
    0x00,0xFF,0x04,0x08,0x07,0x21,0x12,0x0A,
    0x46,0x82,0x7E,0x06,0x0A,0x12,0x31,0x00,
/* C0308 4 */
    0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
    0x00,0xC0,0x30,0x08,0x04,0x02,0x00,0x00,
    0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
    0x00,0x03,0x0C,0x10,0x20,0x40,0x00,0x00,
/* C2567 5 */
    0x00,0x00,0xFC,0x44,0x54,0x54,0x7C,0x55,
    0xD6,0x54,0x7C,0x54,0x54,0x44,0x44,0x00,
    0x80,0x60,0x1F,0x80,0x9F,0x55,0x35,0x15,
    0x1F,0x15,0x15,0x35,0x5F,0x80,0x00,0x00,
/* C2211 6 */
    0x00,0x08,0xE8,0xA8,0xA8,0xA8,0xA8,0xFF,
    0xA8,0xA8,0xA8,0xA8,0xE8,0x0C,0x08,0x00,
    0x00,0x40,0x23,0x12,0x0A,0x06,0x02,0xFF,
    0x02,0x06,0x0A,0x12,0x23,0x60,0x20,0x00,
/* C0309 7 */
    0x00,0x00,0x02,0x04,0x08,0x30,0xC0,0x00,
    0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
    0x00,0x00,0x40,0x20,0x10,0x0C,0x03,0x00,
    0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
/* C5148 8 */
    0x04,0x04,0x04,0x84,0xE4,0x3C,0x27,0x24,
    0x24,0x24,0x24,0xF4,0x24,0x06,0x04,0x00,
    0x04,0x02,0x01,0x00,0xFF,0x09,0x09,0x09,

```

```

    0x09,0x49,0x89,0x7F,0x00,0x00,0x00,0x00,
/* C4762 9 */
    0x00,0xFE,0x02,0x22,0xDA,0x06,0x00,0xFE,
    0x92,0x92,0x92,0x92,0xFF,0x02,0x00,0x00,
    0x00,0xFF,0x08,0x10,0x08,0x07,0x00,0xFF,
    0x42,0x24,0x08,0x14,0x22,0x61,0x20,0x00,
/* C2511 10 */
    0x00,0x00,0x80,0x40,0x30,0x0C,0x00,0xC0,
    0x07,0x1A,0x20,0x40,0x80,0x80,0x80,0x00,
    0x01,0x01,0x20,0x70,0x28,0x24,0x23,0x20,
    0x20,0x28,0x30,0x60,0x00,0x01,0x00,0x00,
/* C4330 11 */
    0x10,0x10,0x92,0x92,0x92,0x92,0x92,0x92,
    0xD2,0x9A,0x12,0x02,0xFF,0x02,0x00,0x00,
    0x00,0x00,0x3F,0x10,0x10,0x10,0x10,0x10,
    0x3F,0x00,0x40,0x80,0x7F,0x00,0x00,0x00,
};
/*****/
/* 定义 ASCII 字库 8 列*16 行 */
/*****/

unsigned char const Ezk[]={
/*-文字:--0x20 */
    0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
    0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
/*-文字:!--0x21 */
    0x00,0x00,0x00,0xF8,0x00,0x00,0x00,0x00,
    0x00,0x00,0x00,0x27,0x00,0x00,0x00,0x00,
/*-文字: "--0x22 */
    0x00,0x08,0x04,0x02,0x08,0x04,0x02,0x00,
    0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
/*-文字:#--0x23 */
    0x40,0x40,0xF8,0x40,0x40,0xF8,0x40,0x00,
    0x04,0x3F,0x04,0x04,0x3F,0x04,0x04,0x00,
/*-文字:$--0x24 */
    0x00,0x70,0x88,0xFC,0x08,0x08,0x30,0x00,
    0x00,0x1C,0x20,0xFF,0x21,0x22,0x1C,0x00,
/*-文字:%--0x25 */
    0xF0,0x08,0xF0,0x80,0x70,0x08,0x00,0x00,
    0x00,0x31,0x0E,0x01,0x1E,0x21,0x1E,0x00,
/*-文字:&--0x26 */
    0x00,0xF0,0x08,0x88,0x70,0x00,0x00,0x00,
    0x1E,0x21,0x23,0x24,0x18,0x16,0x20,0x00,
/*-文字:!--0x27 */
    0x20,0x18,0x00,0x00,0x00,0x00,0x00,0x00,

```

```

0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
/*-文字:(--0x28 */
0x00,0x00,0x00,0x00,0xC0,0x30,0x08,0x04,
0x00,0x00,0x00,0x00,0x03,0x0C,0x10,0x20,
/*-文字:)--0x29 */
0x04,0x08,0x30,0xC0,0x00,0x00,0x00,0x00,
0x20,0x10,0x0C,0x03,0x00,0x00,0x00,0x00,
/*-文字:*--0x2a */
0x40,0x40,0x80,0xF0,0x80,0x40,0x40,0x00,
0x02,0x02,0x01,0x0F,0x01,0x02,0x02,0x00,
/*-文字:++0x2b */
0x00,0x00,0x00,0xE0,0x00,0x00,0x00,0x00,
0x01,0x01,0x01,0x0F,0x01,0x01,0x01,0x00,
/*-文字:,:--0x2c */
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x80,0x60,0x00,0x00,0x00,0x00,0x00,0x00,
/*-文字:---0x2d */
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x01,0x01,0x01,0x01,0x01,0x01,0x01,0x00,
/*-文字:--0x2e */
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x20,0x00,0x00,0x00,0x00,0x00,0x00,
/*-文字:/--0x2f */
0x00,0x00,0x00,0x00,0x00,0xE0,0x18,0x04,
0x00,0x40,0x30,0x0C,0x03,0x00,0x00,0x00,
/*-文字:0-0x30 */
0x00,0xE0,0x10,0x08,0x08,0x10,0xE0,0x00,
0x00,0x0F,0x10,0x20,0x20,0x10,0x0F,0x00,
/*-文字:1--0x31 */
0x00,0x10,0x10,0xF8,0x00,0x00,0x00,0x00,
0x00,0x20,0x20,0x3F,0x20,0x20,0x00,0x00,
/*-文字:2--0x32 */
0x00,0x70,0x08,0x08,0x08,0x88,0x70,0x00,
0x00,0x30,0x28,0x24,0x22,0x21,0x30,0x00,
/*-文字:3--0x33 */
0x00,0x30,0x08,0x88,0x88,0x48,0x30,0x00,
0x00,0x18,0x20,0x20,0x20,0x11,0x0E,0x00,
/*-文字:4--0x34 */
0x00,0x00,0xC0,0x20,0x10,0xF8,0x00,0x00,
0x00,0x07,0x04,0x24,0x24,0x3F,0x24,0x00,
/*-文字:5--0x35 */
0x00,0xF8,0x08,0x88,0x88,0x08,0x08,0x00,
0x00,0x19,0x21,0x20,0x20,0x11,0x0E,0x00,
/*-文字:6--0x36 */
0x00,0xE0,0x10,0x88,0x88,0x18,0x00,0x00,

```

```

0x00,0x0F,0x11,0x20,0x20,0x11,0x0E,0x00,
/*-文字:7--0x37 */
0x00,0x38,0x08,0x08,0xC8,0x38,0x08,0x00,
0x00,0x00,0x00,0x3F,0x00,0x00,0x00,0x00,
/*-文字:8--0x38 */
0x00,0x70,0x88,0x08,0x08,0x88,0x70,0x00,
0x00,0x1C,0x22,0x21,0x21,0x22,0x1C,0x00,
/*-文字:9--0x39 */
0x00,0xE0,0x10,0x08,0x08,0x10,0xE0,0x00,
0x00,0x00,0x31,0x22,0x22,0x11,0x0F,0x00,
/*-文字::-- */
0x00,0x00,0x60,0x60,0x00,0x00,0x00,0x00,
0x00,0x00,0x18,0x18,0x00,0x00,0x00,0x00,
/*-文字:/-- */
0x00,0x00,0x00,0x80,0x00,0x00,0x00,0x00,
0x00,0x00,0x80,0x60,0x00,0x00,0x00,0x00,
/*-文字:<-- */
0x00,0x00,0x80,0x40,0x20,0x10,0x08,0x00,
0x00,0x01,0x02,0x04,0x08,0x10,0x20,0x00,
/*-文字:=-- */
0x40,0x40,0x40,0x40,0x40,0x40,0x40,0x00,
0x04,0x04,0x04,0x04,0x04,0x04,0x04,0x00,
/*-文字:>-- */
0x00,0x08,0x10,0x20,0x40,0x80,0x00,0x00,
0x00,0x20,0x10,0x08,0x04,0x02,0x01,0x00,
/*-文字:~-- */
0x00,0x30,0x08,0x08,0x08,0x88,0x70,0x00,
0x00,0x00,0x00,0x26,0x01,0x00,0x00,0x00,
/*-文字:@-- */
0xC0,0x30,0xC8,0x28,0xE8,0x10,0xE0,0x00,
0x07,0x18,0x27,0x28,0x27,0x28,0x07,0x00,
/*-文字:A-- */
0x00,0x00,0xE0,0x18,0x18,0xE0,0x00,0x00,
0x30,0x0F,0x04,0x04,0x04,0x04,0x0F,0x30,
/*-文字:B-- */
0xF8,0x08,0x08,0x08,0x08,0x90,0x60,0x00,
0x3F,0x21,0x21,0x21,0x21,0x12,0x0C,0x00,
/*-文字:C-- */
0xE0,0x10,0x08,0x08,0x08,0x10,0x60,0x00,
0x0F,0x10,0x20,0x20,0x20,0x10,0x0C,0x00,
/*-文字:D-- */
0xF8,0x08,0x08,0x08,0x08,0x10,0xE0,0x00,
0x3F,0x20,0x20,0x20,0x20,0x10,0x0F,0x00,
/*-文字:E-- */
0x00,0xF8,0x08,0x08,0x08,0x08,0x08,0x00,

```

```

0x00,0x3F,0x21,0x21,0x21,0x20,0x00,
/*-文字:F--*/
0xF8,0x08,0x08,0x08,0x08,0x08,0x08,0x00,
0x3F,0x01,0x01,0x01,0x01,0x00,0x00,
/*-文字:G--*/
0xE0,0x10,0x08,0x08,0x08,0x10,0x60,0x00,
0x0F,0x10,0x20,0x20,0x21,0x11,0x3F,0x00,
/*-文字:H--*/
0x00,0xF8,0x00,0x00,0x00,0x00,0xF8,0x00,
0x00,0x3F,0x01,0x01,0x01,0x01,0x3F,0x00,
/*-文字:I--*/
0x00,0x00,0x00,0xF8,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x3F,0x00,0x00,0x00,0x00,
/*-文字:J--*/
0x00,0x00,0x00,0x00,0x00,0x00,0xF8,0x00,
0x00,0x1C,0x20,0x20,0x20,0x20,0x1F,0x00,
/*-文字:K--*/
0x00,0xF8,0x00,0x80,0x40,0x20,0x10,0x08,
0x00,0x3F,0x01,0x00,0x03,0x04,0x18,0x20,
/*-文字:L--*/
0xF8,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x3F,0x20,0x20,0x20,0x20,0x20,0x20,0x00,
/*-文字:M--*/
0xF8,0xE0,0x00,0x00,0x00,0xE0,0xF8,0x00,
0x3F,0x00,0x0F,0x30,0x0F,0x00,0x3F,0x00,
/*-文字:N--*/
0x00,0xF8,0x30,0xC0,0x00,0x00,0xF8,0x00,
0x00,0x3F,0x00,0x01,0x06,0x18,0x3F,0x00,
/*-文字:O--*/
0x00,0xE0,0x10,0x08,0x08,0x10,0xE0,0x00,
0x00,0x0F,0x10,0x20,0x20,0x10,0x0F,0x00,
/*-文字:P--*/
0xF8,0x08,0x08,0x08,0x08,0x10,0xE0,0x00,
0x3F,0x02,0x02,0x02,0x02,0x01,0x00,0x00,
/*-文字:Q--*/
0x00,0xE0,0x10,0x08,0x08,0x10,0xE0,0x00,
0x00,0x0F,0x10,0x20,0x2C,0x10,0x2F,0x00,
/*-文字:R--*/
0xF8,0x08,0x08,0x08,0x08,0x90,0x60,0x00,
0x3F,0x01,0x01,0x01,0x07,0x18,0x20,0x00,
/*-文字:S--*/
0x60,0x90,0x88,0x08,0x08,0x10,0x20,0x00,
0x0C,0x10,0x20,0x21,0x21,0x12,0x0C,0x00,
/*-文字:T--*/
0x08,0x08,0x08,0xF8,0x08,0x08,0x08,0x00,

```

```

0x00,0x00,0x00,0x3F,0x00,0x00,0x00,0x00,
/*-文字:U-- */
0xF8,0x00,0x00,0x00,0x00,0x00,0xF8,0x00,
0x0F,0x10,0x20,0x20,0x20,0x10,0x0F,0x00,
/*-文字:V-- */
0x18,0xE0,0x00,0x00,0x00,0xE0,0x18,0x00,
0x00,0x01,0x0E,0x30,0x0E,0x01,0x00,0x00,
/*-文字:W-- */
0xF8,0x00,0xC0,0x38,0xC0,0x00,0xF8,0x00,
0x03,0x3C,0x03,0x00,0x03,0x3C,0x03,0x00,
/*-文字:X-- */
0x08,0x30,0xC0,0x00,0xC0,0x30,0x08,0x00,
0x20,0x18,0x06,0x01,0x06,0x18,0x20,0x00,
/*-文字:Y-- */
0x08,0x30,0xC0,0x00,0xC0,0x30,0x08,0x00,
0x00,0x00,0x00,0x3F,0x00,0x00,0x00,0x00,
/*-文字:Z-- */
0x08,0x08,0x08,0x08,0xC8,0x28,0x18,0x00,
0x30,0x2C,0x22,0x21,0x20,0x20,0x20,0x00,
/*-文字:{-- */
0x00,0x00,0x00,0x80,0x7E,0x02,0x00,0x00,
0x00,0x00,0x00,0x00,0x3F,0x20,0x00,0x00,
/*-文字:\-- */
0x00,0x08,0x70,0x80,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x01,0x0E,0x30,0xC0,0x00,
/*-文字:}-- */
0x00,0x02,0x7E,0x80,0x00,0x00,0x00,0x00,
0x00,0x20,0x3F,0x00,0x00,0x00,0x00,0x00,
/*-文字:^-- */
0x00,0x08,0x04,0x02,0x02,0x04,0x08,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
/*-文字:_-- */
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x80,0x80,0x80,0x80,0x80,0x80,0x80,0x80,
/*-文字:`-- */
0x00,0x00,0x02,0x06,0x04,0x08,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
/*-文字:a-- */
0x00,0x00,0x80,0x80,0x80,0x80,0x00,0x00,
0x00,0x19,0x24,0x24,0x24,0x14,0x3F,0x00,
/*-文字:b-- */
0x00,0xF8,0x00,0x80,0x80,0x80,0x00,0x00,
0x00,0x3F,0x11,0x20,0x20,0x20,0x1F,0x00,
/*-文字:c-- */
0x00,0x00,0x80,0x80,0x80,0x80,0x00,0x00,

```



```

0x0E,0x11,0x20,0x20,0x20,0x20,0x11,0x00,
/*-文字:d--*/
0x00,0x00,0x80,0x80,0x80,0x00,0xF8,0x00,
0x00,0x1F,0x20,0x20,0x20,0x11,0x3F,0x00,
/*-文字:e--*/
0x00,0x00,0x80,0x80,0x80,0x00,0x00,0x00,
0x0E,0x15,0x24,0x24,0x24,0x25,0x16,0x00,
/*-文字:f--*/
0x00,0x80,0x80,0xF0,0x88,0x88,0x88,0x00,
0x00,0x00,0x00,0x3F,0x00,0x00,0x00,0x00,
/*-文字:g--*/
0x00,0x00,0x80,0x80,0x80,0x80,0x80,0x00,
0x40,0xB7,0xA8,0xA8,0xA8,0xA7,0x40,0x00,
/*-文字:h--*/
0x00,0xF8,0x00,0x80,0x80,0x80,0x00,0x00,
0x00,0x3F,0x01,0x00,0x00,0x00,0x3F,0x00,
/*-文字:i--*/
0x00,0x00,0x00,0x98,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x3F,0x00,0x00,0x00,0x00,
/*-文字:j--*/
0x00,0x00,0x00,0x00,0x98,0x00,0x00,0x00,
0x00,0x80,0x80,0x80,0x7F,0x00,0x00,0x00,
/*-文字:k--*/
0x00,0xF8,0x00,0x00,0x00,0x80,0x00,0x00,
0x00,0x3F,0x04,0x02,0x0D,0x10,0x20,0x00,
/*-文字:l--*/
0x00,0x00,0x00,0xF8,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x3F,0x00,0x00,0x00,0x00,
/*-文字:m--*/
0x80,0x80,0x80,0x80,0x80,0x80,0x00,0x00,
0x3F,0x00,0x00,0x3F,0x00,0x00,0x3F,0x00,
/*-文字:n--*/
0x00,0x80,0x00,0x80,0x80,0x80,0x00,0x00,
0x00,0x3F,0x01,0x00,0x00,0x00,0x3F,0x00,
/*-文字:o--*/
0x00,0x00,0x80,0x80,0x80,0x00,0x00,0x00,
0x0E,0x11,0x20,0x20,0x20,0x11,0x0E,0x00,
/*-文字:p--*/
0x00,0x80,0x00,0x80,0x80,0x80,0x00,0x00,
0x00,0xFF,0x11,0x20,0x20,0x20,0x1F,0x00,
/*-文字:q--*/
0x00,0x00,0x80,0x80,0x80,0x00,0x80,0x00,
0x00,0x1F,0x20,0x20,0x20,0x11,0xFF,0x00,
/*-文字:r--*/
0x00,0x00,0x80,0x00,0x00,0x80,0x80,0x00,

```

```

0x00,0x00,0x3F,0x01,0x01,0x00,0x00,0x00,
/*-文字:s-- */
0x00,0x00,0x80,0x80,0x80,0x80,0x00,0x00,
0x00,0x13,0x24,0x24,0x24,0x24,0x19,0x00,
/*-文字:t-- */
0x00,0x80,0x80,0xE0,0x80,0x80,0x80,0x00,
0x00,0x00,0x00,0x1F,0x20,0x20,0x20,0x00,
/*-文字:u-- */
0x00,0x80,0x00,0x00,0x00,0x00,0x80,0x00,
0x00,0x1F,0x20,0x20,0x20,0x10,0x3F,0x00,
/*-文字:v-- */
0x80,0x00,0x00,0x00,0x00,0x00,0x80,0x00,
0x00,0x07,0x18,0x20,0x18,0x07,0x00,0x00,
/*-文字:w-- */
0x80,0x00,0x00,0x80,0x00,0x00,0x80,0x00,
0x0F,0x30,0x0E,0x01,0x0E,0x30,0x0F,0x00,
/*-文字:x-- */
0x80,0x00,0x00,0x00,0x00,0x00,0x80,0x00,
0x20,0x11,0x0A,0x04,0x0A,0x11,0x20,0x00,
/*-文字:y-- */
0x80,0x00,0x00,0x00,0x00,0x00,0x80,0x00,
0x00,0x87,0x98,0x60,0x18,0x07,0x00,0x00,
/*-文字:z-- */
0x00,0x80,0x80,0x80,0x80,0x80,0x80,0x00,
0x00,0x30,0x28,0x24,0x22,0x21,0x20,0x00,
/*-文字:{-- */
0x00,0x00,0x00,0x80,0x7E,0x02,0x00,0x00,
0x00,0x00,0x00,0x00,0x3F,0x20,0x00,0x00,
/*-文字:|-- */
0x00,0x00,0x00,0xFF,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0xFF,0x00,0x00,0x00,0x00,
/*-文字:}-- */
0x00,0x02,0x7E,0x80,0x00,0x00,0x00,0x00,
0x00,0x20,0x3F,0x00,0x00,0x00,0x00,0x00,
/*-文字:~-- */
0x00,0x06,0x01,0x01,0x06,0x04,0x03,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
};

```

程序二十九 液晶 CKW19264A 型接口程序（模拟方式）

```

,*****
;连线方式:
;*LCM----S8515*      *LCM----S8515*      *LCM----S8515*      *LCM----S8515*      *
;*DB0----PA0*        *DB4----PA4*        *D/I----PC6*        *CS1----PC4*        *
;*DB1----PA1*        *DB5----PA5*        *R/W----PC7*        *CS2----PC5*        *
;*DB2----PA2*        *DB6----PA6*        *RST----VCC*        *CS3----PD2*        *
;*DB3----PA3*        *DB7----PA7*        *E-----PC3*        *
;S8515 的晶振频率为 4MHz
,*****/
#pragma interrupt_handler timer:7 /*TC1 溢出中断 */
#include <io8515.h>
#include <macros.h>
#include <worddot.h> /* 自定义字符点阵码文件, 存于 include 目录下 */
// #include <math.h> /* 数学运算定义, 没有使用 */
#define Uchar unsigned char

/*****液晶显示器接口引脚定义*****/

#define LCD_E (1 << 3) // PC3----E
#define LCD_DI (1 << 6) // PC6----D/I
#define LCD_RW (1 << 7) // PC7----R/W
#define LCD_CS1 (1 << 4) // PC4----CS1
#define LCD_CS2 (1 << 5) // PC5----CS2
#define LCD_CS3 (1 << 2) // PD2----CS3
#define lcd_set_e() (PORTC |= LCD_E) // 位置位, 输出 1
#define lcd_set_di() (PORTC |= LCD_DI)
#define lcd_set_rw() (PORTC |= LCD_RW)
#define lcd_clear_e() (PORTC &= ~LCD_E) // 位清零, 输出 0
#define lcd_clear_di() (PORTC &= ~LCD_DI)
#define lcd_clear_rw() (PORTC &= ~LCD_RW)
#define lcd_set_cs1() (PORTC |= LCD_CS1) // 片选
#define lcd_set_cs2() (PORTC |= LCD_CS2)
#define lcd_set_cs3() (PORTD |= LCD_CS3)
#define lcd_clear_cs1() (PORTC &= ~LCD_CS1)
#define lcd_clear_cs2() (PORTC &= ~LCD_CS2)
#define lcd_clear_cs3() (PORTD &= ~LCD_CS3)
#define LCD_BUSY 0x80 //LCM 忙判断位
#define lcd_read_status() (PINA &= LCD_BUSY) //LCM 忙判断
#define Datalcm PORTA //数据口

```

```

/*****常用操作命令和参数定义*****/
#define DISPON 0x3f /*显示 on */
#define DISPOFF 0x3e /*显示 off */
#define DISPFIRST 0xc0 /*显示起始行定义 */
#define SETX 0x40 /*X 定位设定指令 (页) */
#define SETY 0xb8 /*Y 定位设定指令 (列) */

/*****显示分区边界位置*****/
#define MODL 0x00 /*左区 */
#define MODM 0x40 /*左区和中区分界 */
#define MODR 0x80 /*中区和右区分界 */
#define LCMLIMIT 0xc0 /*显示区的右边界 */

/*****全局变量定义*****/
Uchar col,row,cbyte,timer1,timer2,statusm; /*列 x,行(页)y,输出数据 */
unsigned int speed=0x7fff;

/*****函数列表*****/
void Lcminit(void); /*液晶模块初始化 */
void Delay(Uchar); /*延时, 入口数为 Ms*/
void lcdbusyL(void); /*busy 判断、等待(左区) */
void lcdbusyM(void); /*busy 判断、等待(中区) */
void lcdbusyR(void); /*busy 判断、等待(右区) */
void Putedot(Uchar x,Uchar y,Uchar flash *Lib,Uchar Order,Uchar widthw);
void Wldata(Uchar); /*数据输出给 LCM */
void Lcmcls( void ); /*LCM 全屏幕清零(填充 0)*/
void wtcorn(void); /*公用 busy 等待 */
void Locatexy(void); /*光标定位 */
void WrcmdL(Uchar); /*左区命令输出 */
void WrcmdM(Uchar); /*中区命令输出 */
void WrcmdR(Uchar); /*右区命令输出 */
void Putstr(Uchar x,Uchar y,Uchar flash *puts,Uchar i); /*字符串输出*/
void Rollscreen(Uchar x); /*屏幕向上滚动演示 */
void Rddata(void); /*从液晶片上读数据 */
void point(void); /*打点 */
void Linexy(Uchar x0,Uchar y0,Uchar xt,Uchar yt);
void main_init(void);
void timer(void);
void circle(Uchar Ox,Uchar Oy,Uchar Rx);
/*****数组列表*****/
Uchar flash Ezk[]; /*ASCII 常规字符点阵码表*/
Uchar flash Hzk[]; /*自用汉字点阵码表 */
Uchar flash STR1[]; /*自定义字符串 */
Uchar flash STR2[]; //flash "=" code(keil c51)

```

```

Uchar flash STR3[];      //
Uchar flash STR4[];      //

/*****
/* 演示主程序          */
*****/

void main(void)

{
    Uchar x=0;
    DDRD  = 0xFF;          /*LCD_CS3;      /*定义输出位          */
    DDRC  = 0xFF;          /*定义为输出口          */
    statusm&=0<<7;
    main_init();
    Delay(5);              /*延时,等待外设准备好          */
    Lcminit();             /*液晶模块初始化,包括全屏幕清屏*/
    Putstr(0,0,STR3,24);    /*第一行字符输出, 24 字节          */
    Putstr(0,2,STR1,12);    /*第二行字符输出, 12 字节 (汉字) */
//    Putstr(0,4,STR3,24); /*第三行字符输出, 24 字节          */
//    Putstr(0,6,STR4,24); /*第四行字符输出, 12 字节          */

    Linexy(0,0,191,0);      /*line (0,0)-(191,0)          */
    Linexy(191,0,191,32);   /*line (191,0)-(191,32) */
    Linexy(191,32,0,32);    /*line (191,32)-(0,32) */
    Linexy(0,32,0,0);       /*line (0,32)-(0,0)          */
    Linexy(1,15,191,15);    /*line (1,15)-(191,15) */
    Linexy(0,63,44,33);     /*line (0,63)-(44,33) */
    Linexy(44,33,191,63);   /*line (44,33)-(191,63) */
    circle(46,49,12);       //画一个圆
    circle(46,49,11);

//    statusm|=1<<7;

    while(1){
//        Rollscreen(x);      /*定位新的显示起始行          */
        x++;
        Delay(20);          /*延时, 控制滚动速度          */
    }

/*****
    初始化 8515 定时寄存器
*****/

void main_init(void)

```

```

{
    TCCR1A = 0x00;
    TCCR1B = 0x00;          /* 停止定时器 1          */
    TCNT1H = 0x00;          /* 清除定时器 1          */
    TCNT1L = 0x00;
    TIMSK = 0x80;           /* 开放定时器 1 溢出中断    */
    SREG |= 0x80;
    TCCR1B = 0x01;          /* 启动定时器 1,预分频比例 1 */
}

```

```

/*****

```

在定时器中断中做多个分级定时

```

*****/

```

```

void timer()

```

```

{
    timer1--;
    if (timer2<0x80) speed+=0x100;
    else speed-=0x200;
    if (statusm&0x80){
        timer2++;
        col = (speed>>8)timer1;
        row = (timer2&0x1f)+32;
        point();}
}

```

```

/*****
/*画圆。数学方程(X-Ox)^2+(Y-Oy)^2=Rx^2          */
*****/

```

```

void circle(Uchar Ox,Uchar Oy,Uchar Rx)

```

```

{
    unsigned int xx,rr,xt,yt,rs;
    yt=Rx;
    rr=Rx*Rx+1;          //补偿 1 修正方形
    rs=(yt+(yt>>1))>>1;  //(0.75)分开 1/8 圆弧来画
    for (xt=0;xt<=rs;xt++)
    {
        xx=xt*xt;
        while ((yt*yt)>(rr-xx))yt--;
        col=Ox+xt;        //第一象限
        row=Oy-yt;
        point();
        col=Ox-xt;        //第二象限
        point();
        row=Oy+yt;        //第三象限
    }
}

```

```

        point();
        col=Ox+xt;           //第四象限
        point();

    /*******45 度镜像画另一半******/

        col=Ox+yt;           //第一象限
        row=Oy-xt;
        point();
        col=Ox-yt;           //第二象限
        point();
        row=Oy+xt;           //第三象限
        point();
        col=Ox+yt;           //第四象限
        point();
    }
}

    /********/
    /*画线。任意方向的斜线,直线数学方程  $aX+bY=1$  */
    /********/
    void Linexy(Uchar x0,Uchar y0,Uchar xt,Uchar yt)
    {
        register Uchar t;
        int xerr=0,yerr=0,delta_x,delta_y,distance;
        int incx,incy;

        delta_x=xt-x0;           /*计算坐标增量 */
        delta_y=yt-y0;
        col = x0;
        row = y0;
        if(delta_x>0) incx=1;           /*设置单步方向 */
        else if( delta_x==0 ) incx=0;           /*垂直线 */
            else {incx=-1;delta_x=-delta_x;}

        if(delta_y>0) incy=1;
        else if( delta_y==0 ) incy=0;           /*水平线 */
            else {incy=-1;delta_y=-delta_y;}

        if( delta_x > delta_y ) distance=delta_x; /*选取基本增量坐标轴*/
        else distance=delta_y;

        for( t=0;t <= distance+1; t++ ) {           /*画线输出 */
            point();           /*画点 */
        }
    }
}

```

```

        xerr += delta_x ;
        yerr += delta_y ;

        if( xerr > distance ) {
            xerr=-distance;
            col+=incx;
        }
        if( yerr > distance ) {
            yerr=-distance;
            row+=incy;
        }
    }
}

/*****
/* 画点 */
*****/

void point(void)
{
    Uchar    xl,y1,x,y;
    xl=col;
    y1=row;
    row=y1>>3;    /*取 Y 方向分页地址 */
    Rddata();
    y=y1&0x07;    /*字节内位置计算 */
    Wrcmda(cbyte1<<y); /*画上屏幕 */
    col=xl;        /*恢复 xy 坐标 */
    row=y1;
}

/*****
/* 屏幕滚动定位 */
*****/

void Rollscreen(Uchar x)
{
    cbyte =  DISPFIRSTlx; /*定义显示起始行为 x?*/
    WrcmdL(cbyte);
    WrcmdM(cbyte);
    WrcmdR(cbyte);
}

/*****
/* 一个字串的输出 */
*****/

```



```

void Putstr(Uchar x,Uchar y,Uchar flash *puts,Uchar i)
{
    Uchar j,W,wordx;
    for (j=0;j<i;j++)
    {
        wordx = puts[j];Delay(2);
        if (wordx&0x80)
        {
            Putedot(x,y,Hzk,wordx&0x7f,16); /*只保留低 7 位 */
        }
        else Putedot(x,y,Ezk,wordx-0x20,8); /*ascii 码表从 0x20 开始*/
        x=col;
        y=row;
    }
}

/*****
/* 字符点阵码数据输出 */
*****/
void Putedot(Uchar x,Uchar y,Uchar flash *Lib,Uchar Order,Uchar widthw)
{
    Uchar i;
    int xi; /*偏移量, 字符量少的可以定义为 Uchar */
    col = x; /*暂存 x,y 坐标, 已备下半个字符使用 */
    row = y;
    xi=Order * widthw<<1; /*每个字符 widthw 列*/
    /*****上半个字符输出*****/
    for(i=0;i<widthw;i++)
    {
        cbyte = Lib[xi]; /*取点阵码, rom 数组 */
        Wrddata(cbyte); /*写输出一字节 */
        xi++;
        col++;
        if (col==LCMLIMIT){col=0;row+=2;}; /*下一列, 如果列越界换行*/
        if (row>7) row=0; /*如果行越界, 返回首行 */
    }
    /*上半个字符输出结束 */

    col = x; /*列对齐 */
    row = y+1; /*指向下半个字符行 */
    /*****下半个字符输出*****/
    for(i=0;i<widthw;i++)
    {
        cbyte = Lib[xi]; /*取点阵码 */
        Wrddata(cbyte); /*写输出一字节 */
        xi++;
    }
}

```

```

        col++;
        if (col==LCMLIMIT){col=0;row+=2;};      /*下一列，如果列越界换行*/
        if (row>7) row=1;                        /*如果行越界，返回首行 */
    }                                              /*下半个字符输出结束 */
    row=y;
}                                                  /*整个字符输出结束 */

/*****
/*  清屏，全屏幕清零          */
*****/
void Lcmcls( void )
{
    for(row=0;row<8;row++)
        for(col=0;col<LCMLIMIT;col++) Wrddata(0);
}

/*****
/*  从液晶片上读数据，保留在全局变量中      */
*****/

void Rddata(void)
{
    Locatexy();      /*坐标定位，返回时保留分区状态不变 */
    DDRA = 0;        /*改变 PA 口的状态，作为输入口 */
    Datacm=0xFF;
    lcd_set_di();    /*数据 */
    lcd_set_rw();    /*读数据 */
    lcd_set_e();
    NOP();           /*读入到 LCM */
    cbyte = PINA;    /*虚读一次 */
    lcd_clear_e();
    Locatexy();      /*坐标定位，返回时保留分区状态不变 */
    DDRA = 0;
    Datacm=0xFF;
    lcd_set_di();    /*数据 */
    lcd_set_rw();    /*读数据 */
    lcd_set_e();
    NOP();           /*读入到 LCM */
    cbyte = PINA;    /*从数据口读数据，真读 */
    lcd_clear_e();NOP();
    DDRA = 0xFF; /*改变 PA 口的状态，作为输出口 */
}

/*****
/*  数据写输出          */
*****/

```

```
/******
```

```
void WrdData(Uchar X)
```

```
{
    Locatexy();    /*坐标定位, 保留分区状态不变*/
    wtrcom();
    lcd_set_di();    /*数据输出*/
    lcd_clear_rw();
    NOP();          /*写输出 */
    DataLcm = X;    /*数据输出到数据口 */
    lcd_set_e();    /*LCM 读入*/
    NOP();
    lcd_clear_e();
}
```

```
/******
```

```
/* 命令输出到左区控制口 */
```

```
/******
```

```
void WrcmdL(Uchar X)
```

```
{
    lcdbusyL();    /*确定分区, 返回时保留分区状态不变*/
    lcd_clear_di();    /*命令操作 */
    lcd_clear_rw();NOP();    /*写输出 */
    DataLcm = X;    /*数据写到数据口 */
    lcd_set_e();NOP();lcd_clear_e(); /*LCM 读入*/
}
```

```
/******
```

```
/* 命令输出到中区控制口 */
```

```
/******
```

```
void WrcmdM(Uchar X)
```

```
{
    lcdbusyM();    /*确定分区, 保留分区状态不变*/
    lcd_clear_di();    /*命令操作 */
    lcd_clear_rw();NOP();    /*写输出 */
    DataLcm = X;    /*命令写到数据口*/
    lcd_set_e();NOP();lcd_clear_e(); /*LCM 读入*/
}
```

```
/******
```

```
/* 命令输出到右区控制口 */
```

```
/******
```

```

void WrcmdR(Uchar X)
{
    lcdbusyR();           /*确定分区, 保留分区状态不变 */
    lcd_clear_di();       /*命令操作      */
    lcd_clear_rw();NOP(); /*写输出      */
    Datalcm = X;          /*命令输出到数据口 */
    lcd_set_e();NOP();lcd_clear_e(); /*读入到 LCM */
}

```

*****/

/* 分区操作允许等待,返回时保留分区选择状态 */

*****/

```

void lcdbusyL(void)
{
    lcd_clear_cs1(); /*CLR   CS1      */
    lcd_set_cs2();   /*SETB  CS2      */
    lcd_set_cs3();   /*SETB  CS3      */
    wtcom();         /* waiting for  enable */
}

```

```

void lcdbusyM(void)
{
    lcd_set_cs1();   /*SETB  CS1      */
    lcd_clear_cs2(); /*CLR   CS2      */
    lcd_set_cs3();   /*SETB  CS3      */
    wtcom();         /* waiting for  enable */
}

```

```

void lcdbusyR(void)
{
    lcd_set_cs1();   /*SETB  CS1      */
    lcd_set_cs2();   /*SETB  CS2      */
    lcd_clear_cs3(); /*CLR   CS3      */
    wtcom();         /* waiting for  enable */
}

```

```

void wtcom(void)
{
    DDRA = 0;
    lcd_clear_di(); /*CLR   DI      */
    lcd_set_rw();   /*SETB  RW      */
    Datalcm = 0xFF; /*MOV   DATA_LCM,0FFH */
    lcd_set_e();NOP();
    while(lcd_read_status());
    lcd_clear_e();
}

```

```

        DDRA=0xFF;
    }

    /*******
    /*根据设定的坐标数据, 定位 LCM 上的下一个操作单元位置 */
    /*******
void Locatexy(void)
{
    Uchar x,y;
    switch (col&0xc0)          /* col.and.0xc0 */
    {
        /*条件分支执行 */
        case 0: {lcdbusyL();break;} /*左区 */
        case 0x40: {lcdbusyM();break;} /*中区 */
        case 0x80: {lcdbusyR();break;} /*右区 */
    }
    x = col&0x3f;SETX;          /* col.and.0x3f.or.setx */
    y = row&0x07;SETY;          /* row.and.0x07.or.sety */
    wcom(); /* waiting for enable */
    lcd_clear_di(); /*CLR DI */
    lcd_clear_rw(); /*CLR RW */
    NOP();
    DataLcm = y; /*MOV P0,Y */
    lcd_set_e();NOP();lcd_clear_e();
    wcom(); /* waiting for enable */
    lcd_clear_di(); /*CLR DI */
    lcd_clear_rw(); /*CLR RW */
    NOP();
    DataLcm = x; /*MOV P0,X */
    lcd_set_e();NOP();lcd_clear_e();
}

    /*******
    /*液晶屏初始化 */
    /*******

void Lcminit(void)
{
    cbyte = DISPOFF; /*关闭显示屏 */
    WrcmdL(cbyte);
    WrcmdM(cbyte);
    WrcmdR(cbyte);
    cbyte = DISPON; /*打开显示屏 */
    WrcmdL(cbyte);
    WrcmdM(cbyte);
    WrcmdR(cbyte);

```

```

        cbyte =  DISPFIRST;    /*定义显示起始行为零    */
        WrcmdL(cbyte);
        WrcmdM(cbyte);
        WrcmdR(cbyte);
        Lcmcls();    /*清屏    */
    }
    /***/
    /* 延时    */
    /***/
    void Delay(Uchar MS)
    {
        timer1=MS;
        while(timer1);
    }

    /***/
    //定义字符串数组    */
    /***/

    Uchar flash STR1[]=
    {
        0x80,0x81,0x82,0x83,0x84,0x85,
        0x86,0x87,0x88,0x89,0x8a,0x8B
    };

    Uchar flash STR2[]="Our friend over the wold";
    Uchar flash STR3[]="Program by ICCAVR V6.21B";
    Uchar flash STR4[]="Thank you 1234567890";

```

程序三十 I²C 总线驱动程序

说明:

用两个普通 I/O 模拟 I²C 总线:

包括 100kHz(T=10μs)的标准模式(慢速模式)选择;

和 400kHz(T=2.5μs)的快速模式选择;

默认 11.0592MHz 的晶振;

```

10 _____*/
11
12 #ifndef SDA
13 #define SDA P0_0
14 #define SCL P0_1
15 #endif

```

```

16
17 extern uchar SystemError;
18
19 #define uchar unsigned char
20 #define uint unsigned int
21 #define Byte unsigned char
22 #define Word unsigned int
23 #define bool bit
24 #define true 1
25 #define false 0
26
27 #define SomeNOP(); _nop_();_nop_();_nop_();_nop_();
28
29 /**-----
30 调用方式: void I2CStart(void) @ 2001/07/0 4
31 函数说明: 私有函数, I2C专用
32 -----*/
33 void I2CStart(void)
34 {
35 EA=0;
36 SDA=1; SCL=1; SomeNOP();//INI
37 SDA=0; SomeNOP();//START
38 SCL=0;
39 }
40
41 /**-----
42 调用方式: void I2CStop(void) @ 2001/07/0 4
43 函数说明: 私有函数, I2C专用
44 -----*/
45 void I2CStop(void)
46 {
47 SCL=0; SDA=0; SomeNOP();//INI
48 SCL=1; SomeNOP(); SDA=1; //STOP
49 EA=1;
50 }
51
52 /**-----
53 调用方式: bit I2CAck(void) @ 2001/07/0 4
54 函数说明: 私有函数, I2C专用, 等待从器件接收方的应答
55 -----*/
56 bool WaitAck(void)
57 {
58 uchar errtime=255;//因故障接收方无ACK, 超时值为255。
59 SDA=1;SomeNOP();
60 SCL=1;SomeNOP();

```

```

61 while(SDA) {errtime--; if (!errtime) {I2CStop();SystemError=0x11;return false;}}
62 SCL=0;
63 return true;
64 }
65
66 /**-----
67 调用方式: void SendAck(void) @ 2001/07/0 4
68 函数说明: 私有函数, I2C专用, 主器件为接收方, 从器件为发送方时, 应答信号。
69 -----*/
70 void SendAck(void)
71 {
72 SDA=0; SomeNOP();
73 SCL=1; SomeNOP();
74 SCL=0;
75 }
76
77 /**-----
78 调用方式: void SendAck(void) @ 2001/07/0 4
79 函数说明: 私有函数, I2C专用, 主器件为接收方, 从器件为发送方时, 非应答信号。
80 **-----
81 void SendNotAck(void)
82 {
83 SDA=1; SomeNOP();
84 SCL=1; SomeNOP();
85 SCL=0;
86 }
87
88 /**-----
89 调用方式: void I2CSend(uchar ch) @ 2001/07/0 5
90 函数说明: 私有函数, I2C专用
91 -----*/
92 void I2CSendByte(Byte ch)
93 {
94 uchar i=8;
95 while (i--)
96 {
97 SCL=0;_nop_();
98 SDA=(bit)(ch&0x80); ch<<=1; SomeNOP();
99 SCL=1; SomeNOP();
100 }
101 SCL=0;
102 }
103
104
105 调用方式: uchar I2CReceive(void) @ 2001/07/0 5

```



```

106 函数说明：私有函数，I2C专用
107
108 Byte I2CReceiveByte(void)
109 {
110     uchar i=8;
111     Byte ddata=0;
112     SDA=1;
113     while (i--)
114     {
115         ddata<<=1;
116         SCL=0;SomeNOP();
117         SCL=1;SomeNOP();
118         ddata|=SDA;
119     }
120     SCL=0;
121     return ddata;
122 }
123
124
125 //-----
126 //开始PCF8563T驱动程序
127 /**-----
128 调用方式：void GetPCF8563(uchar firsttype,uchar count,uchar *buff)
129 函数说明：读取时钟芯片PCF8563的时间，设置要读的第一个时间类型firsttype，并设置读取
130 的字节数，则会一次把时间读取到buff中。顺序是：
131 0x02:秒/0x03:分/0x04:小时/0x05:日/0x06:星期/0x07:月(世纪)/0x08:年
132 -----*/
133 void GetPCF8563(uchar firsttype,uchar count,uchar *buff)
134 {
135     uchar i;
136     I2CStart();
137     I2CSendByte(0xA2);
138     WaitAck();
139     I2CSendByte(firsttype);
140     WaitAck();
141
142     I2CStart();
143     I2CSendByte(0xA3);
144     WaitAck();
145
146     for (i=0;i<count;i++)
147     {
148         buff[i]=I2CReceiveByte();
149         if (i!=count-1) SendAck();//除最后一个字节外，其他都要从MASTER发应答。
150     }

```

```

151
152 SendNotAck();
153 I2CStop();
154 }
155
156
157 /**-----
158 调用方式: void SetPCF8563(uchar timetype,uchar value) @ 2001/08/07
159 函数说明: 调整时钟。timetype是要改的时间类型, value是新设置的时间值(BCD格式)。
160 0x02:秒/0x03:分/0x04:小时/0x05:日/0x06:星期/0x07:月(世纪)/0x08:年
161 -----*/
162 void SetPCF8563(uchar timetype,uchar value)
163 {
164 I2CStart();
165 I2CSendByte(0xA2);
166 WaitAck();
167 I2CSendByte(timetype);
168 WaitAck();
169 I2CSendByte(value);
170 WaitAck();
171 I2CStop();
172 }
173
174 /**-----
175 调用方式: void SetAlarmHour(uchar count) @ 2001/08/07
176 函数说明: 设置报警闹钟在一天的第count点报警。例如: count=23, 则在晚上11点报警。
177 -----*/
178 void SetAlarm(uchar alarmtype,uchar count)
179 {
180 SetPCF8563(0x01,0x02);
181 SetPCF8563(alarmtype,count);
182 }
183
184 /**-----
185 调用方式: void CleanAlarm(void) @ 2001/08/07
186 函数说明: 清除所有报警设置。
187
188 void CleanAlarm(void)
189 {
190 SetPCF8563(0x01,0x00);
191 SetPCF8563(0x09,0x80);
192 SetPCF8563(0x0A,0x80);
193 SetPCF8563(0x0B,0x80);
194 SetPCF8563(0x0C,0x80);
195 // SetPCF8563(0x0D,0x00);

```

```

196 // SetPCF8563(0x0E,0x03);
197 }
198
199
200
201 调用方式: uchar read1380(uchar command)
202 函数说明: read1380()返回当前时间,command指要返回的时间类型。
203 秒:81H 分钟:83H 小时:85H 日期:87H 星期:89H 星期几:8BH 年:8DH
204 205 uchar read1380 (uchar command)
206 {
207     uchar time;
208     GetPCF8563(command,1,&time);
209     return time;
210 }
211
212
213 调用方式: void write1380(uchar command ,uchar time)
214 函数说明: write1380()往HT1380写命令和数据,command是命令字,time是后写入的数据
215 -----*/
216 void write1380(uchar command ,uchar time)
217 {
218     SetPCF8563(command,time);
219 }
220
221
222
223 调用方式: void time_display(uchar x0,uchar y0)
224 函数说明: time_display()在指定的x0,y0坐标,以00:00:00格式显示当前时间。
225
226 //uchar time[]="00:11:11";
227
228 void time_display(uchar x0,uchar y0,bit type) //液晶时间显示
229 {
230     uchar time[]="00:00:00";
231     uchar con[3];
232     uchar time_type;
233     GetPCF8563(0x02,3,con);
234
235     time[0]=(con[2]>>4)+'0';
236     time[1]=(con[2]&0x0f)+'0';
237     time[3]=(con[1]>>4)+'0';
238     time[4]=(con[1]&0x0f)+'0';
239     time[6]=(con[0]>>4)+'0';
240     time[7]=(con[0]&0x0f)+'0';
241

```

```

242 time[8]=0;
243 if(type==1)
244 {
245 time_type=0xff;
246 }
247 else
248 {
249 time_type=0;
250 }
251 dipchar0(x0,y0,F57,1,time_type,time);
252 }
253

```

程序三十一 240128 型液晶代码

/*写汉字液晶子程序

本例程未使用 T6963 的文本模式，使用程序填入字模也足够快。程序以 Youth 所提供的 51 例程移植过来，同时对有些地方做了简化处理，增加了画线画圆的例程，T6963 的画点有专用指令，所以不用读屏就可以直接画点。

```

;*****
;连线图： 液晶屏分为 8 行*15 列汉字，使用总线接口方式。 *
;*LCM----S8515*      *LCM----S8515*      *LCM----S8515*      *LCM----S8515* *
;*DB0----PA0*        *DB4----PA4*        *Rd ----/Rd*        *Cd ----PC0* *
;*DB1----PA1*        *DB5----PA5*        *Wr ----/Wr*        *CE ----PC1* *
;*DB2----PA2*        *DB6----PA6*        *RST----VCC*        *FS ----Vcc* *
;*DB3----PA3*        *DB7----PA7* *
;S8515 的晶振频率为 8MHz，尝试使用 11.0592MHz 超频，发现偶尔会丢失数据 *
;*****/

#include <io8515.h>
#include <macros.h>

#define ulong    unsigned long
#define uint     unsigned int
#define uchar    unsigned char

// ASCII 字符控制代码解释定义
#define STX      0x02
#define ETX      0x03
#define EOT      0x04
#define ENQ      0x05

```

```

#define BS 0x08
#define CR 0x0D
#define LF 0x0A
#define DLE    0x10
#define ETB    0x17
#define SPACE  0x20
#define COMMA   0x2C

```

```

#define TRUE  1
#define FALSE 0

```

```

#define HIGH  1
#define LOW   0

```

```

// T6963C 端口定义由汇编语言程序定义外部端口
extern uchar LCMDW,LCMCW;      //0xf000 数据口
                                //0xf100 命令口

```

```

// T6963C 命令定义

```

```

#define LC_CUR_POS    0x21    // 光标位置设置
#define LC_CGR_POS    0x22    // CGRAM 偏置地址设置
#define LC_ADD_POS    0x24    // 地址指针位置
#define LC_TXT_STP    0x40    // 文本区首址
#define LC_TXT_WID    0x41    // 文本区宽度
#define LC_GRH_STP    0x42    // 图形区首址
#define LC_GRH_WID    0x43    // 图形区宽度
#define LC_MOD_OR      0x80    // 显示方式: 逻辑“或”
#define LC_MOD_XOR     0x81    // 显示方式: 逻辑“异或”
#define LC_MOD_AND     0x82    // 显示方式: 逻辑“与”
#define LC_MOD_TCH     0x83    // 显示方式: 文本特征
#define LC_DIS_SW      0x90    // 显示开关: D0=1/0:光标闪烁启用/禁用;
                                // D1=1/0:光标显示启用/禁用;
                                // D2=1/0:文本显示启用/禁用;
                                // D3=1/0:图形显示启用/禁用;

#define LC_CUR_SHP     0xA0    // 光标形状选择: 0xA0-0xA7 表示光标占的行数
#define LC_AUT_WR      0xB0    // 自动写设置
#define LC_AUT_RD      0xB1    // 自动读设置
#define LC_AUT_OVR     0xB2    // 自动读/写结束
#define LC_INC_WR      0xC0    // 数据一次写, 地址加 1
#define LC_INC_RD      0xC1    // 数据一次读, 地址加 1
#define LC_DEC_WR      0xC2    // 数据一次写, 地址减 1
#define LC_DEC_RD      0xC3    // 数据一次读, 地址减 1
#define LC_NOC_WR      0xC4    // 数据一次写, 地址不变
#define LC_NOC_RD      0xC5    // 数据一次读, 地址不变
#define LC_SCN_RD      0xE0    // 屏读

```

```
#define LC_SCN_CP      0xE8      // 屏拷贝
#define LC_BIT_OP      0xF0      // 位操作:
                                // D0-D2: 定义 D0-D7 位; D3: 1 置位; 0: 清除
```

```
uchar const uPowArr[] = {0x01,0x02,0x04,0x08,0x10,0x20,0x40,0x80};
```

```
// ASCII 字模宽度及高度定义
```

```
#define ASC_CHR_WIDTH      8
#define ASC_CHR_HEIGHT    16
```

```
// ASCII 字模, 显示为 8*16
```

```
char flash ASC_MSK[96*16] = {
```

```
    0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00, /*--  --*/
    0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
    0x00,0x00,0x00,0x18,0x3C,0x3C,0x3C,0x18, /*--  !  --*/
    0x18,0x00,0x18,0x18,0x00,0x00,0x00,0x00,
    0x00,0x00,0x00,0x66,0x66,0x66,0x00,0x00, /*--  "  --*/
    0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
    0x00,0x00,0x00,0x36,0x36,0x7F,0x36,0x36, /*--  #  --*/
    0x36,0x7F,0x36,0x36,0x00,0x00,0x00,0x00,
    0x00,0x18,0x18,0x3C,0x66,0x60,0x30,0x18, /*--  $  --*/
    0x0C,0x06,0x66,0x3C,0x18,0x18,0x00,0x00,
    0x00,0x00,0x70,0xD8,0xDA,0x76,0x0C,0x18, /*--  %  --*/
    0x30,0x6E,0x5B,0x1B,0x0E,0x00,0x00,0x00,
    0x00,0x00,0x00,0x38,0x6C,0x6C,0x38,0x60, /*--  &  --*/
    0x6F,0x66,0x66,0x3B,0x00,0x00,0x00,0x00,
    0x00,0x00,0x00,0x18,0x18,0x18,0x00,0x00, /*--  '  --*/
    0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
    0x00,0x00,0x00,0x0C,0x18,0x18,0x30,0x30, /*--  (  --*/
    0x30,0x30,0x30,0x18,0x18,0x0C,0x00,0x00,
    0x00,0x00,0x00,0x30,0x18,0x18,0x0C,0x0C, /*--  )  --*/
    0x0C,0x0C,0x0C,0x18,0x18,0x30,0x00,0x00,
    0x00,0x00,0x00,0x00,0x00,0x36,0x1C,0x7F, /*--  *  --*/
    0x1C,0x36,0x00,0x00,0x00,0x00,0x00,0x00,
    0x00,0x00,0x00,0x00,0x00,0x18,0x18,0x7E, /*--  +  --*/
    0x18,0x18,0x00,0x00,0x00,0x00,0x00,0x00,
    0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00, /*--  ,  --*/
    0x00,0x00,0x1C,0x1C,0x0C,0x18,0x00,0x00,
    0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x7E, /*--  -  --*/
    0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
    0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00, /*--  .  --*/
    0x00,0x00,0x1C,0x1C,0x00,0x00,0x00,0x00,
    0x00,0x00,0x00,0x06,0x06,0x0C,0x0C,0x18, /*--  /  --*/
    0x18,0x30,0x30,0x60,0x60,0x00,0x00,0x00,
```

```

0x00,0x00,0x00,0x1E,0x33,0x37,0x37,0x33, /*-- 0 --*/
0x3B,0x3B,0x33,0x1E,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x0C,0x1C,0x7C,0x0C,0x0C, /*-- 1 --*/
0x0C,0x0C,0x0C,0x0C,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x3C,0x66,0x66,0x06,0x0C, /*-- 2 --*/
0x18,0x30,0x60,0x7E,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x3C,0x66,0x66,0x06,0x1C, /*-- 3 --*/
0x06,0x66,0x66,0x3C,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x30,0x30,0x36,0x36,0x36, /*-- 4 --*/
0x66,0x7F,0x06,0x06,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x7E,0x60,0x60,0x60,0x7C, /*-- 5 --*/
0x06,0x06,0x0C,0x78,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x1C,0x18,0x30,0x7C,0x66, /*-- 6 --*/
0x66,0x66,0x66,0x3C,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x7E,0x06,0x0C,0x0C,0x18, /*-- 7 --*/
0x18,0x30,0x30,0x30,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x3C,0x66,0x66,0x76,0x3C, /*-- 8 --*/
0x6E,0x66,0x66,0x3C,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x3C,0x66,0x66,0x66,0x66, /*-- 9 --*/
0x3E,0x0C,0x18,0x38,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x1C,0x1C,0x00, /*-- : --*/
0x00,0x00,0x1C,0x1C,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x1C,0x1C,0x00, /*-- ; --*/
0x00,0x00,0x1C,0x1C,0x0C,0x18,0x00,0x00,
0x00,0x00,0x00,0x06,0x0C,0x18,0x30,0x60, /*-- < --*/
0x30,0x18,0x0C,0x06,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x7E,0x00, /*-- = --*/
0x7E,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x60,0x30,0x18,0x0C,0x06, /*-- > --*/
0x0C,0x18,0x30,0x60,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x3C,0x66,0x66,0x0C,0x18, /*-- ? --*/
0x18,0x00,0x18,0x18,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x7E,0xC3,0xC3,0xCF,0xDB, /*-- @ --*/
0xDB,0xCF,0xC0,0x7F,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x18,0x3C,0x66,0x66,0x66, /*-- A --*/
0x7E,0x66,0x66,0x66,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x7C,0x66,0x66,0x66,0x7C, /*-- B --*/
0x66,0x66,0x66,0x7C,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x3C,0x66,0x66,0x60,0x60, /*-- C --*/
0x60,0x66,0x66,0x3C,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x78,0x6C,0x66,0x66,0x66, /*-- D --*/
0x66,0x66,0x6C,0x78,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x7E,0x60,0x60,0x60,0x7C, /*-- E --*/
0x60,0x60,0x60,0x7E,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x7E,0x60,0x60,0x60,0x7C, /*-- F --*/

```

0x60,0x60,0x60,0x60,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x3C,0x66,0x66,0x60,0x60, /*-- G --*/
 0x6E,0x66,0x66,0x3E,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x66,0x66,0x66,0x66,0x7E, /*-- H --*/
 0x66,0x66,0x66,0x66,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x3C,0x18,0x18,0x18,0x18, /*-- I --*/
 0x18,0x18,0x18,0x3C,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x06,0x06,0x06,0x06, /*-- J --*/
 0x06,0x66,0x66,0x3C,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x66,0x66,0x6C,0x6C,0x78, /*-- K --*/
 0x6C,0x6C,0x66,0x66,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x60,0x60,0x60,0x60, /*-- L --*/
 0x60,0x60,0x60,0x7E,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x63,0x63,0x77,0x6B,0x6B, /*-- M --*/
 0x6B,0x63,0x63,0x63,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x63,0x63,0x73,0x7B,0x6F, /*-- N --*/
 0x67,0x63,0x63,0x63,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x3C,0x66,0x66,0x66,0x66, /*-- O --*/
 0x66,0x66,0x66,0x3C,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x7C,0x66,0x66,0x66,0x7C, /*-- P --*/
 0x60,0x60,0x60,0x60,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x3C,0x66,0x66,0x66,0x66, /*-- Q --*/
 0x66,0x66,0x66,0x3C,0x0C,0x06,0x00,0x00,
 0x00,0x00,0x00,0x7C,0x66,0x66,0x66,0x7C, /*-- R --*/
 0x6C,0x66,0x66,0x66,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x3C,0x66,0x60,0x30,0x18, /*-- S --*/
 0x0C,0x06,0x66,0x3C,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x7E,0x18,0x18,0x18,0x18, /*-- T --*/
 0x18,0x18,0x18,0x18,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x66,0x66,0x66,0x66,0x66, /*-- U --*/
 0x66,0x66,0x66,0x3C,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x66,0x66,0x66,0x66,0x66, /*-- V --*/
 0x66,0x66,0x3C,0x18,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x63,0x63,0x63,0x6B,0x6B, /*-- W --*/
 0x6B,0x36,0x36,0x36,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x66,0x66,0x34,0x18,0x18, /*-- X --*/
 0x2C,0x66,0x66,0x66,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x66,0x66,0x66,0x66,0x3C, /*-- Y --*/
 0x18,0x18,0x18,0x18,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x7E,0x06,0x06,0x0C,0x18, /*-- Z --*/
 0x30,0x60,0x60,0x7E,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x3C,0x30,0x30,0x30,0x30, /*-- [--*/
 0x30,0x30,0x30,0x30,0x30,0x30,0x3C,0x3C,
 0x00,0x00,0x00,0x60,0x60,0x30,0x30,0x18, /*-- \ --*/
 0x18,0x0C,0x0C,0x06,0x06,0x00,0x00,0x00,

0x00,0x00,0x00,0x3C,0x0C,0x0C,0x0C,0x0C, /*--] --*/
 0x0C,0x0C,0x0C,0x0C,0x0C,0x0C,0x3C,0x3C,
 0x00,0x18,0x3C,0x66,0x00,0x00,0x00,0x00, /*-- ^ --*/
 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00, /*-- _ --*/
 0x00,0x00,0x00,0x00,0x00,0x00,0xFF,0xFF,
 0x00,0x38,0x18,0x0C,0x00,0x00,0x00,0x00, /*-- ` --*/
 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x00,0x00,0x3C,0x06,0x06, /*-- a --*/
 0x3E,0x66,0x66,0x3E,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x60,0x60,0x7C,0x66,0x66, /*-- b --*/
 0x66,0x66,0x66,0x7C,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x00,0x00,0x3C,0x66,0x60, /*-- c --*/
 0x60,0x60,0x66,0x3C,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x06,0x06,0x3E,0x66,0x66, /*-- d --*/
 0x66,0x66,0x66,0x3E,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x00,0x00,0x3C,0x66,0x66, /*-- e --*/
 0x7E,0x60,0x60,0x3C,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x1E,0x30,0x30,0x30,0x7E, /*-- f --*/
 0x30,0x30,0x30,0x30,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x00,0x00,0x3E,0x66,0x66, /*-- g --*/
 0x66,0x66,0x66,0x3E,0x06,0x06,0x7C,0x7C,
 0x00,0x00,0x00,0x60,0x60,0x7C,0x66,0x66, /*-- h --*/
 0x66,0x66,0x66,0x66,0x00,0x00,0x00,0x00,
 0x00,0x00,0x18,0x18,0x00,0x78,0x18,0x18, /*-- i --*/
 0x18,0x18,0x18,0x7E,0x00,0x00,0x00,0x00,
 0x00,0x00,0x0C,0x0C,0x00,0x3C,0x0C,0x0C, /*-- j --*/
 0x0C,0x0C,0x0C,0x0C,0x0C,0x0C,0x78,0x78,
 0x00,0x00,0x00,0x60,0x60,0x66,0x66,0x6C, /*-- k --*/
 0x78,0x6C,0x66,0x66,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x78,0x18,0x18,0x18,0x18, /*-- l --*/
 0x18,0x18,0x18,0x7E,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x00,0x00,0x7E,0x6B,0x6B, /*-- m --*/
 0x6B,0x6B,0x6B,0x63,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x00,0x00,0x7C,0x66,0x66, /*-- n --*/
 0x66,0x66,0x66,0x66,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x00,0x00,0x3C,0x66,0x66, /*-- o --*/
 0x66,0x66,0x66,0x3C,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x00,0x00,0x7C,0x66,0x66, /*-- p --*/
 0x66,0x66,0x66,0x7C,0x60,0x60,0x60,0x60,
 0x00,0x00,0x00,0x00,0x00,0x3E,0x66,0x66, /*-- q --*/
 0x66,0x66,0x66,0x3E,0x06,0x06,0x06,0x06,
 0x00,0x00,0x00,0x00,0x00,0x66,0x6E,0x70, /*-- r --*/
 0x60,0x60,0x60,0x60,0x00,0x00,0x00,0x00,
 0x00,0x00,0x00,0x00,0x00,0x3E,0x60,0x60, /*-- s --*/

```

0x3C,0x06,0x06,0x7C,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x30,0x30,0x7E,0x30,0x30, /*-- t --*/
0x30,0x30,0x30,0x1E,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x66,0x66,0x66, /*-- u --*/
0x66,0x66,0x66,0x3E,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x66,0x66,0x66, /*-- v --*/
0x66,0x66,0x3C,0x18,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x63,0x6B,0x6B, /*-- w --*/
0x6B,0x6B,0x36,0x36,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x66,0x66,0x3C, /*-- x --*/
0x18,0x3C,0x66,0x66,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x66,0x66,0x66, /*-- y --*/
0x66,0x66,0x66,0x3C,0x0C,0x18,0xF0,0xF0,
0x00,0x00,0x00,0x00,0x00,0x7E,0x06,0x0C, /*-- z --*/
0x18,0x30,0x60,0x7E,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x0C,0x18,0x18,0x18,0x30, /*-- { --*/
0x60,0x30,0x18,0x18,0x18,0x0C,0x00,0x00,
0x00,0x00,0x00,0x18,0x18,0x18,0x18,0x18, /*-- | --*/
0x18,0x18,0x18,0x18,0x18,0x18,0x18,0x18,
0x00,0x00,0x00,0x30,0x18,0x18,0x18,0x0C, /*-- } --*/
0x06,0x0C,0x18,0x18,0x18,0x30,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x71,0xDB, /*-- ~ --*/
0x8E,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00, /*-- . --*/
0x00,0x00,0x1C,0x1C,0x00,0x00,0x00,0x00
};

```

```

typedef struct typFNT_GB16          // 汉字字模数据结构

```

```

{
    signed char Index[2];
    char Msk[32];
};

```

```

struct typFNT_GB16 flash GB_16[] =

```

```

{    // 显示为 16*16
    "中",    0x01,0x00,0x01,0x00,0x21,0x08,0x3F,0xFC,
            0x21,0x08,0x21,0x08,0x21,0x08,0x21,0x08,
            0x21,0x08,0x3F,0xF8,0x21,0x08,0x01,0x00,
            0x01,0x00,0x01,0x00,0x01,0x00,0x01,0x00,
    "文",    0x02,0x00,0x01,0x00,0x01,0x00,0xFF,0xFE,
            0x08,0x20,0x08,0x20,0x08,0x20,0x04,0x40,
            0x04,0x40,0x02,0x80,0x01,0x00,0x02,0x80,
            0x04,0x60,0x18,0x1E,0xE0,0x08,0x00,0x00,
    "测",    0x40,0x02,0x27,0xC2,0x24,0x42,0x84,0x52,
            0x45,0x52,0x55,0x52,0x15,0x52,0x25,0x52,

```

```

    0x25,0x52,0x25,0x52,0xC5,0x52,0x41,0x02,
    0x42,0x82,0x42,0x42,0x44,0x4A,0x48,0x04,
    "试",    0x00,0x20,0x40,0x28,0x20,0x24,0x30,0x24,
    0x27,0xFE,0x00,0x20,0xE0,0x20,0x27,0xE0,
    0x21,0x20,0x21,0x10,0x21,0x10,0x21,0x0A,
    0x29,0xCA,0x36,0x06,0x20,0x02,0x00,0x00,
};
uchar flash turnf[8] = {7,6,5,4,3,2,1,0};
uchar gCurRow,gCurCol; // 当前行、列存储,行高 16 点,列宽 8 点

/* 取当前行数据 */
uchar fnGetRow(void)
{
    return gCurRow;
}

/* 取当前列数据 */
uchar fnGetCol(void)
{
    return gCurCol;
}

/*****
/* 状态位 STA1,STA0 判断 (读写指令和读写数据) */
/* 在读写数据或者写入命令前必须保证均为 1 */
*****/
uchar fnSTA01(void)
{
    uchar i;

    for(i=10;i>0;i--)
    {
        if((LCMCW & 0x03) == 0x03) // 读取状态
            break;
    }
    return i; // 若返回零,说明错误
}

/*****
/*检查 STA2,如果 STA2=1 为自动读状态 */
*****/
uchar fnSTA2(void)
{
    uchar i;

```

```

        for(i=10;i>0;i--)
        {
            if((LCMCW & 0x04) == 0x04)
                break;
        }
        return i;                // 若返回零, 说明错误
    }

    /**
    /* 状态位 STA3 判断 (STA3 = 1 数据自动写状态) */
    /**
    uchar fnSTA3(void)
    {
        uchar i;

        for(i=10;i>0;i--)
        {
            if((LCMCW & 0x08) == 0x08)
                break;
        }
        return i;                // 若返回零, 说明错误
    }

    /**
    /* 状态位 STA6 判断 (STA6 = 1 屏读/屏拷贝状态) */
    /**
    uchar fnSTA6(void)
    {
        uchar i;

        for(i=10;i>0;i--)
        {
            if((LCMCW & 0x40) == 0x40)
                break;
        }
        return i;                // 若返回零, 说明错误
    }

    /**
    /* 写双参数的指令 */
    /**
    uchar fnPR1(uchar uCmd,uchar uPar1,uchar uPar2)
    {
        if(fnSTA01() == 0)
            return 1;

```

```

    LCMDW = uPar1;
    if(fnSTA01() == 0)
        return 2;
    LCMDW = uPar2;
    if(fnSTA01() == 0)
        return 3;
    LCMCW = uCmd;
    return 0;                // 返回 0 成功
}

/*****
/* 写单参数的指令 */
*****/
uchar fnPR11(uchar uCmd,uchar uPar1)
{
    if(fnSTA01() == 0)
        return 1;
    LCMDW = uPar1;
    if(fnSTA01() == 0)
        return 2;
    LCMCW = uCmd;
    return 0;                // 返回 0 成功
}

/*****
/* 写无参数的指令 */
*****/
uchar fnPR12(uchar uCmd)
{
    if(fnSTA01() == 0)
        return 1;
    LCMCW = uCmd;
    return 0;                // 返回 0 成功
}

/*****
/* 写数据 */
*****/
uchar fnPR13(uchar uData)
{
    if(fnSTA3() == 0)
        return 1;
    LCMDW = uData;
    return 0;                // 返回 0 成功
}

```

```

/*****
/* 读数据 */
*****/
uchar fnPR2(void)
{
    if(fnSTA01() == 0)return 1;        // 获取状态, 如果状态错
    return LCMDW;                     // 返回数据
}

/*****
/* 设置当前地址 */
*****/
void fnSetPos(uchar urow, uchar ucol)
{
    uint iPos;

    iPos = urow * 30 + ucol;
    fnPR1(LC_ADD_POS,iPos & 0xFF,iPos / 256);
    gCurRow = urow;
    gCurCol = ucol;
}

/*****
/* 设置当前显示行、列 */
*****/
void cursor(uchar uRow, uchar uCol)
{
    fnSetPos(uRow * 16, uCol);
}

/*****
/* 清屏 */
*****/
void cls(void)
{
    uint i;

    fnPR1(LC_ADD_POS,0x00,0x00);      // 置地址指针为从零开始
    fnPR12(LC_AUT_WR);                // 自动写
    for(i=0;i<240*128/8;i++)          // 清一屏
    {
        fnSTA3();
        fnPR13(0x0);                  // 写数据, 实际使用时请将 0x55 改成 0x0
    }
}

```

```

        fnPR12(LC_AUT_OVR);           // 自动写结束
        fnPR1(LC_ADD_POS,0x00,0x00);  // 重置地址指针
        gCurRow = 0;                 // 置地址指针存储变量
        gCurCol = 0;
    }

    /*******
    /* LCM 初始化                                     */
    /*******
char fnLCMInit(void)
{
    if(fnPR1(LC_TXT_STP,0x00,0x00) != 0)    // 文本显示区首地址
        return (0xff);
    fnPR1(LC_TXT_WID,0x1E,0x00);           // 文本显示区宽度
    fnPR1(LC_GRH_STP,0x00,0x00);           // 图形显示区首地址
    fnPR1(LC_GRH_WID,0x1E,0x00);           // 图形显示区宽度
    fnPR12(LC_CUR_SHP!0x01);               // 光标形状
    fnPR12(LC_MOD_OR);                     // 显示方式设置
    fnPR12(LC_DIS_SW!0x08);               // 显示开关设置

    return 0;
}

    /*******
    /* ASCII(8*16) 及 汉字(16*16) 显示函数                                     */
    /*******
uchar dprintf(uchar x,uchar y, char *ptr)
{
    char  c1,c2,cData;
    uchar ij,uLen,uRow,uCol;
    uint  k;
    uLen=0;
    i=0;
    uRow = y;
    uCol = x;
    fnSetPos(uRow*16,uCol);                //起点定位
    while (ptr[uLen]!=0){uLen++;};          //探测字符串长度

    while(i<uLen)
    {
        c1 = ptr[i];
        c2 = ptr[i+1];
        //ascii 字符与汉字内码的区别在于 128 做分界, 大于界线的为汉字码
        uRow = fnGetRow();

```

```

uCol = fnGetCol();
if(c1 <=128)                                     // ASCII
{
    for(j=0;j<16;j++)                             //写 16 行
    {
        fnPR12(LC_AUT_WR);                         // 写数据(命令)
        if (c1 >= 0x20)
        {
            if(j < (16-ASC_CHR_HEIGHT) )
                fnPR13(0x00);                     // 写数据 0 输出空
            else
                fnPR13( ASC_MSK[(c1-0x20)*ASC_CHR_HEIGHT+j-(16-ASC_CHR_HEIGHT)] );
        }
        else
        {
            fnPR13(cData);
            fnPR12(LC_AUT_OVR);                     //写数据结束
            fnSetPos(uRow+j+1,uCol);
        }
        if(c1 != BS)                                // 非退格
            uCol++;                                // 列数加 1
    }
else                                                // 中文
{
    for(j=0;j<sizeof(GB_16)/sizeof(GB_16[0]);j++) // 查找定位
    {
        if(c1 == GB_16[j].Index[0] && c2 == GB_16[j].Index[1])
            break;
    }
    for(k=0;k<sizeof(GB_16[0].Msk)/2;k++)
    {
        fnSetPos(uRow+k,uCol);
        fnPR12(LC_AUT_WR);                         // 写数据
        if(j < sizeof(GB_16)/sizeof(GB_16[0]))
        {
            fnPR13(GB_16[j].Msk[k*2]);
            fnPR13(GB_16[j].Msk[k*2+1]);
        }
        else                                        // 未找到该字
        {
            if(k < sizeof(GB_16[0].Msk)/4)
            {
                fnPR13(0x00);
                fnPR13(0x00);
            }
            else

```



```

        {
            fnPR13(0xff);
            fnPR13(0xff);
        }
    }
    fnPR12(LC_AUT_OVR);
}
uCol += 2;
i++;
};
if(uCol >= 30)           // 光标后移
{
    uRow += 16;
    if(uRow < 0x80)
        uCol -= 30;
    else
    {
        uRow = 0;
        uCol = 0;
    }
}
fnSetPos(uRow,uCol);
i++;
}
return uLen;             //返回字符串长度，汉字按2字节计算
}

```

```

/*=====*/
/* 延时          */
/*=====*/

```

```
void shortdelay(uint tt)
```

```

{
    uchar i;
    while (tt)
    {
        i=100;
        while (i)--;
        tt--;
    };
}

```

```

/*****/
/* 画点          */
/*****/

```

```
void point(uchar x,uchar y,uchar s)
```

```

{
uchar x1;
x1=x>>3;           //取 Y 方向分页地址
fnSetPos(y,x1);     //起点定位
    x1 = turnf[ x & 0x07 ];
x1=0xF0|x1ls;       //字节内位置计算
fnPR12(x1);         //画上屏幕, S 显示属性 8 画点 0 擦除点
}

/*****/
/*画线。任意方向的斜线,直线数学方程 aX+bY=1    */
/*****/
void Linexy(uchar x0,uchar y0,uchar xt,uchar yt,uchar s)
{
    register uchar t;
    int xerr=0,yerr=0,delta_x,delta_y,distance;
    int incx,incy,uRow,uCol;

    delta_x = xt-x0;           //计算坐标增量
    delta_y = yt-y0;
    uRow = x0;
    uCol = y0;
    if(delta_x>0) incx=1;       //设置单步方向
    else if( delta_x==0 ) incx=0; //垂直线
        else {incx=-1;delta_x=-delta_x;}

    if(delta_y>0) incy=1;
    else if( delta_y==0 ) incy=0; //水平线
        else {incy=-1;delta_y=-delta_y;}

    if( delta_x > delta_y ) distance=delta_x; //选取基本增量坐标轴
    else distance=delta_y;

    for( t=0;t <= distance+1; t++ )
    {
        point(uRow,uCol,s); //画线输出
        xerr += delta_x      //画点
        yerr += delta_y

        if( xerr > distance )
        {
            xerr-=distance;
            uRow+=incx;
        }
        if( yerr > distance )

```

```

        {
            yerr-=distance;
            uCol+=incy;
        }
    }
}

/*****
/*画圆。数学方程 $(X-O_x)^2+(Y-O_y)^2=R_x^2$  */
*****/

void circle(uchar Ox,uchar Oy,uchar Rx,uchar s)
{
    unsigned int xx,rr,xt,yt,rs,row,col;
    yt=Rx;
    rr=Rx*Rx+1;          //补偿 1 修正方形
    rs=(yt+(yt>1))>>1;   //(0.75)分开 1/8 圆弧来画
    for (xt=0;xt<=rs;xt++)
    {
        xx=xt*xt;
        while ((yt*yt)>(rr-xx))yt--;
        row=Ox+xt;        //第一象限
        col=Oy-yt;
        point(row,col,s);
        row=Ox-xt;        //第二象限
        point(row,col,s);
        col=Oy+yt;        //第三象限
        point(row,col,s);
        row=Ox+xt;        //第四象限
        point(row,col,s);

/*****45 度镜像画另一半*****/

        row=Ox+yt;        //第一象限
        col=Oy-xt;
        point(row,col,s);
        row=Ox-yt;        //第二象限
        point(row,col,s);
        col=Oy+xt;        //第三象限
        point(row,col,s);
        row=Ox+yt;        //第四象限
        point(row,col,s);
    }
}

```

```

void main(void) // 测试用
{
    uchar i;
    shortdelay(1200);
    MCUCR |=BIT(SRE)|BIT(SRW);
    fnLCMInit();
    cls();
    cursor(0,0);
    dprintf(3,3,"This is a test: 中文测试");

    dprintf(0,6,"LCM Exsample use 90S8515&6963");
    dprintf(15,7,"~Xiaoqi~");
    Linexy(10,20,239,110,8);           // 画斜线 1
    Linexy(10,20,217,1,8);             // 斜线 2
    Linexy(239,110,217,1,8);           // 斜线 3
    circle(185,45,40,8);                // 画圆
    circle(185,45,41,8);                // 画同心圆加粗
    shortdelay(24000);

    while(1)
    {
        //变化圆演示，直径不断的变化，由大到小再由小到大来回缩放
        for (i=40;i>5;i--)
        {
            circle(185,45,i+1,0);       //擦除外圆
            circle(185,45,i,8);
            circle(185,45,i-1,8);
            shortdelay(3600);
        }

        shortdelay(8000);

        for (i=5;i<40;i++)
        {
            circle(185,45,i-1,0);       //擦除内圆
            circle(185,45,i,8);
            circle(185,45,i+1,8);
            shortdelay(1800);
        }

        shortdelay(4000);
    }
}

```

程序三十二 飞机游戏

游戏运行在 51 CPU 里, 无需任何外扩的 RAM, 只要 51 CPU 的串口和计算机连起来就可以了。用的 Windows32 的终端仿真程序做接收和发送, 波特率 19200bps, 8bit, 1 停止位, 无校验, 无流控。使用 DEC VT-100[ANSI]终端仿真协议。该终端仿真程序可从 mcu51.126.com 下载。也可用 Windows 98 带的终端仿真程序, 不过一般要安装一下。

源文件用 v5.20 版的 keilC51 编译, 优化等级为 2 级。目标文件 4003 字节, 可以写在 51 芯片内。源程序中还有许多没有用的函数, 删掉后可节省空间。

软件仿真办法: 将计算机的 com1 和 com2 口连接起来 (2-3, 3-2, 5-5), 软件仿真时数据发往 com1 口 (已设置), 在 com2 口用终端仿真接收, 这样可以看到中文输出和处理终端协议。而 keilc51 的串行仿真窗口只可接发英文字符。

```

#define cpw77e58      //定义使用的 MCU
#define xtal221184    //定义使用的晶振
#include <serial.c>    //包含串行处理模块

bit showclocksign;
bit showblocksign;
bit showfensign;
uchar count;
uint second;
uchar paoplace;
uint zongfen;
uchar speed=32;

idata  uint  free[16];    //定义行首处理空间
unsigned char idata *freep; //定义指向该数组的指针
/****
gotoxy(uchar x,y)
{putbyte(0x1b);putbyte('['
);
putbyte((y%100)/10+0x30);
putbyte((y%10)+0x30);
putbyte(',');
putbyte((x%100)/10+0x30);
putbyte((x%10)+0x30);
putbyte('H');
}

/*****
clrscr()
```

```

(gotoxy(0,0); putbyte(0x1b); putstring("[J"); }

//*****
showpao()
{gotoxy(1,16); putbyte(0x1b); putstring("[K");
gotoxy(paoplace,16); putbyte('I');
gotoxy(1,17); putbyte(0x1b); putstring("[K");
gotoxy(paoplace-1,17);
putbyte('/');putbyte('H');putbyte('\\');

gotoxy(0,24);
}
//*****
uint bdata showp;
sbit outblock=showp^15;
showblock() //showblock
{uchar i,j;uint bdata p;
    uint st;

    gotoxy(0,0);
    for(i=15;i!=0;i--)
    {putbyte('I');
        showp=free[i];
        st=0x8000;
        for(j=0;j!=16;j++)
        {    if ((showp&st)!=0) putbyte('@');
            else putbyte(' ');
            st>>=1;
            //showp=showp>>1;

        }
        putbyte('I');CR;
    }
gotoxy(0,24);
showblocksign=0;
}
//*****
showfen()
{gotoxy(6,19);
//putstring("得分: ");
putint(zongfen,1);
showfensign=0;

//putbyte(' ');

```

```

//puthex(free[15]/256); puthex(free[15]%256);
//gotoxy(0,24);
}

/*****
void main(void)
{uchar idata c,i,j;bit k;
    uint temp,notype;

serial_init(); // ET0=0;
putstring("\nSerial ready!\n");
start: ET0=1;clrscr();
    putstring("        一个飞机游戏"
        "\n
        "\n        键盘控制方法:"
        "\n        操作者不动键盘 10 秒钟进入演示状态, 按'a"
        "\n        左移, 按'd'右移, 按's'发弹, 按'-'降低难度, "
        "\n        按'+'提高难度, 按'p'进入演示状态。"
        "\n        按任何键开始游戏。");
    while(!inbufsign);
    zongfen=0;showblock();showfen();notype=1000;
    gotoxy(0,19);//putstring("lever:    h-hard e-easy\n5-left 6-right 0-shot");
    putstring("得分:\n  级别:    +:高 -:低\n5:左移 6:右移 0:发射");
    gotoxy(7,20);putchar(speed,1);
    for(i=0;i!=16;i++)
    {free[i]=0;
    }
    paoplace=9;showpao();
while(1) {    if (showblocksign) showblock();
    if (showfensign) showfen();
    if (free[0]!=0)
        {ET0=0;notype=1000;free[0]=0;gotoxy(0,22);putstring("结束!再玩一遍?(y/n)");
        while(getbyte()!='y');goto start;//
        }
    notype--;if(notype==10){notype=1000;goto autoplay;}
    delay(100);
    if (notype>900) {gotoxy(0,22);putbyte(0x1b); putstring("[K");}
    if (inbufsign)
    {    c=getbyte();notype=1000;
        switch (c)
        {case '5': //left move
        case 'a':
            paoplace--; if (paoplace==1) paoplace=2;
            showpao();
            break;

```

```

        case '6': //right move
        case 'd':
            paoplace++; if (paoplace==18) paoplace=17;
            showpao();
            break;
        case '=':
            case '+':
                speed--;
            if (speed==0) speed=1;
            gotoxy(7,20);
            putchar(speed,1);
            break;
        case '_':
        case '-':
            speed++;
            if (speed==255) speed=254;
            gotoxy(7,20);
            putchar(speed,1);
            break;
        case 's':
        case '0': //shot
            for(i=15,j=1;i!=0;i--j=16-i)
            { gotoxy(paoplace,i); putbyte('o');delay(50);
              putbyte(8);putbyte(' ');
              //free[i]=0xffff;
              temp=0x0001<<(17-paoplace);
              if( (free[j]&temp)!=0){ free[j] &= (temp^0xffff);zongfen+=10;
showfensign =1;break;}

            }
            break;
            case 'y': goto start;
        case 'p':
autoplay: gotoxy(0,22);putbyte(0x1b); putstring("[K]");putstring("游戏自动演示中。");
            ET0=1;speed=20;gotoxy(7,20);putchar(speed,1);
            do
            {
                if (showblocksign) showblock();
                if (showfensign) showfen();

                for(i=15,j=1;i!=0;i--j=16-i)
                { gotoxy(paoplace,i); putbyte('o');delay(50);
                  putbyte(8);putbyte(' ');
                  temp=0x0001<<(17-paoplace);
                  if( (free[j]&temp)!=0){ free[j] &= (temp^0xffff);zongfen+ =10;
showfensign=1;goto again;}

```



```

        second++; //1S
        if (second==3600) {second=0;showclocksign=1;}
    // if (iftimeover) timeovercount++;
    // else timeovercount=0;
    //
    }
}

```

serial.c 代码

```

//串口中断服务程序，仅需做简单调用即可完成串口输入输出的处理
//出入均设有缓冲区，大小可任意设置。
//可供使用的函数名：
//char getbyte(void);从接收缓冲区取一个 byte,如不想等待则在调用前检测 inbufsign 是否为 1。
//putbyte(char c);放入一个字节到发送缓冲区
//putbytes(unsigned char *outplace,j);放一串数据到发送缓冲区，自定义长度
//putstring(unsigned char code *puts);发送一个定义在程序存储区的字符串到串口
//puthex(unsigned char c);发送一个字节的 hex 码，分成两个字节发。
//putchar(uchar c,uchar j);发送一个字节数据的 asc 码表达方式，需要定义小数点的位置
//putint(uint ui,uchar j);发送一个整型数据的 asc 码表达方式，需要定义小数点的位置
//delay(unsigned char d); 延时 n x 100ns
//getline(char idata *line, unsigned char n); 获取一行数据回车结束，必须定义最大输入字符数
//CR;发送一个回车换行
//*****

#include <w77e58.h>
#define uchar unsigned char
#define uint unsigned int
#define OLEN 10 /* size of serial transmission buffer */
idata unsigned char outbuf[OLEN]; /* storage for transmission buffer */
unsigned char idata *outlast=outbuf; //最后由中断传输出去的字节位置
unsigned char idata *putlast=outbuf; //最后放入发送缓冲区的字节位置
#define ILEN 2 /* size of serial receiving buffer */
idata unsigned char inbuf[ILEN];
unsigned char idata *inlast=inbuf; //最后由中断进入接收缓冲区的字节位置
unsigned char idata *getlast=inbuf; //最后取走的字节位置
bit outbufsign; //输出缓冲区非空标志 有=1
bit inbufsign; //接收缓冲区非空标志 有=1
bit inbufful; //输入缓冲区满标志 满=1
#define CR putstring("\r\n") //CR=回车换行
unsigned char code comready[]="com is ready!";
//*****

//延时 n x 100ns
void delay(unsigned char d) //在源程序开头定义是否用 w77e58 或 22.1184MHz 晶振
{unsigned char j;
do{ d--;

```

```

//110592 & 89c52
#ifndef cpuw77e58
    #ifndef xtal221184
        j=21;           //k=38 cpu80320 100us k=21 cpu 8052
    #else
        j=42;
    #endif
#else
    #ifndef xtal221184
        j=38;
    #else
        j=76;
    #endif
#endif

do {j--;} while(j!=0);
}while(d!=0);
}
//*****
//放入一个字节到发送缓冲区

putbyte(char c)
{uchar i,j;
  ES=0;           /*暂停串行中断，以免数据比较时出错? */
  if (outlast==putlast )
  {
      i=(0-TH1);
      do{i--j=36; do {j--;}while(j!=0); }while(i!=0);    //延时一个字节发送时间
  }
  *putlast=c;      //放字节进入缓冲区
  putlast++;       //发送缓冲区指针加一
  if (putlast==outbuf+OLEN) putlast=outbuf; //指针到了顶部换到底部
  if (!outbufsign) {outbufsign=1;TI=1; } //缓冲区开始为空置为有，启动发送
  ES=1;
}
//*****
//放一串数据到发送缓冲区
putbytes(unsigned char *outplace,unsigned char j)
{
    int i;
    for(i=0;i<j;i++)
    {putbyte(*outplace);
      outplace++;
    }
}
//*****
putchar(uchar c,uchar j)

```

```

{uchar idata free[4];uchar data i;
i=0;
free[i++]= (c/100+0x30);
if (j==3) free[i++]='.';
free[i++]=(c%100)/10+0x30;
if (j==2) free[i++]='.';
if (j==2 && free[i-3]==0x30) free[i-3]=0x20;
free[i++]=(c%10)+0x30;
if (j==1 && free[i-3]==0x30) free[i-3]=0x20;
if (j==1 && free[i-3]==0x20 && free[i-2]==0x30) free[i-2]=0x20;
putbytes(free,i);
}

//*****

putint(uint ui,uchar j)
{uchar idata free[6];
uchar data i;
i=0;
free[i++]=(ui/10000+0x30);
if (j==5) free[i++]='.';
free[i++]=(ui%10000)/1000+0x30;
if (j==4) free[i++]='.';
if (j==4 && free[i-3]==0x30) free[i-3]=0x20;
free[i++]=(ui%1000)/100+0x30;
if (j==3) free[i++]='.';
if (j==3 && free[i-4]==0x30) free[i-4]=0x20;
if (j==3 && free[i-4]==0x20 && free[i-3]==0x30) free[i-3]=0x20;
free[i++]=(ui%100)/10+0x30;
if (j==2) free[i++]='.';
if (j==2 && free[i-5]==0x30) free[i-5]=0x20;
if (j==2 && free[i-5]==0x20 && free[i-4]==0x30) free[i-4]=0x20;
if (j==2 && free[i-5]==0x20 && free[i-4]==0x20 && free[i-3]==0x30) free[i-3]=0x20;
free[i++]=(ui%10+0x30);
if (j==1 && free[i-5]==0x30) free[i-5]=0x20;
if (j==1 && free[i-5]==0x20 && free[i-4]==0x30) free[i-4]=0x20;
if (j==1 && free[i-5]==0x20 && free[i-4]==0x20 && free[i-3]==0x30) free[i-3]=0x20;
if (j==1 && free[i-5]==0x20 && free[i-4]==0x20 && free[i-3]==0x20 && free[i-2]==0x30) free[i-
2]=0x20;
putbytes(free,i);
}

//*****
//发送一个定义在程序存储区的字符串到串口
putstring(unsigned char *puts)
{for (;*puts!=0;puts++) //遇到停止符 0 结束

```

```

putbyte(*puts);
}
//*****
//发送一个字节的 hex 码, 分成两个字节发。
unsigned char code hex_[]={"0123456789ABCDEF"};
puthex(unsigned char c)
{int ch;
ch=(c>>4)&0x0f;
putbyte(hex_[ch]);
ch=c&0x0f;
putbyte(hex_[ch]);
}
//*****
//从接收缓冲区取一个 byte,如不想等待则在调用前检测 inbufsign 是否为 1。
uchar getbyte (void)
{ char idata c ;
while (!inbufsign);          //缓冲区空等待
ES=0;
c= *getlast;                 //取数据
getlast++;                   //最后取走的数据位置加一
inbufful=0;                  //输入缓冲区的满标志清零
if (getlast==inbuf+ILEN) getlast=inbuf; //地址到顶部回到底部
if (getlast==inlast) inbufsign=0;      //地址相等置接收缓冲区空空标志, 再取数前要检该标志
ES=1;
return (c);                  //取回数据
}
//*****
//接收一行数据, 必须定义放数据串的指针位置和大小    del=0x7f,backspace=0x08,cr=0x0d,lf=0x0a
void getline (uchar idata *line, unsigned char n)
{ unsigned char cnt = 0; //定义已接收的长度
char c;
do {
if ((c = getbyte ()) == 0x0d) c = 0x00;          //读一个字节, 如果是回车换成结束符
if (c == 0x08 || c == 0x7f)                      //BACKSPACE 和 DEL 的处理
{ if (cnt != 0)                                  //已经输入退掉一个字符
{cnt--;                                          //总数目减一
line--;                                          //指针减一
putbyte (0x08);                                //屏幕回显的处理
putbyte (' ');
putbyte (0x08);
}
}
else
{ putbyte (*line = c);                          //其他字符取入, 回显
line++;                                          //指针加一
}
} while (c != 0x00);
}

```

```

        cnt++;
    }
} while (cnt < n - 1 && c != 0x00 && c != 0x1b); //数目到了, 回车或 ESC 停止
*line = 0;
//再加上停止符 0
}
//*****
putinbuf(uchar c)
{ES=0; if(!inbufful)
    {*inlast= c; //放入数据
    inlast++; //最后放入的位置加一
    if (inlast==inbuf+ILEN) inlast=inbuf; //地址到顶部回到底部
    if (inlast==getlast) inbufful=1; //接收缓冲区满置满标志
    inbufsign=1;
    }
ES=1;
}
//*****
//串口中断处理
serial() interrupt 4
{ if (TI)
    { TI = 0;
    if (outbufsign)
        //if (putlast==outlast) outbufsign=0;
        //else
        {SBUF=*outlast; //未发送完继续发送
        outlast++; //最后传出去的字节位置加一
        if (outlast==outbuf+OLEN) outlast=outbuf; //地址到顶部回到底部
        if (putlast==outlast) outbufsign=0; //数据发送完置发送缓冲区空标志
        }
    }
if (RI)
{ RI = 0;
    if(!inbufful)
    {
        *inlast= SBUF; //放入数据
        inlast++; //最后放入的位置加一
        inbufsign=1;
        if (inlast==inbuf+ILEN) inlast=inbuf; //地址到顶部回到底部
        if (inlast==getlast) inbufful=1; //接收缓冲区满置满标志
    }
}
}
//*****
//串口初始化
0xfd=19200,0xfa=9600,0xf4=4800,0xe8=2400,0xd0=1200

```

```

serial_init() {
    SCON = 0x50; /* mode 1: 8-bit UART, 开 */
    TMOD |= 0x20; /* timer 1 mode 2: 8-Bit */
    PCON |= 0x80; TH1 = 0xfd; //baud*2 19200 波特率 */
    TR1 = 1; /* timer 1 run */
    ES = 1; REN=1; EA=1; SM2=1; //SM2=1 时收到的第 9 位为 1 才置位 RI 标志
    TMOD |= 0x01; //th1 auto load 2X8,th0 1X16
    TH0=31; TL0=0; //X 32 =1S
    TR0=1; ET0=1;

}
//*****
//测试用主函数
/*
void main(void)
{char c;
idata unsigned char free[16];
unsigned char idata *freep=free;
serial_init();
putstring(crlf);
putstring(comready);
putstring("jdiptuejls;j:klja");
delay(10);

    while(1)
    {
        c=getbyte();
        putbyte(0x20);
        puthex(c);

        switch(c)
        {case 'r':
            putbytes(inbuf,ILEN);
            break;
            case 'g':
                getline(freep,10);
                putbyte(0x20);
                putstring(freep);
                break;
            default:
                putbyte(c);
        }
    }
}
*/

```

W77E58.H 代码

```

#ifndef true
#define true 1
#define false 0
#endif

#ifndef byte
#define byte unsigned char
#define word unsigned int
#define dword unsigned long
#endif

#define clr_bit(bit_name) bit_name=0
#define set_bit(bit_name) bit_name=1

typedef union
{
    word w;
    byte bh;
    byte bl;
} WordType;

typedef union
{
    dword dw;
    byte b[4];
    word w[2];
} DwordType;

/*--BYTE Registers-----*/
sfr P0 = 0x80;
sfr P1 = 0x90;
sfr P2 = 0xA0;
sfr P3 = 0xB0;
#define p0 P0
#define p1 P1
#define p2 P2
#define p3 P3

sfr PSW = 0xD0;
sfr ACC = 0xE0;
sfr B = 0xF0;
sfr SP = 0x81;
sfr DPL = 0x82;
sfr DPH = 0x83;
sfr PCON = 0x87; //PCON.7(SMOD)波特率加倍, PCON.1(PD)掉电方式, PCON.0(IDL)冻结方式

```


//PCON.6(SMOD0)帧错检测允许, PCON.3(GF1)PCON.2(GF0)

sfr TCON = 0x88;//定时控制寄存器

sfr TMOD = 0x89;//gate,c/t,m1,m0*x2 定时器方式 GATE=1 时只有 intx=1 时才可以开放定时器 x;

//c/t=1 时计数方式, =0 时定时方式。m1m0=00 时 13 位计数, =01 时 16 位=10

时自装入 8 位,

//11 时定时器 0 分两个, 定时器 1 停止。

sfr TL0 = 0x8A;

sfr TL1 = 0x8B;

sfr TH0 = 0x8C;

sfr TH1 = 0x8D;

sfr IE = 0xA8;

sfr IP = 0xB8;//中断优先级

sfr SCON = 0x98;//串口控制与状态

sfr SBUF = 0x99;

/*--W77E58 Extensions-----*/

sfr T2CON = 0xC8;

sfr T2MOD = 0xC9;//HC5,HC4,HC3,HC2,T2CR,-,T2OE,DCEN

//hc5-hc2=1 时外中断 5-2 标志硬件自动清除。

//t2cr=1 捕获完成时自动复位

//t2oe=1 定时器 2 输出允许

//dcen=1 减计数允许, 结合外部输入 t2ex 使用, 16 位自装入模式

sfr RCAP2L = 0xCA;//重装的预置数, 捕获的输出

sfr RCAP2H = 0xCB;

sfr TL2 = 0xCC;

sfr TH2 = 0xCD;

sfr DPL1 = 0x84;

sfr DPH1 = 0x85;

sfr DPS = 0x86;

sfr CKCON = 0x8E; //wd1,wd0,t2m,t1m,t0m,md2,md1,md0

//wd1wd0:看门狗计数 00=2¹⁷,01=2²⁰,10=2²³,11=2²⁶

//t2m,t1m,t0m 等于 1 时时钟 4 分频

//md2,md1,md0 MOVX 执行机器周期 000=2, 001=3...111=9

sfr EXIF = 0x91; //ie5,ie4,ie3,ie2,XT/RG,RGMD,RGSL,-

//ie5,ie3=1 外中断下跳变中断标志

//ie4,ie3=2 外中断上跳变中断标志

sfr P4 = 0xA5; //低四位有效

sfr SADDR = 0xA9; //从机串口 0 地址

sfr SADDR1 = 0xAA; //从机串口 1 地址

sfr SADEN = 0xB9; //串口 0 地址屏蔽, 等于 0 时所有地址都会引起中断

sfr SADEN1 = 0xBA; //串口 0 地址屏蔽, 等于 0 时所有地址都会引起中断

sfr SCON1 = 0xC0;

sfr SBUF1 = 0xC1;

```

sfr ROMMAP= 0xc2; //ROMMAP.7 为等待信号使能, 用 movx 指令时, wait 脚为 p4.0
sfr PMR    = 0xc4;  //CD1,CD0,SWB,-,XTOFF,ALE-OFF,-DME0
                    //cd0cd1=0 时钟不变, =1-1/4, =2-1/64, =3=1/1024
                    //swb=1 强制 4 分频, 外中断或串口中断唤醒
                    //aleoff=1 时 ale 信号终止, 外部内存访问时自动唤醒
                    //dme0=1 时内部 1kram 使能

sfr STATUS= 0xc5; //- ,HIP,LIP,XTUP,SPTA1,SPRA1,SPTA0,SPRA0
                    //hip=1 正在处理高优先级中断
                    //lip=1 正在处理低优先级中断
                    //spta1 串口 0 正在发送数据
                    //spral 串口 0 正在接收数据
                    //spta0 串口 0 正在发送数据
                    //spra0 串口 0 正在接收数据

sfr TA      = 0xc7; //保护位

sfr WDCON = 0xd8; //SMOD_1,POR,-,WDIF,WTRF,EWT,RWT
                    //smod1 加倍串口 1 的波特率,pro 电源复位标志
                    //por 电源复位标志
                    //wdif 看门狗定时中断标志, 软清
                    //wtrf 看门狗定时器复位标志, 看门狗引起复位后, 该位置 1
                    //ewt 允许看门狗定时器自动复位
                    //rwt 复位看门狗定时器, 如果看门狗溢出还没有被复位计数器, 将会引起中断, 再过 512
周期将复位

sfr EIE     = 0xe8; //- , -, -, EWDI,EX5,EX4,EX3,EX2
                    //ewdi=1, 允许看门狗中断?
                    //ex5-ex2; 外部中断允许

sfr EIP     = 0xf8; //- , -, -, pwdi, px5,px4,px3,px2
                    //pwdi=1 看门狗中断优先
                    //px5-px2=1 外部中断优先

/*--BIT Registers-----*/
/* PSW */
sbit CY     = 0xD7;
sbit AC     = 0xD6;
sbit F0     = 0xD5;
sbit RS1    = 0xD4;
sbit RS0    = 0xD3;
sbit OV     = 0xD2;
sbit P      = 0xD0;

/* TCON */
sbit TF1    = 0x8F; //定时器 1 溢出标志, 自动清零
sbit TR1    = 0x8E; //timer1 运行控制位
sbit TF0    = 0x8D; //定时器 0 溢出标志, 自动清零
sbit TR0    = 0x8C; //timer0 运行控制位
sbit IE1    = 0x8B; //外中断 1 跳变中断请求标志, 自动清零

```

```

sbit IT1    = 0x8A;//中断 1 跳变检测使能
sbit IE0    = 0x89;//外中断 0 跳变中断请求标志，自动清零
sbit IT0    = 0x88;//中断 0 跳变检测使能

```

```

/*  IE      */
sbit EA     = 0xAF;//
sbit ES     = 0xAC;
sbit ET1    = 0xAB;
sbit EX1    = 0xAA;
sbit ET0    = 0xA9;
sbit EX0    = 0xA8;

```

```

/*  IP      */
sbit PS     = 0xBC;//串口 0 中断优先设定
sbit PT1    = 0xBB;//定时器 1 中断优先
sbit PX1    = 0xBA;//外中断 1
sbit PT0    = 0xB9;//定时器 0 中断优先
sbit PX0    = 0xB8;//外中断 0

```

```

/*  P3      */
sbit RD     = 0xB7;
sbit WR     = 0xB6;
sbit T1     = 0xB5;
sbit T0     = 0xB4;
sbit INT1   = 0xB3;
sbit INT0   = 0xB2;
sbit TXD    = 0xB1;
sbit RXD    = 0xB0;

```

```

/*  SCON */

```

```

sbit SM0    = 0x9F;//SM0/FE  串口 0 方式 0 选择或帧错标志，人工清零

```

```

sbit SM1    = 0x9E;//串口 1 工作方式选择 0-4/12clk 同步，1-10 位可设 bps 异步，2-64/32clk 异步，3-11 位可变 bps 异步

```

```

sbit SM2    = 0x9D;//允许方式 2 和 3 的多机通讯控制位。方式 2、3 中=1 时接收到的第 9 位为 0 时不启动接收中断 ri 标志

```

//方式 1 中，=1 时只有收到有效停止时才启动 ri，方式 0 中=1 将波特率提高 3 倍。

```

sbit REN    = 0x9C;//允许接收标志
sbit TB8    = 0x9B;//方式 23 中要发送的第 9 位。
sbit RB8    = 0x9A;//方式 23 中接收到的第 9 位。
sbit TI     = 0x99;//发送中断标志
sbit RI     = 0x98;//接收中断标志

```

```

/*..W77E58 Extensions-----*/

```

```

/* PSW      */
sbit F1     = 0xd1;

```

```

/* IE      */
sbit ET2   = 0xAD;//定时器 2 允许
sbit ES1   = 0xAE;//串口 1 优先

/* IP      */
sbit PT2   = 0xBD;//定时器 2 优先
sbit PS1   = 0xBE;//串口 1 优先

/* P1      */
sbit T2     = 0x90;//计数器 2 输入端口
sbit T2EX   = 0x91;//计数器捕获触发
sbit RXD1   = 0x92;//串口 1 入
sbit TXD1   = 0x93;//串口 2 出
sbit INT2   = 0x94;//外中断 2, 上跳变触发
sbit INT3   = 0x95;//外中断 3, 下跳变触发
sbit INT4   = 0x96;//外中断 4, 上跳变触发
sbit INT5   = 0x97;//外中断 5, 下跳变触发

/* T2CON   */
sbit TF2    = 0xCF;//定时器 2 溢出标志, 软清除
sbit EXF2   = 0xCE;//定时器 2 外部标志, 当 exen2=1 且 t2ex 引脚负跳变引起捕获或重载时置位, 软清。
sbit RCLK   = 0xCD;//接收时钟标志, =1 时串口 0 用定时器 2 溢出做时钟。
sbit TCLK   = 0xCC;//发送时钟标志, =1 时串口 0 用定时器 2 溢出做时钟。
sbit EXEN2  = 0xCB;//定时器 2 外部允许标志, =1 时若未用作波特率发生器, t2ex 脚的负跳变引起捕获
sbit TR2    = 0xCA;//定时器 2 运行控制位
sbit C_T2   = 0xC9;//计数/定时选择, =1 时计数
sbit CP_RL  = 0xC8;//捕获/重载标志, =1 时,外部允许时, vt2ex 负跳变发生捕获
//=0 时 溢出或外部允许时, vt2ex 负跳变发生重载

/* SCON1   */ //注释参考 SCON
sbit SM0_1  = 0xC7;
sbit SM1_1  = 0xC6;
sbit SM2_1  = 0xC5;
sbit REN_1  = 0xC4;
sbit TB8_1  = 0xC3;
sbit RB8_1  = 0xC2;
sbit TI_1   = 0xC1;
sbit RI_1   = 0xC0;

/* WDCON   */
sbit SMOD_1 = 0xDF;//加倍串口 1 的波特率,pro 电源复位标志
sbit POR    = 0xDE;//电源复位标志
sbit WDIF   = 0xDB;//看门狗定时中断标志, 软清
sbit WTRF   = 0xDA;//看门狗定时器复位标志, 看门狗引起复位后, 该位置 1

```

```

sbit EWT    = 0xD9; // 允许看门狗定时器自动复位
sbit RWT    = 0xD8; // 复位看门狗定时器，如果看门狗溢出还没有被复位计数器，将会引起中断，再过
512 周期将复位

```

```

/* EIE      */
sbit EWDI   = 0xec; // 允许看门狗中断
sbit EX5    = 0xeb; // ex5-ex2; 外部中断允许
sbit EX4    = 0xea;
sbit EX3    = 0xe9;
sbit EX2    = 0xe8;

/* EIP      */
sbit PWDI   = 0xfc; // pwdi=1 看门狗中断优先
sbit PX5    = 0xfb; // px5-px2=i 外部中断优先
sbit PX4    = 0xfa;
sbit PX3    = 0xf9;
sbit PX2    = 0xf8;

```

程序三十三 PC 键代码

```

/*****
Chip type           : AT90
Clock frequency     : 8.000000 MHz
Memory model        : Tiny
Internal SRAM size   : 256
External SRAM size   : 0
Data Stack size     : 64
*****/

#include <90s4434.h>
#include "kb.h"
#include <delay.h>

// Alphanumeric LCD Module functions
#asm
.equ __lcd_port=0x15
#endasm
#include <lcd.h>

// Declare your global variables here

```

```

void main(void)
{

// 声时局域变量

// I/O 端口初始化
// Port A
PORTA=0x00;
DDRA=0x00;

// Port B
PORTB=0x00;
DDRB=0x00;

// Port C
PORTC=0x00;
DDRC=0x00;

// Port D
PORTD=0x00;
DDRD=0x00;

// Timer/Counter 0 initialization
// Clock source: System Clock
// Clock value: Timer 0 Stopped
// Mode: Output Compare
// OC0 output: Disconnected
TCCR0=0x00;
TCNT0=0x00;

// Timer/Counter 1 initialization
// Clock source: System Clock
// Clock value: Timer 1 Stopped
// Mode: Output Compare
// OC1A output: Discon.
// OC1B output: Discon.
// Noise Canceler: Off
// Input Capture on Falling Edge
TCCR1A=0x00;
TCCR1B=0x00;
TCNT1H=0x00;
TCNT1L=0x00;
OCR1AH=0x00;
OCR1AL=0x00;
OCR1BH=0x00;

```

```

OCR1BL=0x00;

// Timer/Counter 2 initialization
// Clock source: System Clock
// Clock value: Timer 2 Stopped
// Mode: Output Compare
// OC2 output: Disconnected
TCCR2=0x00;
ASSR=0x00;
TCNT2=0x00;
OCR2=0x00;

// External Interrupt(s) initialization
// INT0: On
// INT1: Off
GIMSK= 0x40;      // 开 INT0 interrupt
MCUCR=0x00;

// Timer(s)/Counter(s) Interrupt(s) initialization
TIMSK=0x00;

// Analog Comparator initialization
// Analog Comparator: Off
// Analog Comparator Input Capture by Timer/Counter 1: Off
ACSR=0x80;

// LCD module initialization
//lcd_init(20);
lcd_init(16);

InitKeyBoard();      // 初始化键盘接收

while(1)
{
    key = getchar_kb();
    lcd_putchar(key);
    delay_ms(5);

}

}

```

KB.C 代码

```

/*****

```

```

** low level kexboard routines

```

```

VERSION 1.0

```

```

*****/
#include "kb.h"
#include "scancodes.h"

#define BUFF_SIZE 64

unsigned char edge, bitcount;           // 0 = neg.  1 = pos.

unsigned char kb_buffer[BUFF_SIZE];
unsigned char *inpt, *outpt;
unsigned char buffcnt;

void InitKeyBoard(void)
{
    inpt = kb_buffer;                  // 初始化缓冲
    outpt = kb_buffer;
    buffcnt = 0;

    MCUCR = 2;                         // 下降沿 INTO 中断
    edge = 0;                          // 0 为下降沿, 1 为上升沿
    bitcount = 11;
    #asm("sei")                        // 中断开
}

interrupt [EXT_INT0] void INT0_interrupt(void)
{
    static unsigned char data;          // Holds the received scan code

    if(bitcount < 11 && bitcount > 2)  // Bit 3 to 10 is data. Parity bit,
    {                                   // start and stop bits are ignored.
        data = (data >> 1);
        if(PIND & 8)
            data = data | 0x80;        // Store a '1'
    }

    if(--bitcount == 0)                // All bits received
    {
        decode(data);
        bitcount = 11;
    }
}

void decode(unsigned char sc)

```



```

{
    static unsigned char is_up=0, shift = 0, mode = 0;
    unsigned char i;

    if (!is_up)                // Last data received was the up-key identifier
    {
        switch (sc)
        {
            case 0xF0 :        // The up-key identifier
                is_up = 1;
                break;

            case 0x12 :        // Left SHIFT
                shift = 1;
                break;

            case 0x59 :        // Right SHIFT
                shift = 1;
                break;

            case 0x05 :        // F1
                if(mode == 0)
                    mode = 1;    // Enter scan code mode
                if(mode == 2)
                    mode = 3;    // Leave scan code mode
                break;

            default:
                if(mode == 0 || mode == 3)                // If ASCII mode
                {
                    if(!shift)                            // If shift not pressed,
                    {                                    // do a table look-up
                        for(i = 0; unshifted[i][0]!=sc && unshifted[i][0]; i++);
                        if (unshifted[i][0] == sc) {
                            put_kbbuff(unshifted[i][1]);
                        }
                    } else {                                // If shift pressed
                        for(i = 0; shifted[i][0]!=sc && shifted[i][0]; i++);
                        if (shifted[i][0] == sc) {
                            put_kbbuff(shifted[i][1]);
                        }
                    }
                } else{                                    // Scan code mode
                    print_hexbyte(sc);                    // Print scan code
                    put_kbbuff(' ');
                }
            }
        }
    }
}

```

```

        }
        break;
    }
} else {
    is_up = 0; // Two 0xF0 in a row not allowed
    switch (sc)
    {
        case 0x12 : // Left SHIFT
            shift = 0;
            break;

        case 0x59 : // Right SHIFT
            shift = 0;
            break;

        case 0x05 : // F1
            if(mode == 1)
                mode = 2;
            if(mode == 3)
                mode = 0;
            break;
        case 0x06 : // F2
            // clr();
            break;
    }
}

}

}

void put_kbbuff(unsigned char c)
{
    if (buffcnt < BUFF_SIZE) // If buffer not full
    {
        *inpt = c; // Put character into buffer
        inpt++; // Increment pointer

        buffcnt++;

        if (inpt >= kb_buffer + BUFF_SIZE) // Pointer wrapping
            inpt = kb_buffer;
    }
}

int getchar_kb(void)

```

```

{
    int byte;
    while(buffcnt == 0);           // Wait for data

    byte = *outpt;                 // Get byte
    outpt++;                       // Increment pointer

    if (outpt >= kb_buffer + BUFF_SIZE) // Pointer wrapping
        outpt = kb_buffer;

    buffcnt--;                     // Decrement buffer count

    return byte;
}

void print_hexbyte(unsigned char i)
{
    unsigned char h, l;

    h = i & 0xF0;                  // High nibble
    h = h >> 4;
    h = h + '0';

    if (h > '9')
        h = h + 7;

    l = (i & 0x0F) + '0';          // Low nibble
    if (l > '9')
        l = l + 7;

    put_kbbuff(h);
    put_kbbuff(l);
}

```

KB.H 代码

```

// Keyboard communication routines
#ifndef __KB_INCLUDED
#define __KB_INCLUDED

#include <90s4434.h>

#define CLOCK 2
#define DATAPIN 3

#define ISC00 0

```

```
#define ISC01 1
```

```
void InitKeyBoard(void);  
interrupt [EXT_INT0] void INT0_interrupt(void);  
void decode(unsigned char sc);  
void put_kbbuff(unsigned char c);  
int getchar_kb(void);  
void print_hexbyte(unsigned char i);  
#endif
```

SCANCODE.H 代码

```
// Unshifted characters  
flash unsigned char unshifted[68][2] = {  
0x0d,9,  
0x0e,'l',  
0x15,'q',  
0x16,'l',  
0x1a,'z',  
0x1b,'s',  
0x1c,'a',  
0x1d,'w',  
0x1e,'2',  
0x21,'c',  
0x22,'x',  
0x23,'d',  
0x24,'e',  
0x25,'4',  
0x26,'3',  
0x29,' ',  
0x2a,'v',  
0x2b,'f',  
0x2c,'t',  
0x2d,'r',  
0x2e,'5',  
0x31,'n',  
0x32,'b',  
0x33,'h',  
0x34,'g',  
0x35,'y',  
0x36,'6',  
0x39,',',  
0x3a,'m',  
0x3b,'j',  
0x3c,'u',  
0x3d,'7',
```

```

0x3e,'8',
0x41,',',
0x42,'k',
0x43,'i',
0x44,'o',
0x45,'0',
0x46,'9',
0x49,',',
0x4a,',',
0x4b,'l',
0x4c,'?',
0x4d,'p',
0x4e,'+',
0x52,'?',
0x54,'?',
0x55,'w',
0x5a,13,
0x5b,'?',
0x5d,'w',
0x61,'<',
0x66,8,
0x69,'l',
0x6b,'4',
0x6c,'7',
0x70,'0',
0x71,',',
0x72,'2',
0x73,'5',
0x74,'6',
0x75,'8',
0x79,'+',
0x7a,'3',
0x7b,',',
0x7c,'*',
0x7d,'9',
0,0
};

```

程序三十四 拼音输入法模块

拼音输入法查询函数: `unsigned char code * py_ime(unsigned char input_py_val[])`; 其中, `input_py_val` 为已输入的拼音码字符串头指针, 返回值为中文的起始地址, 当为 0 时, 查询失败。

应用举例:

```
{
    unsigned char input_string[]={ "bang" };
    unsigned char chines_string[100];
    sprintf(chines_string, "%s", py_ime(input_string));
}

/* * 编译环境: Franklin 3.3.4    */
#include <string.h>
#include <reg51.h>

/* 拼音输入法汉字排列表 */
unsigned char code PY_mb_a    []={ "阿啊" };
unsigned char code PY_mb_ai   []={ "哎哀埃挨皑癌矮蔼艾爱隘碍" };
unsigned char code PY_mb_an   []={ "安氨鞍俺岸按案胺暗" };
unsigned char code PY_mb_ang  []={ "肮昂盎" };
unsigned char code PY_mb_ao   []={ "凹敖熬翱袄傲奥澳懊" };
unsigned char code PY_mb_ba   []={ "八巴叭扒吧芭疤捌芭拔跋把靶坝爸罢霸" };
unsigned char code PY_mb_bai  []={ "白百佰柏摆败拜裨" };
unsigned char code PY_mb_ban  []={ "扳班般颁斑搬板版办半伴扮拌拌瓣" };
unsigned char code PY_mb_bang []={ "邦帮梆绑榜膀蚌傍棒谤磅磅" };
unsigned char code PY_mb_bao  []={ "包包胞褒雹宝饱保堡报抱豹鲍暴爆剥薄瀑" };
unsigned char code PY_mb_bei  []={ "卑杯悲碑北贝狈备背狈倍被惫焙辈" };
unsigned char code PY_mb_ben  []={ "奔本笨笨奔" };
unsigned char code PY_mb_beng []={ "崩绷甬泵迸蹦" };
unsigned char code PY_mb_bi   []={ "逼鼻比彼笔鄙币必毕闭庇庇陛毙敝痹莠弊碧蔽壁避臂" };
unsigned char code PY_mb_bian []={ "边编鞭贬扁卞便变遍辨辩辩" };
unsigned char code PY_mb_biao []={ "彪标膘表" };
unsigned char code PY_mb_bie  []={ "憋鳖别瘪" };
unsigned char code PY_mb_bin  []={ "宾彬斌滨濒宾" };
unsigned char code PY_mb_bing []={ "冰兵丙秉柄炳饼并病" };
unsigned char code PY_mb_bo   []={ "拨波玻钵脖菠播伯驳帛泊勃铂舶博渤搏箔膊卜" };
unsigned char code PY_mb_bu   []={ "补哺捕不布步怖部埠簿" };
unsigned char code PY_mb_ca   []={ "擦" };
unsigned char code PY_mb_cai  []={ "猜才材财裁采彩睬睬菜蔡" };
unsigned char code PY_mb_can  []={ "参餐残蚕惭惨灿" };
unsigned char code PY_mb_cang []={ "仓沧苍舱藏" };
unsigned char code PY_mb_cao  []={ "操糙曹槽草" };
unsigned char code PY_mb_ce   []={ "册侧厕测策" };
unsigned char code PY_mb_ceng []={ "层蹭曾" };
unsigned char code PY_mb_cha  []={ "叉插查茬茶搽察碴岔讬差刹" };
unsigned char code PY_mb_chai []={ "拆柴豺" };
unsigned char code PY_mb_chan []={ "搀搀谗馋缠蟾产铲阐颤" };
unsigned char code PY_mb_chang []={ "昌猖肠尝偿常厂场敞畅倡唱" };
unsigned char code PY_mb_chao []={ "抄钞超巢朝嘲潮吵炒绰" };
```

unsigned char code PY_mb_che []={"车扯彻掣撤澈"};
 unsigned char code PY_mb_chen []={"郴尘臣忱沉宸陈晨衬趁"};
 unsigned char code PY_mb_cheng []={"称撑成呈承诚城乘惩程澄橙逞骋秤"};
 unsigned char code PY_mb_chi []={"吃痴弛池驰迟持尺侈齿耻斥赤炽翅"};
 unsigned char code PY_mb_chong []={"充冲虫崇宠"};
 unsigned char code PY_mb_chou []={"抽仇绸畴愁稠筹酬踌丑瞅臭"};
 unsigned char code PY_mb_chu []={"出初厨滁锄雏橱躇础储楚处搐触矗畜"};
 unsigned char code PY_mb_chuai []={"揣"};
 unsigned char code PY_mb_chuan []={"川穿传船椽喘串"};
 unsigned char code PY_mb_chuang []={"闯疮窗床创"};
 unsigned char code PY_mb_chui []={"吹炊垂捶锤"};
 unsigned char code PY_mb_chun []={"春椿纯唇淳醇蠢"};
 unsigned char code PY_mb_chuo []={"戳"};
 unsigned char code PY_mb_ci []={"疵词茨瓷慈辞磁雌此次刺赐"};
 unsigned char code PY_mb_cong []={"囱从匆葱聪丛"};
 unsigned char code PY_mb_cou []={"凑"};
 unsigned char code PY_mb_cu []={"粗促醋簇"};
 unsigned char code PY_mb_cuan []={"蹿窜篡"};
 unsigned char code PY_mb_cui []={"崔催摧脆淬粹粹翠"};
 unsigned char code PY_mb_cun []={"村存寸"};
 unsigned char code PY_mb_cuo []={"搓磋撮挫措错"};
 unsigned char code PY_mb_da []={"搭达答瘩打大"};
 unsigned char code PY_mb_dai []={"呆歹傣代待怠殆贷袋逮戴"};
 unsigned char code PY_mb_dan []={"丹单担耽耽胆掸旦但诞弹淡蛋氮"};
 unsigned char code PY_mb_dang []={"当挡党荡档"};
 unsigned char code PY_mb_dao []={"刀导岛倒捣祷蹈到悼盗道稻"};
 unsigned char code PY_mb_de []={"得德的"};
 unsigned char code PY_mb_deng []={"灯登蹬等邓凳瞪"};
 unsigned char code PY_mb_di []={"低堤滴狄迪敌涤笛嫡底抵地弟帝递第蒂蒂"};
 unsigned char code PY_mb_dian []={"掂滇颠典点碘电佃甸店垫惦淀奠殿殿"};
 unsigned char code PY_mb_diao []={"刁刁凋凋雕吊钓掉"};
 unsigned char code PY_mb_die []={"爹跌迭谍叠碟蝶"};
 unsigned char code PY_mb_ding []={"丁叮叮钉顶鼎订定锭"};
 unsigned char code PY_mb_diu []={"丢"};
 unsigned char code PY_mb_dong []={"东冬董懂动冻恫恫栋洞"};
 unsigned char code PY_mb_dou []={"都兜斗抖陡逗痘"};
 unsigned char code PY_mb_du []={"督毒读牍独堵赌睹妒杜肚度渡镀"};
 unsigned char code PY_mb_duan []={"端短段断缎锻"};
 unsigned char code PY_mb_dui []={"堆队对兑"};
 unsigned char code PY_mb_dun []={"吨敦墩蹲盾钝顿遁"};
 unsigned char code PY_mb_duo []={"哆哆夺掇朵垛躲剝堕舵惰蹉"};
 unsigned char code PY_mb_e []={"讹俄娥峨陂蛾额厄扼恶饿鄂遏"};
 unsigned char code PY_mb_en []={"恩"};
 unsigned char code PY_mb_er []={"儿而尔耳洱饵二贰"};
 unsigned char code PY_mb_fa []={"发乏伐罚阀筏法珐"};

unsigned char code PY_mb_fan []={"帆番翻藩凡矾钒烦樊繁反返犯泛饭范贩"};
 unsigned char code PY_mb_fang []={"方坊芳防妨房肪访纺放"};
 unsigned char code PY_mb_fei []={"飞非啡菲肥匪诽吠废沸肺费"};
 unsigned char code PY_mb_fen []={"分纷纷芬氛酚坟焚粉份奋忿愤粪"};
 unsigned char code PY_mb_feng []={"丰风枫封疯峰烽锋蜂冯逢缝讽风奉"};
 unsigned char code PY_mb_fo []={"佛"};
 unsigned char code PY_mb_fou []={"否"};
 unsigned char code PY_mb_fu []={"夫肤孵敷弗伏扶拂服俘氟浮涪符袱幅福辐抚甫府斧俯釜辅腑腐父
 讣付妇负附咐阜复赴副傅富赋缚腹覆"};
 unsigned char code PY_mb_ga []={"嘎噶"};
 unsigned char code PY_mb_gai []={"该改钙盖溉概"};
 unsigned char code PY_mb_gan []={"干甘杆肝柑竿秆赶敢感赣"};
 unsigned char code PY_mb_gang []={"冈刚岗纲肛缸钢港杠"};
 unsigned char code PY_mb_gao []={"皋羔高膏篙糕搞稿搞告"};
 unsigned char code PY_mb_ge []={"戈疙哥咯鸽割搁歌阁革格葛隔个各咯咯"};
 unsigned char code PY_mb_gei []={"给"};
 unsigned char code PY_mb_gen []={"根跟"};
 unsigned char code PY_mb_geng []={"更庚耕羹埂耿梗"};
 unsigned char code PY_mb_gong []={"工弓公功攻供宫恭躬龚巩汞拱共贡"};
 unsigned char code PY_mb_gou []={"勾沟钩狗苟构购垢够"};
 unsigned char code PY_mb_gu []={"估咕姑孤沽菇辜箍古谷股骨蛊鼓固故顾雇"};
 unsigned char code PY_mb_gua []={"瓜刮刚寡挂褂"};
 unsigned char code PY_mb_guai []={"乖拐怪"};
 unsigned char code PY_mb_guan []={"关观官冠棺馆管贯惯灌罐"};
 unsigned char code PY_mb_guang []={"光广逛"};
 unsigned char code PY_mb_gui []={"归圭龟规闺珪瑰轨诡癸鬼刽柜贵桂跪"};
 unsigned char code PY_mb_gun []={"棍滚棍"};
 unsigned char code PY_mb_guo []={"郭锅国果裹过"};
 unsigned char code PY_mb_ha []={"蛤哈"};
 unsigned char code PY_mb_hai []={"孩骸海亥骇害氩"};
 unsigned char code PY_mb_han []={"酣惑含邯函涵寒韩罕喊汉汗旱悍焊憾撼翰"};
 unsigned char code PY_mb_hang []={"杭航行"};
 unsigned char code PY_mb_hao []={"毫豪嚎壕好郝号浩耗"};
 unsigned char code PY_mb_he []={"呵喝禾合何和河阍核荷涸盒荷贺褐赫鹤"};
 unsigned char code PY_mb_hei []={"黑嘿"};
 unsigned char code PY_mb_hen []={"痕很狠恨"};
 unsigned char code PY_mb_heng []={"亨亨恒横衡"};
 unsigned char code PY_mb_hong []={"轰哄烘弘红宏洪虹鸿"};
 unsigned char code PY_mb_hou []={"候喉猴吼后厚候"};
 unsigned char code PY_mb_hu []={"乎呼忽弧狐胡壶湖葫瑚糊瑚虎唬互户护沪"};
 unsigned char code PY_mb_hua []={"花华哗滑猾化划画话"};
 unsigned char code PY_mb_huai []={"怀徊淮槐坏"};
 unsigned char code PY_mb_huan []={"欢还环桓缓幻宦唤换涣患焕痪"};
 unsigned char code PY_mb_huang []={"荒慌皇凰黄惶惶蝗磺簧恍晃慌幌"};
 unsigned char code PY_mb_hui []={"灰恢挥辉徽回徊悔卉汇会讳绘诲烺贿晦秽惠慧"};

unsigned char code PY_mb_hun []={"昏荤浑魂混"};

unsigned char code PY_mb_huo []={"豁活伙伙或货获祸惑霍"};

unsigned char code PY_mb_ji []={"讥讥饥圾机肌鸡迹姬积基绩缉畸箕稽激及吉汲级即极急疾棘集嫉辑籍几己挤脊计记伎妓忌技际剂季既济继寂寄悸祭蓟冀藉"};

unsigned char code PY_mb_jia []={"加夹佳枷家嘉荚颊甲贾钾价驾架假嫁稼挟"};

unsigned char code PY_mb_jian []={"奸尖坚歼间肩艰监笺箴煎拣俭柬捡减剪检简碱见件建贱剑荐贱健涧舰渐溅践鉴键箭"};

unsigned char code PY_mb_jiang []={"江姜将浆僵疆讲奖桨蒋匠降酱"};

unsigned char code PY_mb_jiao []={"交郊娇饶骄胶椒焦蕉礁角狡皎较轿脚较搅剿缴叫轿较教窖酵觉嚼"};

};

unsigned char code PY_mb_jie []={"阶皆接秸揭街节劫杰洁结捷睫截竭姐解介戒芥届界疥诫借"};

unsigned char code PY_mb_jin []={"巾今斤金津筋襟仪紧谨锦尽劲近进晋浸烬禁靳"};

unsigned char code PY_mb_jing []={"京经茎荆惊晶秸梗兢精鲸并颈景警净径径竟敬靖境静镜"};

unsigned char code PY_mb_jiong []={"炯冢"};

unsigned char code PY_mb_jiu []={"纠究揪九久灸玖韭酒旧臼咎疚厥救就舅"};

unsigned char code PY_mb_ju []={"居拘狙驹疽鞠局桔菊咀沮举矩句扌拒具炬俱惧据距锯聚踞"};

unsigned char code PY_mb_juan []={"娟娟娟卷倦绢眷"};

unsigned char code PY_mb_jue []={"概决决抉绝倔掘爵攫"};

unsigned char code PY_mb_jun []={"军君均钧菌俊郡峻浚竣竣"};

unsigned char code PY_mb_ka []={"咖喀卡"};

unsigned char code PY_mb_kai []={"开揩凯慨楷"};

unsigned char code PY_mb_kan []={"槛刊勘堪坎砍看"};

unsigned char code PY_mb_kang []={"康慷糠扛亢抗炕"};

unsigned char code PY_mb_kao []={"考拷烤靠"};

unsigned char code PY_mb_ke []={"珂苛柯科棵颗磕壳咳可渴克刻客课"};

unsigned char code PY_mb_ken []={"肯垦恳啃"};

unsigned char code PY_mb_keng []={"吭坑"};

unsigned char code PY_mb_kong []={"空孔恐控"};

unsigned char code PY_mb_kou []={"抠口扣寇"};

unsigned char code PY_mb_ku []={"枯哭窟苦库裤酷"};

unsigned char code PY_mb_kua []={"夸垮垮跨跨"};

unsigned char code PY_mb_kuai []={"块快快筷"};

unsigned char code PY_mb_kuan []={"宽款"};

unsigned char code PY_mb_kuang []={"匡筐狂况旷矿框眶"};

unsigned char code PY_mb_kui []={"亏岢盍窺奎葵魁傀愧溃溃"};

unsigned char code PY_mb_kun []={"坤昆捆困"};

unsigned char code PY_mb_kuo []={"扩括阔廓"};

unsigned char code PY_mb_la []={"垃拉啦喇腊蜡辣"};

unsigned char code PY_mb_lai []={"来莱赖"};

unsigned char code PY_mb_lan []={"兰拦栏婪阑蓝澜澜篮览揽揽懒烂滥"};

unsigned char code PY_mb_lang []={"郎狼廊琅榔朗浪"};

unsigned char code PY_mb_lao []={"捞劳牢佬佬佬烙酪"};

unsigned char code PY_mb_le []={"乐勒了"};

unsigned char code PY_mb_lei []={"雷雷垒磊蕾儡肋泪类累擂"};

unsigned char code PY_mb_leng []={"棱楞冷"};

unsigned char code PY_mb_li []={"厘梨狸离莉犁漓璃黎篱礼李里哩理鲤力厉立吏丽利励沥例隶俐
 荔栗砾粒悝痢"};
 unsigned char code PY_mb_lian []={"连帘怜涟莲联廉镰敛脸练炼恋链"};
 unsigned char code PY_mb_liang []={"俩良凉梁粮粱两亮谅辆晾量"};
 unsigned char code PY_mb_liao []={"潦辽疗聊僚寥廖撩僚僚料撖"};
 unsigned char code PY_mb_lie []={"列劣烈猎裂"};
 unsigned char code PY_mb_lin []={"邻林临淋琳霖磷鳞凛吝赁拎"};
 unsigned char code PY_mb_ling []={"伶灵岭玲凌铃陵菱零龄领令另"};
 unsigned char code PY_mb_liu []={"溜刘流留琉硫榴榴瘤柳六"};
 unsigned char code PY_mb_long []={"龙咙笼聋隆窿陇垄拢"};
 unsigned char code PY_mb_lou []={"娄楼搂篓陋漏"};
 unsigned char code PY_mb_lu []={"露卢庐芦炉颅卤虏掳鲁陆录赂鹿禄碌戮潞麓"};
 unsigned char code PY_mb_luan []={"挛恋挛滦卵乱"};
 unsigned char code PY_mb_lue []={"掠咯"};
 unsigned char code PY_mb_lun []={"抡仑伦沦纶轮论"};
 unsigned char code PY_mb_luo []={"罗萝逻箩箩螺裸洛络骆落"};
 unsigned char code PY_mb_lv []={"滤驴吕侣旅铝屡缕履律虑率绿氯"};
 unsigned char code PY_mb_ma []={"妈麻马玛码蚂骂吗嘛"};
 unsigned char code PY_mb_mai []={"埋买迈麦卖脉"};
 unsigned char code PY_mb_man []={"蛮慢瞒满曼慢慢漫蔓"};
 unsigned char code PY_mb_mang []={"忙芒盲茫莽氓"};
 unsigned char code PY_mb_mao []={"猫毛矛茅锚卯柳茂冒贸帽貌"};
 unsigned char code PY_mb_me []={"么"};
 unsigned char code PY_mb_mei []={"没枚玫眉梅媒煤霉每美镁妹昧媚寐"};
 unsigned char code PY_mb_men []={"门闷们"};
 unsigned char code PY_mb_meng []={"萌盟檬猛蒙猛孟梦"};
 unsigned char code PY_mb_mi []={"弥迷谜魅糜靡米眯泌觅秘密幕蜜"};
 unsigned char code PY_mb_mian []={"眠绵棉免勉婉冕緬面"};
 unsigned char code PY_mb_miao []={"苗描瞄秒渺藐妙庙"};
 unsigned char code PY_mb_mie []={"灭蔑"};
 unsigned char code PY_mb_min []={"民皿抿闽悯敏"};
 unsigned char code PY_mb_ming []={"名明鸣铭螟命"};
 unsigned char code PY_mb_miu []={"谬"};
 unsigned char code PY_mb_mo []={"貉摸摹模膜摩磨磨魔抹末沫陌莫寞漠墨默"};
 unsigned char code PY_mb_mou []={"牟谋某"};
 unsigned char code PY_mb_mu []={"母亩牡姆拇木目牧募慕幕睦慕暮穆"};
 unsigned char code PY_mb_na []={"拿哪那纳娜纳呐"};
 unsigned char code PY_mb_nai []={"乃奶氛奈耐"};
 unsigned char code PY_mb_nan []={"男南难"};
 unsigned char code PY_mb_nang []={"囊"};
 unsigned char code PY_mb_nao []={"挠恼脑闹淖"};
 unsigned char code PY_mb_ne []={"呢"};
 unsigned char code PY_mb_nei []={"内馁"};
 unsigned char code PY_mb_nen []={"嫩"};
 unsigned char code PY_mb_neng []={"能"};

unsigned char code PY_mb_ni []={"妮尼泥倪霓你拟逆匿溺腻"};
 unsigned char code PY_mb_nian []={"拈年捻撵碾念蔫"};
 unsigned char code PY_mb_niang []={"娘酿"};
 unsigned char code PY_mb_niao []={"鸟尿"};
 unsigned char code PY_mb_nie []={"捏涅聂啮镊镍孽"};
 unsigned char code PY_mb_nin []={"您"};
 unsigned char code PY_mb_ning []={"宁拧柠柠凝泞"};
 unsigned char code PY_mb_niu []={"牛扭纽钮"};
 unsigned char code PY_mb_nong []={"农浓脓弄"};
 unsigned char code PY_mb_nu []={"奴努怒"};
 unsigned char code PY_mb_nuan []={"暖"};
 unsigned char code PY_mb_nue []={"疟虐"};
 unsigned char code PY_mb_nuo []={"挪诺糯糯"};
 unsigned char code PY_mb_nv []={"女"};
 unsigned char code PY_mb_o []={"哦"};
 unsigned char code PY_mb_ou []={"欧殴鸥呕偶藕沤"};
 unsigned char code PY_mb_pa []={"趴啪爬耙琶帕怕"};
 unsigned char code PY_mb_pai []={"拍排排牌派湃"};
 unsigned char code PY_mb_pan []={"潘攀盘磐判叛盼畔"};
 unsigned char code PY_mb_pang []={"乓庞旁榜胖"};
 unsigned char code PY_mb_pao []={"抛刨咆炮袍跑泡"};
 unsigned char code PY_mb_pei []={"胚胚陪培赔裴沛佩配"};
 unsigned char code PY_mb_pen []={"喷盆"};
 unsigned char code PY_mb_peng []={"抨砰烹朋彭棚棚蓬鹏澎篷膨捧碰"};
 unsigned char code PY_mb_pi []={"辟批坯披砒磅霹皮毗疲啤琵琶匹痞屁僻臂"};
 unsigned char code PY_mb_pian []={"片偏篇骗"};
 unsigned char code PY_mb_piao []={"漂飘瓢票"};
 unsigned char code PY_mb_pie []={"撇瞥"};
 unsigned char code PY_mb_pin []={"拼贫频品聘"};
 unsigned char code PY_mb_ping []={"平平评凭坪苹屏瓶萍"};
 unsigned char code PY_mb_po []={"坡泼颇婆迫破粕魄"};
 unsigned char code PY_mb_pou []={"剖"};
 unsigned char code PY_mb_pu []={"脯仆扑铺莆葡蒲朴圃埔浦普谱曝"};
 unsigned char code PY_mb_qi []={"七沏妻柒凄栖戚期欺漆祁齐其奇歧祈脐崎畦骑棋旗乞企岂启起气
 讫迄卉汽泣契砌器"};

unsigned char code PY_mb_qia []={"掐恰洽"};
 unsigned char code PY_mb_qian []={"千仟扞迁钎牵铅谦签前钱钳乾潜黔浅遣谴欠堑嵌歉"};
 unsigned char code PY_mb_qiang []={"呛羌枪腔强墙蔷抢"};
 unsigned char code PY_mb_qiao []={"悄敲锹橇乔侨桥瞧巧俏峭窍翘撬鞘"};
 unsigned char code PY_mb_qie []={"切茄且怯窃"};
 unsigned char code PY_mb_qin []={"亲侵钦芹秦琴禽勤擒寝沁"};
 unsigned char code PY_mb_qing []={"青氢轻倾卿清情晴氛擎顷请庆"};
 unsigned char code PY_mb_qiong []={"穷琼"};
 unsigned char code PY_mb_qiu []={"丘邱秋囚求沔茜球"};
 unsigned char code PY_mb_qu []={"区曲驱屈蛆驱趋渠取娶龋去趣"};

unsigned char code PY_mb_quan []={"圈全权泉拳痊醅颧犬劝券"};
 unsigned char code PY_mb_que []={"缺缺瘸却雀确鹌榷"};
 unsigned char code PY_mb_qun []={"裙群"};
 unsigned char code PY_mb_ran []={"然燃冉染"};
 unsigned char code PY_mb_rang []={"瓢嚷壤攘让"};
 unsigned char code PY_mb_rao []={"饶扰绕"};
 unsigned char code PY_mb_re []={"惹热"};
 unsigned char code PY_mb_ren []={"人仁壬忍刃认任纫妊韧"};
 unsigned char code PY_mb_reng []={"扔仍"};
 unsigned char code PY_mb_ri []={"日"};
 unsigned char code PY_mb_rong []={"戎绒茸荣容蓉蓉融冗"};
 unsigned char code PY_mb_rou []={"柔揉肉"};
 unsigned char code PY_mb_ru []={"茹茹儒孺蠕汝乳辱入褥"};
 unsigned char code PY_mb_ruan []={"阮软"};
 unsigned char code PY_mb_rui []={"蕊锐瑞"};
 unsigned char code PY_mb_run []={"闰润"};
 unsigned char code PY_mb_ruo []={"若弱"};
 unsigned char code PY_mb_sa []={"撒洒萨"};
 unsigned char code PY_mb_sai []={"塞腮腮赛"};
 unsigned char code PY_mb_san []={"三叁伞散"};
 unsigned char code PY_mb_sang []={"桑嗓丧"};
 unsigned char code PY_mb_sao []={"骚骚扫嫂"};
 unsigned char code PY_mb_se []={"色涩瑟"};
 unsigned char code PY_mb_sen []={"森"};
 unsigned char code PY_mb_seng []={"僧"};
 unsigned char code PY_mb_sha []={"杀沙纱砂莎傻啥煞厦"};
 unsigned char code PY_mb_shai []={"筛晒"};
 unsigned char code PY_mb_shan []={"山删杉衫珊煽闪陕汕苫扇善缮擅膳贍棚"};
 unsigned char code PY_mb_shang []={"伤商墒裳晌赏上尚"};
 unsigned char code PY_mb_shao []={"梢梢烧稍勺苟韶少邵绍哨"};
 unsigned char code PY_mb_she []={"奢除舌蛇舍设社射涉赦慑摄"};
 unsigned char code PY_mb_she []={"申伸身呻绅娠殄深神沈审婢肾甚渗慎什"};
 unsigned char code PY_mb_sheng []={"升生声牲牲甥绳省圣盛剩"};
 unsigned char code PY_mb_shi []={"匙尸失师虱诗施狮湿十石时识实拾蚀食史矢使始驶屎士氏世仕市
 示式事侍势视试饰室恃拭是柿适逝释嗜誓噤似"};
 unsigned char code PY_mb_shou []={"收手守首寿受兽售授瘦"};
 unsigned char code PY_mb_shu []={"书抒叔枢殊梳淑疏舒输蔬孰赎熟暑黍署鼠蜀薯曙术戍束述树竖
 恕庶数墅漱属"};
 unsigned char code PY_mb_shua []={"刷耍"};
 unsigned char code PY_mb_shuai []={"衰摔甩帅"};
 unsigned char code PY_mb_shuan []={"拴栓"};
 unsigned char code PY_mb_shuang []={"双霜爽"};
 unsigned char code PY_mb_shui []={"谁水税睡"};
 unsigned char code PY_mb_shun []={"吮顺舜瞬"};
 unsigned char code PY_mb_shuo []={"说烁朔硕"};

unsigned char code PY_mb_si []={"丝司私思斯嘶嘶死已四寺伺伺嗣肆"};
 unsigned char code PY_mb_song []={"松怱耸讼宋诵送颂"};
 unsigned char code PY_mb_sou []={"嗽搜艘艘"};
 unsigned char code PY_mb_su []={"苏酥俗诉肃素速粟塑溯慄"};
 unsigned char code PY_mb_suan []={"酸蒜算"};
 unsigned char code PY_mb_sui []={"虽绥隋随髓岁崇遂碎隧穗"};
 unsigned char code PY_mb_sun []={"孙损笋"};
 unsigned char code PY_mb_suo []={"唆梭蓑缩所索琐锁"};
 unsigned char code PY_mb_ta []={"她他它塌塔獭挞踏蹋"};
 unsigned char code PY_mb_tai []={"胎台抬苔太汰态泰酖"};
 unsigned char code PY_mb_tan []={"坍贪摊滩摊坛谈痰谭潭檀坦袒毯叹炭探碳"};
 unsigned char code PY_mb_tang []={"汤唐堂棠塘塘膛糖倘淌躺烫趟"};
 unsigned char code PY_mb_tao []={"涛绺掏滔逃桃陶淘萄讨套"};
 unsigned char code PY_mb_te []={"特"};
 unsigned char code PY_mb_teng []={"疼腾誊滕"};
 unsigned char code PY_mb_ti []={"剔梯梯踢啼提题蹄体屉剃涕惕替嚏"};
 unsigned char code PY_mb_tian []={"天添山恬甜填腆舔"};
 unsigned char code PY_mb_tiao []={"调挑条迢眺跳"};
 unsigned char code PY_mb_tie []={"贴铁帖"};
 unsigned char code PY_mb_ting []={"厅汀听烺廷亨庭停挺艇"};
 unsigned char code PY_mb_tong []={"通同彤桐铜童酗瞳统捅桶筒痛"};
 unsigned char code PY_mb_tou []={"偷头投透"};
 unsigned char code PY_mb_tu []={"凸秃突图徒涂途屠土吐兔"};
 unsigned char code PY_mb_tuan []={"湍团"};
 unsigned char code PY_mb_tui []={"推颓腿退蜕褪"};
 unsigned char code PY_mb_tun []={"囤吞屯臀"};
 unsigned char code PY_mb_tuo []={"托拖脱驮陀驼妥桶拓唾"};
 unsigned char code PY_mb_wa []={"哇娃挖洼蛙瓦袜"};
 unsigned char code PY_mb_wai []={"歪外"};
 unsigned char code PY_mb_wan []={"弯湾碗丸完玩顽惋宛挽晚婉惋碗万腕"};
 unsigned char code PY_mb_wang []={"汪亡王网往枉妄忘旺望"};
 unsigned char code PY_mb_wei []={"危威微巍为韦围违桅惟维淮伟伪尾纬苇萎卫未位味畏胃尉
 谓喂渭蔚慰魏"};
 unsigned char code PY_mb_wen []={"温瘟文纹闻蚊吻紊稳问"};
 unsigned char code PY_mb_weng []={"翁嗡瓮"};
 unsigned char code PY_mb_wo []={"挝窝窝蜗我沃卧握斡"};
 unsigned char code PY_mb_wu []={"乌污呜巫屋诬鸪无毋吴吾芜梧五午伍坞武侮捂舞勿务戊物误悟
 晤雾"};
 unsigned char code PY_mb_xi []={"夕汐西吸希析矽息栖悉惜烯晒晰犀稀溪锡熄熙嘻膝习席袭媳橄
 洗喜戏系细隙"};
 unsigned char code PY_mb_xia []={"吓瞎吓侠狭狭暇辖霞下吓夏"};
 unsigned char code PY_mb_xian []={"仙先纤掀掀鲜闲弦贤涎舷衔嫌显险县现线限宪陷馅羨献腺"};
 unsigned char code PY_mb_xiang []={"乡相香厢湘箱襄镶详祥翔享响想向巷项象像橡"};
 unsigned char code PY_mb_xiao []={"宵消萧硝销霄霄霄小晓孝肖哮效校笑啸"};

unsigned char code PY_mb_xie []={"些楔歇蝎协邪胁斜谐携鞋写泄泻卸屑械谢懈蟹"};
 unsigned char code PY_mb_xin []={"心忻芯辛欣铎新薪信衅"};
 unsigned char code PY_mb_xing []={"兴星惺猩腥刑邢形型醒杏姓幸性"};
 unsigned char code PY_mb_xiong []={"凶兄匈汹胸雄熊"};
 unsigned char code PY_mb_xiu []={"宿休修肴朽秀绣袖锈嗅"};
 unsigned char code PY_mb_xu []={"戌须虚嘘需墟徐许旭序叙恤绪续酗婿絮蓄吁"};
 unsigned char code PY_mb_xuan []={"轩宣喧玄悬旋选癣绚眩"};
 unsigned char code PY_mb_xue []={"削靴薛穴学雪血"};
 unsigned char code PY_mb_xun []={"勋熏寻巡旬驯询循训讯汛迅逊殉"};
 unsigned char code PY_mb_ya []={"丫压呀押鸦鸭牙芽蚜崖涯衙哑雅亚讶"};
 unsigned char code PY_mb_yan []={"咽烟淹焉阉延严言岩沿炎研盐阎蜒颜奄衍掩眼演厌彦砚唁宴艳
 验谚堰焰雁燕"};
 unsigned char code PY_mb_yang []={"央殃秧鸯扬羊阳杨佯疡洋仰养氧痒样漾"};
 unsigned char code PY_mb_yao []={"饶妖腰邀尧姚窑瑶遥瑶咬药要耀钥"};
 unsigned char code PY_mb_ye []={"椰噎爷耶也冶野业叶曳页夜掖液腋"};
 unsigned char code PY_mb_yi []={"一伊衣医依依壹揖仪夷沂宜姨胰移遗颐疑彝乙已矣蚁倚椅义亿
 忆艺议亦屹异役抑译邑易经诣疫益诘翌逸意溢肄裔毅翼臆"};
 unsigned char code PY_mb_yin []={"因姻茵荫音殷吟寅淫银尹引饮隐印"};
 unsigned char code PY_mb_ying []={"应英婴纓纓鹰迎盈莹莹莹莹蝇赢颖影映硬"};
 unsigned char code PY_mb_yo []={"哟"};
 unsigned char code PY_mb_yong []={"佣拥雍雍雍永咏泳勇涌惠蛹踊用"};
 unsigned char code PY_mb_you []={"优忧幽悠尤由犹邮油铀游友有酉又右幼佑诱釉"};
 unsigned char code PY_mb_yu []={"迂淤渝于予余盂鱼俞娱渔隅愉逾愚榆虞舆与宇屿羽雨禹语玉驭
 芋郁狱峪浴预域欲喻寓御裕遇愈誉豫"};
 unsigned char code PY_mb_yuan []={"冤鸳渊元员园垣原袁援缘源猿辕远苑怨院愿"};
 unsigned char code PY_mb_yue []={"约月岳悦阅跃粤越"};
 unsigned char code PY_mb_yun []={"云匀郇耘允陨孕运晕酝韵蕴"};
 unsigned char code PY_mb_za []={"匝杂砸咋"};
 unsigned char code PY_mb_zai []={"灾哉栽宰载再在仔"};
 unsigned char code PY_mb_zan []={"咱攢暂赞"};
 unsigned char code PY_mb_zang []={"脏脏葬"};
 unsigned char code PY_mb_zao []={"遭糟凿早枣蚤澡藻灶灶皂造噪躁躁"};
 unsigned char code PY_mb_ze []={"则择泽责"};
 unsigned char code PY_mb_zei []={"贼"};
 unsigned char code PY_mb_zen []={"怎"};
 unsigned char code PY_mb_zeng []={"增憎赠"};
 unsigned char code PY_mb_zha []={"渣渣扎札轧闸侧眨乍炸炸榨榨"};
 unsigned char code PY_mb_zhai []={"斋摘宅翟窄债寨"};
 unsigned char code PY_mb_zhan []={"沾毡粘詹瞻斩展盏崭辗占战栈站绽湛蘸"};
 unsigned char code PY_mb_zhang []={"长张章彰漳樟涨掌丈仗帐杖胀账障漳"};
 unsigned char code PY_mb_zhao []={"招招找沼召兆赵照罩肇爪"};
 unsigned char code PY_mb_zhe []={"遮折哲蜇撤者锗这浙蔗着"};
 unsigned char code PY_mb_zhen []={"贞针侦珍真砧斟甄臻诊枕疹阵振镇震帧"};
 unsigned char code PY_mb_zheng []={"争征怔挣狰睁蒸拯整正证郑政症"};
 unsigned char code PY_mb_zhi []={"之支汁芝吱枝知织肢脂蜘执侄直值职植殖止只旨址纸指趾至志制

帜治炙质峙挚秩致掷痔窒智滞稚置");

unsigned char code PY_mb_zhong []={"中忠终盅钟衷肿种仲众重"};

unsigned char code PY_mb_zhou []={"州舟洵周洲粥轴肘帚咒宙昼皱骤"};

unsigned char code PY_mb_zhu []={"朱诛株殊诸猪蛛竹烛逐主拄煮嘱瞩住助注贮驻柱祝著蛀筑铸"};

unsigned char code PY_mb_zhua []={"抓"};

unsigned char code PY_mb_zhuai []={"拽"};

unsigned char code PY_mb_zhuan []={"专砖转撰篆"};

unsigned char code PY_mb_zhuang []={"妆庄桩装壮状幢撞"};

unsigned char code PY_mb_zhui []={"追椎锥坠缀赘"};

unsigned char code PY_mb_zhun []={"谆准"};

unsigned char code PY_mb_zhuo []={"卓拙捉桌灼茁浊酌啄琢"};

unsigned char code PY_mb_zi []={"孜兹咨姿资淄滋籽子紫滓字自渍"};

unsigned char code PY_mb_zong []={"宗综棕踪鬃总纵"};

unsigned char code PY_mb_zou []={"邹走奏揍"};

unsigned char code PY_mb_zu []={"租足卒族诅阻组祖"};

unsigned char code PY_mb_zuan []={"赚纂钻"};

unsigned char code PY_mb_zui []={"嘴最罪醉"};

unsigned char code PY_mb_zun []={"尊遵"};

unsigned char code PY_mb_zuo []={"昨左佐作坐座做"};

//=====

/* 拼音输入法查询码表 */

unsigned char code PY_index_a[][8]={

{ " ", 0x00, 0x00 },
{ "i", 0x05, 0x00 },
{ "n", 0x20, 0x00 },
{ "ng", 0x33, 0x00 },
{ "o", 0x3A, 0x00 };

unsigned char code PY_index_b[][8]={

{ "a", 0x4D, 0x00 },
{ "ai", 0x70, 0x00 },
{ "an", 0x81, 0x00 },
{ "ang", 0xA0, 0x00 },
{ "ao", 0xB9, 0x00 },
{ "ei", 0xDE, 0x00 },
{ "en", 0xFD, 0x00 },
{ "eng", 0x08, 0x01 },
{ "i", 0x15, 0x01 },
{ "ian", 0x44, 0x01 },
{ "iao", 0x5D, 0x01 },
{ "ie", 0x66, 0x01 },
{ "in", 0x6F, 0x01 },
{ "ing", 0x7C, 0x01 },

```

        {"o"    ",0x8F,0x01},
        {"u"    ",0xB8,0x01}};

unsigned char code PY_index_c[][8]={
        {"a"    ",0xCD,0x01},
        {"ai"   ",0xD0,0x01},
        {"an"   ",0xE7,0x01},
        {"ang"  ",0xF6,0x01},
        {"ao"   ",0x01,0x02},
        {"e"    ",0x0C,0x02},
        {"eng"  ",0x17,0x02},
        {"ha"   ",0x1E,0x02},
        {"hai"  ",0x37,0x02},
        {"han"  ",0x3E,0x02},
        {"hang" ",0x53,0x02},
        {"hao"  ",0x6C,0x02},
        {"he"   ",0x81,0x02},
        {"hen"  ",0x8E,0x02},
        {"heng" ",0xA3,0x02},
        {"hi"   ",0xC2,0x02},
        {"hong" ",0xE1,0x02},
        {"hou"  ",0xEC,0x02},
        {"hu"   ",0x05,0x03},
        {"huai" ",0x28,0x03},
        {"huan" ",0x2B,0x03},
        {"huang" ",0x3A,0x03},
        {"hui"  ",0x45,0x03},
        {"hun"  ",0x50,0x03},
        {"huo"  ",0x5F,0x03},
        {"i"    ",0x62,0x03},
        {"ong"  ",0x7B,0x03},
        {"ou"   ",0x88,0x03},
        {"u"    ",0x8B,0x03},
        {"uan"  ",0x94,0x03},
        {"ui"   ",0x9B,0x03},
        {"un"   ",0xAC,0x03},
        {"uo"   ",0xB3,0x03}};

```

```

unsigned char code PY_index_d[][8]={
        {"a"    ",0xC0,0x03},
        {"ai"   ",0xCD,0x03},
        {"an"   ",0xE6,0x03},
        {"ang"  ",0x05,0x04},
        {"ao"   ",0x10,0x04},
        {"e"    ",0x29,0x04},
        {"eng"  ",0x30,0x04},
        {"i"    ",0x3F,0x04},

```



```

        {"ian"  ",0x64,0x04},
        {"iao"  ",0x85,0x04},
        {"ie"   ",0x96,0x04},
        {"ing"  ",0xA5,0x04},
        {"iu"   ",0xB8,0x04},
        {"ong"  ",0xBB,0x04},
        {"ou"   ",0xD0,0x04},
        {"u"    ",0xE1,0x04},
        {"uan"  ",0xFE,0x04},
        {"ui"   ",0x0B,0x05},
        {"un"   ",0x14,0x05},
        {"uo"   ",0x25,0x05}};

unsigned char code PY_index_e[][8]={
        {""     ",0x3E,0x05},
        {"n"    ",0x59,0x05},
        {"r"    ",0x5C,0x05}};

unsigned char code PY_index_f[][8]={
        {"a"     ",0x6D,0x05},
        {"an"    ",0x7E,0x05},
        {"ang"   ",0xA1,0x05},
        {"ei"    ",0xB8,0x05},
        {"en"    ",0xD1,0x05},
        {"eng"   ",0xF0,0x05},
        {"o"     ",0x0F,0x06},
        {"ou"    ",0x12,0x06},
        {"u"     ",0x15,0x06}};

unsigned char code PY_index_g[][8]={
        {"a"     ",0x6E,0x06},
        {"ai"    ",0x73,0x06},
        {"an"    ",0x80,0x06},
        {"ang"   ",0x97,0x06},
        {"ao"    ",0xAA,0x06},
        {"e"     ",0xBF,0x06},
        {"ei"    ",0xE2,0x06},
        {"en"    ",0xE5,0x06},
        {"eng"   ",0xEA,0x06},
        {"ong"   ",0xF9,0x06},
        {"ou"    ",0x18,0x07},
        {"u"     ",0x2B,0x07},
        {"ua"    ",0x50,0x07},
        {"uai"   ",0x5D,0x07},
        {"uan"   ",0x64,0x07},
        {"uang"  ",0x7B,0x07},
        {"ui"    ",0x82,0x07},
        {"un"    ",0xA3,0x07},

```

```

        {"uo"    ",0xAA,0x07}};

unsigned char code PY_index_h[][8]={
    {"a"      ",0xB7,0x07},
    {"ai"     ",0xBC,0x07},
    {"an"     ",0xCB,0x07},
    {"ang"    ",0xF2,0x07},
    {"ao"     ",0xF9,0x07},
    {"e"      ",0x0C,0x08},
    {"ei"     ",0x2F,0x08},
    {"en"     ",0x34,0x08},
    {"eng"    ",0x3D,0x08},
    {"ong"    ",0x48,0x08},
    {"ou"     ",0x5B,0x08},
    {"u"      ",0x6A,0x08},
    {"ua"     ",0x8F,0x08},
    {"uai"    ",0xA2,0x08},
    {"uan"    ",0xAD,0x08},
    {"uang"   ",0xCA,0x08},
    {"ui"     ",0xE7,0x08},
    {"un"     ",0x12,0x09},
    {"uo"     ",0x1F,0x09}};

unsigned char code PY_index_j[][8]={
    {"i"      ",0x34,0x09},
    {"ia"     ",0xA1,0x09},
    {"ian"    ",0xC6,0x09},
    {"iang"   ",0x15,0x0A},
    {"iao"    ",0x30,0x0A},
    {"ie"     ",0x69,0x0A},
    {"in"     ",0x9C,0x0A},
    {"ing"    ",0xC5,0x0A},
    {"iong"   ",0xF8,0x0A},
    {"iu"     ",0xFD,0x0A},
    {"u"      ",0x20,0x0B},
    {"uan"    ",0x55,0x0B},
    {"ue"     ",0x64,0x0B},
    {"un"     ",0x77,0x0B}};

unsigned char code PY_index_k[][8]={
    {"a"      ",0x8E,0x0B},
    {"ai"     ",0x95,0x0B},
    {"an"     ",0xA0,0x0B},
    {"ang"    ",0xAF,0x0B},
    {"ao"     ",0xBE,0x0B},
    {"e"      ",0xC7,0x0B},
    {"en"     ",0xE6,0x0B},
    {"eng"    ",0xEF,0x0B},

```

```

        {"ong"  ",0xF4,0x0B},
        {"ou"   ",0xFD,0x0B},
        {"u"    ",0x06,0x0C},
        {"ua"   ",0x15,0x0C},
        {"uai"  ",0x20,0x0C},
        {"uan"  ",0x29,0x0C},
        {"uang" ",0x2E,0x0C},
        {"ui"   ",0x3F,0x0C},
        {"un"   ",0x56,0x0C},
        {"uo"   ",0x5F,0x0C}};

```

unsigned char code PY_index_l[][8]={

```

        {"a"    ",0x68,0x0C},
        {"ai"   ",0x77,0x0C},
        {"an"   ",0x7E,0x0C},
        {"ang"  ",0x9D,0x0C},
        {"ao"   ",0xAC,0x0C},
        {"e"    ",0xBF,0x0C},
        {"ei"   ",0xC6,0x0C},
        {"eng"  ",0xDD,0x0C},
        {"i"    ",0xE4,0x0C},
        {"ian"  ",0x29,0x0D},
        {"iang" ",0x46,0x0D},
        {"iao"  ",0x5F,0x0D},
        {"ie"   ",0x78,0x0D},
        {"in"   ",0x83,0x0D},
        {"ing"  ",0x9C,0x0D},
        {"iu"   ",0xB9,0x0D},
        {"ong"  ",0xD0,0x0D},
        {"ou"   ",0xE3,0x0D},
        {"u"    ",0xF0,0x0D},
        {"uan"  ",0x19,0x0E},
        {"ue"   ",0x26,0x0E},
        {"un"   ",0x2B,0x0E},
        {"uo"   ",0x3A,0x0E},
        {"v"    ",0x53,0x0E}};

```

unsigned char code PY_index_m[][8]={

```

        {"a"    ",0x70,0x0E},
        {"ai"   ",0x83,0x0E},
        {"an"   ",0x90,0x0E},
        {"ang"  ",0xA3,0x0E},
        {"ao"   ",0xB0,0x0E},
        {"e"    ",0xC9,0x0E},
        {"ei"   ",0xCC,0x0E},
        {"en"   ",0xED,0x0E},
        {"eng"  ",0xF4,0x0E},

```

```

        {"i"    ",0x05,0x0F},
        {"ian"  ",0x22,0x0F},
        {"iao"  ",0x35,0x0F},
        {"ie"   ",0x46,0x0F},
        {"in"   ",0x4B,0x0F},
        {"ing"  ",0x58,0x0F},
        {"iu"   ",0x65,0x0F},
        {"o"    ",0x68,0x0F},
        {"ou"   ",0x8D,0x0F},
        {"u"    ",0x94,0x0F}
    };

    unsigned char code PY_index_n[][8]={
        {"a"    ",0xB3,0x0F},
        {"ai"   ",0xC2,0x0F},
        {"an"   ",0xCD,0x0F},
        {"ang"  ",0xD4,0x0F},
        {"ao"   ",0xD7,0x0F},
        {"e"    ",0xE2,0x0F},
        {"ei"   ",0xE5,0x0F},
        {"en"   ",0xEA,0x0F},
        {"eng"  ",0xED,0x0F},
        {"i"    ",0xF0,0x0F},
        {"ian"  ",0x07,0x10},
        {"iang" ",0x16,0x10},
        {"iao"  ",0x1B,0x10},
        {"ie"   ",0x20,0x10},
        {"in"   ",0x2F,0x10},
        {"ing"  ",0x32,0x10},
        {"iu"   ",0x3F,0x10},
        {"ong"  ",0x48,0x10},
        {"u"    ",0x51,0x10},
        {"uan"  ",0x58,0x10},
        {"ue"   ",0x5B,0x10},
        {"uo"   ",0x60,0x10},
        {"v"    ",0x69,0x10}
    };

    unsigned char code PY_index_o[][8]={
        {""      ",0x6C,0x10},
        {"u"     ",0x6F,0x10}
    };

    unsigned char code PY_index_p[][8]={
        {"a"      ",0x7E,0x10},
        {"ai"     ",0x8D,0x10},
        {"an"     ",0x9A,0x10},
        {"ang"    ",0xAB,0x10},
        {"ao"     ",0xB6,0x10},
        {"ei"     ",0xC5,0x10},
        {"en"     ",0xD8,0x10},

```

```

        {"eng"  ",0xDD,0x10},
        {"i"    ",0xFA,0x10},
        {"ian"  ",0x1F,0x11},
        {"iao"  ",0x28,0x11},
        {"ie"   ",0x31,0x11},
        {"in"   ",0x36,0x11},
        {"ing"  ",0x41,0x11},
        {"o"    ",0x54,0x11},
        {"ou"   ",0x65,0x11},
        {"u"    ",0x68,0x11}};

unsigned char code PY_index_q[][8]={
        {"i"    ",0x87,0x11},
        {"ia"   ",0xD0,0x11},
        {"ian"  ",0xD7,0x11},
        {"iang" ",0x04,0x12},
        {"iao"  ",0x15,0x12},
        {"ie"   ",0x34,0x12},
        {"in"   ",0x3F,0x12},
        {"ing"  ",0x56,0x12},
        {"iong" ",0x71,0x12},
        {"iu"   ",0x76,0x12},
        {"u"    ",0x87,0x12},
        {"uan"  ",0xA2,0x12},
        {"ue"   ",0xB9,0x12},
        {"un"   ",0xCA,0x12}};

unsigned char code PY_index_r[][8]={
        {"an"   ",0xCF,0x12},
        {"ang"  ",0xD8,0x12},
        {"ao"   ",0xE3,0x12},
        {"e"    ",0xEA,0x12},
        {"en"   ",0xEF,0x12},
        {"eng"  ",0x04,0x13},
        {"i"    ",0x09,0x13},
        {"ong"  ",0x0C,0x13},
        {"ou"   ",0x21,0x13},
        {"u"    ",0x28,0x13},
        {"uan"  ",0x3D,0x13},
        {"ui"   ",0x42,0x13},
        {"un"   ",0x49,0x13},
        {"uo"   ",0x4E,0x13}};

unsigned char code PY_index_s[][8]={
        {"a"    ",0x53,0x13},
        {"ai"   ",0x5A,0x13},
        {"an"   ",0x63,0x13},
        {"ang"  ",0x6C,0x13},

```

```

{"ao"  ",0x73,0x13},
{"e"   ",0x7C,0x13},
{"en"  ",0x83,0x13},
{"eng" ",0x86,0x13},
{"ha"  ",0x89,0x13},
{"hai" ",0x9C,0x13},
{"han" ",0xA1,0x13},
{"hang",0xC4,0x13},
{"hao" ",0xD5,0x13},
{"he"  ",0xEC,0x13},
{"hen" ",0x05,0x14},
{"heng",0x28,0x14},
{"hi"  ",0x3F,0x14},
{"hou" ",0xA0,0x14},
{"hu"  ",0xB5,0x14},
{"hua" ",0xF8,0x14},
{"huai",0xFD,0x14},
{"huan",0x06,0x15},
{"huang",0x0B,0x15},
{"hui" ",0x12,0x15},
{"hun" ",0x1B,0x15},
{"huo" ",0x24,0x15},
{"i"   ",0x2D,0x15},
{"ong" ",0x4C,0x15},
{"ou"  ",0x5D,0x15},
{"u"   ",0x66,0x15},
{"uan" ",0x7D,0x15},
{"ui"  ",0x84,0x15},
{"un"  ",0x9B,0x15},
{"uo"  ",0xA2,0x15}};

```

unsigned char code PY_index_t[][8]={

```

{"a"   ",0xB3,0x15},
{"ai"  ",0xC6,0x15},
{"an"  ",0xD9,0x15},
{"ang" ",0xFE,0x15},
{"ao"  ",0x19,0x16},
{"e"   ",0x30,0x16},
{"eng" ",0x33,0x16},
{"i"   ",0x3C,0x16},
{"ian" ",0x5B,0x16},
{"iao" ",0x6C,0x16},
{"ie"  ",0x79,0x16},
{"ing" ",0x80,0x16},
{"ong" ",0x95,0x16},
{"ou"  ",0xB0,0x16},

```

```

        {"u"      "0xB9,0x16},
        {"uan"    "0xD0,0x16},
        {"ui"     "0xD5,0x16},
        {"un"     "0xE2,0x16},
        {"uo"     "0xEB,0x16}};

unsigned char code PY_index_w[][8]={
        {"a"      "0x02,0x17},
        {"ai"     "0x11,0x17},
        {"an"     "0x16,0x17},
        {"ang"    "0x39,0x17},
        {"ei"     "0x4E,0x17},
        {"en"     "0x91,0x17},
        {"eng"    "0xA6,0x17},
        {"o"      "0xAD,0x17},
        {"u"      "0xC0,0x17}};

unsigned char code PY_index_x[][8]={
        {"i"      "0xFB,0x17},
        {"ia"     "0x40,0x18},
        {"ian"    "0x59,0x18},
        {"iang"   "0x90,0x18},
        {"iao"    "0xB9,0x18},
        {"ie"     "0xDC,0x18},
        {"in"     "0x05,0x19},
        {"ing"    "0x1A,0x19},
        {"iong"   "0x37,0x19},
        {"iu"     "0x46,0x19},
        {"u"      "0x5B,0x19},
        {"uan"    "0x82,0x19},
        {"ue"     "0x97,0x19},
        {"un"     "0xA6,0x19}};

unsigned char code PY_index_y[][8]={
        {"a"      "0xC3,0x19},
        {"an"     "0xE4,0x19},
        {"ang"    "0x27,0x1A},
        {"ao"     "0x4A,0x1A},
        {"e"      "0x6D,0x1A},
        {"i"      "0x8C,0x1A},
        {"in"     "0xF7,0x1A},
        {"ing"    "0x18,0x1B},
        {"o"      "0x3D,0x1B},
        {"ong"    "0x40,0x1B},
        {"ou"     "0x5F,0x1B},
        {"u"      "0x88,0x1B},
        {"uan"    "0xE1,0x1B},
        {"ue"     "0x0A,0x1C},

```

```

        {"un  ",0x1D,0x1C}};
unsigned char code PY_index_z[][8]={
        {"a  ",0x36,0x1C},
        {"ai  ",0x3F,0x1C},
        {"an  ",0x50,0x1C},
        {"ang ",0x59,0x1C},
        {"ao  ",0x60,0x1C},
        {"e   ",0x7D,0x1C},
        {"ei  ",0x86,0x1C},
        {"en  ",0x89,0x1C},
        {"eng ",0x8C,0x1C},
        {"ha  ",0x93,0x1C},
        {"hai ",0xAE,0x1C},
        {"han ",0xBD,0x1C},
        {"hang",0xE0,0x1C},
        {"hao ",0x01,0x1D},
        {"he  ",0x18,0x1D},
        {"hen ",0x2F,0x1D},
        {"heng",0x52,0x1D},
        {"hi  ",0x6F,0x1D},
        {"hong",0xC6,0x1D},
        {"hou ",0xDD,0x1D},
        {"hu  ",0xFA,0x1D},
        {"hua ",0x2F,0x1E},
        {"huai",0x32,0x1E},
        {"huan",0x35,0x1E},
        {"huang",0x40,0x1E},
        {"hui  ",0x51,0x1E},
        {"hun  ",0x5E,0x1E},
        {"huo  ",0x63,0x1E},
        {"i    ",0x78,0x1E},
        {"ong  ",0x95,0x1E},
        {"ou   ",0xA4,0x1E},
        {"u    ",0xAD,0x1E},
        {"uan  ",0xBE,0x1E},
        {"ui   ",0xC5,0x1E},
        {"un   ",0xCE,0x1E},
        {"uo   ",0xD3,0x1E}};
unsigned char code PY_index_end[][8]={
        "    ",0,0};

```

/*变量声明*/

unsigned int code py_mb_begin=(unsigned int)&PY_mb_a[0];

unsigned char code (* code PY_index_pointer[27])[8]={


```

PY_index_a,PY_index_b,PY_index_c,PY_index_d,PY_index_e,PY_index_f,PY_index_g,PY_index_h,
PY_index_j,PY_index_j,PY_index_k,PY_index_l,PY_index_m,PY_index_n,PY_index_o,PY_index_p,
PY_index_q,PY_index_r,PY_index_s,PY_index_t,PY_index_w,PY_index_w,PY_index_w,PY_index_x,
PY_index_y,PY_index_z,PY_index_end};

/* 函数声明*/
/*"拼音输入法查询函数, input_py 为已输入的拼音码, 返回值是中文的起始地址, 当为 0 时, 查询失
败"
unsigned char code * py_ime(unsigned char input_py_val[]);

/*主程序*/

unsigned char code * py_ime(unsigned char input_py_val[])
{
    unsigned char code (* xdata p1)[8],(* xdata p2)[8],(* xdata p3)[8];
    unsigned char xdata i=1;
    if (input_py_val[0]==0) return(0);
                                /*"如果输入空字符返回 0"
    if (input_py_val[0]=='i') return(0);
    if (input_py_val[0]=='u') return(0);
    if (input_py_val[0]=='v') return(0);
    p1=p2=PY_index_pointer[input_py_val[0]-0x61];
                                /*"计算入口树根"
    p3=PY_index_pointer[input_py_val[0]-0x60];
                                /*"设置指针界限"
    if (p1==0) return(0);

                                /*"查询失败返回 0"
    while (p1<p3) if ((*p1)[0]==input_py_val[1]) {p2=p1;break;} else p1++;
                                /*"查询第二个拼音"
    p1=p2;
    while (p1<p3)

                                /*"阶梯法查询余下拼音"
        if (((*p1)[i]==input_py_val[i+1])&&((*p1)[i-1]==input_py_val[i]))
        {
            p2=p1;
            i++;
        }
        else p1++;
    return((unsigned char code *)((*p2)[6]+(*p2)[7]*256+py_mb_begin));    /*"返回查询结果首地址"
}

```

程序三十五 串行口代码

```
#include "s_uart.h"
#include "init.h"
#include "stdio.h"

sbit WDT=0x90^4;    //P1.4
void sputch(char ch);
char sgetch(void);
void sputs(char code *str);
void main()
{
    code char HEX[]="0123456789ABCDEF";
    unsigned char i;

    SimuUARTinit();
    //serial_baud_19200;
    TL0=0XFD;
    while(1)
    {
        sputch(sgetch());
        //sputs(HEX);
        //sputch(sgetch());
        /*
        sputch(HEX[TL0>>4]);
        sputch(HEX[TL0&0x0f]);
        sputch(' ');
        */
        WDT=!WDT;
    }
}

void sputch(char ch)
{
    while(!tTI);
    tSBUF=ch;
    tTI=0;
}

void sputs(char code *str)
{
    while(*str)
    {
```

```

        if(rRI) sputch(sgetch());
        sputch(*str++);
    }
}
char sgetch(void)
{
    while(!rRI) WDT=!WDT;;
    rRI=0;
    return(rSBUF);
}

```

INIT.H 代码

```

//Timer/Counter initialize
#define timer0_13bit          TMOD&=0xf0
#define timer0_16bit          TMOD&=0xf0;TMODl=0x01
#define timer0_auto_reload    TMOD&=0xf0;TMODl=0x02
#define timer1_13bit          TMOD&=0x0f
#define timer1_16bit          TMOD&=0x0f;TMODl=0x10
#define timer1_auto_reload    TMOD&=0x0f;TMODl=0x20
#define timer2_auto_reload    CP_RL=0;
#define timer2_capture        CP_RL=1;

#define timer2_extern_enable   EXEN2=1;
#define timer2_extern_disable EXEN2=0;

#define timer0_stop           TR0=0
#define timer1_stop           TR1=0
#define timer2_stop           TR2=0;

#define timer0_start          TR0=1
#define timer1_start          TR1=1
#define timer2_start          TR2=1;

#define enable()              EA=1
#define disable()             EA=0

#define int_timer0()          TF0=1
#define int_timer1()          TF1=1
#define int_timer2()          TF2=1

#define int1_priority_high     PX1=1
#define int1_priority_low      PX1=0

#define int0_priority_high     PX0=1

```

```

#define int0_priority_low      PX0=0

#define serial_priority_high   PS=1
#define serial_priority_low    PS=0

#define serial1_priority_high  PS1=1
#define serial1_priority_low   PS1=0

#define timer0_priority_high   PT0=1
#define timer0_priority_low    PT0=0

#define timer2_priority_high   PT2=1
#define timer2_priority_low    PT2=0

#define int0_falling_edge      IT0=1
#define int1_falling_edge      IT1=1

#define int0_int_enable        EX0=1
#define int1_int_enable        EX1=1
#define timer0_int_enable      ET0=1
#define timer1_int_enable      ET1=1
#define timer2_int_enable      ET2=1
#define serial_int_enable      ES=1
#define serial1_int_enable     ES1=1

#define int0_int_disable       EX0=0
#define int1_int_disable       EX1=0
#define timer0_int_disable     ET0=0
#define timer1_int_disable     ET1=0
#define timer2_int_disable     ET2=0
#define serial_int_disable     ES=0
#define serial1_int_disable    ES1=0

//Define the baud rate generator
#define serial_baud_double     PCON=PCON|0x80;
//The following is both of the two serial port,use the same baud rate
#define serial_baud_1200       TMOD&=0x0f;TMOD|=0x20;TH1=0xe8;TR1=1
#define serial_baud_2400       TMOD&=0x0f;TMOD|=0x20;TH1=0xf4;TR1=1
#define serial_baud_4800       TMOD&=0x0f;TMOD|=0x20;TH1=0xfa;TR1=1
#define serial_baud_9600       TMOD&=0x0f;TMOD|=0x20;TH1=0xfd;TR1=1
#define serial_baud_19200      TMOD&=0x0f;TMOD|=0x20;TH1=0xfd;TR1=1;PCON=PCON|0x80

//The follwing is serial port use differant baud rate
//

```

```

//          OSC          OSC=11.0592          345600
//          T2 = 0      -  -----  ===== 0 -  -----
//          32 * BAUD_RATE          BAUD_RATE
//
//
#define serial0_baud_1200    T2CON=0x34;RCAP2H=0xfe;RCAP2L=0xe0
#define serial0_baud_2400    T2CON=0x34;RCAP2H=0xff;RCAP2L=0x70
#define serial0_baud_4800    T2CON=0x34;RCAP2H=0xff;RCAP2L=0xb8
#define serial0_baud_9600    T2CON=0x34;RCAP2H=0xff;RCAP2L=0xdc
#define serial0_baud_19200    T2CON=0x34;RCAP2H=0xff;RCAP2L=0xee

#define serial1_baud_1200    TMOD&=0x0f;TMOD|=0x20;TH1=0xe8;TR1=1
#define serial1_baud_2400    TMOD&=0x0f;TMOD|=0x20;TH1=0xf4;TR1=1
#define serial1_baud_4800    TMOD&=0x0f;TMOD|=0x20;TH1=0xfa;TR1=1
#define serial1_baud_9600    TMOD&=0x0f;TMOD|=0x20;TH1=0xfd;TR1=1
#define serial1_baud_19200    TMOD&=0x0f;TMOD|=0x20;TH1=0xfd;TR1=1;WDCON=WDCON|0x80

#define serial_uart8          SM0=0;SM1=1;SM2=0
#define serial0_uart8          SM0=0;SM1=1;SM2=0
#define serial1_uart8          SM0_1=0;SM1_1=1;SM2_1=0
#define serial_uart9          SM0=1;SM1=1;TR1=1

#define serial_receive_enable  REN=1
#define serial0_receive_enable REN=1
#define serial1_receive_enable REN_1=1

#define timer2_speed_3          CKCON|=0x20
#define timer1_speed_3          CKCON|=0x10
#define timer0_speed_3          CKCON|=0x08

// #define use_inter_SRAM        PMR|=0x05
// #define use_extern_PORT        PMR&=(0x05^0xff)
#define use_inter_SRAM          PMR|=0x01
#define use_extern_PORT          PMR&=(0x01^0xff)
// #define ALE_out_disable        PMR|=0x04
// #define ALE_out_enable          PMR&=(0x04^0xff)

#define movx_ins_9              CKCON|=0x03;

```

UART.H 代码

```
#include "REG51.h"
```

```

//-----
#define BAUD_RATE 2400
// #define BAUD_RATE 1200

```

```

#define RELOAD(TIMER,SVALUE)  TIMER+=SVALUE+1
sbit  tTXD=P3^7;
sbit  rRXD=P3^2;

bit   tTI;
bit   rRI;

data  unsigned char rSBUF;
data  unsigned char tSBUF;

data  unsigned char rSBUF0;
data  unsigned char RxdCnt;
data  unsigned char rSmpCnt;

data  unsigned char TxdCnt;
data  unsigned char tSmpCnt;
//-----

void SimuUARTinit(void)
{
    tTI=1;
    tTXD=1;
    rRXD=1;

    TMOD&=0xf0;
    TMOD|=0x03;
    ET0=1;
    ET1=1;
    TR0=1;
    IT0=1;
    EX0=1;
    TR1=1;
    EA=1;
}
//-----
void IntTH0(void)  interrupt 3
{
    #if BAUD_RATE==2400
        RELOAD(TH0,0x80);
    #endif
    if(tSmpCnt-- == 0)
    {
        tSmpCnt=2;
    }
}

```

```

        if(!TI) return;
        switch(TxdCnt++)
        {
            case 0:
                tTXD=0;
                break;
            case 9:
                tTXD=1;
                tTI=1;
                TxdCnt=0;
                break;
            default:
                tTXD=tSBUF&0x01;
                tSBUF>>=1;
                break;
        }
    }
}

//-----
void RxdInt0(void) interrupt 0
{
    #if BAUD_RATE==2400
        TL0=0xef;
    #else
        TL0=0xaf;
    #endif
    TF0=0;
    rSmpCnt=1;
}

//-----
void IntTL0(void) interrupt 1
{
    #if BAUD_RATE==2400
        RELOAD(TL0,1);
    #endif
    if(rSmpCnt-- == 0)
    {
        rSmpCnt=2;
        switch(RxdCnt++)
        {
            case 0:
                if(rRXD==1) RxdCnt=0;
                break;
            default:
                if(RxdCnt>9){ RxdCnt=0;return;}
        }
    }
}

```

```

        rSBUF0>>=1;
        rSBUF0|=rRXD?0x80:0;
        break;
    case 9:
        RxdCnt=0;
        if(rRXD==0) return;
        rSBUF=rSBUF0;
        rRI=1;
    }
}
//-----

```

程序三十六 蛇游戏代码

```

#include "w77e58.h"
#include "vt100.h"
#include "init.h"
#include "stdio.h"
#include "stdlib.h"

typedef struct {
    unsigned char x;
    unsigned char y;
}SnakeType;

#define UP 0xf0
#define DOWN 0x0f
//-----
#define LEFT 0x00
#define RIGHT 0xff

#define L_SHEET (2*2+1)
#define R_SHEET (13*2+1)

#define T_SHEET 5
#define B_SHEET 16
//-----

void init(void);
void delay(unsigned int);
void put_block(char,char);
void hide_block(char,char);

```



```

bit  flash_snake(unsigned char);
void  putch(char);
bit  kbhit(void);
char  getch(void);
void  clr_wchdog(void){}
void  put_bug(void);
void  init_snake(void);
void  putstr(char xdata *);
void  flash_data(void);
void  type_kbd(char code*);
//-----
data  dword    GameScore;
data  dword    HighScore;
data  byte  direct;
data  word     SnakeLen;
xdata  SnakeType snake[400];
xdata  byte  print_buf[20];

data  byte  Level;
data  byte  bug_x,bug_y;

bit  TimerEnd;
bit  GameOver;
bit  GameStart;
bit  blink_flag0;
bit  blink_flag1;
bit  flashed;
bit  type_flag;

//-----
void  main()
{
    data  byte  ch,ch1;
    bit  scoreadd;

    init();
    put_bug();
    while(1)
    {
        REN_1=1;
        if(kbhit())
        {
            ch1=ch;
            ch=getch();
            ch=ch&(0x20^0xff);

```

```

switch(ch)
{
case 'W':
    ch=UP;
    if(~ch == ch1) ch=ch1;
    else {TimerEnd=1;scoreadd=1;}
break;
case 'S':
    ch=DOWN;
    if(~ch == ch1) ch=ch1;
    else {TimerEnd=1;scoreadd=1;}
break;
case 'A':
    ch=LEFT;
    if(~ch == ch1) ch=ch1;
    else {TimerEnd=1;scoreadd=1;}
break;
case 'D':
    ch=RIGHT;
    if(~ch == ch1) ch=ch1;
    else {TimerEnd=1;scoreadd=1;}
break;
case 'V':
    if(GameOver)
    {
        init_snake();
        GameOver=0;
        GameScore=0;
        //GameStart=0;
    }
    if(!GameStart)
    {
        del_line(4);
        gotoxy(L_SHEET,4);
        fputstr("Playing game....");
        del_line(24);
        gotoxy(L_SHEET,24);
        fputstr("Option| UP(&W) DOWN(&S) LEFT(&A) RIGHT(D) &PAUSE");
    }
    GameStart=1;ch=1;
    break;
case 'P':
    if(!GameStart) break;
    while(!kbhit())
    {

```

```

                                gotoxy(L_SHEET,4);
                                type_kbd("Game pause, any key to continue...");
                                del_line(4);
                                }
                                gotoxy(L_SHEET,4);
                                fputstr("Playing game....");
                                break;
case 'N':
    if(!GameStart && Level>0) Level--;
    flash_data();
    ch=1;
    break;
case 'U':
    if(!GameStart && Level<16) Level++;
    flash_data();
default:
    ch=1;
    }
}
if(!GameStart)
{
    del_line(4);
    gotoxy(L_SHEET,4);
    if(GameOver) type_kbd("Game Over,");
    type_kbd("Press Enter to start ");
}
if(!GameOver && GameStart)
{
    if(blink_flag0)
    {
        hide_block(bug_x,bug_y);
        blink_flag0=0;
    }
    if(blink_flag1)
    {
        put_block(bug_x,bug_y);
        blink_flag1=0;
    }
}

if( TimerEnd && GameStart && !GameOver)
{
    TimerEnd=0;
    if(scoreadd)
    {

```

```

        scoreadd=0;
        GameScore += (20+Level*2);
    }
    if(!flash_snake(ch))
    {
        if(GameScore>HighScore) HighScore=GameScore;
        GameScore=0;
        GameOver=1;
        GameStart=0;
        hide_block(bug_x,bug_y);
        del_line(24);
        gotoxy(L_SHEET,24);
        fputstr("Optionl Level &UP/DOW&N");
    }
    flash_data();
}
}
}
//-----
void time_int(void) interrupt 1
{
    static byte tcc;

    TH0=0x80;
    if((tcc&0x03)==0) blink_flag0=1;
    if((tcc&0x03)==2) blink_flag1=1;

    if( ++tcc >= (16-Level+4) )
    {
        tcc=0;
        TimerEnd=!flashed;
        flashed=0;
    }
    if(type_flag ) TimerEnd=1;
}
//-----
void init(void)
{
    char i;
    char xdata*xptr;

    EA=0;
    use_inter_SRAM;
    xptr=(char xdata*)0x400;
    while(xptr) *xptr--=0;

```

```

timer0_16bit;
timer0_start;
ET0=1;
serial_uart8;
REN_1=1;
serial_baud_9600;
TI_1=1;
Level=8;
fputstr("#");
clrscr();
for(i=L_SHEET;i<=R_SHEET;i+=2)
{
    put_block(i,T_SHEET);
    put_block(i,B_SHEET);
}
for(i=T_SHEET+1;i<B_SHEET;i++)
{
    put_block(L_SHEET,i);
    put_block(R_SHEET,i);
}
fputstr("$");
gotoxy(L_SHEET,1);fputstr("#Cupidity snake version 1.00      $");
gotoxy(L_SHEET,2);fputstr("High Score:");
gotoxy(L_SHEET,3);fputstr("Score      :");
gotoxy(L_SHEET+30,2);fputstr("Tuched:");
gotoxy(L_SHEET+30,3);fputstr("Level :");
gotoxy(L_SHEET,24);
fputstr("Option| Level &UP/DOW&N");
snake[0].x=0;
init_snake();
flash_data();
GameStart=0;
GameOver=0;
EA=1;
}
//-----
void init_snake(void)
{
    unsigned char i;

    i=0;
    while(snake[i].x!=0)
    {
        hide_block(snake[i].x,snake[i].y);
        i++;
    }
}

```

```

    }
    for(i=0;i<7;i++)
    {
        snake[i].x=L_SHEET+20;
        snake[i].y=T_SHEET+4+i;
    }
    snake[7].x=0;
    direct=LEFT;
    for(i=0;snake[i].x!=0;i++)
        put_block(snake[i].x,snake[i].y);
    SnakeLen=7;

}

//-----
void put_bug(void)
{
    data byte i,ch;
re_find:
    do{
        ch=rand()%76;
    }while((ch<=L_SHEET || ch>=R_SHEET) || 0==(ch&0x01));
    bug_x=ch;
    do{
        ch=rand()%24;
    }while(ch<=T_SHEET || ch>=B_SHEET);
    for(i=0;snake[i].x!=0;i++)
    {
        if(ch==snake[i].y && bug_x==snake[i].x)
            goto re_find;
    }
    bug_y=ch;
    put_block(bug_x,bug_y);
}

//-----
bit flash_snake(unsigned char dir1)
{
    data unsigned char x0,y0,i,j;

    if(dir1==1) dir1=direct;
    else
    {
        if(direct == ~dir1)    dir1=direct;
        else direct=dir1;
    }

```

```

}
switch(dir1)
{
case UP:
    y0=snake[0].y-1;
    x0=snake[0].x;
    break;
case DOWN:
    y0=snake[0].y+1;
    x0=snake[0].x;
    break;
case LEFT:
    x0=snake[0].x-2;
    y0=snake[0].y;
    break;
case RIGHT:
    x0=snake[0].x+2;
    y0=snake[0].y;
    break;
}
if( (y0<=T_SHEET)||(y0>=B_SHEET)||
(x0<=L_SHEET)||(x0>=R_SHEET))return(0);

for(i=0;snake[i].x!=0;i++)
{
    if(x0==snake[i].x && y0==snake[i].y) return(0);
}
hide_block(x0,y0);
put_block(x0,y0);
hide_block(snake[i-1].x,snake[i-1].y);
if(bug_x==x0 && bug_y==y0)
{
    SnakeLen++;
    GameScore +=(SnakeLen*5 + Level*3);
    if((SnakeLen%15)==0 && Level<16) Level++;
    snake[i].x=snake[i-1].x;
    snake[i].y=snake[i-1].y;
    snake[i+1].x=0;
    i++;

    for(j=0;snake[j].x!=0;j++)
    {
        hide_block(snake[j].x,snake[j].y);
        put_block(snake[j].x,snake[j].y);
    }
}

```

```

        delay(0x200);
    }
    put_bug();
}

while(--i) snake[i]=snake[i-1];
snake[0].x=x0;
snake[0].y=y0;
flashed=1;
return(1);
}
//-----
void put_block(char x,char y)
{
    gotoxy(x,y);
    fputstr("■");
    delay(100);
}
//-----
void hide_block(char x,char y)
{
    gotoxy(x,y);
    fputstr(" ");
}
//-----
bit kbhit(void)
{
    return(RI_1);
}
//-----
void putstr(char xdata*str)
{
    while(*str)
    {
        putchar(*str++);
    }
}
//-----
void flash_data(void)
{
    gotoxy(L_SHEET+sizeof("High Score: "),2);
    sprintf(print_buf,"%-10ld",HighScore);putstr(print_buf);
    gotoxy(L_SHEET+sizeof("High Score: "),3);
    sprintf(print_buf,"%-10ld",GameScore);putstr(print_buf);
    gotoxy(L_SHEET+30+sizeof("Tuched: "),2);

```



```

    sprintf(print_buf,"%3d",SnakeLen);putstr(print_buf);
    gotoxy(L_SHEET+30+sizeof("Tuched: "),3);
    sprintf(print_buf,"%3d",(int)(char)Level);putstr(print_buf);

}
//-----
void type_kbd(char code *str)
{
    data byte ch,i=0;

    type_flag=1;
    flashed=1;
    while( (ch=str[i]) !=0)
    {
        if(kbhit()) break;
        if(TimerEnd)
        {
            TimerEnd=0;
            putch(ch);
            if(ch=="\n") fputstr("\r   ");
            i++;
        }
        clr_wchdog();
    }
    type_flag=0;
}
//-----

```

INIT.H 代码

```

//Timer/Counter initialize
#define timer0_13bit          TMOD&=0xf0
#define timer0_16bit          TMOD&=0xf0;TMOD|=0x01
#define timer0_auto_reload    TMOD&=0xf0;TMOD|=0x02
#define timer1_13bit          TMOD&=0x0f
#define timer1_16bit          TMOD&=0x0f;TMOD|=0x10
#define timer1_auto_reload    TMOD&=0x0f;TMOD|=0x20
#define timer2_auto_reload    CP_RL=0;
#define timer2_capture        CP_RL=1;

#define timer2_extern_enable   EXEN2=1;
#define timer2_extern_disable EXEN2=0;

#define timer0_stop            TR0=0
#define timer1_stop            TR1=0
#define timer2_stop            TR2=0;

```

```

#define timer0_start      TR0=1
#define timer1_start      TR1=1
#define timer2_start      TR2=1;

#define    enable()        EA=1
#define    disable()       EA=0

#define    int_timer0()    TF0=1
#define    int_timer1()    TF1=1
#define    int_timer2()    TF2=1

#define int1_priority_high    PX1=1
#define int1_priority_low     PX1=0

#define int0_priority_high    PX0=1
#define int0_priority_low     PX0=0

#define serial_priority_high  PS=1
#define serial_priority_low   PS=0

#define serial1_priority_high    PS1=1
#define serial1_priority_low     PS1=0

#define timer0_priority_high    PT0=1
#define timer0_priority_low     PT0=0

#define timer2_priority_high    PT2=1
#define timer2_priority_low     PT2=0

#define int0_falling_edge      IT0=1
#define int1_falling_edge      IT1=1

#define int0_int_enable        EX0=1
#define int1_int_enable        EX1=1
#define timer0_int_enable      ET0=1
#define timer1_int_enable      ET1=1
#define timer2_int_enable      ET2=1
#define serial_int_enable      ES=1
#define serial1_int_enable     ES1=1

#define int0_int_disable       EX0=0
#define int1_int_disable       EX1=0
#define timer0_int_disable     ET0=0
#define timer1_int_disable     ET1=0

```

```

#define timer2_int_disable      ET2=0
#define serial_int_disable      ES=0
#define serial1_int_disable     ES1=0

//Define the baud rate generater
#define serial_baud_double      PCON=PCON|0x80;
//The following is both of the two serial port,use the same baud rate
#define serial_baud_1200        TMOD&=0x0f;TMOD|=0x20;TH1=0xe8;TR1=1
#define serial_baud_2400        TMOD&=0x0f;TMOD|=0x20;TH1=0xf4;TR1=1
#define serial_baud_4800        TMOD&=0x0f;TMOD|=0x20;TH1=0xfa;TR1=1
#define serial_baud_9600        TMOD&=0x0f;TMOD|=0x20;TH1=0xfd;TR1=1
#define serial_baud_19200
TMOD&=0x0f;TMOD|=0x20;TH1=0xfd;TR1=1;PCON=PCON|0x80

//The follwing is serial port use differant baud rate
//
//          OSC          OSC=11.0592          345600
//      T2 = 0  -  -----  ===== 0  -  -----
//          32 * BAUD_RATE          BAUD_RATE
//
//
#define serial0_baud_1200      T2CON=0x34;RCAP2H=0xfe;RCAP2L=0xe0
#define serial0_baud_2400      T2CON=0x34;RCAP2H=0xff;RCAP2L=0x70
#define serial0_baud_4800      T2CON=0x34;RCAP2H=0xff;RCAP2L=0xb8
#define serial0_baud_9600      T2CON=0x34;RCAP2H=0xff;RCAP2L=0xdc
#define serial0_baud_19200      T2CON=0x34;RCAP2H=0xff;RCAP2L=0xee

#define serial1_baud_1200      TMOD&=0x0f;TMOD|=0x20;TH1=0xe8;TR1=1
#define serial1_baud_2400      TMOD&=0x0f;TMOD|=0x20;TH1=0xf4;TR1=1
#define serial1_baud_4800      TMOD&=0x0f;TMOD|=0x20;TH1=0xfa;TR1=1
#define serial1_baud_9600      TMOD&=0x0f;TMOD|=0x20;TH1=0xfd;TR1=1
#define serial1_baud_19200      TMOD&=0x0f;TMOD|=0x20;TH1=0xfd;TR1=1;WDCON=WDCON|0x80

#define serial_uart8           SM0=0;SM1=1;SM2=0
#define serial0_uart8          SM0=0;SM1=1;SM2=0
#define serial1_uart8          SM0_1=0;SM1_1=1;SM2_1=0
#define serial_uart9           SM0=1;SM1=1;TR1=1

#define serial_receive_enable   REN=1
#define serial0_receive_enable  REN=1
#define serial1_receive_enable  REN_1=1

#define timer2_speed_3          CKCON|=0x20
#define timer1_speed_3          CKCON|=0x10
#define timer0_speed_3          CKCON|=0x08

```

```

#define use_inter_SRAM          PMRl=0x05
#define use_extern_PORT         PMR&=(0x05^0xff)
#define use_inter_SRAM          PMRl=0x01
#define use_extern_PORT         PMR&=(0x01^0xff)
#define ALE_out_disable         PMRl=0x04
#define ALE_out_enable          PMR&=(0x04^0xff)

#define movx_ins_9              CKCONl=0x03;

```

VT100 代碼

```

//-----
// This module use the function VT100 of terminal, screen controll
// By Dept. OF TECHNOLOGY,Skean
//-----

```

```

#define NORMAL_FONT            0x00
#define THICK_FONT             0x01
#define UNDER_LINE_FONT       0x04
#define REVERSE_VIDEO_FONT     0x07

```

```

#define REVERSE_THICK_FONT     0x17
#define REVERSE_UNDERLINE_FONT 0x47

```

```

#define init_terminal          init_trm

```

```

//-----
void clrscr(void);
void gotoxy(unsigned char x,unsigned char y);
void set_font(unsigned char font);
void del_line(unsigned char line);
void init_trm(unsigned char max_line);
//-----
// '@','#,&','$ and '^' supply the format control
//-----
// '@'= THICK_FONT
// '#'= REVERSE_VIDEO_FONT
// '& '= UNDER_LINE_FONT
// '$'= NORMAL_FONT
// '^'= transferred meaning,

```

```

void fputstr(unsigned char code *code_string);

```

X25045.H 代碼

```

/*****
* X25043/45 application ; Procedures

```

```

*   absolute one address access
*****

WARNING: The function with '_' ahead ,user may not use it.as it
used internal
*/
#define ALONE_COMPILE
#ifdef ALONE_COMPILE
    #include "INTRINS.H"
#endif

#ifndef nop2()
#define nop2()  _nop();_nop();_nop()
#endif

#define WREN    0x06
#define WRD1    0x04
#define RDSR    0x05
#define WRSR    0x01
#define READ    0x03
#define WRITE   0x02

#define _SI0x80
#define _SCK    0x40
#define _SO 0x20
#define _CS 0x10

//-----
#ifndef dword
    #define dword    unsigned long
    #define word    unsigned int
    #define byte    unsigned char
    typedef union{
        word    w;
        byte    bh;
        byte    bl;
    }WordType;

    typedef union{
        dword    dw;
        word w[2];
        byte b[4];
    }DwordType;

#endif

```

```

//-----
sfr PORT = 0x90;
void _w_byte(Data)
    char Data;
{
    char i;

    PORT &= (_SCK^0xff);
    for(i=0;i<8;i++)
    {
        nop2();nop2();
        if(Data & 0x80) PORT |= _SI;
        else PORT &= (_SI^0xff);
        nop2();nop2();
        PORT |= _SCK;
        nop2();nop2();
        Data=Data<<1;
        nop2();nop2();
        PORT &= (_SCK^0xff);
        nop2();nop2();
    }
}
//-----
char _r_byte(void)
{
    char i;
    char result;

    result=0;
    for(i=0;i<8;i++)
    {
        nop2();nop2();
        PORT |= _SCK;
        result=result<<1;
        nop2();nop2();
        if((PORT & _SO) != 0)
            result |= 0x01;
        nop2();nop2();
        PORT &= (_SCK^0xff);
        nop2();nop2();
    }
    return(result);
}
//-----
void write_status(status)

```

```

    char status;
{
    PORT &= (_CS^0xff);
    nop2();nop2();
    _w_byte(status);
    PORT |= _CS;
    nop2();nop2();
    return;
}
//-----
void clr_wchdog(void)
{
    PORT &= (_CS^0xff);
    PORT |= _CS;
}
//-----
void wait_free(void)
{
    unsigned int t;

    t=3000;
    while(--t);
}
//-----
void write_reg(_code)
    char _code;
{
    write_status(WREN);
    PORT &= (_CS^0xff);
    nop2();nop2();
    _w_byte(WRSR);
    _w_byte(_code);
    nop2();nop2();
    PORT |= _CS;
    wait_free();
}
//-----

unsigned char read_byte(address)
    unsigned int address;
{
    char result;

    PORT &= (_CS^0xff);    // Chip select
    nop2();nop2();

```

```

    _w_byte((char)(address>255 ? (0x08|READ): READ));
    _w_byte((char)(address & 0x00ff));
    result=_r_byte();
    nop2();nop2();
    PORT |=_CS;

                                // Chip unselect

    return(result);
}
//-----
void write_byte(address,Data)
    unsigned int address;
    char Data;
{
    write_status(WREN);
    nop2();nop2();
    PORT &= (_CS^0xff);
    nop2();nop2();
    _w_byte((unsigned char)(address>255 ? (0x08|WRITE): WRITE));
    _w_byte((unsigned char)(address & 0x00ff));
    _w_byte(Data);
    nop2();nop2();
    PORT |=_CS;
    wait_free();
    return;
}
/*
//-----
unsigned long  read_data(format,address)
    unsigned char  format;
    unsigned int   address;
{
    DwordType  result;

    switch(format&0xdf)
    {
    case 'L':
        result.b[0]=read_byte(address);
        result.b[1]=read_byte(address+1);
        result.b[2]=read_byte(address+2);
        result.b[3]=read_byte(address+3);
        break;
    case 'D':
        result.b[2]=read_byte(address);
        result.b[3]=read_byte(address+1);
        break;
    }
}

```



```

        case 'C':
            result.b[3]=read_byte(address);
            break;
    }
    return(result.dw);
}
//-----
void write_data(format,address,Data)
    unsigned char format;
    unsigned int address;
    DwordType data* Data;
{
    switch(format&0xdf)
    {
        case 'L':
            write_byte(address , Data->b[0]);
            write_byte(address+1, Data->b[1]);
            write_byte(address+2, Data->b[2]);
            write_byte(address+3, Data->b[3]);
            break;
        case 'D':
            write_byte(address , Data->b[2]);
            write_byte(address+1, Data->b[3]);
            break;
        case 'C':
            write_byte(address , Data->b[3]);
            break;
    }
}
}

```

程序三十七 与液晶模块 T6963C 连接代码

```

/* LCM (MGLS-240128TA) 显示程序    */
/* 时钟频率: 22.1184 MHz            */
/* 接口方式: 直接接口方式          */
/* 开发环境: Keil C51 V6.14         */

```

```

#include <absacc.h>
#include <reg52.h>
#include <stdarg.h>
#include <stdio.h>

```

```

#define ulong        unsigned long
#define uint         unsigned int
#define uchar        unsigned char

#define STX          0x02
#define ETX          0x03
#define EOT          0x04
#define ENQ          0x05
#define BS           0x08
#define CR           0x0D
#define LF           0x0A
#define DLE          0x10
#define ETB          0x17
#define SPACE        0x20
#define COMMA        0x2C

#define TRUE         1
#define FALSE        0

#define HIGH         1
#define LOW          0

// T6963C 端口定义
#define LCMDW        XBYTE[0x5000]    // 数据口
#define LCMCW        XBYTE[0x5002]    // 命令口

// T6963C 命令定义
#define LC_CUR_POS   0x21             // 光标位置设置
#define LC_CGR_POS   0x22             // CGRAM 偏置地址设置
#define LC_ADD_POS   0x24             // 地址指针位置
#define LC_TXT_STP   0x40             // 文本区首址
#define LC_TXT_WID   0x41             // 文本区宽度
#define LC_GRH_STP   0x42             // 图形区首址
#define LC_GRH_WID   0x43             // 图形区宽度
#define LC_MOD_OR     0x80             // 显示方式: 逻辑“或”
#define LC_MOD_XOR    0x81             // 显示方式: 逻辑“异或”
#define LC_MOD_AND    0x82             // 显示方式: 逻辑“与”
#define LC_MOD_TCH    0x83             // 显示方式: 文本特征
#define LC_DIS_SW     0x90             // 显示开关: D0=1/0:光标闪烁启用/禁用;
                                        // D1=1/0:光标显示启用/禁用;
                                        // D2=1/0:文本显示启用/禁用;
                                        // D3=1/0:图形显示启用/禁用;
#define LC_CUR_SHP    0xA0             // 光标形状选择: 0xA0-0xA7 表示光标占的行数
#define LC_AUT_WR     0xB0             // 自动写设置

```

```

#define LC_AUT_RD      0xB1    // 自动读设置
#define LC_AUT_OVR     0xB2    // 自动读/写结束
#define LC_INC_WR      0xC0    // 数据一次写, 地址加 1
#define LC_INC_RD      0xC1    // 数据一次读, 地址加 1
#define LC_DEC_WR      0xC2    // 数据一次写, 地址减 1
#define LC_DEC_RD      0xC3    // 数据一次读, 地址减 1
#define LC_NOC_WR      0xC4    // 数据一次写, 地址不变
#define LC_NOC_RD      0xC5    // 数据一次读, 地址不变
#define LC_SCN_RD      0xE0    // 屏读
#define LC_SCN_CP      0xE8    // 屏拷贝
#define LC_BIT_OP       0xF0    // 位操作: D0-D2: 定义 D0-D7 位; D3: 1 置位; 0: 清除

```

```
code uchar const uPowArr[] = {0x01,0x02,0x04,0x08,0x10,0x20,0x40,0x80};
```

// ASCII 字模宽度及高度定义

```

#define ASC_CHR_WIDTH      8
#define ASC_CHR_HEIGHT    12

```

// ASCII 字模, 显示为 8*16

```
char code ASC_MSK[96*12] = {
```

// Terminal9; 此字体下对应的点阵为: 宽 x 高=8x12

```
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xff,0xff,0xff,0xff, // < 0x20 时,打印此字
```

```
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00, // ' '
```

```
0x00,0x0C,0x1E,0x1E,0x1E,0x0C,0x0C,0x00,0x0C,0x0C,0x00,0x00, // '!'
```

```
0x00,0x66,0x66,0x66,0x24,0x00,0x00,0x00,0x00,0x00,0x00,0x00, // '"'
```

```
0x00,0x36,0x36,0x7F,0x36,0x36,0x36,0x7F,0x36,0x36,0x00,0x00, // '#'
```

```
0x0C,0x0C,0x3E,0x03,0x03,0x1E,0x30,0x30,0x1F,0x0C,0x0C,0x00, // '$'
```

```
0x00,0x00,0x00,0x23,0x33,0x18,0x0C,0x06,0x33,0x31,0x00,0x00, // '%'
```

```
0x00,0x0E,0x1B,0x1B,0x0E,0x5F,0x7B,0x33,0x3B,0x6E,0x00,0x00, // '&'
```

```
0x00,0x0C,0x0C,0x0C,0x06,0x00,0x00,0x00,0x00,0x00,0x00, // '''
```

```
0x00,0x30,0x18,0x0C,0x06,0x06,0x06,0x0C,0x18,0x30,0x00,0x00, // '('
```

```
0x00,0x06,0x0C,0x18,0x30,0x30,0x30,0x18,0x0C,0x06,0x00,0x00, // ')'
```

```
0x00,0x00,0x00,0x66,0x3C,0xFF,0x3C,0x66,0x00,0x00,0x00,0x00, // '*'
```

```
0x00,0x00,0x00,0x18,0x18,0x7E,0x18,0x18,0x00,0x00,0x00,0x00, // '+'
```

```
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x1C,0x1C,0x06,0x00, // ','
```

```
0x00,0x00,0x00,0x00,0x00,0x7F,0x00,0x00,0x00,0x00,0x00,0x00, // '.'
```

```
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x1C,0x1C,0x00,0x00, // ':'
```

```
0x00,0x00,0x40,0x60,0x30,0x18,0x0C,0x06,0x03,0x01,0x00,0x00, // '/'
```

```
0x00,0x3E,0x63,0x73,0x7B,0x6B,0x6F,0x67,0x63,0x3E,0x00,0x00, // '0'
```

```
0x00,0x08,0x0C,0x0F,0x0C,0x0C,0x0C,0x0C,0x0C,0x3F,0x00,0x00, // '1'
```

```
0x00,0x1E,0x33,0x33,0x30,0x18,0x0C,0x06,0x33,0x3F,0x00,0x00, // '2'
```

```
0x00,0x1E,0x33,0x30,0x30,0x1C,0x30,0x30,0x33,0x1E,0x00,0x00, // '3'
```

```
0x00,0x30,0x38,0x3C,0x36,0x33,0x7F,0x30,0x30,0x78,0x00,0x00, // '4'
```

```
0x00,0x3F,0x03,0x03,0x03,0x1F,0x30,0x30,0x33,0x1E,0x00,0x00, // '5'
```

```
0x00,0x1C,0x06,0x03,0x03,0x1F,0x33,0x33,0x33,0x1E,0x00,0x00, // '6'
```

```
0x00,0x7F,0x63,0x63,0x60,0x30,0x18,0x0C,0x0C,0x0C,0x00,0x00, // '7'
```

0x00,0x1E,0x33,0x33,0x37,0x1E,0x3B,0x33,0x33,0x1E,0x00,0x00, // '8'
 0x00,0x1E,0x33,0x33,0x33,0x3E,0x18,0x18,0x0C,0x0E,0x00,0x00, // '9'
 0x00,0x00,0x00,0x1C,0x1C,0x00,0x00,0x1C,0x1C,0x00,0x00,0x00, // ':'
 0x00,0x00,0x00,0x1C,0x1C,0x00,0x00,0x1C,0x1C,0x18,0x0C,0x00, // ';'
 0x00,0x30,0x18,0x0C,0x06,0x03,0x06,0x0C,0x18,0x30,0x00,0x00, // '<
 0x00,0x00,0x00,0x00,0x7E,0x00,0x7E,0x00,0x00,0x00,0x00,0x00, // '='
 0x00,0x06,0x0C,0x18,0x30,0x60,0x30,0x18,0x0C,0x06,0x00,0x00, // '>
 0x00,0x1E,0x33,0x30,0x18,0x0C,0x0C,0x00,0x0C,0x0C,0x00,0x00, // '?'
 0x00,0x3E,0x63,0x63,0x7B,0x7B,0x7B,0x03,0x03,0x3E,0x00,0x00, // '@'
 0x00,0x0C,0x1E,0x33,0x33,0x33,0x3F,0x33,0x33,0x33,0x00,0x00, // 'A'
 0x00,0x3F,0x66,0x66,0x66,0x3E,0x66,0x66,0x66,0x3F,0x00,0x00, // 'B'
 0x00,0x3C,0x66,0x63,0x03,0x03,0x03,0x63,0x66,0x3C,0x00,0x00, // 'C'
 0x00,0x1F,0x36,0x66,0x66,0x66,0x66,0x66,0x36,0x1F,0x00,0x00, // 'D'
 0x00,0x7F,0x46,0x06,0x26,0x3E,0x26,0x06,0x46,0x7F,0x00,0x00, // 'E'
 0x00,0x7F,0x66,0x46,0x26,0x3E,0x26,0x06,0x06,0x0F,0x00,0x00, // 'F'
 0x00,0x3C,0x66,0x63,0x03,0x03,0x73,0x63,0x66,0x7C,0x00,0x00, // 'G'
 0x00,0x33,0x33,0x33,0x33,0x3F,0x33,0x33,0x33,0x33,0x00,0x00, // 'H'
 0x00,0x1E,0x0C,0x0C,0x0C,0x0C,0x0C,0x0C,0x0C,0x1E,0x00,0x00, // 'I'
 0x00,0x78,0x30,0x30,0x30,0x30,0x33,0x33,0x33,0x1E,0x00,0x00, // 'J'
 0x00,0x67,0x66,0x36,0x36,0x1E,0x36,0x36,0x66,0x67,0x00,0x00, // '~ 'K'
 0x00,0x0F,0x06,0x06,0x06,0x06,0x46,0x66,0x66,0x7F,0x00,0x00, // 'L'
 0x00,0x63,0x77,0x7F,0x7F,0x6B,0x63,0x63,0x63,0x63,0x00,0x00, // 'M'
 0x00,0x63,0x63,0x67,0x6F,0x7F,0x7B,0x73,0x63,0x63,0x00,0x00, // 'N'
 0x00,0x1C,0x36,0x63,0x63,0x63,0x63,0x63,0x36,0x1C,0x00,0x00, // 'O'
 0x00,0x3F,0x66,0x66,0x66,0x3E,0x06,0x06,0x06,0x0F,0x00,0x00, // 'P'
 0x00,0x1C,0x36,0x63,0x63,0x63,0x73,0x7B,0x3E,0x30,0x78,0x00, // 'Q'
 0x00,0x3F,0x66,0x66,0x66,0x3E,0x36,0x66,0x66,0x67,0x00,0x00, // 'R'
 0x00,0x1E,0x33,0x33,0x03,0x0E,0x18,0x33,0x33,0x1E,0x00,0x00, // 'S'
 0x00,0x3F,0x2D,0x0C,0x0C,0x0C,0x0C,0x0C,0x0C,0x1E,0x00,0x00, // 'T'
 0x00,0x33,0x33,0x33,0x33,0x33,0x33,0x33,0x33,0x1E,0x00,0x00, // 'U'
 0x00,0x33,0x33,0x33,0x33,0x33,0x33,0x1E,0x0C,0x00,0x00, // 'V'
 0x00,0x63,0x63,0x63,0x63,0x6B,0x6B,0x36,0x36,0x36,0x00,0x00, // 'W'
 0x00,0x33,0x33,0x33,0x1E,0x0C,0x1E,0x33,0x33,0x33,0x00,0x00, // 'X'
 0x00,0x33,0x33,0x33,0x33,0x1E,0x0C,0x0C,0x0C,0x1E,0x00,0x00, // 'Y'
 0x00,0x7F,0x73,0x19,0x18,0x0C,0x06,0x46,0x63,0x7F,0x00,0x00, // 'Z'
 0x00,0x3C,0x0C,0x0C,0x0C,0x0C,0x0C,0x0C,0x0C,0x3C,0x00,0x00, // '['
 0x00,0x00,0x01,0x03,0x06,0x0C,0x18,0x30,0x60,0x40,0x00,0x00, // '\'
 0x00,0x3C,0x30,0x30,0x30,0x30,0x30,0x30,0x30,0x3C,0x00,0x00, // ']'
 0x08,0x1C,0x36,0x63,0x00,0x00,0x00,0x00,0x00,0x00,0x00, // '^'
 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xFF,0x00, // '_'
 0x0C,0x0C,0x18,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00, // '`'
 0x00,0x00,0x00,0x00,0x1E,0x30,0x3E,0x33,0x33,0x6E,0x00,0x00, // 'a'
 0x00,0x07,0x06,0x06,0x3E,0x66,0x66,0x66,0x66,0x3B,0x00,0x00, // 'b'
 0x00,0x00,0x00,0x00,0x1E,0x33,0x03,0x03,0x33,0x1E,0x00,0x00, // 'c'
 0x00,0x38,0x30,0x30,0x3E,0x33,0x33,0x33,0x33,0x6E,0x00,0x00, // 'd'

```

0x00,0x00,0x00,0x00,0x1E,0x33,0x3F,0x03,0x33,0x1E,0x00,0x00, // 'e'
0x00,0x1C,0x36,0x06,0x06,0x1F,0x06,0x06,0x06,0x0F,0x00,0x00, // 'f'
0x00,0x00,0x00,0x00,0x6E,0x33,0x33,0x33,0x3E,0x30,0x33,0x1E, // 'g'
0x00,0x07,0x06,0x06,0x36,0x6E,0x66,0x66,0x66,0x67,0x00,0x00, // 'h'
0x00,0x18,0x18,0x00,0x1E,0x18,0x18,0x18,0x18,0x7E,0x00,0x00, // 'i'
0x00,0x30,0x30,0x00,0x3C,0x30,0x30,0x30,0x30,0x33,0x33,0x1E, // 'j'
0x00,0x07,0x06,0x06,0x66,0x36,0x1E,0x36,0x66,0x67,0x00,0x00, // 'k'
0x00,0x1E,0x18,0x18,0x18,0x18,0x18,0x18,0x18,0x7E,0x00,0x00, // 'l'
0x00,0x00,0x00,0x00,0x3F,0x6B,0x6B,0x6B,0x6B,0x63,0x00,0x00, // 'm'
0x00,0x00,0x00,0x00,0x1F,0x33,0x33,0x33,0x33,0x33,0x00,0x00, // 'n'
0x00,0x00,0x00,0x00,0x1E,0x33,0x33,0x33,0x33,0x1E,0x00,0x00, // 'o'
0x00,0x00,0x00,0x00,0x3B,0x66,0x66,0x66,0x66,0x3E,0x06,0x0F, // 'p'
0x00,0x00,0x00,0x00,0x6E,0x33,0x33,0x33,0x33,0x3E,0x30,0x78, // 'q'
0x00,0x00,0x00,0x00,0x37,0x76,0x6E,0x06,0x06,0x0F,0x00,0x00, // 'r'
0x00,0x00,0x00,0x00,0x1E,0x33,0x06,0x18,0x33,0x1E,0x00,0x00, // 's'
0x00,0x00,0x04,0x06,0x3F,0x06,0x06,0x06,0x36,0x1C,0x00,0x00, // 't'
0x00,0x00,0x00,0x00,0x33,0x33,0x33,0x33,0x33,0x6E,0x00,0x00, // 'u'
0x00,0x00,0x00,0x00,0x33,0x33,0x33,0x33,0x1E,0x0C,0x00,0x00, // 'v'
0x00,0x00,0x00,0x00,0x63,0x63,0x6B,0x6B,0x36,0x36,0x00,0x00, // 'w'
0x00,0x00,0x00,0x00,0x63,0x36,0x1C,0x1C,0x36,0x63,0x00,0x00, // 'x'
0x00,0x00,0x00,0x00,0x66,0x66,0x66,0x66,0x3C,0x30,0x18,0x0F, // 'y'
0x00,0x00,0x00,0x00,0x3F,0x31,0x18,0x06,0x23,0x3F,0x00,0x00, // 'z'
0x00,0x38,0x0C,0x0C,0x06,0x03,0x06,0x0C,0x0C,0x38,0x00,0x00, // '{'
0x00,0x18,0x18,0x18,0x18,0x00,0x18,0x18,0x18,0x18,0x00,0x00, // 'p'
0x00,0x07,0x0C,0x0C,0x18,0x30,0x18,0x0C,0x0C,0x07,0x00,0x00, // '}'
0x00,0xCE,0x5B,0x73,0x00,0x00,0x00,0x00,0x00,0x00,0x00, // '~'
};

```

```

typedef struct typFNT_GB16 // 汉字字模显示数据结构

```

```

{
    char Index[2];
    char Msk[32];
};

```

```

struct typFNT_GB16 xdata GB_16[] = { // 显示为 16*16

```

```

    "
    0x01,0x00,0x01,0x00,0x21,0x08,0x3F,0xFC,0x21,0x08,0x21,0x08,0x21,0x08,0x21,0x08,0x3F,0xF8,0x
    21,0x08,0x01,0x00,0x01,0x00,0x01,0x00,0x01,0x00,0x01,0x00,
    "
    0x02,0x00,0x01,0x00,0x01,0x00,0xFF,0xFE,0x08,0x20,0x08,0x20,0x08,0x20,0x04,0x40,0x04,0x40,0x02,0x80,0x
    01,0x00,0x02,0x80,0x04,0x60,0x18,0x1E,0xE0,0x08,0x00,0x00,
    "
    0x40,0x02,0x27,0xC2,0x24,0x42,0x84,0x52,0x45,0x52,0x55,0x52,0x15,0x52,0x25,0x52,0x25,0x52,0x25,0x52,0x
    C5,0x52,0x41,0x02,0x42,0x82,0x42,0x42,0x44,0x4A,0x48,0x04,
    "
    0x00,0x20,0x40,0x28,0x20,0x24,0x30,0x24,0x27,0xFE,0x00,0x20,0xE0,0x20,0x27,0xE0,0x21,0x20,0x21,0x10,0

```

中

文

测

试

```
x21,0x10,0x21,0x0A,0x29,0xCA,0x36,0x06,0x20,0x02,0x00,0x00,
```

```
};
```

```
uchar gCurRow,gCurCol; // 当前行、列存储, 行高 16 点, 列宽 8 点
```

```
uchar fnGetRow(void)
```

```
{
```

```
    return gCurRow;
```

```
}
```

```
uchar fnGetCol(void)
```

```
{
```

```
    return gCurCol;
```

```
}
```

```
uchar fnST01(void) // 状态位 STA1,STA0 判断 (读写指令和读写数据)
```

```
{
```

```
    uchar i;
```

```
    for(i=10;i>0;i--)
```

```
    {
```

```
        if((LCMCW & 0x03) == 0x03)
```

```
            break;
```

```
    }
```

```
    return i; // 若返回零, 说明错误
```

```
}
```

```
uchar fnST2(void) // 状态位 ST2 判断 (数据自动读状态)
```

```
{
```

```
    uchar i;
```

```
    for(i=10;i>0;i--)
```

```
    {
```

```
        if((LCMCW & 0x04) == 0x04)
```

```
            break;
```

```
    }
```

```
    return i; // 若返回零, 说明错误
```

```
}
```

```
uchar fnST3(void) // 状态位 ST3 判断 (数据自动写状态)
```

```
{
```

```
    uchar i;
```

```
    for(i=10;i>0;i--)
```

```
    {
```

```

        if((LCMCW & 0x08) == 0x08)
            break;
    }
    return i;    // 若返回零, 说明错误
}

uchar fnST6(void)        // 状态位 ST6 判断 (屏读/屏拷贝状态)
{
    uchar i;

    for(i=10;i>0;i--)
    {
        if((LCMCW & 0x40) == 0x40)
            break;
    }
    return i;    // 若返回零, 说明错误
}

uchar fnPR1(uchar uCmd,uchar uPar1,uchar uPar2) // 写双参数的指令
{
    if(fnST01() == 0)
        return 1;
    LCMDW = uPar1;
    if(fnST01() == 0)
        return 2;
    LCMDW = uPar2;
    if(fnST01() == 0)
        return 3;
    LCMCW = uCmd;
    return 0;    // 返回 0 成功
}

uchar fnPR11(uchar uCmd,uchar uPar1) // 写单参数的指令
{
    if(fnST01() == 0)
        return 1;
    LCMDW = uPar1;
    if(fnST01() == 0)
        return 2;
    LCMCW = uCmd;
    return 0;    // 返回 0 成功
}

uchar fnPR12(uchar uCmd)        // 写无参数的指令
{

```

```

        if(fnST01() == 0)
            return 1;
        LCMCW = uCmd;
        return 0; // 返回 0 成功
    }

uchar fnPR13(uchar uData)    // 写数据
{
    if(fnST3() == 0)
        return 1;
    LCMDW = uData;
    return 0; // 返回 0 成功
}

uchar fnPR2(void)            // 读数据
{
    if(fnST01() == 0)
        return 1;
    return LCMDW;
}

// 设置当前地址
void fnSetPos(uchar urow, uchar ucol)
{
    uint iPos;

    iPos = urow * 30 + ucol;
    fnPR1(LC_ADD_POS, iPos & 0xFF, iPos / 256);
    gCurRow = urow;
    gCurCol = ucol;
}

// 设置当前显示行、列
void cursor(uchar uRow, uchar uCol)
{
    fnSetPos(uRow * 16, uCol);
}

// 清屏
void cls(void)
{
    uint i;

    fnPR1(LC_ADD_POS, 0x00, 0x00); // 置地址指针
    fnPR12(LC_AUT_WR);             // 自动写

```



```

for(i=0;j<240*30;i++)
{
    fnST3();
    fnPR13(0x00);          // 写数据
}
fnPR12(LC_AUT_OVR);        // 自动写结束
fnPR1(LC_ADD_POS,0x00,0x00); // 重置地址指针
gCurRow = 0;              // 置地址指针存储变量
gCurCol = 0;
}

// LCM 初始化
char fnLCMInit(void)
{
    if(fnPR1(LC_TXT_STP,0x00,0x00) != 0) // 文本显示区首地址
        return -1;
    fnPR1(LC_TXT_WID,0x1E,0x00); // 文本显示区宽度
    fnPR1(LC_GRH_STP,0x00,0x00); // 图形显示区首地址
    fnPR1(LC_GRH_WID,0x1E,0x00); // 图形显示区宽度
    fnPR12(LC_CUR_SHP|0x01); // 光标形状
    fnPR12(LC_MOD_OR); // 显示方式设置
    fnPR12(LC_DIS_SW|0x08); // 显示开关设置

    return 0;
}

// ASCII(8*16) 及 汉字(16*16) 显示函数
uchar dprintf(char *fmt, ...)
{
    va_list arg_ptr;
    char c1,c2,cData;
    char tmpBuf[64];          // LCD 显示数据缓冲区
    uchar i=0,j,uLen,uRow,uCol;
    uint k;

    va_start(arg_ptr, fmt);
    uLen = (uchar)vsprintf(tmpBuf, fmt, arg_ptr);
    va_end(arg_ptr);

    while(i<uLen)
    {
        c1 = tmpBuf[i];
        c2 = tmpBuf[i+1];
        uRow = fnGetRow();
        uCol = fnGetCol();
    }

```

```

if(c1 >= 0)
{
    // ASCII
    if(c1 < 0x20)
    {
        switch(c1)
        {
            case CR:
            case LF:        // 回车或换行
                i++;
                if(uRow < 112)
                    fnSetPos(uRow+16,0);
                else
                    fnSetPos(0,0);
                continue;
            case BS:        // 退格
                if(uCol > 0)
                    uCol--;
                fnSetPos(uRow,uCol);
                cData = 0x00;
                break;
            default:        // 其他
                c1 = 0x1f;
        }
    }
    for(j=0;j<16;j++)
    {
        fnPR12(LC_AUT_WR);        // 写数据
        if(c1 >= 0x1f)
        {
            if(j < (16-ASC_CHR_HEIGHT))
                fnPR13(0x00);
            else
                fnPR13(ASC_MSK[(c1-0x1f)*ASC_CHR_HEIGHT+j-(16-ASC_CHR_HEIGHT)]);
        }
        else
        {
            fnPR13(cData);
            fnPR12(LC_AUT_OVR);
            fnSetPos(uRow+j+1,uCol);
        }
        if(c1 != BS)        // 非退格
            uCol++;
    }
}
else
{
    // 中文
    for(j=0;j<sizeof(GB_16)/sizeof(GB_16[0]);j++)

```

```

        {
            if(c1 == GB_16[j].Index[0] && c2 == GB_16[j].Index[1])
                break;
        }
        for(k=0;k<sizeof(GB_16[0].Msk)/2;k++)
        {
            fnSetPos(uRow+k,uCol);
            fnPR12(LC_AUT_WR);          // 写数据
            if(j < sizeof(GB_16)/sizeof(GB_16[0]))
            {
                fnPR13(GB_16[j].Msk[k*2]);
                fnPR13(GB_16[j].Msk[k*2+1]);
            }
            else                          // 未找到该字
            {
                if(k < sizeof(GB_16[0].Msk)/4)
                {
                    fnPR13(0x00);
                    fnPR13(0x00);
                }
                else
                {
                    fnPR13(0xff);
                    fnPR13(0xff);
                }
            }
            fnPR12(LC_AUT_OVR);
        }
        uCol += 2;
        i++;
    }
    if(uCol >= 30)                        // 光标后移
    {
        uRow += 16;
        if(uRow < 0x80)
            uCol -= 30;
        else
        {
            uRow = 0;
            uCol = 0;
        }
    }
    fnSetPos(uRow,uCol);
    i++;
}

```

```

    return uLen;
}

void main(void) // 测试用
{
    fnLCMInit();
    cls();
    cursor(0,0);
    dprintf("%s","This is a test:中文测试");
}

```

程序三十八 键盘输入法设计草案

利用有限键的键盘实现拼音输入

1 键代表 ABC 2 键代表 DEF 3 键代表 GHI 4 键代表 JKL
 5 键代表 MNO 6 键代表 PQRS 7 键代表 TUV 8 键代表 WXYZ
 9 键代表 " "(空格) 0 键为确认键(该拼音输入结束)

R 键为拼音输入, 字母输入(大写), 字母输入(小写), 数字输入, 字符输入转切键, 即每按一次该键将会切换倒下一状态;

Q 键为下翻页键, W 为上翻页键;

E 键为退格键, 消除错误的输入;

Q 键在拼音未结束, 即未按 0 键时为错误消除键(例如拚 zhuang 却拚成了 zhang, 它可以以一个字母一个字母的消除)。

例: "李"-LI 按 43-0(结束输入标志), 再按 2(从 LI 和 JI 中选 LI), 再按 Q, W 键进行翻页选择"李"(直接按所对应的数字)

编码方式:

此编码与该拼音的第一个字的区位码对应

显示该字及该拼音的下几个字

a-01 b-02 c-03 d-04 e-05 f-06 g-07 h-08 i-09 j-0a k-0b l-0c m-0d n-0e o-0f p-10 q-11 r-12
 s-13 t-14 u-15 v-16 w-17 x-18 y-19 z-1a

汉字索引方式:

例:

拚"工"

音 节	有几种可能	首 区 位	屏幕显示拼音
3, 5, 5, 3, 0, 0, 2,	2 种	25,4,26,68, 0, 0, 0, 0, 0, 0, 0,	7,15,14,7, 8,15,14,7,0,0,0,0,0,0,0,0,0,0,0,0,
g o n g		25,4	
h o n g		26,68	

由索引库提取汉字区位，再由字库提取字模。索引库中有汉字索引表，数字索引表，字母索引表和字符索引表。汉字显示 16×16 ，其他为 8×16 。本程序在电脑上模拟成功 put.exe 为执行文件，Egavga.bgi, hzk16 放同一目录下。若要移植到单片机，可以使用索引表 index.c 另外再编一索引程序即可，一级汉字可放在程序存储其中，也可放在外部 RAM 中，随时调用。

```
#include<graphics.h>
#include<io.h>
#include<stdio.h>
#include<stdlib.h>
#include<conio.h>
#include<index.c>

headdisp(){
unsigned int  gst[25][14]=
{
/*GstBitMap:          db 25,14*/
0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,
0x0,0x0,0x0,0x0,0x0,0x0,0x1a,0xaa,0xa8,0x0,0x0,0x8,0x0,0x0,
0x1,0xaa,0xaa,0x2,0xaa,0xac,0x15,0x55,0x5c,0x0,0x3,0x0,0x0,0x0,
0x5,0x55,0x55,0x5,0x55,0x56,0x2a,0xaa,0xac,0x0,0x20,0x0,0x0,0x0,
0xa,0xaa,0xab,0xa,0xaa,0xab,0x15,0x55,0x5e,0x0,0x50,0x0,0x0,0x0,
0x5,0x5d,0xd5,0x95,0x7f,0xd5,0x3f,0xaf,0xfc,0x4,0x0,0x4,0x0,0x0,
0xa,0xff,0xeb,0x8a,0xff,0xeb,0x3f,0xdf,0xfc,0xb,0x0,0xc8,0x0,0x0,
0x15,0xff,0xf5,0xd5,0xff,0xdf,0xf,0xaf,0xfc,0x25,0x8,0x80,0x0,0x0,
0xa,0xaa,0xbf,0x8a,0xe0,0x1f,0x0,0xae,0x0,0xd4,0x14,0x0,0x0,0x0,
0x15,0xc0,0x1f,0x15,0x55,0x5f,0x0,0x5c,0x0,0xa9,0x48,0x0,0x0,0x0,
0x2b,0xcb,0x6b,0x8a,0xaa,0xa4,0x0,0xbc,0x1,0x50,0x80,0x44,0x0,0x0,
0x15,0xd5,0x97,0x95,0x55,0x54,0x1,0x5c,0x0,0xad,0x42,0x20,0x0,0x0,
0x2b,0x8a,0x6b,0xf,0xff,0xae,0x0,0xbc,0x0,0x52,0x35,0x0,0x0,0x0,
0x17,0x97,0x97,0x7,0xff,0xd6,0x1,0x58,0x0,0xc,0x4a,0x0,0x0,0x0,
0x2b,0x8f,0xef,0x2f,0xff,0xae,0x0,0xbc,0x0,0x16,0xb0,0x0,0x0,0x0,
0x57,0x8f,0x16,0x50,0x0,0x56,0x3,0x78,0x0,0xb,0x48,0x14,0x0,0x0,
0x2a,0xaa,0xdf,0x55,0x55,0x5e,0x2,0xb8,0x0,0xa,0x81,0x40,0x0,0x0,
0x55,0x55,0x6e,0x2a,0xaa,0xae,0x1,0x70,0x0,0x1,0x4c,0x80,0x0,0x0,
0x2a,0xaa,0x9e,0x55,0x55,0x5e,0x2,0xb8,0x0,0x2,0xab,0x0,0x0,0x0,
0x15,0x55,0x7e,0x2a,0xaa,0xbe,0x5,0x70,0x0,0x1,0x54,0x20,0x0,0x0,
0x1f,0xff,0xfc,0x1f,0xff,0xfc,0x3,0xf0,0x0,0x0,0x28,0x0,0x0,0x0,
0xf,0xff,0xf8,0x1f,0xff,0xf8,0x1,0xf0,0x0,0x0,0x50,0x0,0x0,0x0,
0x7,0xff,0xe0,0xf,0xff,0xf0,0x1,0xf0,0x0,0x0,0x0,0x0,0x0,0x0,
0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,
0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,
};
```

```

int x=0;
int y=0;
register m,n,i,j,k=0;
    for(i=0;i<25;i++)
    {
        for(j=0;j<14;j++)
        {
            for(k=0;k<8;k++)
            {
                if(((gst[i][j]>>(7-k))&0x1)!=NULL)
                {
                    putpixel(x+8*j+k,y+i,GREEN);
                }
            }
        }
    }
}

```

```

pydisp()          /*拼音状态显示*/
{
    unsigned int py[5][2]= {38,20,          /*拼*/
                            50,84,          /*音*/
                            42,68,          /*输*/
                            40,75,          /*入*/
                            03,26,};        /*'*/

    int s1;
    for(s1=0;s1<5;s1++)
    {
        puthz(s1*16,50,py[s1][0],py[s1][1],4);
    }
}

```

```

sengdisp()        /*英文输入状态显示(小写)*/
{
    unsigned int syw[7][2]={51,02,          /*英*/
                            46,36,          /*文*/
                            42,68,          /*输*/
                            40,75,          /*入*/
                            48,01,          /*小*/
                            48,20,          /*写*/
                            03,26,};        /*'*/

    int s1;
    for(s1=0;s1<7;s1++)

```

```

    {
        puthz(s1*16,50,syw[s1][0],syw[s1][1],4);
    }
}
bengdisp()          /*英文输入状态显示(大写)*/
{
    unsigned int byw[7][2]={51,02,          /*'英'*/
                            46,36,          /*'文'*/
                            42,68,          /*'输'*/
                            40,75,          /*'入'*/
                            20,83,          /*'大'*/
                            48,20,          /*'写'*/
                            03,26,};        /*'.'*/

    int s1;
    for(s1=0;s1<7;s1++)
    {
        puthz(s1*16,50,byw[s1][0],byw[s1][1],4);
    }
}

numdisp()
{

    unsigned int snum[5][2]={42,93,          /*'数'*/
                             55,54,          /*'字'*/
                             42,68,          /*'输'*/
                             40,75,          /*'入'*/
                             03,26,};        /*'.'*/

    int s1;
    for(s1=0;s1<5;s1++)
    {
        puthz(s1*16,50,snun[s1][0],snun[s1][1],4);
    }
}

zifudisp()
{
    unsigned int zifu[5][2]={
                            55,54,          /*'字'*/
                            23,91,          /*'符'*/
                            42,68,          /*'输'*/
                            40,75,          /*'入'*/

```

```
03,26,});    /*:**/
```

```
int s1;  
    for(s1=0;s1<5;s1++)  
    {  
        puthz(s1*16,50,zifu[s1][0],zifu[s1][1],4);  
    }  
}
```

```
headdisp2()  
{  
    unsigned int head[4][2]={26,03,  
                                45,69,  
                                25,11,  
                                43,30,  
                                };  
}
```

```
int s1;  
    for(s1=0;s1<4;s1++)  
    {  
        puthz((s1+10)*16,0,head[s1][0],head[s1][1],4);  
    }  
}
```

```
puthz(int x,int y,int h,int l,int color)  
{  
    FILE *fp;  
    int kk;  
    char buffer[32];  
    register m,n,i,ii,j,k=0;  
    unsigned char qh,wh;  
    unsigned long int offset;  
    if((fp=fopen("hzk16","rb"))==NULL)  
    {  
        printf("can't open hzk16,please addit!");  
        getch();  
        closegraph();  
        exit(0);  
    }  
    offset=(94*(h-1)+(l-1))*32L;  
    fseek(fp,offset,SEEK_SET);  
    fread(buffer,32,1,fp);  
    for(i=0;i<16;i++)  
    {  
        for(j=0;j<2;j++)  
        {
```



```

        for(k=0;k<8;k++)
        {
            if(((buffer[i*2+j]>>(7-k))&0x1)!=NULL)
            {
                putpixel(x+8*j+k,y+i,color);
            }
        }
    }
    if(x>512)
    {
        x=0;y=y+16;
    }
    fclose(fp);
}

```

```

putnum(int x,int y,int dem,int color)
{
    register m,n,i,ii,j,k=0;
    for(i=0;i<16;i++)
    {
        for(j=0;j<1;j++)
        {
            for(k=0;k<8;k++)
            {
                if(((num[dem][i+j]>>(7-k))&0x1)!=NULL)
                {
                    putpixel(x+8*j+k,y+i,color);
                }
            }
        }
    }
}

```

```

putzimub(int x2,int y2,int dem2,int color2)
{
    register m2,n2,i2,ii2,j2,k2=0;
    for(i2=0;i2<16;i2++)
    {
        for(j2=0;j2<1;j2++)
        {
            for(k2=0;k2<8;k2++)
            {
                if(((beng[dem2][i2+j2]>>(7-k2))&0x1)!=NULL)

```

```

        {
            putpixel(x2+8*j2+k2,y2+i2,color2);
        }
    }
}

putzimus(int x3,int y3,int dem3,int color3)
{
    register m3,n3,i3,ii3,j3,k3=0;
    for(i3=0;i3<16;i3++)
    {
        for(j3=0;j3<1;j3++)
        {
            for(k3=0;k3<8;k3++)
            {
                if(((seng[dem3][i3+j3]>>(7-k3))&0x1)!=NULL)
                {
                    putpixel(x3+8*j3+k3,y3+i3,color3);
                }
            }
        }
    }
}

```

```

putzifu(int x,int y,int dem,int color)
{
    register m,n,i,ii,j,k=0;
    for(i=0;i<16;i++)
    {
        for(j=0;j<1;j++)
        {
            for(k=0;k<8;k++)
            {
                if(((sig[dem][i+j]>>(7-k))&0x1)!=NULL)
                {
                    putpixel(x+8*j+k,y+i,color);
                }
            }
        }
    }
}

```

```

check_key( unsigned char k) /*查看并确认键值*/
{
    int c;
    switch(k)
    {
        case '1':c=1;
        break;
        case '2':c=2;
        break;
        case '3':c=3;
        break;
        case '4':c=4;
        break;
        case '5':c=5;
        break;
        case '6':c=6;
        break;
        case '7':c=7;
        break;
        case '8':c=8;
        break;
        case '9':c=9;
        break;
        case '0':c=0;
        break;
        case 'q':c=10;                /*define F1*/
        break;
        case 'w':c=11;                /*define F2*/
        break;
        case 'e':c=12;                /*define F3*/
        break;
        case 'r':c=13;                /*define F4*/
        break;
        case 'c':
        closegraph();
        exit(0);
        default:
        c=20;
        break;
    }

    return(c);
}

main()
{

```

```

int firqmh,firqml;
int a[6][16];
int key[6],i,j,s,ii,k,m,kkk,kk=0,jj=120,n,nm=0;
int ks,mm,d=0,key1,key2,qmh,qml,ss,kp,kd=0,page=0,page1=0;
int driver=DETECT,mode;
int end;
int endd;
int mu;
int endl;
int sm[6];
int ok;
char kc;
initgraph(&driver,&mode,"");
setcolor(5);
headdisp();
headdisp2();
bar(0,50,500,100);
bar(0,100,500,220);
    for(i=0;i<5;i++)
    {
        line(0,100+i,500,100+i);
    }
    for(i=0;i<5;i++)
    {
        line(0,220+i,500,220+i);
    }
    for(i=0;i<5;i++)
    {
        line(0,45+i,500,45+i);
    }
    for(i=0;i<2;i++)
    {
        line(i,45,i,220);
    }
    for(i=0;i<2;i++)
    {
        line(i+500,45,i+500,220);
    }
top:
    if(kd==4)
    {
        kd=0;
    }    /*键盘状态循环*/
    while(kd==0)    /*汉字输入状态*/
    {

```

```

bar(0,50,500,100);
pydisp();
h1:
    for(;;)
    {
        page=0;
        for(i=0;i<7;i++)
        {
            again:
            key[i]=check_key(getch());    /*取键值*/
            if(key[0]==0)
            {
                goto again;
            }
            switch(key[i])
            {
                /*判断键状态*/
            case 9:kk=kk+8;goto h1;
            case 13:kd++;goto top;    /*是否是状态切换键*/
            case 12:
                kk=kk-8;
                bar(kk,jj,kk+8,jj+16);
                if(kk<0)
                {
                    kk=304;
                    jj=jj-16;
                }
                if(jj<=120)
                {
                    jj=120;
                }
            goto h1;
            case 0 :
                s=check_key(getch());
                goto outhz;    /*汉字输入确认*/
            case 10:i=i-2;if(i<0){i=0;goto top;}
                break;
            case 11:
            case 20:
                goto again;
            }
            if(i==0)
            {
                /*若为第一次读键*/
                for(k=0;k<402;k++)
                {
                    if(hzk_index[k][0]==key[0])    /*搜索索引表*/

```

```

        {
            ok=k;
            goto here;
        }
    }
}
if(i==1)
{
    /*若为第二次读键*/
    mu=0;
    for(k=0;k<402;k++)
    {
        if(hzk_index[k][0]==key[0]&&hzk_index[k][1]==key[1]) /*搜索索引表*/
        {
            mu=1;
            ok=k;
            goto here;
        }
    }
    if(mu==0){sm[i]=ok;goto again;}
}

if(i==2)
{
    /*若为第三次读键*/
    mu=0;
    for(k=0;k<402;k++)
    {
        if(hzk_index[k][0]==key[0]&&hzk_index[k][1]==key[1]&&hzk_index[k][2]
==key[2])/*搜索索引表*/
        {
            ok=k;
            mu=1;
            goto here;
        }
    }
    if(mu==0){sm[i]=ok;goto again;}
}

if(i==3)
{
    /*若为第四次读键*/
    mu=0;
    for(k=0;k<402;k++)
    {
        if(hzk_index[k][0]==key[0]&&hzk_index[k][1]==key[1]&&hzk_index[k][2]==key[2]&&hzk_index[k][3]==key[3])
        {
            ok=k;

```

```

        mu=1;
        goto here;
    }
}
if(mu==0){sm[i]=ok;goto again;}
}

if(i==4)
{
    /*若为第五次读键*/
    mu=0;
    for(k=0;k<402;k++)
    {
if(hzk_index[k][0]==key[0]&&hzk_index[k][1]==key[1]&&hzk_index[k][2]==key[2]&&hzk_index[k][3]==key[3]
&&hzk_index[k][4]==key[4])
        {
            mu=1;
            ok=k;
            goto here;
        }
    }
    if(mu==0){sm[i]=ok;goto again;}
}

if(i==5)
{
    /*若为第六次读键*/
    mu=0;
    for(k=0;k<402;k++)
    {
if(hzk_index[k][0]==key[0]&&hzk_index[k][1]==key[1]&&hzk_index[k][2]==key[2]&&hzk_index[k][3]==key[3]
&&hzk_index[k][4]==key[4]&&hzk_index[k][5]==key[5])
        {
            mu=1;
            ok=k;
            goto here;
        }
    }
    if(mu==0){sm[i]=ok;goto again;}
}

here:
/*显示对应键盘字母所有的可能组合*/
m=hzk_index[k][6];    /*m为所有可能数，它位于索引表每行第7列*/
firqmb=hzk_index[k][7];
firqml=hzk_index[k][8];
bar(0,50,300,100);
pydisp();

```

```

        bar(100,50,200,100);
        for(ks=0;ks<9;ks++)
        {
            putnum(ks*32,70,ks+1,2);
            puthz((ks*2+1)*16,70,firqmh,firqml+ks,2);
        }
        for(n=0;n<m;n++)
        {
            putnum(10*8+n*64,50,n+1,2);
            for(kkk=1;kkk<=i+1;kkk++)
            {
                a[n][kkk]=hzk_index[k][18+(n*(i+1)+kkk)];
                putzimus(11*8+n*64+kkk*8,50,a[n][kkk]-1,2);
            }
        }
    }
}

```

outhz: /*用户选择组合中所有可能中的一种*/

```

nn=++nn ;
    if(mu==1)
    {
        qmh=hzk_index[k][7+(s-1)*2];
        qml=hzk_index[k][7+(s-1)*2+1];
    }
    if(mu==0)
    {
        qmh=hzk_index[sm[i]][7+(s-1)*2];
        qml=hzk_index[sm[i]][7+(s-1)*2+1];
    }
fydisp:
bar(0,50,500,100);
pydisp();
    for(i=0;i<9;i++)
    {
        putnum(i*32,70,i+1,2);
        puthz((i*2+1)*16,70,qmh,qml+i+(9*page),2);
    }
endtop:
end=check_key(getch());
switch(end)
{
    case 11:page=page+1;goto fydisp;
}

```



```

        case 10:page=page-1;goto fydisp;
        case 20:goto endtop;
        case 12:goto endtop;
        case 13:kd++;goto top;
        default:
            break;
    }
    puthz(kk,jj,qmh,qml+end+page*9-1,2);
    kk=kk+16;
    if(kk>300)
    {
        kk=0;
        jj=jj+16;
    }
}
}

```

```

while(kd==1)      /*字母输入状态*/
{
    top1:
    bar(0,50,500,100);
    bengdisp();
    for(;;)
    {
        while(d)
        {
            top2:
            key1=check_key(getch());
            switch(key1)
            {
                case 9:kk=kk+8;goto top2;
                case 13:kd++;goto top;
                case 0: d=!d;goto top1;
                case 20:
                case 11:
                case 10: goto top2;
                case 12:
                kk=kk-8;
                bar(kk,jj,kk+8,jj+16);
                if(kk<0)
                {
                    kk=304;
                    jj=jj-16;
                }
            }
            /*大写字母输入*/
            /*判断键值*/
            /*判断键状态*/
            /*大小写切换*/
        }
    }
}

```

```

        }
        if(jj<=120)
        {
            jj=120;
        }
        goto top2;
    }
    qmh=key1*3-3;
    kp=key1;
    bar(0,50,500,100);
    bengdisp();
    for(mm=0;mm<3;mm++)
    {
        putzimub((mm+15)*8,50,qmh+mm,3);
    }
    key2=check_key(getch());
    switch(key2)
    {
        case 13:kd++;goto top;
        case 0:d=!d;goto top1;
    }
    goto outen;
}

while(!d)
{
    /*小写字母输入*/
top3:
    bar(0,50,500,100);
    sengdisp();
    key1=check_key(getch());    /*判断键值*/
    switch(key1)
    {
        /*判断键状态*/
        case 9:kk=kk+8;goto top3;
        case 13:kd++;goto top;
        case 0: d=!d;goto top1;    /*大小写切换*/
        case 10:
        case 11:
        case 20:goto top3;
        case 12:
            kk=kk-8;
            bar(kk,jj,kk+8,jj+16);
            if(kk<0)
            {
                kk=304;
                jj=jj-16;
            }
        }
    }
}

```

```

        }
        if(jj<=120)
        {
            jj=120;
        }
        goto top3;
    }
    qmh=key1*3-3;
    kp=key1;
    bar(0,50,500,100);
    sengdisp();
    for(mm=0;mm<3;mm++)
    {
        putzimus((mm+15)*8,50,qmh+mm,3);
    }
    key2=check_key(getch());
    switch(key2)
    {
        case 13:kd++;goto top;
        case 0: d=!d;goto top1;
        case 9:kk=kk+8;goto top;
        case 12:
            kk=kk-8;
            bar(kk,jj,kk+8,jj+16);
            if(kk<0)
            {
                kk=304;
                jj=jj-16;
            }
            if(jj<=120)
            {
                jj=120;
            }
            goto top3;
    }
    goto outen;
}

outen: /*从显示的三个字母中选出所需的字母*/
    if(d)
    {
        ii=kp*3-4+key2;
        putzimub(kk,jj,ii,3);
        kk=kk+8;
        if(kk>288)

```

```

        {
            kk=0;
            jj=jj+16;
        }
    }
    if(!d)
    {
        ii=kp*3-4+key2;
        putzimus(kk,jj,ii,3);
        kk=kk+8;
        if(kk>288)
        {
            kk=0;
            jj=jj+16;
        }
    }
}

while(kd==2)          /*数字输入状态*/
{
    bar(0,50,500,100);
    numdisp();
    top4:
    for(;;)
    {
        key1=check_key(getch());          /*判断键值*/
        switch(key1)
        {
            case 13:kd++;goto top;
            case 11:
            case 10:
            case 20:goto top4;
            case 12:kk=kk-8;
            bar(kk,jj,kk+8,jj+16);
            if(kk<0)
            {
                kk=304;
                jj=jj-16;
            }
            if(jj<=120)
            {
                jj=120;
            }
            goto top4;
        }
    }
}

```

```

        putnum(kk,jj,key1,3);
        kk=kk+8;
        if(kk>288)
        {
            kk=0;
            jj=jj+16;
        }
    }
}

while(kd==3)        /*字符输入状态*/
{
    zifudisp:
    bar(0,50,500,100);
    zifudisp();
    bar(100,50,200,100);
    for(ks=0;ks<9;ks++)
    {
        putnum(ks*32,70,ks+1,2);
        putzifu((ks*2+1)*16,70,ks+9*page1,2);
    }
    endtop1:
    end1=check_key(getch());
    switch(end1)
    {
        case 11:page1=page1+1;if(page1>=2){page1=2;}goto zifudisp;
        case 10:page1=page1-1;if(page1<=0){page1=0;}goto zifudisp;
        case 20:goto endtop1;
        case 13:kd++;goto top;
        case 12:
            kk=kk-8;
            bar(kk,jj,kk+8,jj+16);
            if(kk<0)
            {
                kk=304;
                jj=jj-16;
            }
            if(jj<=120)
            {
                jj=120;
            }
            goto endtop1;
    }
    putzifu(kk,jj,end1+page1*9-1,2);
    kk=kk+8;

```


2, 0, 0, 0, 0, 0, 3,	22, 74, 20, 78, 23,	2, 0, 0, 0, 0, 0, 0,	4, 5, 6, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
2, 1, 0, 0, 0, 0, 2,	20, 78, 23, 2,	0, 0, 0, 0, 0, 0, 0, 0,	4, 1, 6, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
2, 1, 3, 0, 0, 0, 1,	20, 84, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	4, 1, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
2, 1, 5, 0, 0, 0, 3,	21, 2, 21, 22, 23,	10, 0, 0, 0, 0, 0, 0, 0,	4, 1, 14, 4, 1, 15, 6, 1, 14, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
2, 1, 5, 3, 0, 0, 2,	21, 17, 23, 27,	0, 0, 0, 0, 0, 0, 0, 0,	4, 1, 14, 7, 6, 1, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
2, 2, 0, 0, 0, 0, 1,	21, 34, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	4, 5, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
2, 2, 3, 0, 0, 0, 1,	23, 38, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	6, 5, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
2, 2, 5, 0, 0, 0, 1,	23, 50, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	6, 5, 14, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
2, 2, 5, 3, 0, 0, 2,	23, 65, 21, 37,	0, 0, 0, 0, 0, 0, 0, 0,	6, 5, 14, 7, 4, 5, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
2, 3, 0, 0, 0, 0, 1,	21, 44, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	4, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
2, 3, 1, 5, 0, 0, 2,	21, 63, 21, 79,	0, 0, 0, 0, 0, 0, 0, 0,	4, 9, 1, 14, 4, 9, 1, 15, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
2, 3, 2, 0, 0, 0, 1,	21, 88, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	4, 9, 5, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
2, 3, 5, 3, 0, 0, 1,	22, 1, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	4, 9, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
2, 3, 7, 0, 0, 0, 1,	22, 10, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	4, 9, 21, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
2, 5, 0, 0, 0, 0, 2,	22, 87, 23, 80,	0, 0, 0, 0, 0, 0, 0, 0,	5, 14, 6, 15, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
2, 5, 5, 3, 0, 0, 1,	22, 11, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	4, 15, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
2, 5, 7, 0, 0, 0, 2,	22, 21, 23, 81,	0, 0, 0, 0, 0, 0, 0, 0,	4, 15, 21, 6, 15, 21, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
2, 6, 0, 0, 0, 0, 1,	22, 88, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	5, 18, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
2, 7, 0, 0, 0, 0, 2,	22, 28, 23, 82,	0, 0, 0, 0, 0, 0, 0, 0,	4, 21, 6, 21, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
2, 7, 1, 5, 0, 0, 1,	22, 43, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	4, 21, 1, 14, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
2, 7, 3, 0, 0, 0, 1,	22, 49, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	4, 21, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
2, 7, 5, 0, 0, 0, 2,	22, 53, 22, 62,	0, 0, 0, 0, 0, 0, 0, 0,	4, 21, 14, 4, 21, 15, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
3, 0, 0, 0, 0, 0, 3,	24, 33, 24, 33, 25,	94, 0, 0, 0, 0, 0, 0, 0,	7, 8, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
3, 1, 0, 0, 0, 0, 2,	24, 33, 25, 94,	0, 0, 0, 0, 0, 0, 0, 0,	7, 1, 8, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
3, 1, 3, 0, 0, 0, 2,	24, 35, 26, 1,	0, 0, 0, 0, 0, 0, 0, 0,	7, 1, 9, 8, 1, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
3, 1, 5, 0, 0, 0, 4,	24, 41, 24, 61, 26,	8, 26, 30, 0, 0, 0, 0,	7, 1, 14, 7, 1, 15, 8, 1, 14, 8, 1, 15, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
3, 1, 5, 3, 0, 0, 2,	24, 52, 26, 27,	0, 0, 0, 0, 0, 0, 0, 0,	7, 1, 14, 7, 8, 1, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
3, 2, 0, 0, 0, 0, 2,	24, 71, 26, 39,	0, 0, 0, 0, 0, 0, 0, 0,	7, 5, 8, 5, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
3, 2, 3, 0, 0, 0, 2,	24, 88, 26, 57,	0, 0, 0, 0, 0, 0, 0, 0,	7, 5, 9, 8, 5, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
3, 2, 5, 0, 0, 0, 2,	24, 89, 26, 59,	0, 0, 0, 0, 0, 0, 0, 0,	7, 5, 14, 8, 5, 14, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
3, 2, 5, 3, 0, 0, 2,	24, 91, 26, 63,	0, 0, 0, 0, 0, 0, 0, 0,	7, 5, 14, 7, 8, 5, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
3, 5, 0, 0, 0, 0, 2,	25, 4, 0,	0, 0, 0, 0, 0, 0, 0, 0,	7, 15, 8, 15, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
3, 5, 5, 0, 0, 0, 2,	25, 4, 0,	0, 0, 0, 0, 0, 0, 0, 0,	7, 15, 14, 8, 15, 14, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
3, 5, 5, 3, 0, 0, 2,	25, 4, 26, 68,	0, 0, 0, 0, 0, 0, 0, 0,	7, 15, 14, 7, 8, 15, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
3, 5, 7, 0, 0, 0, 2,	25, 19, 26, 77,	0, 0, 0, 0, 0, 0, 0, 0,	7, 15, 21, 8, 15, 21, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
3, 7, 0, 0, 0, 0, 2,	25, 28, 26, 84,	0, 0, 0, 0, 0, 0, 0, 0,	7, 21, 8, 21, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
3, 7, 1, 0, 0, 0, 2,	25, 47, 27,	8, 0, 0, 0, 0, 0, 0, 0,	7, 21, 1, 8, 21, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
3, 7, 1, 3, 0, 0, 1,	25, 52,	0, 0, 0, 0, 0, 0, 0, 0,	7, 21, 1, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
3, 7, 1, 5, 0, 0, 2,	25, 55, 27, 22,	0, 0, 0, 0, 0, 0, 0, 0,	7, 21, 1, 14, 8, 21, 1, 14, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
3, 7, 1, 5, 3, 0, 2,	25, 66, 27, 36,	0, 0, 0, 0, 0, 0, 0, 0,	7, 21, 1, 14, 7, 8, 21, 1, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
3, 7, 3, 0, 0, 0, 2,	25, 69, 27, 50,	0, 0, 0, 0, 0, 0, 0, 0,	7, 21, 9, 8, 21, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
3, 7, 5, 0, 0, 0, 4,	25, 85, 27, 71, 25, 88,	27, 77, 0, 0, 0, 0, 7, 21,	14, 8, 21, 14, 7, 21, 15, 8, 21, 15, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
4, 0, 0, 0, 0, 0, 3,	31, 6,	0, 0, 0, 0, 0, 0, 0, 0,	10, 11, 12, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
4, 1, 0, 0, 0, 0, 2,	31, 6, 32, 12,	0, 0, 0, 0, 0, 0, 0, 0,	11, 1, 12, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
4, 1, 3, 0, 0, 0, 2,	31, 10, 32, 19,	0, 0, 0, 0, 0, 0, 0, 0,	11, 1, 9, 12, 1, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,

4, 1, 5, 0, 0, 0, 4,	32,22, 32, 44,31, 15,31,28, 0, 0, 0, 0,12,1,14,12,1,15,11,1,14,11,1,15,0,0,0,0,0,0,0,0,0,0,0,
4, 1, 5, 3, 0, 0, 2,	31,21, 32, 27, 0, 0, 0, 0, 0, 0, 0, 0, 11,1,14,7,12,1,14,7,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 2, 0, 0, 0, 0, 2,	32,53, 31, 32, 0, 0, 0, 0, 0, 0, 0, 0, 12,5,11,5,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 2, 3, 0, 0, 0, 1,	32,55, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 12,5,9,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 2, 5, 0, 0, 0, 2,	31,47, 12, 5, 0, 0, 0, 0, 0, 0, 0, 0, 12,5,14,11,5,14,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 2, 5, 3, 0, 0, 2,	31,51, 32, 66, 0, 0, 0, 0, 0, 0, 0, 0, 11,5,14,7,12,5,14,7,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 3, 0, 0, 0, 0, 2,	27,87, 32, 69, 0, 0, 0, 0, 0, 0, 0, 0, 10,9,12,9,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 3, 1, 0, 0, 0, 2,	28,47, 33, 9, 0, 0, 0, 0, 0, 0, 0, 0, 10,9,1,12,9,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 3, 1, 5, 0, 0, 4,	28,63, 33, 10,29, 22,33, 35,0, 0, 0, 0,10,9,1,14,12,9,1,14,10,9,1,15,12,9,1,15,0,0,0,0,0,0,0,0,
4, 3, 1, 5, 3, 0, 2,	29, 9, 33, 24, 0, 0, 0, 0, 0, 0, 0, 0, 10,9,1,14,7,12,9,1,14,7,0,0,0,0,0,0,0,0,0,0,0,0,
4, 3, 2, 0, 0, 0, 2,	29,50, 33, 48, 0, 0, 0, 0, 0, 0, 0, 0, 10,9,5,12,9,5,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 3, 5, 0, 0, 0, 2,	29,77, 33, 53, 0, 0, 0, 0, 0, 0, 0, 0, 10,9,14,12,9,14,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 3, 5, 3, 0, 0, 2,	30, 3, 33, 65, 0, 0, 0, 0, 0, 0, 0, 0, 10,9,14,7,12,9,14,7,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 3, 5, 5, 3, 0, 1,	30, 28, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 10,9,15,14,7,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 3, 7, 0, 0, 0, 2,	33, 79,30, 30, 0, 0, 0, 0, 0, 0, 0, 0, 12,9,21,10,9,21,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 5, 0, 0, 0, 0, 2,	33, 90, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 12,15,11,15,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 5, 5, 0, 0, 0, 2,	33, 90, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 12,15,14,11,15,14,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 5, 5, 3, 0, 0, 2,	33, 90,31, 53, 0, 0, 0, 0, 0, 0, 0, 0, 12,15,14,7,11,15,14,7,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 7, 0, 0, 0, 0, 4,	31, 61,34, 11,30, 47,31, 68,0, 0, 0, 0, 11,21,12,21,10,21,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 7, 1, 3, 0, 0, 1,	31, 73, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 11,21,1,9,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 7, 1, 5, 0, 0, 3,	31, 77,30, 72,34, 45, 0, 0, 0, 0, 0, 0,11,21,1,14,10,21,1,14,12,21,1,14,0,0,0,0,0,0,0,0,0,0,
4, 7, 1, 5, 3, 0, 1,	31, 79, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 11,21,1,14,7,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 7, 2, 0, 0, 0, 2,	34, 51,30, 79, 0, 0, 0, 0, 0, 0, 0, 0, 12,21,5,10,21,5,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 7, 3, 0, 0, 0, 1,	31, 87, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 11,21,9,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 7, 5, 0, 0, 0, 5, 32,	4, 32, 8,30, 89,34,53,34,60, 0, 0, 11,21,14,11,21,15,10,21,14,12,21,14,12,21,15,0,0,0,0,0,0,0,0,
4, 7, 0, 0, 0, 0, 1,	34, 31, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 12,22,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 0, 0, 0, 0, 0, 3,	37, 22, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 13,14,15,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 1, 0, 0, 0, 0, 2,	34, 72,36, 35, 0, 0, 0, 0, 0, 0, 0, 0, 13,1,14,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 1, 3, 0, 0, 0, 2,	34, 81,36, 42, 0, 0, 0, 0, 0, 0, 0, 0, 13,1,9,14,1,9,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 1, 5, 0, 0, 0, 4,	34, 87,35, 8, 36, 47,36,51, 0, 0, 0, 0, 13,1,14,13,1,15,14,1,14,14,1,15,0,0,0,0,0,0,0,0,0,
5, 1, 5, 3, 0, 0, 2,	35, 2,36, 50, 0, 0, 0, 0, 0, 0, 0, 0, 13,1,14,7,14,1,14,7,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 2, 0, 0, 0, 0, 2,	35, 20,36, 56, 0, 0, 0, 0, 0, 0, 0, 0, 13,5,14,5,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 2, 3, 0, 0, 0, 2,	35, 21,36, 57, 0, 0, 0, 0, 0, 0, 0, 0, 13,5,9,14,5,9,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 2, 5, 0, 0, 0, 2,	35, 37,36, 59, 0, 0, 0, 0, 0, 0, 0, 0, 13,5,14,14,5,14,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 2, 5, 3, 0, 0, 2,	35, 40,36, 60, 0, 0, 0, 0, 0, 0, 0, 0, 13,5,14,7,14,5,14,7,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 3, 0, 0, 0, 0, 2,	35, 48,36, 61, 0, 0, 0, 0, 0, 0, 0, 0, 13,9,14,9,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 3, 1, 5, 0, 0, 4,	35, 62,35, 71,36, 72,36, 81, 0, 0, 0, 0, 0, 13,9,1,14,13,9,1,15,14,9,1,14,14,9,1,15,0,0,0,0,0,0,0,
5, 3, 1, 5, 3, 0, 1,	36, 79, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 14,9,1,14,7,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 3, 2, 0, 0, 0, 2,	35, 79,36, 81, 0, 0, 0, 0, 0, 0, 0, 0, 13,9,5,14,9,5,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 3, 5, 0, 0, 0, 2,	35, 81,36, 90, 0, 0, 0, 0, 0, 0, 0, 0, 13,9,14,14,9,14,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 3, 5, 3, 0, 0, 2,	35, 87,36, 91, 0, 0, 0, 0, 0, 0, 0, 0, 13,9,14,7,14,9,14,7,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 3, 7, 0, 0, 0, 2,	35, 93,37, 3, 0, 0, 0, 0, 0, 0, 0, 0, 13,9,21,14,9,21,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 5, 0, 0, 0, 0, 1,	35, 94, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 13,15,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 5, 7, 0, 0, 0, 1,	36, 17, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 13,15,21,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 5, 5, 3, 0, 0, 1,	37, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 14,15,14,7,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,

5, 7, 0, 0, 0, 0, 3,	36, 20,37, 23,37, 11, 0, 0, 0, 0, 0, 0,	13,21,15,21,14,21,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 7, 1, 5, 0, 0, 1,	37, 15, 0, 0, 0, 0, 0, 0, 0, 0, 0,	14,21,1,14,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 7, 2, 0, 0, 0, 1,	37, 16, 0, 0, 0, 0, 0, 0, 0, 0, 0,	14,21,5,0,
5, 7, 5, 0, 0, 0, 1,	37, 18, 0, 0, 0, 0, 0, 0, 0, 0, 0,	14,21,15,0,
5, 7, 0, 0, 0, 0, 1,	37, 14, 0, 0, 0, 0, 0, 0, 0, 0, 0,	14,22,0,
6, 0, 0, 0, 0, 0, 4,	37, 30, 0, 0, 0, 0, 0, 0, 0, 0, 0,	16,17,18,19,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 1, 0, 0, 0, 0, 2,	37, 30,40, 86, 0, 0, 0, 0, 0, 0, 0,	16,1,19,1,0,
6, 1, 3, 0, 0, 0, 2,	37, 36,40, 89, 0, 0, 0, 0, 0, 0, 0,	16,1,9,19,1,9,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 1, 5, 0, 0, 0, 6, 37, 42,40, 36,37, 55,40, 27,40,93,41, 6, 16,1,14,18,1,15,16,1,15,18,1,14,19,1,14,19,1,15,0,0,0,0,0,0,		
6, 1, 5, 3, 0, 0, 3, 37, 50,40, 31,41, 3, 0, 0, 0, 0, 0, 0, 16,1,14,7,18,1,14,7,19,1,14,7,0,0,0,0,0,0,0,0,0,0,0,0,0,0,		
6, 2, 0, 0, 0, 0, 2,	40, 39,41, 10, 0, 0, 0, 0, 0, 0, 0,	18,5,19,5,0,
6, 2, 3, 0, 0, 0, 1,	37, 62, 0, 0, 0, 0, 0, 0, 0, 0, 0,	16,5,9,0,
6, 2, 5, 0, 0, 0, 3, 37, 71,40, 41,41, 13, 0, 0, 0, 0, 0, 16,5,14,18,5,14,19,5,14,0,0,0,0,0,0,0,0,0,0,0,0,0,		
6, 2, 5, 3, 0, 0, 2, 40, 51,37, 73,41, 14, 0, 0, 0, 0, 0, 18,5,14,7,16,5,14,7,19,5,14,7,0,0,0,0,0,0,0,0,0,0,0,0,0,		
6, 3, 0, 0, 0, 0, 4,	37, 87,38, 58,40, 53,43, 25, 0, 0, 0, 0, 16,9,17,9,18,9,19,9,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,	
6, 3, 1, 0, 0, 0, 2,	38, 94,41, 15, 0, 0, 0, 0, 0, 0, 0,	17,9,1,19,8,1,0,
6, 3, 1, 3, 0, 0, 1,	41, 24, 0, 0, 0, 0, 0, 0, 0, 0, 0,	19,8,1,9,0,
6, 3, 1, 5, 0, 0, 6, 38, 10,38, 14,39, 33, 39, 3,41,26,41,50, 16,9,1,14,16,9,1,15,17,9,1,15,17,9,1,14,19,8,1,14,19,8,1,15,		
6, 3, 1, 5, 3, 0, 2,	39, 25,41, 42, 0, 0, 0, 0, 0, 0, 0,	17,9,1,14,7,19,8,1,14,7,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 2, 0, 0, 0, 3,	38, 18,39, 47,41, 61, 0, 0, 0, 0, 0, 0,	16,9,5,17,9,5,19,8,5,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 2, 5, 0, 0, 1,	41, 73, 0, 0, 0, 0, 0, 0, 0, 0, 0,	19,8,5,14,0,
6, 3, 2, 5, 3, 0, 1,	41, 89, 0, 0, 0, 0, 0, 0, 0, 0, 0,	19,8,5,14,7,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 3, 0, 0, 0, 1,	42, 6, 0, 0, 0, 0, 0, 0, 0, 0, 0,	19,8,9,0,
6, 3, 5, 0, 0, 0, 2,	38, 20, 0, 0, 0, 0, 0, 0, 0, 0, 0,	16,9,14,17,9,14,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 5, 3, 0, 0, 2,	38, 25,39, 64, 0, 0, 0, 0, 0, 0, 0,	16,9,14,7,17,9,14,7,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 5, 5, 3, 0, 1,	39, 77, 0, 0, 0, 0, 0, 0, 0, 0, 0,	17,9,15,14,7,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 5, 7, 0, 0, 1,	42, 53, 0, 0, 0, 0, 0, 0, 0, 0, 0,	19,8,15,21,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 7, 0, 0, 0, 2,	39, 79,42, 63, 0, 0, 0, 0, 0, 0, 0,	17,9,21,19,8,21,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 7, 1, 0, 0, 1,	43, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0,	19,8,21,1,0,
6, 3, 7, 1, 3, 0, 1,	43, 4, 0, 0, 0, 0, 0, 0, 0, 0, 0,	19,8,21,1,9,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 7, 1, 5, 0, 1,	43, 8, 0, 0, 0, 0, 0, 0, 0, 0, 0,	19,8,21,1,14,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 7, 1, 5, 3, 1,	43, 10, 0, 0, 0, 0, 0, 0, 0, 0, 0,	19,8,21,1,14,7,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 7, 3, 0, 0, 1,	43, 13, 0, 0, 0, 0, 0, 0, 0, 0, 0,	19,8,21,9,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 7, 5, 0, 0, 2,	43, 17	

7, 1, 5, 0, 0, 0, 2,	44, 14, 44, 45, 0,	0, 0, 0, 0, 0, 0, 0,	20, 1, 14, 20, 1, 15, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
7, 1, 5, 3, 0, 0, 1,	44, 32, 0,	0, 0, 0, 0, 0, 0, 0, 0,	20, 1, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
7, 2, 0, 0, 0, 0, 1,	44, 56, 0,	0, 0, 0, 0, 0, 0, 0, 0,	20, 5, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
7, 2, 5, 3, 0, 0, 1,	44, 57, 0,	0, 0, 0, 0, 0, 0, 0, 0,	20, 5, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
7, 3, 0, 0, 0, 0, 1,	44, 61, 0,	0, 0, 0, 0, 0, 0, 0, 0,	20, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
7, 3, 1, 5, 0, 0, 2,	44, 76, 44, 84, 0,	0, 0, 0, 0, 0, 0, 0, 0,	20, 9, 1, 14, 20, 9, 1, 15, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
7, 3, 2, 0, 0, 0, 1,	44, 89, 0,	0, 0, 0, 0, 0, 0, 0, 0,	20, 9, 5, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
7, 3, 5, 3, 0, 0, 1,	44, 92, 0,	0, 0, 0, 0, 0, 0, 0, 0,	20, 9, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
7, 5, 5, 3, 0, 0, 1,	45, 8,	0, 0, 0, 0, 0, 0, 0, 0,	20, 15, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
7, 5, 7, 0, 0, 0, 1,	45, 21, 0,	0, 0, 0, 0, 0, 0, 0, 0,	20, 15, 21, 19, 15, 21, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
7, 7, 0, 0, 0, 0, 1,	45, 25, 0,	0, 0, 0, 0, 0, 0, 0, 0,	20, 21, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
7, 7, 1, 5, 0, 0, 1,	45, 36, 0,	0, 0, 0, 0, 0, 0, 0, 0,	20, 21, 1, 14, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
7, 7, 3, 0, 0, 0, 1,	45, 38, 0,	0, 0, 0, 0, 0, 0, 0, 0,	20, 21, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
7, 7, 5, 0, 0, 0, 2,	45, 44, 45, 47, 0,	0, 0, 0, 0, 0, 0, 0, 0,	20, 21, 14, 20, 21, 15, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 0, 0, 0, 0, 0, 4,	45, 58, 0,	0, 0, 0, 0, 0, 0, 0, 0,	23, 24, 25, 26, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 1, 0, 0, 0, 0, 3,	45, 58, 49, 25, 52, 49, 0, 0, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	23, 1, 25, 1, 26, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 1, 3, 0, 0, 0, 2,	45, 65, 52, 52, 0,	0, 0, 0, 0, 0, 0, 0, 0,	23, 1, 9, 26, 1, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 1, 5, 0, 0, 0, 5,	45, 67, 49, 41, 52, 66, 49, 91, 52, 59, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	23, 1, 14, 25, 1, 14, 26, 1, 15, 25, 1, 15, 26, 1, 14, 0, 0, 0, 0, 0, 0, 0, 0,
8, 1, 5, 3, 0, 0, 3,	45, 84, 52, 63, 49, 74, 0, 0, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	23, 1, 14, 7, 26, 1, 14, 7, 25, 1, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 2, 0, 0, 0, 0, 2,	50, 12, 52, 80, 0,	0, 0, 0, 0, 0, 0, 0, 0,	25, 5, 26, 5, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 2, 3, 0, 0, 0, 2,	45, 94, 52, 84, 0,	0, 0, 0, 0, 0, 0, 0, 0,	23, 5, 9, 26, 5, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 2, 5, 0, 0, 0, 2,	47, 33, 52, 85, 0,	0, 0, 0, 0, 0, 0, 0, 0,	23, 5, 14, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 2, 5, 3, 0, 0, 2,	47, 43, 52, 86, 0,	0, 0, 0, 0, 0, 0, 0, 0,	23, 5, 14, 7, 26, 5, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 0, 0, 0, 0, 3,	46, 84, 50, 27, 55, 40, 0,	0, 0, 0, 0, 0, 0, 0, 0,	24, 9, 25, 9, 26, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 1, 0, 0, 0, 2,	47, 25, 52, 90, 0,	0, 0, 0, 0, 0, 0, 0, 0,	24, 9, 1, 26, 8, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 1, 3, 0, 0, 1,	53, 10, 0,	0, 0, 0, 0, 0, 0, 0, 0,	26, 8, 1, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 1, 5, 0, 0, 4,	47, 38, 47, 84, 53, 16, 53, 48, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	24, 9, 1, 14, 24, 9, 1, 15, 26, 8, 1, 14, 26, 8, 1, 15, 0, 0, 0, 0, 0, 0, 0,
8, 3, 1, 5, 3, 0, 2,	47, 64, 53, 33, 0,	0, 0, 0, 0, 0, 0, 0, 0,	24, 9, 1, 14, 7, 26, 8, 1, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 2, 0, 0, 0, 2,	48, 8, 53, 58,	0, 0, 0, 0, 0, 0, 0, 0,	24, 9, 5, 26, 8, 5, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 2, 5, 0, 0, 1,	53, 68, 0,	0, 0, 0, 0, 0, 0, 0, 0,	26, 8, 5, 14, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 2, 5, 3, 0, 1,	53, 84, 0,	0, 0, 0, 0, 0, 0, 0, 0,	26, 8, 5, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 3, 0, 0, 0, 1,	54, 5, 0,	0, 0, 0, 0, 0, 0, 0, 0,	26, 8, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 5, 0, 0, 0, 2,	48, 29, 50, 80, 0,	0, 0, 0, 0, 0, 0, 0, 0,	24, 9, 14, 25, 9, 14, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 5, 3, 0, 0, 2,	48, 39, 51, 2,	0, 0, 0, 0, 0, 0, 0, 0,	24, 9, 14, 7, 25, 9, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 5, 5, 3, 0, 2,	48, 54, 54, 48, 0,	0, 0, 0, 0, 0, 0, 0, 0,	24, 9, 15, 14, 7, 26, 8, 15, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 5, 7, 0, 0, 1,	54, 59, 0,	0, 0, 0, 0, 0, 0, 0, 0,	26, 8, 15, 21, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 7, 0, 0, 0, 2,	48, 61, 54, 73, 0,	0, 0, 0, 0, 0, 0, 0, 0,	24, 9, 21, 26, 8, 21, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 7, 1, 0, 0, 1,	55, 5, 0,	0, 0, 0, 0, 0, 0, 0, 0,	26, 8, 21, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 7, 1, 3, 0, 1,	55, 7, 0,	0, 0, 0, 0, 0, 0, 0, 0,	26, 8, 21, 1, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 7, 1, 5, 0, 1,	55, 8, 0,	0, 0, 0, 0, 0, 0, 0, 0,	26, 8, 21, 1, 14, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 7, 1, 5, 3, 1,	55, 14, 0,	0, 0, 0, 0, 0, 0, 0, 0,	26, 8, 21, 1, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 7, 3, 0, 0, 1,	55, 21, 0,	0, 0, 0, 0, 0, 0, 0, 0,	26, 8, 21, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 7, 5, 0, 0, 2,	55, 27, 55, 29, 0,	0, 0, 0, 0, 0, 0, 0, 0,	26, 8, 21, 14, 26, 8, 21, 15, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 5, 0, 0, 0, 0, 1,	46, 46, 0,	0, 0, 0, 0, 0, 0, 0, 0,	23, 15, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 5, 5, 3, 0, 0, 2,	51, 20, 55, 55, 0,	0, 0, 0, 0, 0, 0, 0, 0,	25, 15, 14, 7, 26, 15, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,


```

/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x0C,0x0C,0x00,0x00,0x1C,0x04,0x04,0x04,0x04,0x04,0x44,0x78,

/*-- 文字: k  --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0xC0,0x40,0x40,0x40,0x4E,0x48,0x50,0x68,0x48,0x44,0xEE,0x00,0x00,

/*-- 文字: l  --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x70,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x7C,0x00,0x00,

/*-- 文字: m  --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xFE,0x49,0x49,0x49,0x49,0x49,0xED,0x00,0x00,

/*-- 文字: n  --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xDC,0x62,0x42,0x42,0x42,0x42,0xE7,0x00,0x00,

/*-- 文字: o  --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x3C,0x42,0x42,0x42,0x42,0x42,0x3C,0x00,0x00,

/*-- 文字: p  --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xD8,0x64,0x42,0x42,0x42,0x44,0x78,0x40,0xE0,

/*-- 文字: q  --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x1E,0x22,0x42,0x42,0x42,0x22,0x1E,0x02,0x07,

/*-- 文字: r  --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xEE,0x32,0x20,0x20,0x20,0x20,0xF8,0x00,0x00,

/*-- 文字: s  --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x3E,0x42,0x40,0x3C,0x02,0x42,0x7C,0x00,0x00,

/*-- 文字: t  --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x00,0x00,0x10,0x10,0x7C,0x10,0x10,0x10,0x10,0x10,0x0C,0x00,0x00,

/*-- 文字: u  --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/

```

```

0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xC6,0x42,0x42,0x42,0x42,0x46,0x3B,0x00,0x00,

/*-- 文字: v --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xE7,0x42,0x24,0x24,0x28,0x10,0x10,0x00,0x00,

/*-- 文字: w --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xD7,0x92,0x92,0xAA,0xAA,0x44,0x44,0x00,0x00,

/*-- 文字: x --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x6E,0x24,0x18,0x18,0x18,0x24,0x76,0x00,0x00,

/*-- 文字: y --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xE7,0x42,0x24,0x24,0x28,0x18,0x10,0x10,0xE0,

/*-- 文字: z --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x7E,0x44,0x08,0x10,0x10,0x22,0x7E,0x00,0x00,
};

unsigned int beng[26][16]={

/*-- 文字: A --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x10,0x10,0x18,0x28,0x28,0x24,0x3C,0x44,0x42,0x42,0xE7,0x00,0x00,

/*-- 文字: B --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xF8,0x44,0x44,0x44,0x78,0x44,0x42,0x42,0x42,0x44,0xF8,0x00,0x00,

/*-- 文字: C --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x3E,0x42,0x42,0x80,0x80,0x80,0x80,0x80,0x42,0x44,0x38,0x00,0x00,

/*-- 文字: D --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xF8,0x44,0x42,0x42,0x42,0x42,0x42,0x42,0x44,0xF8,0x00,0x00,

/*-- 文字: E --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/

```

```

0x00,0x00,0x00,0xFC,0x42,0x48,0x48,0x78,0x48,0x48,0x40,0x42,0x42,0xFC,0x00,0x00,

/*-- 文字: F --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xFC,0x42,0x48,0x48,0x78,0x48,0x48,0x40,0x40,0x40,0xE0,0x00,0x00,

/*-- 文字: G --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x3C,0x44,0x44,0x80,0x80,0x80,0x8E,0x84,0x44,0x44,0x38,0x00,0x00,

/*-- 文字: H --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xE7,0x42,0x42,0x42,0x42,0x7E,0x42,0x42,0x42,0x42,0xE7,0x00,0x00,

/*-- 文字: I --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x7C,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x7C,0x00,0x00,

/*-- 文字: J --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x3E,0x08,0x08,0x08,0x08,0x08,0x08,0x08,0x08,0x08,0x88,0xF0,

/*-- 文字: K --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xEE,0x44,0x48,0x50,0x70,0x50,0x48,0x48,0x44,0xEE,0x00,0x00,

/*-- 文字: L --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xE0,0x40,0x40,0x40,0x40,0x40,0x40,0x40,0x40,0xFE,0x00,0x00,

/*-- 文字: M --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xEE,0x6C,0x6C,0x6C,0x6C,0x54,0x54,0x54,0x54,0x54,0xD6,0x00,0x00,

/*-- 文字: N --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xC7,0x62,0x62,0x52,0x52,0x4A,0x4A,0x4A,0x46,0x46,0xE2,0x00,0x00,

/*-- 文字: O --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x38,0x44,0x82,0x82,0x82,0x82,0x82,0x82,0x44,0x38,0x00,0x00,

/*-- 文字: P --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xFC,0x42,0x42,0x42,0x42,0x7C,0x40,0x40,0x40,0x40,0xE0,0x00,0x00,

```

```

/*-- 文字: Q --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x38,0x44,0x82,0x82,0x82,0x82,0xB2,0xCA,0x4C,0x38,0x06,0x00,

/*-- 文字: R --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xFC,0x42,0x42,0x42,0x7C,0x48,0x48,0x44,0x44,0x42,0xE3,0x00,0x00,

/*-- 文字: S --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x3E,0x42,0x42,0x40,0x20,0x18,0x04,0x02,0x42,0x42,0x7C,0x00,0x00,

/*-- 文字: T --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xFE,0x92,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x38,0x00,0x00,

/*-- 文字: U --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xE7,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x3C,0x00,0x00,

/*-- 文字: V --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xE7,0x42,0x42,0x44,0x24,0x24,0x28,0x28,0x18,0x10,0x10,0x00,0x00,

/*-- 文字: W --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xD6,0x92,0x92,0x92,0x92,0xAA,0xAA,0x6C,0x44,0x44,0x44,0x00,0x00,
/*-- 文字: X --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xE7,0x42,0x24,0x24,0x18,0x18,0x18,0x24,0x24,0x42,0xE7,0x00,0x00,

/*-- 文字: Y --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xEE,0x44,0x44,0x28,0x28,0x10,0x10,0x10,0x10,0x10,0x38,0x00,0x00,

/*-- 文字: Z --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x7E,0x84,0x04,0x08,0x08,0x10,0x20,0x20,0x42,0x42,0xFC,0x00,0x00,

};

```

```

unsigned int num[10][16]={

```

```

/*-- 文字: 0 --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x18,0x24,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x24,0x18,0x00,0x00,

/*-- 文字: 1 --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x10,0x70,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x7C,0x00,0x00,

/*-- 文字: 2 --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x3C,0x42,0x42,0x42,0x04,0x04,0x08,0x10,0x20,0x42,0x7E,0x00,0x00,

/*-- 文字: 3 --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x3C,0x42,0x42,0x04,0x18,0x04,0x02,0x02,0x42,0x44,0x38,0x00,0x00,

/*-- 文字: 4 --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x04,0x0C,0x14,0x24,0x24,0x44,0x44,0x7E,0x04,0x04,0x1E,0x00,0x00,

/*-- 文字: 5 --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x7E,0x40,0x40,0x40,0x58,0x64,0x02,0x02,0x42,0x44,0x38,0x00,0x00,

/*-- 文字: 6 --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x1C,0x24,0x40,0x40,0x58,0x64,0x42,0x42,0x42,0x24,0x18,0x00,0x00,

/*-- 文字: 7 --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x7E,0x44,0x44,0x08,0x08,0x10,0x10,0x10,0x10,0x10,0x10,0x00,0x00,

/*-- 文字: 8 --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x3C,0x42,0x42,0x42,0x24,0x18,0x24,0x42,0x42,0x42,0x3C,0x00,0x00,

/*-- 文字: 9 --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x18,0x24,0x42,0x42,0x42,0x26,0x1A,0x02,0x02,0x24,0x38,0x00,0x00,

};

unsigned int sig[27][16]={
/*-- 文字: , --*/

```



```

/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x60,0x60,0x20,0xC0,

/*-- 文字: . --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x60,0x60,0x00,0x00,

/*-- 文字: / --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x01,0x02,0x02,0x04,0x04,0x08,0x08,0x10,0x10,0x20,0x20,0x40,0x40,0x00,

/*-- 文字: \ --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x40,0x40,0x20,0x20,0x10,0x10,0x10,0x08,0x08,0x04,0x04,0x02,0x02,

/*-- 文字: ; --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x10,0x00,0x00,0x00,0x00,0x10,0x10,0x20,

/*-- 文字: : --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x18,0x18,0x00,0x00,0x00,0x00,0x18,0x18,0x00,0x00,

/*-- 文字: < --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x02,0x04,0x08,0x10,0x20,0x40,0x20,0x10,0x08,0x04,0x02,0x00,0x00,

/*-- 文字: > --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x40,0x20,0x10,0x08,0x04,0x02,0x04,0x08,0x10,0x20,0x40,0x00,0x00,

/*-- 文字: ? --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x3C,0x42,0x42,0x62,0x02,0x04,0x08,0x08,0x00,0x18,0x18,0x00,0x00,

/*-- 文字: " --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x12,0x36,0x24,0x48,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,

/*-- 文字: ' --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x60,0x60,0x20,0xC0,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,

/*-- 文字: [ --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/

```

```

0x00,0x1E,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x1E,0x00,

/*-- 文字: | --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x78,0x08,0x08,0x08,0x08,0x08,0x08,0x08,0x08,0x08,0x08,0x78,0x00,

/*-- 文字: { --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x03,0x04,0x04,0x04,0x04,0x04,0x08,0x04,0x04,0x04,0x04,0x04,0x03,0x00,

/*-- 文字: } --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x60,0x10,0x10,0x10,0x10,0x10,0x08,0x10,0x10,0x10,0x10,0x10,0x60,0x00,

/*-- 文字: ! --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x00,0x18,0x18,0x00,0x00,

/*-- 文字: @ --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x38,0x44,0x5A,0xAA,0xAA,0xAA,0xAA,0xB4,0x42,0x44,0x38,0x00,0x00,

/*-- 文字: # --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x24,0x24,0x24,0xFE,0x48,0x48,0x48,0xFE,0x48,0x48,0x48,0x00,0x00,

/*-- 文字: $ --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x10,0x38,0x54,0x54,0x50,0x30,0x18,0x14,0x14,0x54,0x54,0x38,0x10,0x10,

/*-- 文字: % --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x44,0xA4,0xA8,0xA8,0xA8,0x54,0x1A,0x2A,0x2A,0x2A,0x44,0x00,0x00,

/*-- 文字: & --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x30,0x48,0x48,0x48,0x50,0x6E,0xA4,0x94,0x88,0x89,0x76,0x00,0x00,

/*-- 文字: * --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x10,0x10,0xD6,0x38,0x38,0xD6,0x10,0x10,0x00,0x00,0x00,0x00,

/*-- 文字: ( --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x02,0x04,0x08,0x08,0x10,0x10,0x10,0x10,0x10,0x10,0x08,0x08,0x04,0x02,0x00,

```

```

/*-- 文字: ) --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x40,0x20,0x10,0x10,0x08,0x08,0x08,0x08,0x08,0x08,0x10,0x10,0x20,0x40,0x00,

/*-- 文字: - --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x7F,0x00,0x00,0x00,0x00,0x00,0x00,0x00,

/*-- 文字: + --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x10,0x10,0x10,0x10,0xFE,0x10,0x10,0x10,0x10,0x00,0x00,0x00,

/*-- 文字: = --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x00,0x00,0xFE,0x00,0x00,0x00,0xFE,0x00,0x00,0x00,0x00,0x00,
};

```

INDEX1 代码

```

unsigned int hzk_index[437][25]={
1, 0, 0, 0, 0, 0, 3,    16, 1,    1, 2, 3, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 0, 0, 0, 0, 2,    16, 37,   2, 1, 3, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 0, 0, 0, 0, 0,    18, 33,   0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 3, 0, 0, 0, 2,    16, 55,   2, 1, 9, 3, 1, 9, 0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 3, 0, 0, 0, 0,    18, 44,   0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 5, 0, 0, 0, 4,    16, 63,   2, 1, 14, 2, 1, 15, 3, 1, 14, 3, 1, 15, 0, 0, 0, 0,
1, 1, 5, 0, 0, 0, 0,    16, 90,   0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 5, 0, 0, 0, 0,    18, 45,   0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 5, 0, 0, 0, 0,    18, 57,   0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 5, 3, 0, 0, 2,    16, 78,   2, 1, 14, 7, 3, 1, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 5, 3, 0, 0, 0,    18, 53,   0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
1, 2, 0, 0, 0, 0, 1,    18, 62,   3, 5, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
1, 2, 3, 0, 0, 0, 1,    17, 13,   2, 5, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
1, 2, 5, 0, 0, 0, 1,    17, 28,   2, 5, 14, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
1, 2, 5, 3, 0, 0, 2,    17, 32,   2, 5, 14, 7, 3, 5, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0,
1, 2, 5, 3, 0, 0, 0,    18, 67,   0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
1, 3, 0, 0, 0, 0, 4,    16, 3,    1, 9, 2, 9, 3, 9, 3, 8, 0, 0, 0, 0, 0, 0, 0, 0,
1, 3, 0, 0, 0, 0, 0,    17, 38,   0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
1, 3, 0, 0, 0, 0, 0,    20, 35,   0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
1, 3, 1, 0, 0, 0, 1,    18, 69,   3, 8, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
1, 3, 1, 3, 0, 0, 1,    18, 80,   3, 8, 1, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
1, 3, 1, 5, 0, 0, 4,    17, 62,   2, 9, 1, 14, 2, 9, 1, 15, 3, 8, 1, 14, 3, 8, 1, 15,
1, 3, 1, 5, 0, 0, 0,    17, 74,   0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
1, 3, 1, 5, 0, 0, 0,    18, 83,   0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,

```

1, 3, 1, 5, 0, 0, 0,	19, 12,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
1, 3, 1, 5, 3, 0, 1,	18, 93,	3,8,1,14,7,0,0,0,0,0,0,0,0,0,0,
1, 3, 2, 0, 0, 0, 2,	17, 78,	2,9,5,3,8,5,0,0,0,0,0,0,0,0,0,
1, 3, 2, 0, 0, 0, 0,	19, 21,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
1, 3, 2, 5, 0, 0, 1,	19, 28,	3,8,5,14,0,0,0,0,0,0,0,0,0,0,0,
1, 3, 2, 5, 3, 0, 1,	19, 37,	3,8,5,14,7,0,0,0,0,0,0,0,0,0,0,
1, 3, 3, 0, 0, 0, 1,	19, 52,	3,8,9,0,0,0,0,0,0,0,0,0,0,0,0,
1, 3, 5, 0, 0, 0, 2,	17, 82,	2,9,14,3,8,15,0,0,0,0,0,0,0,0,0,
1, 3, 5, 3, 0, 0, 1,	17, 91,	2,9,14,7,0,0,0,0,0,0,0,0,0,0,0,
1, 3, 5, 5, 3, 0, 1,	19, 68,	3,8,15,14,7,0,0,0,0,0,0,0,0,0,0,
1, 3, 5, 7, 0, 0, 1,	19, 73,	3,8,15,21,0,0,0,0,0,0,0,0,0,0,0,
1, 3, 7, 0, 0, 0, 1,	19, 85,	3,8,21,0,0,0,0,0,0,0,0,0,0,0,0,
1, 3, 7, 1, 3, 0, 1,	20, 7,	3,8,21,1,9,0,0,0,0,0,0,0,0,0,0,
1, 3, 7, 1, 5, 0, 1,	20, 8,	3,8,21,1,14,0,0,0,0,0,0,0,0,0,0,
1, 3, 7, 1, 5, 3, 1,	20, 15,	3,8,21,1,14,7,0,0,0,0,0,0,0,0,0,
1, 3, 7, 3, 0, 0, 1,	20, 21,	3,8,21,9,0,0,0,0,0,0,0,0,0,0,0,
1, 3, 7, 5, 0, 0, 2,	20, 26,	3,8,21,14,3,8,21,15,0,0,0,0,0,0,0,
1, 3, 7, 5, 0, 0, 0,	20, 33,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
1, 5, 0, 0, 0, 0, 3,	16, 16,	1,14,1,15,2,15,0,0,0,0,0,0,0,0,0,
1, 5, 0, 0, 0, 0, 0,	16, 28,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
1, 5, 0, 0, 0, 0, 0,	18, 3,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
1, 5, 5, 3, 0, 0, 1,	20, 47,	3,15,14,7,0,0,0,0,0,0,0,0,0,0,0,
1, 5, 7, 0, 0, 0, 1,	20, 53,	3,15,21,0,0,0,0,0,0,0,0,0,0,0,0,
1, 7, 0, 0, 0, 0, 2,	18, 22,	2,21,3,21,0,0,0,0,0,0,0,0,0,0,0,
1, 7, 0, 0, 0, 0, 0,	20, 54,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
1, 7, 1, 5, 0, 0, 1,	20, 58,	3,21,1,14,0,0,0,0,0,0,0,0,0,0,0,
1, 7, 3, 0, 0, 0, 1,	20, 61,	3,21,9,0,0,0,0,0,0,0,0,0,0,0,0,
1, 7, 5, 0, 0, 0, 2,	20, 69,	3,21,14,3,21,15,0,0,0,0,0,0,0,0,0,
1, 7, 5, 0, 0, 0, 0,	20, 72,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
2, 0, 0, 0, 0, 0, 3,	22, 74,	4,5,6,0,0,0,0,0,0,0,0,0,0,0,0,
2, 1, 0, 0, 0, 0, 2,	20, 78,	4,1,6,1,0,0,0,0,0,0,0,0,0,0,0,
2, 1, 0, 0, 0, 0, 0,	23, 2,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
2, 1, 3, 0, 0, 0, 1,	20, 84,	4,1,9,0,0,0,0,0,0,0,0,0,0,0,0,
2, 1, 5, 0, 0, 0, 3,	21, 2,	4,1,14,4,1,15,6,1,14,0,0,0,0,0,0,
2, 1, 5, 0, 0, 0, 0,	21, 22,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
2, 1, 5, 0, 0, 0, 0,	23, 10,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
2, 1, 5, 3, 0, 0, 2,	21, 17,	4,1,14,7,6,1,14,7,0,0,0,0,0,0,0,
2, 1, 5, 3, 0, 0, 0,	23, 27,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
2, 2, 0, 0, 0, 0, 1,	21, 34,	4,5,0,0,0,0,0,0,0,0,0,0,0,0,0,
2, 2, 3, 0, 0, 0, 1,	23, 38,	6,5,9,0,0,0,0,0,0,0,0,0,0,0,0,
2, 2, 5, 0, 0, 0, 1,	23, 50,	6,5,14,0,0,0,0,0,0,0,0,0,0,0,0,
2, 2, 5, 3, 0, 0, 2,	23, 65,	6,5,14,7,4,5,14,7,0,0,0,0,0,0,0,
2, 2, 5, 3, 0, 0, 0,	21, 37,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
2, 3, 0, 0, 0, 0, 1,	21, 44,	4,9,0,0,0,0,0,0,0,0,0,0,0,0,0,
2, 3, 1, 5, 0, 0, 2,	21, 63,	4,9,1,14,4,9,1,15,0,0,0,0,0,0,0,

2, 3, 1, 5, 0, 0, 0,	21, 79,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
2, 3, 2, 0, 0, 0, 1,	21, 88,	4,9,5,0,0,0,0,0,0,0,0,0,0,0,0,
2, 3, 5, 3, 0, 0, 1,	22, 1,	4,9,14,7,0,0,0,0,0,0,0,0,0,0,0,
2, 3, 7, 0, 0, 0, 1,	22, 10,	4,9,21,0,0,0,0,0,0,0,0,0,0,0,0,
2, 5, 0, 0, 0, 0, 2,	22, 87,	5,14,6,15,0,0,0,0,0,0,0,0,0,0,0,
2, 5, 0, 0, 0, 0, 0,	23, 80,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
2, 5, 5, 3, 0, 0, 1,	22, 11,	4,15,14,7,0,0,0,0,0,0,0,0,0,0,0,
2, 5, 7, 0, 0, 0, 2,	22, 21,	4,15,21,6,15,21,0,0,0,0,0,0,0,0,0,
2, 5, 7, 0, 0, 0, 0,	23, 81,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
2, 6, 0, 0, 0, 0, 1,	22, 88,	5,18,0,0,0,0,0,0,0,0,0,0,0,0,0,
2, 7, 0, 0, 0, 0, 2,	22, 28,	4,21,6,21,0,0,0,0,0,0,0,0,0,0,0,
2, 7, 0, 0, 0, 0, 0,	23, 82,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
2, 7, 1, 5, 0, 0, 1,	22, 43,	4,21,1,14,0,0,0,0,0,0,0,0,0,0,0,
2, 7, 3, 0, 0, 0, 1,	22, 49,	4,21,9,0,0,0,0,0,0,0,0,0,0,0,0,
2, 7, 5, 0, 0, 0, 2,	22, 53,	4,21,14,4,21,15,0,0,0,0,0,0,0,0,0,
2, 7, 5, 0, 0, 0, 0,	22, 62,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
3, 0, 0, 0, 0, 0, 3,	24, 33,	7,8,9,0,0,0,0,0,0,0,0,0,0,0,0,
3, 1, 0, 0, 0, 0, 2,	24, 33,	7,1,8,1,0,0,0,0,0,0,0,0,0,0,0,
3, 1, 0, 0, 0, 0, 0,	25, 94,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
3, 1, 3, 0, 0, 0, 2,	24, 35,	7,1,9,8,1,9,0,0,0,0,0,0,0,0,0,
3, 1, 3, 0, 0, 0, 0,	26, 1,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
3, 1, 5, 0, 0, 0, 4,	24, 41,	7,1,14,7,1,15,8,1,14,8,1,15,0,0,0,0,
3, 1, 5, 0, 0, 0, 0,	24, 61,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
3, 1, 5, 0, 0, 0, 0,	26, 8,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
3, 1, 5, 0, 0, 0, 0,	26, 30,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
3, 1, 5, 3, 0, 0, 2,	24, 52,	7,1,14,7,8,1,14,7,0,0,0,0,0,0,0,
3, 1, 5, 3, 0, 0, 0,	26, 27,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
3, 2, 0, 0, 0, 0, 2,	24, 71,	7,5,8,5,0,0,0,0,0,0,0,0,0,0,0,
3, 2, 0, 0, 0, 0, 0,	26, 39,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
3, 2, 3, 0, 0, 0, 2,	24, 88,	7,5,9,8,5,9,0,0,0,0,0,0,0,0,0,
3, 2, 3, 0, 0, 0, 0,	26, 57,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
3, 2, 5, 0, 0, 0, 2,	24, 89,	7,5,14,8,5,14,0,0,0,0,0,0,0,0,0,
3, 2, 5, 0, 0, 0, 0,	26, 59,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
3, 2, 5, 3, 0, 0, 2,	24, 91,	7,5,14,7,8,5,14,7,0,0,0,0,0,0,0,
3, 2, 5, 3, 0, 0, 0,	26, 63,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
3, 5, 0, 0, 0, 0, 2,	25, 4,	7,15,8,15,0,0,0,0,0,0,0,0,0,0,0,
3, 5, 5, 0, 0, 0, 2,	25, 4,	7,15,14,8,15,14,0,0,0,0,0,0,0,0,0,
3, 5, 5, 3, 0, 0, 2,	25, 4,	7,15,14,7,8,15,14,7,0,0,0,0,0,0,0,
3, 5, 5, 3, 0, 0, 0,	26, 68,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
3, 5, 7, 0, 0, 0, 2,	25, 19,	7,15,21,8,15,21,0,0,0,0,0,0,0,0,0,
3, 5, 7, 0, 0, 0, 0,	26, 77,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
3, 7, 0, 0, 0, 0, 2,	25, 28,	7,21,8,21,0,0,0,0,0,0,0,0,0,0,0,
3, 7, 0, 0, 0, 0, 0,	26, 84,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
3, 7, 1, 0, 0, 0, 2,	25, 47,	7,21,1,8,21,1,0,0,0,0,0,0,0,0,0,
3, 7, 1, 0, 0, 0, 0,	27, 8,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,

3, 7, 1, 3, 0, 0, 1,	25, 52,	7,21,1,9,0,0,0,0,0,0,0,0,0,0,0,
3, 7, 1, 5, 0, 0, 2,	25, 55,	7,21,1,14,8,21,1,14,0,0,0,0,0,0,0,
3, 7, 1, 5, 0, 0, 0,	27, 22,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
3, 7, 1, 5, 3, 0, 2,	25, 66,	7,21,1,14,7,8,21,1,14,7,0,0,0,0,0,0,
3, 7, 1, 5, 3, 0, 0,	27, 36,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
3, 7, 3, 0, 0, 0, 2,	25, 69,	7,21,9,8,21,9,0,0,0,0,0,0,0,0,0,
3, 7, 3, 0, 0, 0, 0,	27, 50,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
3, 7, 5, 0, 0, 0, 4,	25, 85,	7,21,14,8,21,14,7,21,15,8,21,15,0,0,0,0,
3, 7, 5, 0, 0, 0, 0,	27, 71,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
3, 7, 5, 0, 0, 0, 0,	25, 88,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
3, 7, 5, 0, 0, 0, 0,	27, 77,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 0, 0, 0, 0, 0, 3,	31, 6,	10,11,12,0,0,0,0,0,0,0,0,0,0,0,0,
4, 1, 0, 0, 0, 0, 2,	31, 6,	11,1,12,1,0,0,0,0,0,0,0,0,0,0,0,
4, 1, 0, 0, 0, 0, 0,	32, 12,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 1, 3, 0, 0, 0, 2,	31, 10,	11,1,9,12,1,9,0,0,0,0,0,0,0,0,0,
4, 1, 3, 0, 0, 0, 0,	32, 19,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 1, 5, 0, 0, 0, 4,	32, 22,	12,1,14,12,1,15,11,1,14,11,1,15,0,0,0,0,
4, 1, 5, 0, 0, 0, 0,	32, 44,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 1, 5, 0, 0, 0, 0,	31, 15,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 1, 5, 0, 0, 0, 0,	31, 28,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 1, 5, 3, 0, 0, 2,	31, 21,	11,1,14,7,12,1,14,7,0,0,0,0,0,0,0,
4, 1, 5, 3, 0, 0, 0,	32, 27,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 2, 0, 0, 0, 0, 2,	32, 53,	12,5,11,5,0,0,0,0,0,0,0,0,0,0,0,
4, 2, 0, 0, 0, 0, 0,	31, 32,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 2, 3, 0, 0, 0, 1,	32, 55,	12,5,9,0,0,0,0,0,0,0,0,0,0,0,0,
4, 2, 5, 0, 0, 0, 2,	31, 47,	12,5,14,11,5,14,0,0,0,0,0,0,0,0,0,
4, 2, 5, 3, 0, 0, 2,	31, 51,	11,5,14,7,12,5,14,7,0,0,0,0,0,0,0,
4, 2, 5, 3, 0, 0, 0,	32, 66,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 3, 0, 0, 0, 0, 2,	27, 87,	10,9,12,9,0,0,0,0,0,0,0,0,0,0,0,
4, 3, 0, 0, 0, 0, 0,	32, 69,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 3, 1, 0, 0, 0, 2,	28, 47,	10,9,1,12,9,1,0,0,0,0,0,0,0,0,0,
4, 3, 1, 0, 0, 0, 0,	33, 9,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 3, 1, 5, 0, 0, 4,	28, 63,	10,9,1,14,12,9,1,14,10,9,1,15,12,9,1,15,
4, 3, 1, 5, 0, 0, 0,	33, 10,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 3, 1, 5, 0, 0, 0,	29, 22,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 3, 1, 5, 0, 0, 0,	33, 35,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 3, 1, 5, 3, 0, 2,	29, 9,	10,9,1,14,7,12,9,1,14,7,0,0,0,0,0,0,
4, 3, 1, 5, 3, 0, 0,	33, 24,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 3, 2, 0, 0, 0, 2,	29, 50,	10,9,5,12,9,5,0,0,0,0,0,0,0,0,0,
4, 3, 2, 0, 0, 0, 0,	33, 48,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 3, 5, 0, 0, 0, 2,	29, 77,	10,9,14,12,9,14,0,0,0,0,0,0,0,0,0,
4, 3, 5, 0, 0, 0, 0,	33, 53,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 3, 5, 3, 0, 0, 2,	30, 3,	10,9,14,7,12,9,14,7,0,0,0,0,0,0,0,
4, 3, 5, 3, 0, 0, 0,	33, 65,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 3, 5, 5, 3, 0, 1,	30, 28,	10,9,15,14,7,0,0,0,0,0,0,0,0,0,0,

4, 3, 7, 0, 0, 0, 2,	33, 79,	12,9,21,10,9,21,0,0,0,0,0,0,0,0,0,
4, 3, 7, 0, 0, 0, 0,	30, 30,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 5, 0, 0, 0, 0, 2,	33, 90,	12,15,11,15,0,0,0,0,0,0,0,0,0,0,
4, 5, 5, 0, 0, 0, 2,	33, 90,	12,15,14,11,15,14,0,0,0,0,0,0,0,0,
4, 5, 5, 3, 0, 0, 2,	33, 90,	12,15,14,7,11,15,14,7,0,0,0,0,0,0,
4, 5, 5, 3, 0, 0, 0,	31, 53,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 7, 0, 0, 0, 0, 3,	31, 61,	11,21,12,21,10,21,0,0,0,0,0,0,0,0,
4, 7, 0, 0, 0, 0, 0,	34, 11,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 7, 0, 0, 0, 0, 0,	30, 47,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 7, 1, 0, 0, 0, 0,	31, 68,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 7, 1, 3, 0, 0, 1,	31, 73,	11,21,1,9,0,0,0,0,0,0,0,0,0,0,
4, 7, 1, 5, 0, 0, 3,	31, 77,	11,21,1,14,10,21,1,14,12,21,1,14,0,0,0,
4, 7, 1, 5, 0, 0, 0,	30, 72,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 7, 1, 5, 0, 0, 0,	34, 45,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 7, 1, 5, 3, 0, 1,	31, 79,	11,21,1,14,7,0,0,0,0,0,0,0,0,0,
4, 7, 2, 0, 0, 0, 2,	34, 51,	12,21,5,10,21,5,0,0,0,0,0,0,0,0,
4, 7, 2, 0, 0, 0, 0,	30, 79,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 7, 3, 0, 0, 0, 1,	31, 87,	11,21,9,0,0,0,0,0,0,0,0,0,0,0,
4, 7, 5, 0, 0, 0, 5,	32, 4,	11,21,14,11,21,15,10,21,14,12,21,14,12,21,15,0,
4, 7, 5, 0, 0, 0, 0,	32, 8,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 7, 5, 0, 0, 0, 0,	30, 89,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 7, 5, 0, 0, 0, 0,	34, 53,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 7, 5, 0, 0, 0, 0,	34, 60,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
4, 7, 0, 0, 0, 0, 1,	34, 31,	12,22,0,0,0,0,0,0,0,0,0,0,0,0,
5, 0, 0, 0, 0, 0, 3,	37, 22,	13,14,15,0,0,0,0,0,0,0,0,0,0,0,
5, 1, 0, 0, 0, 0, 2,	34, 72,	13,1,14,1,0,0,0,0,0,0,0,0,0,0,
5, 1, 0, 0, 0, 0, 0,	36, 35,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 1, 3, 0, 0, 0, 2,	34, 81,	13,1,9,14,1,9,0,0,0,0,0,0,0,0,
5, 1, 3, 0, 0, 0, 0,	36, 42,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 1, 5, 0, 0, 0, 4,	34, 87,	13,1,14,13,1,15,14,1,14,14,1,15,0,0,0,
5, 1, 5, 0, 0, 0, 0,	35, 8,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 1, 5, 0, 0, 0, 0,	36, 47,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 1, 5, 0, 0, 0, 0,	36, 51,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 1, 5, 3, 0, 0, 2,	35, 2,	13,1,14,7,14,1,14,7,0,0,0,0,0,0,
5, 1, 5, 3, 0, 0, 0,	36, 50,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 2, 0, 0, 0, 0, 2,	35, 20,	13,5,14,5,0,0,0,0,0,0,0,0,0,0,
5, 2, 0, 0, 0, 0, 0,	36, 56,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 2, 3, 0, 0, 0, 2,	35, 21,	13,5,9,14,5,9,0,0,0,0,0,0,0,0,
5, 2, 3, 0, 0, 0, 0,	36, 57,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 2, 5, 0, 0, 0, 2,	35, 37,	13,5,14,14,5,14,0,0,0,0,0,0,0,0,
5, 2, 5, 0, 0, 0, 0,	36, 59,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 2, 5, 3, 0, 0, 2,	35, 40,	13,5,14,7,14,5,14,7,0,0,0,0,0,0,
5, 2, 5, 3, 0, 0, 0,	36, 60,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 3, 0, 0, 0, 0, 2,	35, 48,	13,9,14,9,0,0,0,0,0,0,0,0,0,0,
5, 3, 0, 0, 0, 0, 0,	36, 61,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,

5, 3, 1, 5, 0, 0, 4,	35, 62,	13,9,1,14,13,9,1,15,14,9,1,14,14,9,1,15,
5, 3, 1, 5, 0, 0, 0,	35, 71,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 3, 1, 5, 0, 0, 0,	36, 72,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 3, 1, 5, 0, 0, 0,	36, 81,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 3, 1, 5, 3, 0, 1,	36, 79,	14,9,1,14,7,0,0,0,0,0,0,0,0,0,0,0,
5, 3, 2, 0, 0, 0, 2,	35, 79,	13,9,5,14,9,5,0,0,0,0,0,0,0,0,0,0,
5, 3, 2, 0, 0, 0, 0,	36, 81,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 3, 5, 0, 0, 0, 2,	35, 81,	13,9,14,14,9,14,0,0,0,0,0,0,0,0,0,0,
5, 3, 5, 0, 0, 0, 0,	36, 90,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 3, 5, 3, 0, 0, 2,	35, 87,	13,9,14,7,14,9,14,7,0,0,0,0,0,0,0,0,
5, 3, 5, 3, 0, 0, 0,	36, 91,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 3, 7, 0, 0, 0, 2,	35, 93,	13,9,21,14,9,21,0,0,0,0,0,0,0,0,0,0,
5, 3, 7, 0, 0, 0, 0,	37, 3,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 5, 0, 0, 0, 0, 1,	35, 94,	13,15,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 5, 7, 0, 0, 0, 1,	36, 17,	13,15,21,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 5, 5, 3, 0, 0, 1,	37, 7,	14,15,14,7,0,0,0,0,0,0,0,0,0,0,0,0,
5, 7, 0, 0, 0, 0, 3,	36, 20,	13,21,15,21,14,21,0,0,0,0,0,0,0,0,0,0,
5, 7, 0, 0, 0, 0, 0,	37, 23,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 7, 0, 0, 0, 0, 0,	37, 11,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 7, 1, 5, 0, 0, 1,	37, 15,	14,21,1,14,0,0,0,0,0,0,0,0,0,0,0,0,
5, 7, 2, 0, 0, 0, 1,	37, 16,	14,21,5,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 7, 5, 0, 0, 0, 1,	37, 18,	14,21,15,0,0,0,0,0,0,0,0,0,0,0,0,0,
5, 7, 0, 0, 0, 0, 1,	37, 14,	14,22,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 0, 0, 0, 0, 0, 4,	37, 30,	16,17,18,19,0,0,0,0,0,0,0,0,0,0,0,0,
6, 1, 0, 0, 0, 0, 2,	37, 30,	16,1,19,1,0,0,0,0,0,0,0,0,0,0,0,0,
6, 1, 0, 0, 0, 0, 0,	40, 86,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 1, 3, 0, 0, 0, 2,	37, 36,	16,1,9,19,1,9,0,0,0,0,0,0,0,0,0,0,
6, 1, 3, 0, 0, 0, 0,	40, 89,	19,1,9,20,1,9,0,0,0,0,0,0,0,0,0,0,
6, 1, 5, 0, 0, 0, 6,	37, 42,	16,1,14,18,1,15,16,1,15,18,1,14,19,1,14,19,1,15,
6, 1, 5, 0, 0, 0, 0,	40, 36,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 1, 5, 0, 0, 0, 0,	37, 55,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 1, 5, 0, 0, 0, 0,	40, 27,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 1, 5, 0, 0, 0, 0,	40, 93,	19,1,14,19,1,15,20,1,14,20,1,15,0,0,0,0,
6, 1, 5, 0, 0, 0, 0,	41, 6,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 1, 5, 3, 0, 0, 3,	37, 50,	16,1,14,7,18,1,14,7,19,1,14,7,0,0,0,0,
6, 1, 5, 3, 0, 0, 0,	40, 31,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 1, 5, 3, 0, 0, 0,	41, 3,	19,1,14,7,20,1,14,7,0,0,0,0,0,0,0,0,
6, 2, 0, 0, 0, 0, 2,	40, 39,	18,5,19,5,0,0,0,0,0,0,0,0,0,0,0,0,
6, 2, 0, 0, 0, 0, 0,	41, 10,	19,5,20,5,0,0,0,0,0,0,0,0,0,0,0,0,
6, 2, 3, 0, 0, 0, 1,	37, 62,	16,5,9,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 2, 5, 0, 0, 0, 3,	37, 71,	16,5,14,18,5,14,19,5,14,0,0,0,0,0,0,0,
6, 2, 5, 0, 0, 0, 0,	40, 41,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 2, 5, 0, 0, 0, 0,	41, 13,	19,5,14,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 2, 5, 3, 0, 0, 2,	40, 51,	18,5,14,7,16,5,14,7,19,5,14,7,0,0,0,0,
6, 2, 5, 3, 0, 0, 0,	37, 73,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,

6, 2, 5, 3, 0, 0, 0,	41, 14,	19,5,14,7,20,5,14,7,0,0,0,0,0,0,0,
6, 3, 0, 0, 0, 0, 4,	37, 87,	16,9,17,9,18,9,19,9,0,0,0,0,0,0,0,
6, 3, 0, 0, 0, 0, 0,	38, 58,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 0, 0, 0, 0, 0,	40, 53,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 0, 0, 0, 0, 0,	43, 25,	19,9,20,9,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 1, 0, 0, 0, 2,	38, 94,	17,9,1,19,8,1,0,0,0,0,0,0,0,0,0,
6, 3, 1, 0, 0, 0, 0,	41, 15,	19,8,1,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 1, 3, 0, 0, 1,	41, 24,	19,8,1,9,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 1, 5, 0, 0, 6,	38, 10,	16,9,1,14,16,9,1,15,17,9,1,15,17,9,1,14,19,8,1,14,19,8,1,15,
6, 3, 1, 5, 0, 0, 0,	38, 14,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 1, 5, 0, 0, 0,	39, 33,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 1, 5, 0, 0, 0,	39, 3,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 1, 5, 0, 0, 0,	41, 26,	19,8,1,14,19,8,1,15,20,9,1,14,20,9,1,15,
6, 3, 1, 5, 0, 0, 0,	41, 50,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 1, 5, 3, 0, 2,	39, 25,	17,9,1,14,7,19,8,1,14,7,0,0,0,0,0,0,
6, 3, 1, 5, 3, 0, 0,	41, 42,	19,8,1,14,7,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 2, 0, 0, 0, 3,	38, 18,	16,9,5,17,9,5,19,8,5,0,0,0,0,0,0,0,
6, 3, 2, 0, 0, 0, 0,	39, 47,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 2, 0, 0, 0, 0,	41, 61,	19,8,5,20,9,5,0,0,0,0,0,0,0,0,0,0,
6, 3, 2, 5, 0, 0, 1,	41, 73,	19,8,5,14,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 2, 5, 3, 0, 1,	41, 89,	19,8,5,14,7,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 3, 0, 0, 0, 1,	42, 6,	19,8,9,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 5, 0, 0, 0, 2,	38, 20,	16,9,14,17,9,14,0,0,0,0,0,0,0,0,0,0,
6, 3, 5, 0, 0, 0, 0,	39, 53,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 5, 3, 0, 0, 2,	38, 25,	16,9,14,7,17,9,14,7,0,0,0,0,0,0,0,0,
6, 3, 5, 3, 0, 0, 0,	39, 64,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 5, 5, 3, 0, 1,	39, 77,	17,9,15,14,7,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 5, 7, 0, 0, 1,	42, 53,	19,8,15,21,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 7, 0, 0, 0, 2,	39, 79,	17,9,21,19,8,21,0,0,0,0,0,0,0,0,0,0,
6, 3, 7, 0, 0, 0, 0,	42, 63,	19,8,21,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 7, 1, 0, 0, 1,	43, 2,	19,8,21,1,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 7, 1, 3, 0, 1,	43, 4,	19,8,21,1,9,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 7, 1, 5, 0, 1,	43, 8,	19,8,21,1,14,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 7, 1, 5, 3, 1,	43, 10,	19,8,21,1,14,7,0,0,0,0,0,0,0,0,0,0,
6, 3, 7, 3, 0, 0, 1,	43, 13,	19,8,21,9,0,0,0,0,0,0,0,0,0,0,0,0,
6, 3, 7, 5, 0, 0, 2,	43, 17,	19,8,21,14,19,8,21,15,0,0,0,0,0,0,0,0,
6, 3, 7, 5, 0, 0, 0,	43, 21,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 5, 0, 0, 0, 0, 1,	38, 34,	16,15,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 5, 5, 3, 0, 0, 2,	40, 54,	18,15,14,7,19,15,14,7,0,0,0,0,0,0,0,0,
6, 5, 5, 3, 0, 0, 0,	43, 41,	19,15,14,7,20,15,14,7,0,0,0,0,0,0,0,0,
6, 5, 7, 0, 0, 0, 3,	38, 42,	16,15,21,18,15,21,19,15,21,0,0,0,0,0,0,0,
6, 5, 7, 0, 0, 0, 0,	40, 64,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 5, 7, 0, 0, 0, 0,	43, 49,	19,15,21,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 7, 0, 0, 0, 0, 4,	38, 43,	16,21,17,21,18,21,19,21,0,0,0,0,0,0,0,
6, 7, 0, 0, 0, 0, 0,	39, 87,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,

6, 7, 0, 0, 0, 0, 0,	40, 67,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 7, 0, 0, 0, 0, 0,	43, 52,	19,21,20,21,0,0,0,0,0,0,0,0,0,0,0,
6, 7, 1, 5, 0, 0, 3,	40, 6,	17,21,1,14,18,21,1,14,19,21,1,14,0,0,0,0,
6, 7, 1, 5, 0, 0, 0,	40, 77,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 7, 1, 5, 0, 0, 0,	43, 65,	19,21,1,14,20,21,1,14,0,0,0,0,0,0,0,
6, 7, 2, 0, 0, 0, 1,	40, 17,	17,21,5,0,0,0,0,0,0,0,0,0,0,0,0,
6, 7, 3, 0, 0, 0, 2,	40, 79,	18,21,9,19,21,9,0,0,0,0,0,0,0,0,0,
6, 7, 3, 0, 0, 0, 0,	43, 68,	19,21,9,20,21,9,0,0,0,0,0,0,0,0,0,
6, 7, 5, 0, 0, 0, 5,	40, 26,	17,21,14,18,21,14,18,21,15,19,21,14,19,21,15,0,
6, 7, 5, 0, 0, 0, 0,	40, 82,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 7, 5, 0, 0, 0, 0,	40, 84,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
6, 7, 5, 0, 0, 0, 0,	43, 79,	19,21,14,19,21,15,20,21,14,20,21,15,0,0,0,0,
6, 7, 5, 0, 0, 0, 0,	43, 82,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
7, 0, 0, 0, 0, 0, 3,	43, 90,	20,21,22,0,0,0,0,0,0,0,0,0,0,0,0,
7, 1, 0, 0, 0, 0, 1,	43, 90,	20,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
7, 1, 3, 0, 0, 0, 1,	44, 5,	20,1,9,0,0,0,0,0,0,0,0,0,0,0,0,0,
7, 1, 5, 0, 0, 0, 2,	44, 14,	20,1,14,20,1,15,0,0,0,0,0,0,0,0,0,0,
7, 1, 5, 0, 0, 0, 0,	44, 45,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
7, 1, 5, 3, 0, 0, 1,	44, 32,	20,1,14,7,0,0,0,0,0,0,0,0,0,0,0,0,
7, 2, 0, 0, 0, 0, 1,	44, 56,	20,5,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
7, 2, 5, 3, 0, 0, 1,	44, 57,	20,5,14,7,0,0,0,0,0,0,0,0,0,0,0,0,
7, 3, 0, 0, 0, 0, 1,	44, 61,	20,9,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
7, 3, 1, 5, 0, 0, 2,	44, 76,	20,9,1,14,20,9,1,15,0,0,0,0,0,0,0,0,
7, 3, 1, 5, 0, 0, 0,	44, 84,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
7, 3, 2, 0, 0, 0, 1,	44, 89,	20,9,5,0,0,0,0,0,0,0,0,0,0,0,0,0,
7, 3, 5, 3, 0, 0, 1,	44, 92,	20,9,14,7,0,0,0,0,0,0,0,0,0,0,0,0,
7, 5, 5, 3, 0, 0, 1,	45, 8,	20,15,14,7,0,0,0,0,0,0,0,0,0,0,0,0,
7, 5, 7, 0, 0, 0, 1,	45, 21,	20,15,21,19,15,21,0,0,0,0,0,0,0,0,0,0,
7, 7, 0, 0, 0, 0, 1,	45, 25,	20,21,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
7, 7, 1, 5, 0, 0, 1,	45, 36,	20,21,1,14,0,0,0,0,0,0,0,0,0,0,0,0,
7, 7, 3, 0, 0, 0, 1,	45, 38,	20,21,9,0,0,0,0,0,0,0,0,0,0,0,0,0,
7, 7, 5, 0, 0, 0, 2,	45, 44,	20,21,14,20,21,15,0,0,0,0,0,0,0,0,0,0,
7, 7, 5, 0, 0, 0, 0,	45, 47,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
8, 0, 0, 0, 0, 0, 4,	45, 58,	23,24,25,26,
8, 5, 0, 0, 0, 0, 1,	46, 46,	23,15,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
8, 7, 0, 0, 0, 0, 2,	48, 70,	23,21,24,21,0,0,0,0,0,0,0,0,0,0,0,0,
8, 7, 0, 0, 0, 0, 0,	47, 55,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
8, 7, 1, 5, 0, 0, 1,	48, 89,	24,21,1,14,0,0,0,0,0,0,0,0,0,0,0,0,
8, 7, 2, 0, 0, 0, 1,	49, 5,	24,21,5,0,0,0,0,0,0,0,0,0,0,0,0,0,
8, 7, 5, 0, 0, 0, 1,	49, 11,	24,21,14,0,0,0,0,0,0,0,0,0,0,0,0,0,
8, 0, 0, 0, 0, 0, 2,	49, 25,	25,26,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
8, 1, 0, 0, 0, 0, 3,	45, 58,	23,1,25,1,26,1,0,0,0,0,0,0,0,0,0,0,
8, 1, 0, 0, 0, 0, 0,	49, 25,	25,1,26,1,0,0,0,0,0,0,0,0,0,0,0,0,
8, 1, 0, 0, 0, 0, 0,	52, 49,	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
8, 1, 3, 0, 0, 0, 2,	45, 65,	23,1,9,26,1,9,0,0,0,0,0,0,0,0,0,0,

8, 1, 3, 0, 0, 0, 0,	52, 52,	26, 1, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 1, 5, 0, 0, 0, 5,	45, 67,	23, 1, 14, 25, 1, 14, 26, 1, 15, 25, 1, 15, 26, 1, 14,
8, 1, 5, 0, 0, 0, 0,	49, 41,	25, 1, 14, 26, 1, 15, 25, 1, 15, 26, 1, 14, 0, 0, 0, 0,
8, 1, 5, 0, 0, 0, 0,	52, 66,	0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 1, 5, 0, 0, 0, 0,	49, 91,	0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 1, 5, 0, 0, 0, 0,	52, 59,	0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 1, 5, 3, 0, 0, 3,	45, 84,	23, 1, 14, 7, 26, 1, 14, 7, 25, 1, 14, 7,
8, 1, 5, 3, 0, 0, 0,	52, 63,	26, 1, 14, 7, 25, 1, 14, 7, 0, 0, 0, 0, 0, 0, 0,
8, 1, 5, 3, 0, 0, 0,	49, 74,	0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 2, 0, 0, 0, 0, 2,	50, 12,	25, 5, 26, 5, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 2, 0, 0, 0, 0, 0,	52, 80,	0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 2, 3, 0, 0, 0, 2,	45, 94,	23, 5, 9, 26, 5, 9, 0, 0, 0, 0, 0, 0, 0, 0,
8, 2, 3, 0, 0, 0, 0,	52, 84,	26, 5, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 2, 5, 0, 0, 0, 2,	47, 33,	23, 5, 14, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 2, 5, 0, 0, 0, 0,	52, 85,	26, 5, 14, 26, 5, 14, 0, 0, 0, 0, 0, 0, 0, 0,
8, 2, 5, 3, 0, 0, 2,	47, 43,	23, 5, 14, 7, 26, 5, 14, 7, 0, 0, 0, 0, 0, 0, 0,
8, 2, 5, 3, 0, 0, 0,	52, 86,	26, 5, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 0, 0, 0, 0, 3,	46, 84,	24, 9, 25, 9, 26, 9, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 0, 0, 0, 0, 0,	50, 27,	25, 9, 26, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 0, 0, 0, 0, 0,	55, 40,	0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 1, 0, 0, 0, 2,	47, 25,	24, 9, 1, 26, 8, 1, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 1, 0, 0, 0, 0,	52, 90,	26, 8, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 1, 3, 0, 0, 1,	53, 10,	26, 8, 1, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 1, 5, 0, 0, 4,	47, 38,	24, 9, 1, 14, 24, 9, 1, 15, 26, 8, 1, 14, 26, 8, 1, 15,
8, 3, 1, 5, 0, 0, 0,	47, 84,	0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 1, 5, 0, 0, 0,	53, 16,	26, 8, 1, 14, 26, 8, 1, 15, 0, 0, 0, 0, 0, 0, 0,
8, 3, 1, 5, 0, 0, 0,	53, 48,	0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 1, 5, 3, 0, 2,	47, 64,	24, 9, 1, 14, 7, 26, 8, 1, 14, 7, 0, 0, 0, 0, 0, 0,
8, 3, 1, 5, 3, 0, 0,	53, 33,	26, 8, 1, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 2, 0, 0, 0, 2,	48, 8,	24, 9, 5, 26, 8, 5, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 2, 0, 0, 0, 0,	53, 58,	26, 8, 5, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 2, 5, 0, 0, 1,	53, 68,	26, 8, 5, 14, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 2, 5, 3, 0, 1,	53, 84,	26, 8, 5, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 3, 0, 0, 0, 1,	54, 5,	26, 8, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 5, 0, 0, 0, 2,	48, 29,	24, 9, 14, 25, 9, 14, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 5, 0, 0, 0, 0,	50, 80,	25, 9, 14, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 5, 3, 0, 0, 2,	48, 39,	24, 9, 14, 7, 25, 9, 14, 7, 0, 0, 0, 0, 0, 0, 0,
8, 3, 5, 3, 0, 0, 0,	51, 2,	25, 9, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 5, 5, 3, 0, 2,	48, 54,	24, 9, 15, 14, 7, 26, 8, 15, 14, 7, 0, 0, 0, 0, 0, 0,
8, 3, 5, 5, 3, 0, 0,	54, 48,	26, 8, 15, 14, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 5, 7, 0, 0, 1,	54, 59,	26, 8, 15, 21, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 7, 0, 0, 0, 2,	48, 61,	24, 9, 21, 26, 8, 21, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 7, 0, 0, 0, 0,	54, 73,	26, 8, 21, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8, 3, 7, 1, 0, 0, 1,	55, 5,	26, 8, 21, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,

```

8, 3, 7, 1, 3, 0, 1,    55, 7,    26,8,21,1,9,0,0,0,0,0,0,0,0,0,0,
8, 3, 7, 1, 5, 0, 1,    55, 8,    26,8,21,1,14,0,0,0,0,0,0,0,0,0,0,
8, 3, 7, 1, 5, 3, 1,    55, 14,   26,8,21,1,14,7,0,0,0,0,0,0,0,0,0,
8, 3, 7, 3, 0, 0, 1,    55, 21,   26,8,21,9,0,0,0,0,0,0,0,0,0,0,
8, 3, 7, 5, 0, 0, 2,    55, 27,   26,8,21,14,26,8,21,15,0,0,0,0,0,0,0,0,
8, 3, 7, 5, 0, 0, 0,    55, 29,   0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
8, 5, 5, 3, 0, 0, 2,    51, 20,   25,15,14,7,26,15,14,7,0,0,0,0,0,0,0,0,
8, 5, 5, 3, 0, 0, 0,    55, 55,   0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
8, 5, 7, 0, 0, 0, 2,    51, 36,   25,15,21,26,15,21,0,0,0,0,0,0,0,0,0,0,
8, 5, 7, 0, 0, 0, 0,    55, 62,   0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
8, 7, 0, 0, 0, 0, 2,    55, 66,   26,21,25,21,0,0,0,0,0,0,0,0,0,0,0,0,
8, 7, 0, 0, 0, 0, 0,    51, 56,   0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
8, 7, 1, 5, 0, 0, 2,    52, 7,    25,21,1,14,26,21,1,14,0,0,0,0,0,0,0,0,
8, 7, 1, 5, 0, 0, 0,    55, 74,   0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
8, 7, 2, 0, 0, 0, 1,    52, 27,   25,21,5,0,0,0,0,0,0,0,0,0,0,0,0,0,
8, 7, 3, 0, 0, 0, 1,    55, 76,   26,21,9,0,0,0,0,0,0,0,0,0,0,0,0,0,
8, 7, 5, 0, 0, 0, 3,    52, 37,   25,21,14,26,21,14,26,21,15,0,0,0,0,0,0,0,0,
8, 7, 5, 0, 0, 0, 0,    55, 80,   0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
8, 7, 5, 0, 0, 0, 0,    55, 82,   0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
};

```

```

unsigned int seng[26][16]={

```

```

/*-- 文字: a --*/

```

```

/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/

```

```

0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x3C,0x42,0x1E,0x22,0x42,0x42,0x3F,0x00,0x00,

```

```

/*-- 文字: b --*/

```

```

/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/

```

```

0x00,0x00,0x00,0xC0,0x40,0x40,0x40,0x58,0x64,0x42,0x42,0x42,0x64,0x58,0x00,0x00,

```

```

/*-- 文字: c --*/

```

```

/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/

```

```

0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x1C,0x22,0x40,0x40,0x40,0x22,0x1C,0x00,0x00,

```

```

/*-- 文字: d --*/

```

```

/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/

```

```

0x00,0x00,0x00,0x06,0x02,0x02,0x02,0x1E,0x22,0x42,0x42,0x42,0x26,0x1B,0x00,0x00,

```

```

/*-- 文字: e --*/

```

```

/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/

```

```

0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x3C,0x42,0x7E,0x40,0x40,0x42,0x3C,0x00,0x00,

```

```

/*-- 文字: f --*/

```

```

/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/

```

```

0x00,0x00,0x00,0x0F,0x11,0x10,0x10,0x7E,0x10,0x10,0x10,0x10,0x10,0x7C,0x00,0x00,

```

```

/*-- 文字: g --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x3E,0x44,0x44,0x38,0x40,0x3C,0x42,0x42,0x3C,

/*-- 文字: h --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xC0,0x40,0x40,0x40,0x5C,0x62,0x42,0x42,0x42,0x42,0xE7,0x00,0x00,

/*-- 文字: i --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x30,0x30,0x00,0x00,0x70,0x10,0x10,0x10,0x10,0x7C,0x00,0x00,

/*-- 文字: j --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x0C,0x0C,0x00,0x00,0x1C,0x04,0x04,0x04,0x04,0x04,0x44,0x78,

/*-- 文字: k --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xC0,0x40,0x40,0x4E,0x48,0x50,0x68,0x48,0x44,0xEE,0x00,0x00,

/*-- 文字: l --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x70,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x7C,0x00,0x00,

/*-- 文字: m --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xFE,0x49,0x49,0x49,0x49,0x49,0xED,0x00,0x00,

/*-- 文字: n --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xDC,0x62,0x42,0x42,0x42,0x42,0xE7,0x00,0x00,

/*-- 文字: o --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x3C,0x42,0x42,0x42,0x42,0x42,0x3C,0x00,0x00,

/*-- 文字: p --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xD8,0x64,0x42,0x42,0x42,0x44,0x78,0x40,0xE0,

/*-- 文字: q --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x1E,0x22,0x42,0x42,0x42,0x22,0x1E,0x02,0x07,

/*-- 文字: r --*/

```

```

/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xEE,0x32,0x20,0x20,0x20,0x20,0xF8,0x00,0x00,

/*-- 文字: s  --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x3E,0x42,0x40,0x3C,0x02,0x42,0x7C,0x00,0x00,

/*-- 文字: t  --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x00,0x00,0x10,0x10,0x7C,0x10,0x10,0x10,0x10,0x10,0x0C,0x00,0x00,

/*-- 文字: u  --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xC6,0x42,0x42,0x42,0x42,0x46,0x3B,0x00,0x00,

/*-- 文字: v  --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xE7,0x42,0x24,0x24,0x28,0x10,0x10,0x00,0x00,

/*-- 文字: w  --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xD7,0x92,0x92,0xAA,0xAA,0x44,0x44,0x00,0x00,

/*-- 文字: x  --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x6E,0x24,0x18,0x18,0x18,0x24,0x76,0x00,0x00,

/*-- 文字: y  --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xE7,0x42,0x24,0x24,0x28,0x18,0x10,0x10,0xE0,

/*-- 文字: z  --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x7E,0x44,0x08,0x10,0x10,0x22,0x7E,0x00,0x00,
};

unsigned int beng[26][16]={
/*-- 文字: A  --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x10,0x10,0x18,0x28,0x28,0x24,0x3C,0x44,0x42,0x42,0xE7,0x00,0x00,

/*-- 文字: B  --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0xF8,0x44,0x44,0x44,0x78,0x44,0x42,0x42,0x42,0x44,0xF8,0x00,0x00,

```

```

/*-- 文字: C --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x3E,0x42,0x42,0x80,0x80,0x80,0x80,0x42,0x44,0x38,0x00,0x00,

/*-- 文字: D --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xF8,0x44,0x42,0x42,0x42,0x42,0x42,0x42,0x44,0xF8,0x00,0x00,

/*-- 文字: E --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xFC,0x42,0x48,0x48,0x78,0x48,0x48,0x40,0x42,0x42,0xFC,0x00,0x00,

/*-- 文字: F --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xFC,0x42,0x48,0x48,0x78,0x48,0x48,0x40,0x40,0x40,0xE0,0x00,0x00,

/*-- 文字: G --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x3C,0x44,0x44,0x80,0x80,0x80,0x8E,0x84,0x44,0x44,0x38,0x00,0x00,

/*-- 文字: H --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xE7,0x42,0x42,0x42,0x42,0x7E,0x42,0x42,0x42,0x42,0xE7,0x00,0x00,

/*-- 文字: I --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x7C,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x7C,0x00,0x00,

/*-- 文字: J --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x3E,0x08,0x08,0x08,0x08,0x08,0x08,0x08,0x08,0x08,0x88,0xF0,

/*-- 文字: K --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xEE,0x44,0x48,0x50,0x70,0x50,0x48,0x48,0x44,0x44,0xEE,0x00,0x00,

/*-- 文字: L --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xE0,0x40,0x40,0x40,0x40,0x40,0x40,0x40,0x40,0x42,0xFE,0x00,0x00,

/*-- 文字: M --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xEE,0x6C,0x6C,0x6C,0x6C,0x54,0x54,0x54,0x54,0xD6,0x00,0x00,

```

```

/*-- 文字: N --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xC7,0x62,0x62,0x52,0x52,0x4A,0x4A,0x4A,0x46,0x46,0xE2,0x00,0x00,

/*-- 文字: O --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x38,0x44,0x82,0x82,0x82,0x82,0x82,0x82,0x82,0x44,0x38,0x00,0x00,

/*-- 文字: P --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xFC,0x42,0x42,0x42,0x42,0x7C,0x40,0x40,0x40,0x40,0xE0,0x00,0x00,

/*-- 文字: Q --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x38,0x44,0x82,0x82,0x82,0x82,0x82,0xB2,0xCA,0x4C,0x38,0x06,0x00,

/*-- 文字: R --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xFC,0x42,0x42,0x42,0x7C,0x48,0x48,0x44,0x44,0x42,0xE3,0x00,0x00,

/*-- 文字: S --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x3E,0x42,0x42,0x40,0x20,0x18,0x04,0x02,0x42,0x42,0x7C,0x00,0x00,

/*-- 文字: T --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xFE,0x92,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x38,0x00,0x00,

/*-- 文字: U --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xE7,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x3C,0x00,0x00,

/*-- 文字: V --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xE7,0x42,0x42,0x44,0x24,0x24,0x28,0x28,0x18,0x10,0x10,0x00,0x00,

/*-- 文字: W --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xD6,0x92,0x92,0x92,0x92,0xAA,0xAA,0x6C,0x44,0x44,0x44,0x00,0x00,
/*-- 文字: X --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0xE7,0x42,0x24,0x24,0x18,0x18,0x18,0x24,0x24,0x42,0xE7,0x00,0x00,

/*-- 文字: Y --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/

```


0x00,0x00,0x00,0xEE,0x44,0x44,0x28,0x28,0x10,0x10,0x10,0x10,0x10,0x38,0x00,0x00,

/*-- 文字: Z --*/

/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/

0x00,0x00,0x00,0x7E,0x84,0x04,0x08,0x08,0x10,0x20,0x20,0x42,0x42,0xFC,0x00,0x00,

};

unsigned int num[10][16]={

/*-- 文字: 0 --*/

/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/

0x00,0x00,0x00,0x18,0x24,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x24,0x18,0x00,0x00,

/*-- 文字: 1 --*/

/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/

0x00,0x00,0x00,0x10,0x70,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x7C,0x00,0x00,

/*-- 文字: 2 --*/

/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/

0x00,0x00,0x00,0x3C,0x42,0x42,0x42,0x04,0x04,0x08,0x10,0x20,0x42,0x7E,0x00,0x00,

/*-- 文字: 3 --*/

/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/

0x00,0x00,0x00,0x3C,0x42,0x42,0x04,0x18,0x04,0x02,0x02,0x42,0x44,0x38,0x00,0x00,

/*-- 文字: 4 --*/

/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/

0x00,0x00,0x00,0x04,0x0C,0x14,0x24,0x24,0x44,0x44,0x7E,0x04,0x04,0x1E,0x00,0x00,

/*-- 文字: 5 --*/

/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/

0x00,0x00,0x00,0x7E,0x40,0x40,0x40,0x58,0x64,0x02,0x02,0x42,0x44,0x38,0x00,0x00,

/*-- 文字: 6 --*/

/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/

0x00,0x00,0x00,0x1C,0x24,0x40,0x40,0x58,0x64,0x42,0x42,0x42,0x24,0x18,0x00,0x00,

/*-- 文字: 7 --*/

/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/

0x00,0x00,0x00,0x7E,0x44,0x44,0x08,0x08,0x10,0x10,0x10,0x10,0x10,0x00,0x00,

/*-- 文字: 8 --*/

/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/

0x00,0x00,0x00,0x3C,0x42,0x42,0x42,0x24,0x18,0x24,0x42,0x42,0x3C,0x00,0x00,

```

/*-- 文字: 9 --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x18,0x24,0x42,0x42,0x26,0x1A,0x02,0x02,0x24,0x38,0x00,0x00,

);

unsigned int sig[27][16]={
/*-- 文字: , --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x60,0x60,0x20,0xC0,

/*-- 文字: . --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x60,0x60,0x00,0x00,

/*-- 文字: / --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x01,0x02,0x02,0x04,0x04,0x08,0x08,0x10,0x10,0x20,0x20,0x40,0x40,0x00,

/*-- 文字: \ --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x40,0x40,0x20,0x20,0x10,0x10,0x10,0x08,0x08,0x04,0x04,0x04,0x02,0x02,

/*-- 文字: ; --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x10,0x00,0x00,0x00,0x00,0x00,0x10,0x10,0x20,

/*-- 文字: : --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x18,0x18,0x00,0x00,0x00,0x00,0x18,0x18,0x00,0x00,

/*-- 文字: < --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x02,0x04,0x08,0x10,0x20,0x40,0x20,0x10,0x08,0x04,0x02,0x00,0x00,

/*-- 文字: > --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x40,0x20,0x10,0x08,0x04,0x02,0x04,0x08,0x10,0x20,0x40,0x00,0x00,

/*-- 文字: ? --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x3C,0x42,0x42,0x62,0x02,0x04,0x08,0x08,0x00,0x18,0x18,0x00,0x00,

/*-- 文字: " --*/

```

```

/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x12,0x36,0x24,0x48,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
/*-- 文字: ' --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x60,0x60,0x20,0xC0,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
/*-- 文字: [ --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x1E,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x1E,0x00,
/*-- 文字: ] --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x78,0x08,0x08,0x08,0x08,0x08,0x08,0x08,0x08,0x08,0x08,0x08,0x78,0x00,
/*-- 文字: { --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x03,0x04,0x04,0x04,0x04,0x04,0x08,0x04,0x04,0x04,0x04,0x04,0x03,0x00,
/*-- 文字: } --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x60,0x10,0x10,0x10,0x10,0x10,0x08,0x10,0x10,0x10,0x10,0x10,0x60,0x00,
/*-- 文字: ! --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x10,0x10,0x10,0x10,0x10,0x10,0x10,0x00,0x18,0x18,0x00,0x00,
/*-- 文字: @ --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x38,0x44,0x5A,0xAA,0xAA,0xAA,0xAA,0xB4,0x42,0x44,0x38,0x00,0x00,
/*-- 文字: # --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x24,0x24,0x24,0xFE,0x48,0x48,0x48,0xFE,0x48,0x48,0x00,0x00,
/*-- 文字: $ --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x10,0x38,0x54,0x54,0x50,0x30,0x18,0x14,0x14,0x54,0x54,0x38,0x10,0x10,
/*-- 文字: % --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/
0x00,0x00,0x00,0x44,0xA4,0xA8,0xA8,0xA8,0x54,0x1A,0x2A,0x2A,0x44,0x00,0x00,
/*-- 文字: & --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16  --*/

```

```

0x00,0x00,0x00,0x30,0x48,0x48,0x48,0x50,0x6E,0xA4,0x94,0x88,0x89,0x76,0x00,0x00,

/*-- 文字: * --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x10,0x10,0xD6,0x38,0x38,0xD6,0x10,0x10,0x00,0x00,0x00,0x00,

/*-- 文字: ( --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x02,0x04,0x08,0x08,0x10,0x10,0x10,0x10,0x10,0x08,0x08,0x04,0x02,0x00,

/*-- 文字: ) --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x40,0x20,0x10,0x10,0x08,0x08,0x08,0x08,0x08,0x10,0x10,0x20,0x40,0x00,

/*-- 文字: - --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x7F,0x00,0x00,0x00,0x00,0x00,0x00,0x00,

/*-- 文字: + --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x10,0x10,0x10,0x10,0xFE,0x10,0x10,0x10,0x10,0x00,0x00,0x00,

/*-- 文字: = --*/
/*-- 宋体 12; 此字体下对应的点阵为: 宽 x 高=8x16 --*/
0x00,0x00,0x00,0x00,0x00,0x00,0xFE,0x00,0x00,0x00,0xFE,0x00,0x00,0x00,0x00,
};

```

程序三十九 16*4 液晶汉字代码

```

;连线方式:
;*LCM---89C52* *LCM---89C52* *LCM-----89C52* *LCM-----89C52* *
;*DB0---P0.0* *DB4---P0.4* *D/I-----P2.6* *CS1-----P2.4* *
;*DB1---P0.1* *DB5---P0.5* *R/W-----P2.7* *CS2-----P2.5* *
;*DB2---P0.2* *DB6---P0.6* *RST-----VCC* *CS3-----P3.2* *
;*DB3---P0.3* *DB7---P0.7* *E-----P2.3* *
;89C52 的晶振频率为 12MHz *
;*****
//画线部分请参照 avr 的 c 程序。
/*#pragma src /*生成 ASM 文件开关,必要时打开 */
#include
#include

```

```

#include

#define    Uchar unsigned char

/*****液晶显示器接口引脚定义*****/

sbit  Elcm=    P2^3;           //
sbit  CS1LCM=  P2^4;           //
sbit  CS2LCM=  P2^5;           //
sbit  CS3LCM=  P3^2;           /*这个连接只是做实验的临时接法。*/
sbit  Dilcm=   P2^6;           //
sbit  Rwlcm=   P2^7;           //
sfr  Datalcm=  0x80;           //数据口

/*****常用操作命令和参数定义*****/
#define      DISPON          0x3f      /*显示 on          */
#define      DISPOFF 0x3e      /*显示 off          */
#define      DISPFIRST      0xc0      /*显示起始行定义    */
#define      SETX            0x40      /*X 定位设定指令(页)*/
#define      SETY            0xb8      /*Y 定位设定指令(列)*/
#define      Lcdbusy  0x80      /*LCM 忙判断位      */

/*****显示分区边界位置*****/
#define      MODL            0x00      /*左区              */
#define      MODM            0x40      /*左区和中区分界    */
#define      MODR            0x80      /*中区和右区分界    */
#define      LCMLIMIT      0xc0      /*显示区的右边界    */

/*****全局变量定义*****/
Uchar col,row,cbyte;           /*列 x,行(页)y,输出数据 */
bit xy;                         /*画线方向标志: 1 水平 */

/*****函数列表*****/
void Lcinit(void);              /*液晶模块初始化      */
void Delay(Uchar);              /*延时, 入口数为 Ms    */
void lcdbusyL(void);            /*busy 判断、等待(左区)*/
void lcdbusyM(void);            /*busy 判断、等待(中区)*/
void lcdbusyR(void);            /*busy 判断、等待(右区)*/
void Putedot(Uchar);            /*半角字符输出          */
void Putcdot(Uchar);            /*全角(汉字)输出          */
void Wrdats(Uchar);             /*数据输出给 LCM        */
void Lcmcls( void );            /*LCM 全屏幕清零(填充 0)*/
void wtdcom(void);              /*公用 busy 等待        */
void Locatxy(void);             /*光标定位              */
void WrcmdL(Uchar);            /*左区命令输出          */

```

```

void WrcmdM(Uchar);          /*中区命令输出          */
void WrcmdR(Uchar);          /*右区命令输出          */
void Putstr(Uchar *puts,Uchar i);    /*中英文字符串输出      */
void Rollscreen(Uchar x);      /*屏幕向上滚动演示      */
void Rddata(void);            /* 从液晶片上读数据      */
void Linehv(Uchar length);     /*横(竖)方向画线        */
void point(void);             /*打点                    */
void Linexy(Uchar endx,Uchar endy);

/*****数组列表*****/
Uchar code Ezk[];             /*ASCII 常规字符点阵码表 */
Uchar code Hzk[];             /*自用汉字点阵码表      */
Uchar code STR1[];            /*自定义字符串          */
Uchar code STR2[];            //
Uchar code STR3[];            //
Uchar code STR4[];            //

/*****
/* 演示主程序          */
*****/
void main(void)

{
    Uchar x;
    col=0;
    row=0;
    Delay(40);                /*延时大约 40ms, 等待外设准备好*/
    Lcminit();                /*液晶模块初始化, 包括全屏幕清屏*/
    Putstr(STR2,24);          /*第一行字符输出, 24 字节      */
    col=0;
    row=2;
    Putstr(STR1,12);          /*第二行字符输出, 12 字节      */
    col=0;
    row=4;
    Putstr(STR3,24);          /*第三行字符输出, 24 字节      */
    col=0;
    row=6;
    Putstr(STR4,24);          /*第四行字符输出, 12 字节      */
    x=0;
    col=0;
    row=0;
    xy = 1;                    /*方向标志。定为水平方向      */
    Linehv(192);              /*画一条横线(0,0)-(191,0)      */
    col=0;

```

```

row=15;
xy = 1;
Linehv(192);          /*画一条横线(0,15)-(191,15)    */
col=0;
row=32;
xy = 1;
Linehv(192);          /*画一条横线(0,32)-(191,32)    */
col=0;
row=1;
xy = 0;               /*方向标志。定为垂直方向      */
Linehv(31);           /*画一条竖线(0,1)-(0,31)      */
col=191;
row=1;
xy = 0;
Linehv(31);           /*画一条竖线(191,1)-(191,31)  */
col=0;               /*设定斜线的起点坐标          */
row=63;
Linexy(44,31);        /*画一段斜线(0,63)-(44,31)    */
col=44;
row=31;
Linexy(190,62);       /*继续画斜线(44,31)-(191,63)  */
while(1){
    Rollscreen(x);     /*定位新的显示起始行 */
    x++;
    Delay(100);        /*延时，控制滚动速度 */
};
}

```

```

/*****
/*画线。任意方向的斜线，不支持垂直的或水平线  */
*****/

```

```

void Linexy(Uchar endx,Uchar endy)
{
    register Uchar t;
    int xerr=0,yerr=0,delta_x,delta_y,distance;
    Uchar incx,incy;

    /* compute the distance in both directions */
    delta_x=endx-col;
    delta_y=endy-row;

    /* compute the direction of the increment ,
       an increment of "0" means either a vertical or horizontal lines */

```

```

    if(delta_x>0) incx=1;
    else if( delta_x==0 ) incx=0;
        else incx=-1;

    if(delta_y>0) incy=1;
    else if( delta_y==0 ) incy=0;
        else incy=-1;

/* determine which distance is greater */
    delta_x = cabs( delta_x );
    delta_y = cabs( delta_y );

    if( delta_x > delta_y ) distance=delta_x;
    else distance=delta_y;

/* draw the line */
    for( t=0;t <= distance+1; t++ ) {
        point();
        xerr += delta_x ;
        yerr += delta_y ;
        if( xerr > distance ) {
            xerr-=distance;
            col+=incx;
        }
        if( yerr > distance ) {
            yerr-=distance;
            row+=incy;
        }
    }
}

/*****
/*画线。只提供 X 或 Y 方向的，不支持斜线 */
*****/
void Linehv(Uchar length)
{
    Uchar xs,ys;
    if (xy){ys = col;
        for (xs=0;xs>3;
            Rddata();
            y=y1&0x07;
            x=0x01;
            /*取 Y 方向分页地址 */
            /*字节内位置计算 */

```



```

        x=x<7) row=0;                /*如果行越界, 返回首行      */
    }                                /*上半个字符输出结束 */

    col = bakerx;                    /*列对齐                */
    row = bakery+1;                  /*指向下半个字符行      */
                                    /*下半个字符输出, 8列    */
    for(i=0;i7) row=1;              /*如果行越界, 返回首行  */
    }                                /*下半个字符输出结束 */
    row=bakery;
}                                    /*整个字符输出结束    */

/*****
/* 全角字符点阵码数据输出          */
*****/

void Putcdot(Uchar Order)
{
    Uchar i,bakerx,bakery;          /*共定义 3 个局部变量      */
    int x;                          /*偏移量, 字符量少的可以定义为 UCHAR */
    bakerx = col;                   /*暂存 x,y 坐标, 已备下半个字符使用 */
    bakery = row;
    x=Order * 0x20;                 /*每个字符 32 字节        */

                                    /*上半个字符输出, 16列    */
    for(i=0;i6) row=0;              /*如果行越界, 返回首行    */
    }                                /*上半个字符输出结束 */

                                    /*下半个字符输出, 16列    */

    col = bakerx;
    row = bakery+1;
    for(i=0;i7) row=1;              /*如果行越界, 返回首行    */
    }                                /*下半个字符输出结束 */
    row = bakery;
}                                    /*整个字符输出结束 */

/*****
/* 清屏, 全屏幕清零                */
*****/

void Lcmcls( void )
{
    for(row=0;row

```

程序四十 智能化家电控制

```
#include <reg8751.h>
#include <math.h>
#include <intrins.h>
#include <stdio.h>

sbit cd=P0^2;
sbit wr=P0^0;
sbit rd=P0^1;
sbit reset=P0^3;

sbit back_light=P3^6;

unsigned int time,adds;
unsigned char x,y;

/* 忙标志 */
#pragma disable
unsigned char busy(void) {
    unsigned char dat;
    cd=1;rd=1;wr=1;
    P1=0xff;
    rd=0;
    dat=P1;
    rd=1;
    return(dat);
}
/* 数据,指令读写判别 */
#pragma disable
void p1(void) {
    while ((busy()&3)!=3) {}
}
/* 数据自动读判别 */
#pragma disable
void p2(void) {
    while ((busy()&4)!=4) {}
}
/* 数据自动写判别 */
#pragma disable
void p3(void) {
```

```

        while ((busy() & 8) != 8) {}
    }
    /* 控制指令 */
    #pragma disable
    void ctrl(unsigned char dat) {
        p1();
        cd=1;
        wr=0;
        P1=dat;
        wr=1;
    }
    /* 写数据 */
    #pragma disable
    void write(unsigned char dat) {
        p1();
        cd=0;
        wr=0;
        P1=dat;
        wr=1;
        cd=1;
    }
    /* 自动写 */
    #pragma disable
    void autowrite(unsigned char dat) {
        p3();
        cd=0;
        wr=0;
        P1=dat;
        wr=1;
        cd=1;
    }
    /* 读数据 */
    #pragma disable
    unsigned char read(void) {
        unsigned char dat;
        p1();
        cd=0;
        P1=0xff;
        rd=0;
        dat=P1;
        rd=1;
        cd=1;
        return(dat);
    }
    /* 自动读数据 */

```

```

#pragma disable
unsigned char autoread(void) {
    unsigned char dat;
    p2();
    cd=0;
    P1=0xff;
    rd=0;
    dat=P1;
    rd=1;
    cd=1;
    return(dat);
}
/* 显示图形和文本 */
#pragma disable
void dp(unsigned char d) { /*显示*/
    write(d);ctrl(0xc0);
}
/* 设定图形 x,y 值 */
#pragma disable
void ag(unsigned char x,unsigned char y) { /*地址*/
    unsigned int xy;
    xy=y;
    xy=xy*16+x+256;
    write(xy&0xff);write(xy/256);ctrl(0x24);
}
/* 设定文本 x,y 值 */
#pragma disable
void at(unsigned char x,unsigned char y) { /*地址*/
    write(y*16+x);write(0);ctrl(0x24);
}
/* 点亮一点 */
#pragma disable
void setb(unsigned char d) {
    ctrl(0xf8ld);
}
/* 清除一点 */
#pragma disable
void clrb(unsigned char d) {
    ctrl(0xf0ld);
}
/* x,y 处显示光标 */
#pragma disable
void ab(unsigned char x,unsigned char y) { /*光标*/
    ctrl(0x97); /*光标开*/
    write(x);write(y);ctrl(0x21);
}

```

```

}
/* 取消光标 */
#pragma disable
void noab(void) {
    ctrl(0x9c);
}

/* lcd 初始化 */
void init(void) {
    reset=0;
    reset=1;
    write(0x0);write(0);ctrl(0x40);    /*文本首址*/
    write(0x10);write(0x0);ctrl(0x41); /*文本区域*/
    write(0x0);write(0x1);ctrl(0x42);  /*图形首址*/
    write(0x10);write(0x0);ctrl(0x43);  /*图形区域*/
    ctrl(0x81);    /*显示方式*/
    ctrl(0x90);    /*显示开关*/
    ctrl(0xa0);    /*光标形状*/
    at(0,0);
    ctrl(0xb0);    /*自动写方式*/
    for (adds=0;adds<1280;adds++) {
        autowrite(0x0);
    }
    ctrl(0xb2);    /*结束自动写方式 */
    ctrl(0x9c);
}

unsigned char code asc16[]={
0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
0,0,126,129,165,129,129,189,153,129,129,126,0,0,0,0,
0,0,126,255,219,255,255,195,231,255,255,126,0,0,0,0,
0,0,0,0,108,254,254,254,254,124,56,16,0,0,0,0,
0,0,0,0,16,56,124,254,124,56,16,0,0,0,0,
0,0,0,24,60,60,231,231,231,24,24,60,0,0,0,0,
0,0,0,24,60,126,255,255,126,24,24,60,0,0,0,0,
0,0,0,0,0,24,60,60,24,0,0,0,0,0,
255,255,255,255,255,255,231,195,195,231,255,255,255,255,255,
0,0,0,0,0,60,102,66,66,102,60,0,0,0,0,
255,255,255,255,255,195,153,189,189,153,195,255,255,255,255,
0,0,30,14,26,50,120,204,204,204,204,120,0,0,0,0,
0,0,60,102,102,102,102,60,24,126,24,24,0,0,0,0,
0,0,63,51,63,48,48,48,48,112,240,224,0,0,0,0,
0,0,127,99,127,99,99,99,99,103,231,230,192,0,0,0,
0,0,0,24,24,219,60,231,60,219,24,24,0,0,0,0,
0,128,192,224,240,248,254,248,240,224,192,128,0,0,0,0,

```

0,2,6,14,30,62,254,62,30,14,6,2,0,0,0,0,
 0,0,24,60,126,24,24,24,126,60,24,0,0,0,0,0,
 0,0,102,102,102,102,102,102,102,0,102,102,0,0,0,0,
 0,0,127,219,219,219,123,27,27,27,27,27,0,0,0,0,
 0,124,198,96,56,108,198,198,108,56,12,198,124,0,0,0,
 0,0,0,0,0,0,0,254,254,254,254,0,0,0,0,
 0,0,24,60,126,24,24,24,126,60,24,126,0,0,0,0,
 0,0,24,60,126,24,24,24,24,24,24,24,0,0,0,0,
 0,0,24,24,24,24,24,24,24,126,60,24,0,0,0,0,
 0,0,0,0,0,24,12,254,12,24,0,0,0,0,0,0,
 0,0,0,0,0,48,96,254,96,48,0,0,0,0,0,0,
 0,0,0,0,0,192,192,192,254,0,0,0,0,0,0,
 0,0,0,0,0,40,108,254,108,40,0,0,0,0,0,0,
 0,0,0,0,16,56,56,124,124,254,254,0,0,0,0,0,
 0,0,0,0,254,254,124,124,56,56,16,0,0,0,0,0,
 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
 0,0,24,60,60,60,24,24,24,0,24,24,0,0,0,0,
 0,102,102,102,36,0,0,0,0,0,0,0,0,0,0,0,
 0,0,0,108,108,254,108,108,108,254,108,108,0,0,0,0,
 24,24,124,198,194,192,124,6,6,134,198,124,24,24,0,0,
 0,0,0,0,194,198,12,24,48,96,198,134,0,0,0,0,
 0,0,56,108,108,56,118,220,204,204,204,118,0,0,0,0,
 0,48,48,48,96,0,0,0,0,0,0,0,0,0,0,0,
 0,0,12,24,48,48,48,48,48,48,24,12,0,0,0,0,
 0,0,48,24,12,12,12,12,12,12,24,48,0,0,0,0,
 0,0,0,0,0,102,60,255,60,102,0,0,0,0,0,0,
 0,0,0,0,0,24,24,126,24,24,0,0,0,0,0,0,
 0,0,0,0,0,0,0,0,24,24,24,48,0,0,0,0,
 0,0,0,0,0,0,0,254,0,0,0,0,0,0,0,0,
 0,0,0,0,0,0,0,0,0,24,24,0,0,0,0,
 0,0,0,0,2,6,12,24,48,96,192,128,0,0,0,0,
 0,0,56,108,198,198,214,214,198,198,108,56,0,0,0,0,
 0,0,24,56,120,24,24,24,24,24,24,126,0,0,0,0,
 0,0,124,198,6,12,24,48,96,192,198,254,0,0,0,0,
 0,0,124,198,6,6,60,6,6,6,198,124,0,0,0,0,
 0,0,12,28,60,108,204,254,12,12,12,30,0,0,0,0,
 0,0,254,192,192,192,252,6,6,6,198,124,0,0,0,0,
 0,0,56,96,192,192,252,198,198,198,198,124,0,0,0,0,
 0,0,254,198,6,6,12,24,48,48,48,48,0,0,0,0,
 0,0,124,198,198,198,124,198,198,198,124,0,0,0,0,
 0,0,124,198,198,198,126,6,6,6,12,120,0,0,0,0,
 0,0,0,0,24,24,0,0,0,24,24,0,0,0,0,0,
 0,0,0,0,24,24,0,0,0,24,24,48,0,0,0,0,
 0,0,0,6,12,24,48,96,48,24,12,6,0,0,0,0,
 0,0,0,0,0,126,0,0,126,0,0,0,0,0,0,0,0,

0,0,0,96,48,24,12,6,12,24,48,96,0,0,0,0,
 0,0,124,198,198,12,24,24,24,0,24,24,0,0,0,0,
 0,0,0,124,198,198,222,222,222,220,192,124,0,0,0,0,
 0,0,16,56,108,198,198,254,198,198,198,198,0,0,0,0,
 0,0,252,102,102,102,124,102,102,102,102,252,0,0,0,0,
 0,0,60,102,194,192,192,192,192,194,102,60,0,0,0,0,
 0,0,248,108,102,102,102,102,102,102,108,248,0,0,0,0,
 0,0,254,102,98,104,120,104,96,98,102,254,0,0,0,0,
 0,0,254,102,98,104,120,104,96,96,96,240,0,0,0,0,
 0,0,60,102,194,192,192,222,198,198,102,58,0,0,0,0,
 0,0,198,198,198,198,254,198,198,198,198,198,0,0,0,0,
 0,0,60,24,24,24,24,24,24,24,24,60,0,0,0,0,
 0,0,30,12,12,12,12,12,204,204,204,120,0,0,0,0,
 0,0,230,102,102,108,120,120,108,102,102,230,0,0,0,0,
 0,0,240,96,96,96,96,96,98,102,254,0,0,0,0,
 0,0,198,238,254,254,214,198,198,198,198,198,0,0,0,0,
 0,0,198,230,246,254,222,206,198,198,198,198,0,0,0,0,
 0,0,124,198,198,198,198,198,198,198,198,124,0,0,0,0,
 0,0,252,102,102,102,124,96,96,96,96,240,0,0,0,0,
 0,0,124,198,198,198,198,198,198,214,222,124,12,14,0,0,
 0,0,252,102,102,102,124,108,102,102,102,230,0,0,0,0,
 0,0,124,198,198,96,56,12,6,198,198,124,0,0,0,0,
 0,0,126,126,90,24,24,24,24,24,24,60,0,0,0,0,
 0,0,198,198,198,198,198,198,198,198,198,124,0,0,0,0,
 0,0,198,198,198,198,198,198,198,108,56,16,0,0,0,0,
 0,0,198,198,198,198,214,214,214,254,238,108,0,0,0,0,
 0,0,198,198,108,124,56,56,124,108,198,198,0,0,0,0,
 0,0,102,102,102,102,60,24,24,24,24,60,0,0,0,0,
 0,0,254,198,134,12,24,48,96,194,198,254,0,0,0,0,
 0,0,60,48,48,48,48,48,48,48,60,0,0,0,0,
 0,0,0,128,192,224,112,56,28,14,6,2,0,0,0,0,
 0,0,60,12,12,12,12,12,12,12,12,60,0,0,0,0,
 16,56,108,198,0,0,0,0,0,0,0,0,0,0,0,0,
 0,0,0,0,0,0,0,0,0,0,0,0,255,0,0,
 48,48,24,0,0,0,0,0,0,0,0,0,0,0,0,
 0,0,0,0,120,12,124,204,204,204,118,0,0,0,0,
 0,0,224,96,96,120,108,102,102,102,102,124,0,0,0,0,
 0,0,0,0,124,198,192,192,192,198,124,0,0,0,0,
 0,0,28,12,12,60,108,204,204,204,204,118,0,0,0,0,
 0,0,0,0,124,198,254,192,192,198,124,0,0,0,0,
 0,0,56,108,100,96,240,96,96,96,96,240,0,0,0,0,
 0,0,0,0,118,204,204,204,204,204,124,12,204,120,0,
 0,0,224,96,96,108,118,102,102,102,102,230,0,0,0,0,
 0,0,24,24,0,56,24,24,24,24,24,60,0,0,0,0,
 0,0,6,6,0,14,6,6,6,6,6,6,102,102,60,0,


```

//144
/*湿 4210*/0,71,52,20,135,100,36,15,49,233,37,35,33,63,32,0,
8,252,8,8,248,8,8,248,49,36,40,48,36,254,0,0,
//146
/*亮 3333*/1,127,0,31,16,16,31,0,127,143,8,8,8,16,96,0,
8,252,16,248,16,16,240,0,127,244,32,32,34,34,30,0,
//148
/*警 3015*/20,127,16,63,85,29,3,255,31,0,31,0,31,16,31,0,
64,124,200,40,16,110,4,254,31,0,240,0,240,16,240,0,
//150
/*告 2470*/1,17,17,31,17,33,1,255,31,16,16,16,16,31,16,0,
0,0,16,248,0,0,4,254,31,16,16,16,16,240,16,0,
//152
/*星 4839*/0,63,32,63,32,63,1,33,63,65,191,1,1,255,0,0,
8,252,8,248,8,248,0,8,63,16,248,0,4,254,0,0
};

/* 显示 8*16 点阵英文字母,反白 */
void da(unsigned char n,bit un) {
    unsigned char i;
    unsigned int k=n;
    for (i=0;i<16;i++) {
        ag(x,y*8+i);
        if (un==0) dp(asc16[k*16+i]); else dp(255-asc16[k*16+i]);
    }
    if ((++x)>=16) {
        x=0;
        y+=2;
        if (y>=8) y=0;
    }
}

/* 反白文本/取消 */
void da8(unsigned char n,bit un) {
    unsigned char i,j;
    for (j=0;j<n;j++) {
        for (i=0;i<8;i++) {
            ag(x,y*8+i);
            if (un==0) dp(0xff); else dp(0x0);
        }
        if ((++x)>=16) {
            x=0;
            y+=1;
            if (y>=8) y=0;
        }
    }
}

```

```

}
/* 划线 */
void line(unsigned char x0,unsigned char y0,unsigned char x1,unsigned char y1) {
    unsigned char x=x1-x0,y=y1-y0,xx,temp;
    float yy;
    if (x>y) {
        yy=y0;
        for (xx=x0;xx<x1;xx++) {
            yy+=(float)y/x;
            ag((xx/8),yy);
            setb((7-xx&7));
        }
    } else {
        yy=x0;
        for (xx=y0;xx<y1;xx++) {
            yy+=(float)x/y;
            ag(yy/8,xx);
            setb((7-(char)yy&7));
        }
    }
}

```

```

//=====

```

```

/* 串行数据输出输入 */

```

```

//=====

```

```

sbit clock_dat=P3^4;
sbit clock_clk=P3^3;
sbit clock_rst=P3^5;
sbit sda=P3^3;
sbit scl=P3^4;
unsigned char bdata a;
sbit a0=a^0;
sbit a1=a^1;
sbit a2=a^2;
sbit a3=a^3;
sbit a4=a^4;
sbit a5=a^5;
sbit a6=a^6;
sbit a7=a^7;

```

```

void s_out(unsigned char dd) {
    a=dd;
    clock_dat=a0;clock_clk=1;clock_clk=0;
    clock_dat=a1;clock_clk=1;clock_clk=0;

```

```

        clock_dat=a2;clock_clk=1;clock_clk=0;
        clock_dat=a3;clock_clk=1;clock_clk=0;
        clock_dat=a4;clock_clk=1;clock_clk=0;
        clock_dat=a5;clock_clk=1;clock_clk=0;
        clock_dat=a6;clock_clk=1;clock_clk=0;
        clock_dat=a7;clock_clk=1;clock_clk=0;
    }
    unsigned char s_in(void) {
        clock_dat=1;
    //    clock_clk=1;clock_clk=0;a0=clock_dat;
        a0=clock_dat;
        clock_clk=1;clock_clk=0;a1=clock_dat;
        clock_clk=1;clock_clk=0;a2=clock_dat;
        clock_clk=1;clock_clk=0;a3=clock_dat;
        clock_clk=1;clock_clk=0;a4=clock_dat;
        clock_clk=1;clock_clk=0;a5=clock_dat;
        clock_clk=1;clock_clk=0;a6=clock_dat;
        clock_clk=1;clock_clk=0;a7=clock_dat;
        return(a);
    }
    //=====
    /*    时钟    */
    //=====

    unsigned char read_clock(unsigned char ord) {
        unsigned char i,dd=0;
        clock_rst=1;
        s_out(ord);
        dd=s_in();
        clock_rst=0;
        return(dd);
    }
    void write_clock(unsigned char ord,unsigned char dd) {
        unsigned char i;
        clock_rst=1;
        s_out(ord);
        s_out(dd);
        clock_rst=0;
    }

    //=====
    void s24(void) {
        scl=0;sda=1;scl=1;sda=0;scl=0;
    }
    void p24(void) {

```

```

    sda=0;scl=1;sda=1;
}

```

```

unsigned char rd24(void) {
    unsigned char i,dd;
    dd=0;
    sda=1;
    for (i=8;i!=0;i--) {
        dd<<=1;
        scl=1;
        if (sda==1) ddi=1;
        scl=0;
    }
    sda=1;
    scl=1;
    scl=0;
    return(dd);
}

```

```

void wd24(unsigned char dd) {
    a=dd;
    sda=a7;scl=1;scl=0;
    _nop();_nop();
    sda=a6;scl=1;scl=0;
    _nop();_nop();
    sda=a5;scl=1;scl=0;
    _nop();_nop();
    sda=a4;scl=1;scl=0;
    _nop();_nop();
    sda=a3;scl=1;scl=0;
    _nop();_nop();
    sda=a2;scl=1;scl=0;
    _nop();_nop();
    sda=a1;scl=1;scl=0;
    _nop();_nop();
    sda=a0;scl=1;scl=0;
    _nop();_nop();
    sda=1;
    scl=1;
}

```

```

void write24(unsigned int address,unsigned char dd){
    s24();wd24(0xa0);wd24((address/256)<<1);scl=0;wd24(address);scl=0;wd24(dd);scl=0;p24();
    while (1) {
        s24();

```

```

        wd24(0xa0);
        if (sda==0) break;
        scl=0;
    }
}
unsigned char read24(unsigned int address){
    unsigned char dd;
    s24();wd24(0xa0);wd24(address/256);scl=0;wd24(address);scl=0;
    s24();wd24(0xa1);scl=0;dd=rd24();p24();return(dd);
}

```

/* 红外线键值转换*/

```

unsigned char code inf_key[]={
    0xb8,0x47, //tape
    0x38,0xc7, //tuner
    0x78,0x87, //cd
    0x48,0xb7, //band
    0xc8,0x37, //lm
    0x30,0xcf, //att
    0x2,0xfd, //up
    0x82,0x7d, //down
    0x42,0xbd, //left
    0xc2,0x3d, //right
    0xd0,0x2f, //dec
    0x50,0xaf //inc
};

```

```

#define k_tape 1
#define k_tuner 2
#define k_cd 3
#define k_band 4
#define k_lm 5
#define k_att 6
#define k_u 7
#define k_d 8
#define k_l 9
#define k_r 10
#define k_dec 11
#define k_inc 12

```

```

sbit p_light=P0^4;
sbit p_temp=P0^5;
unsigned char light_time,temp_time,l_t,t_t;

```

/* 定时器 0 中断 */

```

unsigned char inf_time,key,key_inf,key_push,key_low,key_high,k[4];

```

```

int_t00() {
}

void int_t0(void) interrupt 1 {
    unsigned char i;
    TH0=0xf8;
    time++;
    /* 红外线判别 */
    if (inf_time==0) {
        if ((k[0]==0xb5)&&(k[1]==0x4a)) {
            for (i=0;i<24;i+=2) {
                if ((k[2]==inf_key[i])&&(k[3]==inf_key[i+1])) {
                    key_inf=i/2+1;
                    break;
                }
            }
            if (i>=24) key_inf=0;
        } else key_inf=0;
    }
    if ((++inf_time)>0x7f) {
        inf_time=0x7f;
        key_inf=0;
    }

    if (key_inf!=0) {
        key_push=key_inf;
        if ((++key_low)>250) {
            key_low=250;
            key=key_push+0x10;
        } else if (key_high!=0) {
            key=key_push;
            key_high=0;
        }
    } else {
        key_low=0;
        if ((++key_high)>10) {
            key_high=10;
            key=0;
        }
    }

    /* 软件 a/d 转换 */
    if (p_light==0) light_time++;
    else {
        p_light=0;
        _nop();
    }
}

```

```

        p_light=1;
        l_t=light_time;
        light_time=0;
    }
    if (p_temp==0) temp_time++;
    else {
        p_temp=0;
        _nop_();
        p_temp=1;
        t_t=temp_time;
        temp_time=0;
    }
}

unsigned char inf_n,inf_d;

/* 红外线检测,当时间间隔大于 4ms 时纪录 */
/* 红外线格式 32ms 一段,共 32b 信息,0 用 768usl-表达,1780usl-表达,
   每一段用 24ms+5ms+3ms 隔开
*/

void int_ex(void) interrupt 0 {
    unsigned char th,i;
//    bit i;
    TR1=0;
    th=TH1;
    TH1=TL1=0;
    TR1=1;
    inf_n++;
    if ((th>=7)&&(th<=9)) i=0;
    else if ((th>=15)&&(th<=17)) i=1;
    else {inf_n=0;inf_d=0;}
    if (inf_n!=0) {
        inf_d<<=1;
        inf_d|=i;
    }
    switch (inf_n) {
        case 8:k[0]=inf_d;break;
        case 16:k[1]=inf_d;break;
        case 24:k[2]=inf_d;break;
        case 32:k[3]=inf_d;inf_d=0;inf_n=0;inf_time=0;break;
    }
    IE0=0;
}

void int_t1(void) interrupt 3 {

```

```

        TH1=0x40;
    }
    /* 显示红外线检测信息 */
    void disp_inf(void) {
    /*  unsigned char i,j,d[4];
        while (1) {
            at(0,0);
            for (j=0;j<4;j++) {
                sprintf(d,"%4x",(unsigned int)k[j]);
                for (i=0;i<4;i++) dp(d[i]-0x20);
            }
            sprintf(d,"%4x",(unsigned int)key);
            for (i=0;i<4;i++) dp(d[i]-0x20);
            sprintf(d,"%4x",(unsigned int)t_t);
            if (key==1) back_light=0;
            if (key==2) back_light=1;
        }
    */
    }

#define c_h 0x85
#define c_m 0x83
#define c_s 0x81
#define c_day 0x87
#define c_mon 0x89
#define c_week 0x8b
#define c_year 0x8d
#define c_lock 0x8e

void k_w(void) {
    while (key!=0) {}
}
/* 显示日历时钟 */
void disp_clock(void) {
    unsigned char i,n;
    x=0;y=0;
    da(152,0);
    da(153,0);
    da(134,0);
    da(135,0);

    da(' ',0);
    i=read_clock(0x8b);
    da(i+0x30,0);

```



```

at(7,0);
dp('1'-0x20);
dp('2'-0x20);
dp(':-0x20);
dp('5'-0x20);
dp('9'-0x20);
dp(' '-0x20);
dp('o'-0x20);
dp('n'-0x20);

```

```

at(7,1);
dp('1'-0x20);
dp('2'-0x20);
dp(':-0x20);
dp('5'-0x20);
dp('9'-0x20);
dp(' '-0x20);
dp('o'-0x20);
dp('f'-0x20);
dp('f'-0x20);

```

```

line(53,0,53,18);
line(0,18,128,18);

```

```

x=0;y=3;
da(128,0);
da(129,0);
da(130,0);
da(131,0);
x=0;y=6;

```

```

da(132,0);
da(133,0);
da(134,0);
da(135,0);

```

```

da(' ',0);
da('1',0);
da('9',0);
i=read_clock(0x8d);
da(i/16+0x30,0);
da(i%16+0x30,0);
da('-',0);
i=read_clock(0x89);
da(i/16+0x30,0);

```

```

da(i%16+0x30,0);
da('-',0);
i=read_clock(0x87);
da(i/16+0x30,0);
da(i%16+0x30,0);

x=13;
y=1;
while (1) {
    x=6;y=3;
    i=read_clock(0x85);
    da(i/16+0x30,0);
    da(i%16+0x30,0);
    da(':',0);
    i=read_clock(0x83);
    da(i/16+0x30,0);
    da(i%16+0x30,0);
    da(':',0);
    i=read_clock(0x81);
    da(i/16+0x30,0);
    da(i%16+0x30,0);

    if (key==6) {
        back_light=!back_light;
        while (key!=0) {}
    }
    //seting
    if (key==k_band) {
        k_w();
        n=0;
        while (1) {
            switch (n) {
                case 0:
                    while (1) {
                        k_w();
                        i=read_clock(c_h);
                        x=6,y=3;
                        da(i/16+0x30,1);da(i%16+0x30,1);
                        if (key==k_inc) {
                            k_w();
                            if ((++i)>0x23) i=0;
                            write_clock(c_h-1,i);
                        } else if (key==k_dec) {
                            k_w();
                            if ((--i)>23) i=23;

```



```

    char *p;
//    for (i=0;i<1024;i++) dd[i]=1;
//    for (i=0;i<1024;i++) if (dd[i]==1) j++;

    i=1236;
    at(0,0);

    sprintf(p, "%u", i);

    at(0,0);
    for (k=0;k<20;k++) {
        j=*p++;
//        dp((* p)+0x20);
//        dp(k+0x10);
//        p++;
    }
    while (1) {}
}

void main(void) {
    unsigned char ad;
    TMOD=0x11;
    EA=1;
    ET0=1;TR0=1;
    EX0=IT0=1;
    TR1=1;ET1=1;
    init();
    write_clock(0x8e,0);
    write_clock(0x80,0);

    write_clock(0x8e,0);
    write_clock(0x80,0);

//    disp_inf();
//    hrd();
    disp_clock();
    while (1) {}

}

```

附录 A MCS-51 单片机定点运算子程序库

说明:

① 多字节定点操作数: 用[R0]或[R1]来表示存放在由 R0 或 R1 指示的连续单元中的数据, 地址小的单元存放数据的高字节。例如: [R0]=123456H, 若(R0)=30H, 则(30H)=12H, (31H)=34H, (32H)=56H。

② 运算精度: 单次定点运算精度为结果最低位的当量值。

③ 工作区: 数据工作区固定在 PSW、A、B、R2~R7, 用户只要不在工作区中存放无关的或非消耗性的信息, 程序就具有较好的透明性。

④ 左规: 左规格化, 每左规一次, 被开方数左移两位。

1. 多字节 BCD 码加法

标 号: BCDA

入口条件: 字节数在 R7 中, 被加数在[R0]中, 加数在[R1]中。

出口信息: 和在[R0]中, 最高位进位在 CY 中。

影响资源: PSW、A、R2 堆栈需求: 2 字节

BCDA: MOV A,R7 ; 取字节数至 R2 中

MOV R2,A

ADD A,R0 ; 初始化数据指针

MOV R0,A

MOV A,R2

ADD A,R1

MOV R1,A

CLR C

BCD1: DEC R0 ; 调整数据指针

DEC R1

MOV A,@R0

ADDC A,@R1 ; 按字节相加

DA A ; 十进制调整

MOV @R0,A ; 和存回[R0]中

DJNZ R2,BCD1 ; 处理完所有字节

RET

2. 多字节 BCD 码减法

标 号: BCDB

入口条件: 字节数在 R7 中, 被减数在[R0]中, 减数在[R1]中。

出口信息: 差在[R0]中, 最高位借位在 CY 中。

影响资源: PSW、A、R2、R3 堆栈需求: 6 字节

BCDB: LCALL NEG1 ; 减数[R1]十进制取补

LCALL BCDA ; 按多字节 BCD 码加法处理

CPL C ; 将补码加法的进位标志转换成借位标志

MOV F0,C ; 保护借位标志

LCALL NEG1 ; 恢复减数[R1]的原始值
 MOV C,F0 ; 恢复借位标志
 RET
 NEG1: MOV A,R0 ; [R1]十进制取补子程序入口
 XCH A,R1 ; 交换指针
 XCH A,R0
 LCALL NEG ; 通过[R0]实现[R1]取补
 MOV A,R0
 XCH A,R1 ; 换回指针
 XCH A,R0
 RET

3. 多字节 BCD 码取补

标 号: NEG

入口条件: 字节数在 R7 中, 操作数在[R0]中。

出口信息: 结果仍在[R0]中。

影响资源: PSW、A、R2、R3 堆栈需求: 2 字节

NEG: MOV A,R7 ; 取(字节数减-)至 R2 中

DEC A

MOV R2,A

MOV A,R0 ; 保护指针

MOV R3,A

NEG0: CLR C

MOV A,#99H

SUBB A,@R0 ; 按字节十进制取补

MOV @R0,A ; 存回[R0]中

INC R0 ; 调整数据指针

DJNZ R2,NEG0 ; 处理完(R2)字节

MOV A,#9AH ; 最低字节单独取补

SUBB A,@R0

MOV @R0,A

MOV A,R3 ; 恢复指针

MOV R0,A

RET

4. 多字节 BCD 码左移十进制一位(乘十)

标 号: BRLN

入口条件: 字节数在 R7 中, 操作数在[R0]中。

出口信息: 结果仍在[R0]中, 移出的十进制最高位在 R3 中。

影响资源: PSW、A、R2、R3 堆栈需求: 2 字节

BRLN: MOV A,R7 ; 取字节数至 R2 中

MOV R2,A

ADD A,R0 ; 初始化数据指针

MOV R0,A

MOV R3,#0 ; 工作单元初始化
 BRL1: DEC R0 ; 调整数据指针
 MOV A,@R0 ; 取一字节
 SWAP A ; 交换十进制高低位
 MOV @R0,A ; 存回
 MOV A,R3 ; 取低字节移出的十进制高位
 XCHD A,@R0 ; 换出本字节的十进制高位
 MOV R3,A ; 保存本字节的十进制高位
 DJNZ R2,BRL1 ; 处理完所有字节
 RET

5. 双字节二进制无符号数乘法

标 号: MULD

入口条件: 被乘数在 R2、R3 中, 乘数在 R6、R7 中。

出口信息: 乘积在 R2、R3、R4、R5 中。

影响资源: PSW、A、B、R2~R7 堆栈需求: 2 字节

MULD: MOV A,R3 ; 计算 R3 乘 R7

MOV B,R7

MUL AB

MOV R4,B ; 暂存部分积

MOV R5,A

MOV A,R3 ; 计算 R3 乘 R6

MOV B,R6

MUL AB

ADD A,R4 ; 累加部分积

MOV R4,A

CLR A

ADDC A,B

MOV R3,A

MOV A,R2 ; 计算 R2 乘 R7

MOV B,R7

MUL AB

ADD A,R4 ; 累加部分积

MOV R4,A

MOV A,R3

ADDC A,B

MOV R3,A

CLR A

RLC A

XCH A,R2 ; 计算 R2 乘 R6

MOV B,R6

MUL AB

ADD A,R3 ; 累加部分积

MOV R3,A

```
MOV A,R2
ADDC A,B
MOV R2,A
RET
```

6. 双字节二进制无符号数平方

标 号: MUL2

入口条件: 待平方数在 R2、R3 中。

出口信息: 结果在 R2、R3、R4、R5 中。

影响资源: PSW、A、B、R2~R5 堆栈需求: 2 字节

```
MUL2: MOV A,R3 ; 计算 R3 平方
MOV B,A
MUL AB
MOV R4,B ; 暂存部分积
MOV R5,A
MOV A,R2 ; 计算 R2 平方
MOV B,A
MUL AB
XCH A,R3 ; 暂存部分积, 并换出 R2 和 R3
XCH A,B
XCH A,R2
MUL AB ; 计算  $2 \times R2 \times R3$ 
CLR C
RLC A
XCH A,B
RLC A
JNC MU20
INC R2 ; 累加溢出量
MU20: XCH A,B ; 累加部分积
ADD A,R4
MOV R4,A
MOV A,R3
ADDC A,B
MOV R3,A
CLR A
ADDC A,R2
MOV R2,A
RET
```

7. 双字节二进制无符号数除法

标 号: DIVD

入口条件: 被除数在 R2、R3、R4、R5 中, 除数在 R6、R7 中。

出口信息: OV=0 时, 双字节商在 R2、R3 中, OV=1 时溢出。

影响资源: PSW、A、B、R1~R7 堆栈需求: 2 字节


```

DIVD: CLR C ; 比较被除数和除数
MOV A,R3
SUBB A,R7
MOV A,R2
SUBB A,R6
JC DVD1
SETB OV ; 溢出
RET
DVD1: MOV B,#10H ; 计算双字节商
DVD2: CLR C ; 部分商和余数同时左移一位
MOV A,R5
RLC A
MOV R5,A
MOV A,R4
RLC A
MOV R4,A
MOV A,R3
RLC A
MOV R3,A
XCH A,R2
RLC A
XCH A,R2
MOV F0,C ; 保存溢出位
CLR C
SUBB A,R7 ; 计算 (R2R3-R6R7)
MOV R1,A
MOV A,R2
SUBB A,R6
ANL C,/F0 ; 结果判断
JC DVD3
MOV R2,A ; 够减, 存放新的余数
MOV A,R1
MOV R3,A
INC R5 ; 商的低位置一
DVD3: DJNZ B,DVD2 ; 计算完十六位商 (R4R5)
MOV A,R4 ; 将商移到 R2R3 中
MOV R2,A
MOV A,R5
MOV R3,A
CLR OV ; 设立成功标志
RET

```

8. 双字节二进制无符号数除以单字节二进制数

标 号: D457

入口条件：被除数在 R4、R5 中，除数在 R7 中。

出口信息：OV=0 时，单字节商在 R3 中，OV=1 时溢出。

影响资源：PSW、A、R3~R7 堆栈需求：2 字节

D457: CLR C

MOV A,R4

SUBB A,R7

JC DV50

SETB OV ; 商溢出

RET

DV50: MOV R6,#8 ; 求平均值 (R4R5 / R7 → R3)

DV51: MOV A,R5

RLC A

MOV R5,A

MOV A,R4

RLC A

MOV R4,A

MOV F0,C

CLR C

SUBB A,R7

ANL C,F0

JC DV52

MOV R4,A

DV52: CPL C

MOV A,R3

RLC A

MOV R3,A

DJNZ R6,DV51

MOV A,R4 ; 四舍五入

ADD A,R4

JC DV53

SUBB A,R7

JC DV54

DV53: INC R3

DV54: CLR OV

RET

9. 三字节二进制无符号数除以单字节二进制数

标 号：DV31

入口条件：被除数在 R3、R4、R5 中，除数在 R7 中。

出口信息：OV=0 时，双字节商在 R4、R5 中，OV=1 时溢出。

影响资源：PSW、A、B、R2~R7 堆栈需求：2 字节

DV31: CLR C

```

MOV A,R3
SUBB A,R7
JC DV30
SETB OV ; 商溢出
RET
DV30: MOV R2,#10H ; 求 R3R4R5 / R7 → R4R5
DM23: CLR C
MOV A,R5
RLC A
MOV R5,A
MOV A,R4
RLC A
MOV R4,A
MOV A,R3
RLC A
MOV R3,A
MOV F0,C
CLR C
SUBB A,R7
ANL C,/F0
JC DM24
MOV R3,A
INC R5
DM24: DJNZ R2,DM23
MOV A,R3 ; 四舍五入
ADD A,R3
JC DM25
SUBB A,R7
JC DM26
DM25: INC R5
MOV A,R5
JNZ DM26
INC R4
DM26: CLR OV
RET ; 商在 R4R5 中

```

10. 双字节二进制有符号数乘法（补码）

标 号: MULS

入口条件: 被乘数在 R2、R3 中, 乘数在 R6、R7 中。

出口信息: 乘积在 R2、R3、R4、R5 中。

影响资源: PSW、A、B、R2~R7 堆栈需求: 4 字节

```
MULS: MOV R4,#0 ; 清零 R4R5
```

```
MOV R5,#0
```

```
LCALL MDS ; 计算结果的符号和两个操作数的绝对值
```

LCALL MUL0 : 计算两个绝对值的乘积

SJMP MDSE : 用补码表示结果

11. 双字节二进制有符号数除法(补码)

标 号: DIVS

入口条件: 被除数在 R2、R3、R4、R5 中, 除数在 R6、R7 中。

出口信息: OV=0 时商在 R2、R3 中, OV=1 时溢出。

影响资源: PSW、A、B、R1~R7 堆栈需求: 5 字节

DIVS: LCALL MDS : 计算结果的符号和两个操作数的绝对值

PUSH PSW : 保存结果的符号

LCALL DIVD : 计算两个绝对值的商

JNB OV,DVS1 : 溢出否?

POP ACC : 溢出, 放去结果的符号, 保留溢出标志

RET

DVS1: POP PSW : 未溢出, 取出结果的符号

MOV R4,#0

MOV R5,#0

MDSE: JB F0,MDS2 : 用补码表示结果

CLR OV : 结果为正, 原码即补码, 计算成功

RET

MDS: CLR F0 : 结果符号初始化

MOV A,R6 : 判断第二操作数的符号

JNB ACC.7,MDS1 : 为正, 不必处理

CPL F0 : 为负, 结果符号取反

XCH A,R7 : 第二操作数取补, 得到其绝对值

CPL A

ADD A,#1

XCH A,R7

CPL A

ADDC A,#0

MOV R6,A

MDS1: MOV A,R2 : 判断第一操作数或运算结果的符号

JNB ACC.7,MDS3: 为正, 不必处理

CPL F0 : 为负, 结果符号取反

MDS2: MOV A,R5 : 求第一操作数的绝对值或运算结果的补码

CPL A

ADD A,#1

MOV R5,A

MOV A,R4

CPL A

ADDC A,#0

MOV R4,A

MOV A,R3

CPL A

```

ADDC A,#0
MOV R3,A
MOV A,R2
CPL A
ADDC A,#0
MOV R2,A
MDS3: CLR OV ; 运算成功
RET

```

12. 双字节二进制无符号数开平方（快速）

标 号: SH2

入口条件: 被开方数在 R2、R3 中。

出口信息: 平方根仍在 R2、R3 中，整数部分的位数为原数的一半，其余为小数。

影响资源: PSW、A、B、R2~R7 堆栈需求: 2 字节

```

SH2: MOV A,R2
ORL A,R3
JNZ SH20
RET ; 被开方数为零，不必运算
SH20: MOV R7,#0 ; 左规次数初始化
MOV A,R2
SH22: ANL A,#0C0H ; 被开方数高字节小于 40H 否?
JNZ SQRH ; 不小于 40H，左规格化完成，转开方过程
CLR C ; 每左规一次，被开方数左移两位
MOV A,R3
RLC A
MOV F0,C
CLR C
RLC A
MOV R3,A
MOV A,R2
MOV ACC.7,C
MOV C,F0
RLC A
RLC A
MOV R2,A
INC R7 ; 左规次数加一
SJMP SH22 ; 继续左规

```

13. 四字节二进制无符号数开平方（快速）

标 号: SH4

入口条件: 被开方数在 R2、R3、R4、R5 中。

出口信息: 平方根在 R2、R3 中，整数部分的位数为原数的一半，其余为小数。

影响资源: PSW、A、B、R2~R7 堆栈需求: 2 字节

```

SH4: MOV A,R2

```

```

ORL A,R3
ORL A,R4
ORL A,R5
JNZ SH40
RET ; 被开方数为零, 不必运算
SH40: MOV R7,#0 ; 左规次数初始化
MOV A,R2
SH41: ANL A,#0C0H ; 被开方数高字节小于 40H 否?
JNZ SQRH ; 不小于 40H, 左规格化完成
MOV R6,#2 ; 每左规一次, 被开方数左移两位
SH42: CLR C ; 被开方数左移一位
MOV A,R5
RLC A
MOV R5,A
MOV A,R4
RLC A
MOV R4,A
MOV A,R3
RLC A
MOV R3,A
MOV A,R2
RLC A
MOV R2,A
DJNZ R6,SH42 ; 被开方数左移完两位
INC R7 ; 左规次数加一
SJMP SH41 ; 继续左规
SQRH: MOV A,R2 ; 规格化后高字节按折线法分为三个区间
ADD A,#57H
JC SQR2
ADD A,#45H
JC SQR1
ADD A,#24H
MOV B,#0E3H ; 第一区间的斜率
MOV R4,#80H ; 第一区间的平方根基数
SJMP SQR3
SQR1: MOV B,#0B2H ; 第二区间的斜率
MOV R4,#0A0H ; 第二区间的平方根基数
SJMP SQR3
SQR2: MOV B,#8DH ; 第三区间的斜率
MOV R4,#0D0H ; 第三区间的平方根基数
SQR3: MUL AB ; 与区间基点的偏移量乘区间斜率
MOV A,B
ADD A,R4 ; 累加到平方根的基数上
MOV R4,A
MOV B,A

```

MUL AB ; 求当前平方根的幂
 XCH A,R3 ; 求偏移量 (存放在 R2R3 中)
 CLR C
 SUBB A,R3
 MOV R3,A
 MOV A,R2
 SUBB A,B
 MOV R2,A
 SQR4: SETB C ; 用减奇数法校正一个字节平方根
 MOV A,R4 ; 当前平方根的两倍加一存入 R5R6 中
 RLC A
 MOV R6,A
 CLR A
 RLC A
 MOV R5,A
 MOV A,R3 ; 偏移量小于该奇数否?
 SUBB A,R6
 MOV B,A
 MOV A,R2
 SUBB A,R5
 JC SQR5 ; 小于, 校正结束, 已达到一个字节的精度
 INC R4 ; 不小于, 平方根加一
 MOV R2,A ; 保存新的偏移量
 MOV R3,B
 SJMP SQR4 ; 继续校正
 SQR5: MOV A,R4 ; 将一个字节精度的根存入 R2
 XCH A,R2
 RRC A
 MOV F0,C ; 保存最终偏移量的最高位
 MOV A,R3
 MOV R5,A ; 将最终偏移量的低八位存入 R5 中
 MOV R4,#8 ; 通过 (R5R6 / R2) 求根的低字节
 SQR6: CLR C
 MOV A,R3
 RLC A
 MOV R3,A
 CLR C
 MOV A,R5
 SUBB A,R2
 JB F0,SQR7
 JC SQR8
 SQR7: MOV R5,A
 INC R3
 SQR8: CLR C
 MOV A,R5

```

RLC A
MOV R5,A
MOV F0,C
DJNZ R4,SQR6 ; 根的第二字节计算完, 在 R3 中
MOV A,R7 ; 取原被开方数的左规次数
JZ SQRE ; 未左规, 开方结束
SQR9: CLR C ; 按左规次数右移平方根, 得到实际根
MOV A,R2
RRC A
MOV R2,A
MOV A,R3
RRC A
MOV R3,A
DJNZ R7,SQR9
SQRE: RET

```

14. 单字节十六进制数转换成双字节 ASCII 码

标 号: HASC

入口条件: 待转换的单字节十六进制数在累加器 A 中。

出口信息: 高四位的 ASCII 码在 A 中, 低四位的 ASCII 码在 B 中。

影响资源: PSW、A、B 堆栈需求: 4 字节

```

HASC: MOV B,A ; 暂存待转换的单字节十六进制数
LCALL HAS1 ; 转换低四位
XCH A,B ; 存放低四位的 ASCII 码
SWAP A ; 准备转换高四位
HAS1: ANL A,#0FH ; 将累加器的低四位转换成 ASCII 码
ADD A,#90H
DA A
ADDC A,#40H
DA A
RET

```

15. ASCII 码转换成十六进制数

标 号: ASCH

入口条件: 待转换的 ASCII 码 (30H~39H 或 41H~46H) 在 A 中。

出口信息: 转换后的十六进制数 (00H~0FH) 仍在累加器 A 中。

影响资源: PSW、A 堆栈需求: 2 字节

```

ASCH: CLR C
SUBB A,#30H
JNB ACC.4,ASH1
SUBB A,#7
ASH1: RET

```


16. 单字节十六进制整数转换成单字节 BCD 码整数

标 号: HB CD

入口条件: 待转换的单字节十六进制整数在累加器 A 中。

出口信息: 转换后的 BCD 码整数 (十位和个位) 仍在累加器 A 中, 百位在 R3 中。

影响资源: PSW、A、B、R3 堆栈需求: 2 字节

HB CD: MOV B, #100 ; 分离出百位, 存放在 R3 中

DIV AB

MOV R3, A

MOV A, #10 ; 余数继续分离十位和个位

XCH A, B

DIV AB

SWAP A

ORL A, B ; 将十位和个位拼装成 BCD 码

RET

17. 双字节十六进制整数转换成双字节 BCD 码整数

标 号: HB 2

入口条件: 待转换的双字节十六进制整数在 R6、R7 中。

出口信息: 转换后的三字节 BCD 码整数在 R3、R4、R5 中。

影响资源: PSW、A、R2~R7 堆栈需求: 2 字节

HB 2: CLR A ; BCD 码初始化

MOV R3, A

MOV R4, A

MOV R5, A

MOV R2, #10H ; 转换双字节十六进制整数

HB 3: MOV A, R7 ; 从高端移出待转换数的一位到 CY 中

RLC A

MOV R7, A

MOV A, R6

RLC A

MOV R6, A

MOV A, R5 ; BCD 码带进位自身相加, 相当于乘 2

ADDC A, R5

DA A ; 十进制调整

MOV R5, A

MOV A, R4

ADDC A, R4

DA A

MOV R4, A

MOV A, R3

ADDC A, R3

MOV R3, A ; 双字节十六进制数的万位数不超过 6, 不用调整

DJNZ R2, HB 3 ; 处理完 16bit

RET

18. 单字节十六进制小数转换成单字节 BCD 码小数

标 号: HBD

入口条件: 待转换的单字节十六进制小数在累加器 A 中。

出口信息: CY=0 时转换后的 BCD 码小数仍在 A 中。CY=1 时原小数接近整数 1。

影响资源: PSW、A、B 堆栈需求: 2 字节

```
HBD: MOV B,#100 ; 原小数扩大一百倍
      MUL AB
      RLC A ; 余数部分四舍五入
      CLR A
      ADDC A,B
      MOV B,#10 ; 分离出十分位和百分位
      DIV AB
      SWAP A
      ADD A,B ; 拼装成单字节 BCD 码小数
      DA A ; 调整后若有进位, 原小数接近整数 1
      RET
```

19. 双字节十六进制小数转换成双字节 BCD 码小数

标 号: HBD2

入口条件: 待转换的双字节十六进制小数在 R2、R3 中。

出口信息: 转换后的双字节 BCD 码小数仍在 R2、R3 中。

影响资源: PSW、A、B、R2、R3、R4、R5 堆栈需求: 6 字节

```
HBD2: MOV R4,#4 ; 四位十进制码
      HBD3: MOV A,R3 ; 原小数扩大十倍
      MOV B,#10
      MUL AB
      MOV R3,A
      MOV R5,B
      MOV A,R2
      MOV B,#10
      MUL AB
      ADD A,R5
      MOV R2,A
      CLR A
      ADDC A,B
      PUSH ACC ; 保存溢出的一位十进制码
      DJNZ R4,HBD3 ; 计算完四位十进制码
      POP ACC ; 取出万分位
      MOV R3,A
      POP ACC ; 取出千分位
      SWAP A
      ORL A,R3 ; 拼装成低字节 BCD 码小数
      MOV R3,A
```

POP ACC ; 取出百分位
 MOV R2,A
 POP ACC ; 取出十分位
 SWAP A
 ORL A,R2 ; 拼装成高字节 BCD 码小数
 MOV R2,A
 RET

20. 单字节 BCD 码整数转换成单字节十六进制整数

标 号: BCDH

入口条件: 待转换的单字节 BCD 码整数在累加器 A 中。

出口信息: 转换后的单字节十六进制整数仍在累加器 A 中。

影响资源: PSW、A、B、R4 堆栈需求: 2 字节

BCDH: MOV B,#10H ; 分离十位和个位
 DIV AB
 MOV R4,B ; 暂存个位
 MOV B,#10 ; 将十位转换成十六进制
 MUL AB
 ADD A,R4 ; 按十六进制加上个位
 RET

21. 双字节 BCD 码整数转换成双字节十六进制整数

标 号: BH2

入口条件: 待转换的双字节 BCD 码整数在 R2、R3 中。

出口信息: 转换后的双字节十六进制整数仍在 R2、R3 中。

影响资源: PSW、A、B、R2、R3、R4 堆栈需求: 4 字节

BH2: MOV A,R3 ; 将低字节转换成十六进制
 LCALL BCDH
 MOV R3,A
 MOV A,R2 ; 将高字节转换成十六进制
 LCALL BCDH
 MOV B,#100 ; 扩大一百倍
 MUL AB
 ADD A,R3 ; 和低字节按十六进制相加
 MOV R3,A
 CLR A
 ADDC A,B
 MOV R2,A
 RET

22. 单字节 BCD 码小数转换成单字节十六进制小数

标 号: BHD

入口条件: 待转换的单字节 BCD 码数在累加器 A 中。

出口信息：转换后的单字节十六进制小数仍在累加器 A 中。

影响资源：PSW、A、R2、R3 堆栈需求：2 字节

BHD: MOV R2,#8 ; 准备计算一个字节小数

BHD0: ADD A,ACC ; 按十进制倍增

DA A

XCH A,R3

RLC A ; 将进位标志移入结果中

XCH A,R3

DJNZ R2,BHD0 ; 共计算 8bit 小数

ADD A,#0B0H ; 剩余部分达到 0.50 否?

JNC BHD1 ; 四舍

INC R3 ; 五入

BHD1: MOV A,R3 ; 取结果

RET

23. 双字节 BCD 码小数转换成双字节十六进制小数

标 号：BHD2

入口条件：待转换的双字节 BCD 码小数在 R4、R5 中。

出口信息：转换后的双字节十六进制小数在 R2、R3 中。

影响资源：PSW、A、R2~R6 堆栈需求：2 字节

BHD2: MOV R6,#10H ; 准备计算两个字节小数

BHD3: MOV A,R5 ; 按十进制倍增

ADD A,R5

DA A

MOV R5,A

MOV A,R4

ADDC A,R4

DA A

MOV R4,A

MOV A,R3 ; 将进位标志移入结果中

RLC A

MOV R3,A

MOV A,R2

RLC A

MOV R2,A

DJNZ R6,BHD3 ; 共计算 16bit 小数

MOV A,R4

ADD A,#0B0H ; 剩余部分达到 0.50 否?

JNC BHD4 ; 四舍

INC R3 ; 五入

MOV A,R3

JNZ BHD4

INC R2

BHD4: RET

24. 求单字节十六进制无符号数据块的极值

标 号: MM

入口条件: 数据块的首址在 DPTR 中, 数据个数在 R7 中。

出口信息: 最大值在 R6 中, 地址在 R2R3 中; 最小值在 R7 中, 地址在 R4R5 中。

影响资源: PSW、A、B、R1~R7 堆栈需求: 4 字节

```
MM: MOV B,R7 ; 保存数据个数
MOVX A,@DPTR ; 读取第一个数据
MOV R6,A ; 作为最大值的初始值
MOV R7,A ; 也作为最小值的初始值
MOV A,DPL ; 取第一个数据的地址
MOV R3,A ; 作为最大值存放地址的初始值
MOV R5,A ; 也作为最小值存放地址的初始值
MOV A,DPH
MOV R2,A
MOV R4,A
MOV A,B ; 取数据个数
DEC A ; 减一, 得到需要比较的次数
JZ MME ; 只有一个数据, 不需要比较
MOV R1,A ; 保存比较次数
PUSH DPL ; 保护数据块的首址
PUSH DPH
MM1: INC DPTR ; 指向一个新的数据
MOVX A,@DPTR ; 读取这个数据
MOV B,A ; 保存
SETB C ; 与最大值比较
SUBB A,R6
JC MM2 ; 不超过当前最大值, 保持当前最大值
MOV R6,B ; 超过当前最大值, 更新最大值存放地址
MOV R2,DPH ; 同时更新最大值存放地址
MOV R3,DPL
SJMP MM3
MM2: MOV A,B ; 与最小值比较
CLR C
SUBB A,R7
JNC MM3 ; 大于或等于当前最小值, 保持当前最小值
MOV R7,B ; 更新最小值
MOV R4,DPH ; 更新最小值存放地址
MOV R5,DPL
MM3: DJNZ R1,MM1 ; 处理完全部数据
POP DPH ; 恢复数据首址
POP DPL
MME: RET
```

25. 求单字节十六进制有符号数据块的极值

标 号: MMS

入口条件: 数据块的首址在 DPTR 中, 数据个数在 R7 中。

出口信息: 最大值在 R6 中, 地址在 R2R3 中; 最小值在 R7 中, 地址在 R4R5 中。

影响资源: PSW、A、B、R1~R7 堆栈需求: 4 字节

```
MMS: MOV B,R7 ; 保存数据个数
MOVX A,@DPTR ; 读取第一个数据
MOV R6,A ; 作为最大值的初始值
MOV R7,A ; 也作为最小值的初始值
MOV A,DPL ; 取第一个数据的地址
MOV R3,A ; 作为最大值存放地址的初始值
MOV R5,A ; 也作为最小值存放地址的初始值
MOV A,DPH
MOV R2,A
MOV R4,A
MOV A,B ; 取数据个数
DEC A ; 减一, 得到需要比较的次数
JZ MMSE ; 只有一个数据, 不需要比较
MOV R1,A ; 保存比较次数
PUSH DPL ; 保护数据块的首址
PUSH DPH
MMS1: INC DPTR ; 调整数据指针
MOVX A,@DPTR ; 读取一个数据
MOV B,A ; 保存
SETB C ; 与最大值比较
SUBB A,R6
JZ MMS4 ; 相同, 不更新最大值
JNB OV,MMS2 ; 差未溢出, 符号位有效
CPL ACC.7 ; 差溢出, 符号位取反
MMS2: JB ACC.7,MMS4; 差为负, 不更新最大值
MOV R6,B ; 更新最大值
MOV R2,DPH ; 更新最大值存放地址
MOV R3,DPL
SJMP MMS7
MMS4: MOV A,B ; 与最小值比较
CLR C
SUBB A,R7
JNB OV,MMS6 ; 差未溢出, 符号位有效
CPL ACC.7 ; 差溢出, 符号位取反
MMS6: JNB ACC.7,MMS7; 差为正, 不更新最小值
MOV R7,B ; 更新最小值
MOV R4,DPH ; 更新最小值存放地址
MOV R5,DPL
MMS7: DJNZ R1,MMS1 ; 处理完全部数据
```

POP DPH ; 恢复数据首址

POP DPL

MMSE: RET

26. 顺序查找 (ROM) 单字节表格

标 号: FDS1

入口条件: 待查找的内容在 A 中, 表格首址在 DPTR 中, 表格的字节数在 R7 中。

出口信息: OV=0 时, 顺序号在累加器 A 中; OV=1 时, 未找到。

影响资源: PSW、A、B、R2、R6 堆栈需求: 2 字节

FDS1: MOV B,A ; 保存待查找的内容

MOV R2,#0 ; 顺序号初始化 (指向表首)

MOV A,R7 ; 保存表格的长度

MOV R6,A

FD11: MOV A,R2 ; 按顺序号读取表格内容

MOVC A,@A+DPTR

CJNE A,B,FD12; 与待查找的内容比较

CLR OV ; 相同, 查找成功

MOV A,R2 ; 取对应的顺序号

RET

FD12: INC R2 ; 指向表格中的下一个内容

DJNZ R6,FD11 ; 查完全部表格内容

SETB OV ; 未查找到, 失败

RET

27. 顺序查找 (ROM) 双字节表格

标 号: FDS2

入口条件: 查找内容在 R4、R5 中, 表格首址在 DPTR 中, 数据总个数在 R7 中。

出口信息: OV=0 时顺序号在累加器 A 中, 地址在 DPTR 中; OV=1 时未找到。

影响资源: PSW、A、R2、R6、DPTR 堆栈需求: 2 字节

FDS2: MOV A,R7 ; 保存表格中数据的个数

MOV R6,A

MOV R2,#0 ; 顺序号初始化 (指向表首)

FD21: CLR A ; 读取表格内容的高字节

MOVC A,@A+DPTR

XRL A,R4 ; 与待查找内容的高字节比较

JNZ FD22

MOV A,#1 ; 读取表格内容的低字节

MOVC A,@A+DPTR

XRL A,R5 ; 与待查找内容的低字节比较

JNZ FD22

CLR OV ; 相同, 查找成功

MOV A,R2 ; 取对应的顺序号

RET

FD22: INC DPTR ; 指向下一个数据

```

INC DPTR
INC R2 ; 顺序号加一
DJNZ R6,FD21 ; 查完全部数据
SETB OV ; 未查找到,失败
RET

```

28. 对分查找 (ROM) 单字节无符号增序数据表格

标 号: FDD1

入口条件: 待查找的内容在累加器 A 中, 表格首址在 DPTR 中, 字节数在 R7 中。

出口信息: OV=0 时, 顺序号在累加器 A 中; OV=1 时, 未找到。

影响资源: PSW、A、B、R2、R3、R4 堆栈需求: 2 字节

```

FDD1: MOV B,A ; 保存待查找的内容
MOV R2,#0 ; 区间低端指针初始化 (指向第一个数据)
MOV A,R7
DEC A
MOV R3,A ; 区间高端指针初始化 (指向最后一个数据)
FD61: CLR C ; 判断区间大小
MOV A,R3
SUBB A,R2
JC FD69 ; 区间消失, 查找失败
RRC A ; 取区间大小的一半
ADD A,R2 ; 加上区间的低端
MOV R4,A ; 得到区间的中心
MOVC A,@A+DPTR; 读取该点的内容
CJNE A,B,FD65; 与待查找的内容比较
CLR OV ; 相同, 查找成功
MOV A,R4 ; 取顺序号
RET
FD65: JC FD68 ; 该点的内容比待查找的内容大否?
MOV A,R4 ; 偏大, 取该点位置
DEC A ; 减一
MOV R3,A ; 作为新的区间高端
SJMP FD61 ; 继续查找
FD68: MOV A,R4 ; 偏小, 取该点位置
INC A ; 加一
MOV R2,A ; 作为新的区间低端
SJMP FD61 ; 继续查找
FD69: SETB OV ; 查找失败
RET

```

29. 对分查找 (ROM) 双字节无符号增序数据表格

标 号: FDD2

入口条件: 查找内容在 R4、R5 中, 表格首址在 DPTR 中, 数据个数在 R7 中。

出口信息: OV=0 时顺序号在累加器 A 中, 址在 DPTR 中; OV=1 时未找到。

影响资源: PSW、A、B、R1~R7、DPTR 堆栈需求: 2 字节

FDD2: MOV R2,#0 ; 区间低端指针初始化 (指向第一个数据)

MOV A,R7

DEC A

MOV R3,A ; 区间高端指针初始化, 指向最后一个数据

MOV R6,DPH ; 保存表格首址

MOV R7,DPL

FD81: CLR C ; 判断区间大小

MOV A,R3

SUBB A,R2

JC FD89 ; 区间消失, 查找失败

RRC A ; 取区间大小的一半

ADD A,R2 ; 加上区间的低端

MOV R1,A ; 得到区间的中心

MOV DPH,R6

CLR C ; 计算区间中心的地址

RLC A

JNC FD82

INC DPH

FD82: ADD A,R7

MOV DPL,A

JNC FD83

INC DPH

FD83: CLR A ; 读取该点的内容的高字节

MOVC A,@A+DPTR

MOV B,R4 ; 与待查找内容的高字节比较

CJNE A,B,FD84: 不相同

MOV A,#1 ; 读取该点的内容的低字节

MOVC A,@A+DPTR

MOV B,R5

CJNE A,B,FD84: 与待查找内容的低字节比较

MOV A,R1 ; 取顺序号

CLR OV ; 查找成功

RET

FD84: JC FD86 ; 该点的内容比待查找的内容大否?

MOV A,R1 ; 偏大, 取该点位置

DEC A ; 减一

MOV R3,A ; 作为新的区间高端

SJMP FD81 ; 继续查找

FD86: MOV A,R1 ; 偏小, 取该点位置

INC A ; 加一

MOV R2,A ; 作为新的区间低端

SJMP FD81 ; 继续查找

FD89: MOV DPH,R6 ; 相同, 恢复首址

MOV DPL,R7

SETB OV : 查找失败

RET

30. 求单字节十六进制无符号数据块的平均值

标 号: DDM1

入口条件: 数据块的首址在 DPTR 中, 数据个数在 R7 中。

出口信息: 平均值在累加器 A 中。

影响资源: PSW、A、R2~R6 堆栈需求: 4 字节

```
DDM1: MOV A,R7 ; 保存数据个数
MOV R2,A
PUSH DPH
PUSH DPL
CLR A : 初始化累加和
MOV R4,A
MOV R5,A
DM11: MOVX A,@DPTR ; 读取一个数据
ADD A,R5 : 累加到累加和中
MOV R5,A
JNC DM12
INC R4
DM12: INC DPTR ; 调整指针
DJNZ R2,DM11 ; 累加完全部数据
LCALL D457 : 求平均值 (R4R5 / R7 → R3)
MOV A,R3 ; 取平均值
POP DPL
POP DPH
RET
```

31. 求双字节十六进制无符号数据块的平均值

标 号: DDM2

入口条件: 数据块的首址在 DPTR 中, 双字节数据总个数在 R7 中。

出口信息: 平均值在 R4、R5 中。

影响资源: PSW、A、R2~R6 堆栈需求: 4 字节

```
DDM2: MOV A,R7 ; 保存数据个数
MOV R2,A : 初始化数据指针
PUSH DPL ; 保持首址
PUSH DPH
CLR A : 初始化累加和
MOV R3,A
MOV R4,A
MOV R5,A
DM20: MOVX A,@DPTR ; 读取一个数据的高字节
MOV B,A
INC DPTR
```

MOVX A,@DPTR ; 读取一个数据的低字节
 INC DPTR
 ADD A,R5 ; 累加到累加和中
 MOV R5,A
 MOV A,B
 ADDC A,R4
 MOV R4,A
 JNC DM21
 INC R3
 DM21: DJNZ R2,DM20 ; 累加完全部数据
 POP DPH ; 恢复首址
 POP DPL
 LJMP DV31 ; 求 R3R4R5 / R7 → R4R5, 得到平均值

32. 求单字节数据块的（异或）校验和

标 号: XR1

入口条件: 数据块的首址在 DPTR 中, 数据的个数在 R6、R7 中。

出口信息: 校验和在累加器 A 中。

影响资源: PSW、A、B、R4~R7 堆栈需求: 2 字节

XR1: MOV R4,DPH ; 保存数据块的首址
 MOV R5,DPL
 MOV A,R7 ; 双字节计数器调整
 JZ XR10
 INC R6
 XR10: MOV B,#0 ; 校验和初始化
 XR11: MOVX A,@DPTR ; 读取一个数据
 XRL B,A ; 异或运算
 INC DPTR ; 指向下一个数据
 DJNZ R7,XR11 ; 双字节计数器减一
 DJNZ R6,XR11
 MOV DPH,R4 ; 恢复数据首址
 MOV DPL,R5
 MOV A,B ; 取校验和
 RET

33. 求双字节数据块的（异或）校验和

标 号: XR2

入口条件: 数据块的首址在 DPTR 中, 双字节数据总个数在 R6、R7 中。

出口信息: 校验和在 R2、R3 中。

影响资源: PSW、A、R2~R7 堆栈需求: 2 字节

XR2: MOV R4,DPH ; 保存数据块的首址
 MOV R5,DPL
 MOV A,R7 ; 双字节计数器调整
 JZ XR20

```

INC R6
XR20: CLR A ; 校验和初始化
MOV R2,A
MOV R3,A
XR21: MOVX A,@DPTR ; 读取一个数据的高字节
XRL A,R2 ; 异或运算
MOV R2,A
INC DPTR
MOVB A,@DPTR ; 读取一个数据的低字节
XRL A,R3 ; 异或运算
MOV R3,A
INC DPTR ; 指向下一个数据
DJNZ R7,XR21 ; 双字节计数器减一
DJNZ R6,XR21
MOV DPH,R4 ; 恢复数据首址
MOV DPL,R5
RET

```

34. 单字节无符号数据块排序（增序）

标 号: SORT

入口条件: 数据块的首址在 R0 中, 字节数在 R7 中。

出口信息: 完成排序（增序）

影响资源: PSW、A、R2~R6 堆栈需求: 2 字节

```

SORT: MOV A,R7
MOV R5,A ; 比较次数初始化
SRT1: CLR F0 ; 交换标志初始化
MOV A,R5 ; 取上遍比较次数
DEC A ; 本遍比上遍减少一次
MOV R5,A ; 保存本遍次数
MOV R2,A ; 复制到计数器中
JZ SRT5 ; 若为零, 排序结束
MOV A,R0 ; 保存数据指针
MOV R6,A
SRT2: MOV A,@R0 ; 读取一个数据
MOV R3,A
INC R0 ; 指向下一个数据
MOV A,@R0 ; 再读取一个数据
MOV R4,A
CLR C
SUBB A,R3 ; 比较两个数据的大小
JNC SRT4 ; 顺序正确（增序或相同），不必交换
SETB F0 ; 设立交换标志
MOV A,R3 ; 将两个数据交换位置

```

```

MOV @R0,A
DEC R0
MOV A,R4
MOV @R0,A
INC R0 ; 指向下一个数据
SRT4: DJNZ R2,SRT2 ; 完成本遍的比较次数
MOV A,R6 ; 恢复数据首址
MOV R0,A
JB F0,SRT1 ; 本遍若进行过交换, 则需继续排序
SRT5: RET ; 排序结束
END

```

附录 B MCS-51 单片机浮点运算子程序库

说明:

① 双字节定点操作数: 用[R0]或[R1]来表示存放在由 R0 或 R1 指示的连续单元中的数据, 地址小的单元存放高字节。如果[R0]=1234H, 若(R0)=30H, 则(30H)=12H, (31H)=34H。

② 二进制浮点操作数: 用三个字节表示, 第一个字节的最高位为数符, 其余七位为阶码(补码形式), 第二字节为尾数的高字节, 第三字节为尾数的低字节, 尾数用双字节纯小数(原码)来表示。当尾数的最高位为 1 时, 便称为规格化浮点数, 简称操作数。在程序说明中, 也用[R0]或[R1]来表示 R0 或 R1 指示的浮点操作数, 例如: 当[R0]=-6.000 时, 则二进制浮点数表示为 83C000H。若(R0)=30H, 则(30H)=83H, (31H)=0C0H, (32H)=00H。

③ 十进制浮点操作数: 用三个字节表示, 第一个字节的最高位为数符, 其余七位为阶码(二进制补码形式), 第二字节为尾数的高字节, 第三字节为尾数的低字节, 尾数用双字节 BCD 码纯小数(原码)来表示。当十进制数的绝对值大于 1 时, 阶码就等于整数部分的位数, 如 876.5 的阶码是 03H, -876.5 的阶码是 83H; 当十进制数的绝对值小于 1 时, 阶码就等于 80H 减去小数点后面零的个数, 例如 0.00382 的阶码是 7EH, -0.00382 的阶码是 0FEH。在程序说明中, 用[R0]或[R1]来表示 R0 或 R1 指示的十进制浮点操作数。例如有一个十进制浮点操作数存放在 30H、31H、32H 中, 数值是 -0.07315, 即 -0.7315 乘以 10 的 -1 次方, 则(30H)=0FFH, 31H=73H, (32H)=15H。若用[R0]来指向它, 则应使(R0)=30H。

④ 运算精度: 单次定点运算精度为结果最低位的当量值; 单次二进制浮点算术运算的精度优于十万分之一; 单次二进制浮点超越函数运算的精度优于万分之一; BCD 码浮点数本身的精度比较低(万分之一到千分之一), 不宜作为运算的操作数, 仅用于输入或输出时的数制转换。不管那种数据格式, 随着连续运算的次数增加, 精度都会下降。

⑤ 工作区: 数据工作区固定在 A、B、R2~R7, 数符或标志工作区固定在 PSW 和 23H 单元(位 1CH~1FH)。在浮点系统中, R2、R3、R4 和位 1FH 为第一工作区, R5、R6、R7 和位 1EH 为第二工作区。用户只要不在工作区中存放无关的或非消耗性的信息, 程序就具有较好的透明性。

⑥ 子程序调用范例: 由于本程序库特别注意了各子程序接口的相容性, 很容易采用积木方式(或流水线方式)完成一个公式的计算。以浮点运算为例: 计算 $y = L_n \sqrt{|\sin(ab/c+d)|}$

已知: $a=-123.4$; $b=0.7577$; $c=56.34$; $d=1.276$; 它们分别存放在 30H、33H、36H、39H 开始的连续三个单元中。用 BCD 码浮点数表示时, 分别为 $a=831234H$; $b=007577H$; $c=025634H$; $d=011276H$ 。求解过程: 通过调用 BTOF 子程序, 将各变量转换成二进制浮点操作数, 再进行各种运算, 最后调用 FTOB 子程序,

还原成十进制形式，供输出使用。程序如下：

```
TEST: MOV R0,#39H ; 指向 BCD 码浮点操作数 d
LCALL BTOF ; 将其转换成二进制浮点操作数
MOV R0,#36H ; 指向 BCD 码浮点操作数 c
LCALL BTOF ; 将其转换成二进制浮点操作数
MOV R0,#33H ; 指向 BCD 码浮点操作数 b
LCALL BTOF ; 将其转换成二进制浮点操作数
MOV R0,#30H ; 指向 BCD 码浮点操作数 a
LCALL BTOF ; 将其转换成二进制浮点操作数
MOV R1,#33H ; 指向二进制浮点操作数 b
LCALL FMUL ; 进行浮点乘法运算
MOV R1,#36H ; 指向二进制浮点操作数 c
LCALL FDIV ; 进行浮点除法运算
MOV R1,#39H ; 指向二进制浮点操作数 d
LCALL FADD ; 进行浮点加法运算
LCALL FSIN ; 进行浮点正弦运算
LCALL FABS ; 进行浮点绝对值运算
LCALL FSQR ; 进行浮点开平方运算
LCALL FLN ; 进行浮点对数运算
LCALL FTOB ; 将结果转换成 BCD 码浮点数
STOP: LJMP STOP
END
```

运行结果，[R0]=804915H，即 $y=-0.4915$ ，比较精确的结果应该是-0.491437。

1. 浮点数格式化

标 号：FSDT

入口条件：待格式化浮点操作数在[R0]中。

出口信息：已格式化浮点操作数仍在[R0]中。

影响资源：PSW、A、R2、R3、R4、位 1FH 堆栈需求：6 字节

FSDT: LCALL MVR0 ; 将待格式化操作数传送到第一工作区中

LCALL RLN ; 通过左规完成格式化

LJMP MOV0 ; 将已格式化浮点操作数传回到[R0]中

2. 浮点数加法

标 号：FADD

入口条件：被加数在[R0]中，加数在[R1]中。

出口信息：OV=0 时，和仍在[R0]中，OV=1 时，溢出。

影响资源：PSW、A、B、R2~R7、位 1EH、1FH 堆栈需求：6 字节

FADD: CLR F0 ; 设立加法标志

SJMP AS ; 计算代数和

3. 浮点数减法

标 号：FSUB

入口条件：被减数在[R0]中，减数在[R1]中。

出口信息: OV=0 时, 差仍在[R0]中, OV=1 时, 溢出。

影响资源: PSW、A、B、R2~R7、位 1EH、1FH 堆栈需求: 6 字节

FSUB: SETB F0 ; 设立减法标志

AS: LCALL MVR1 ; 计算代数和。先将[R1]传送到第二工作区

MOV C,F0 ; 用加减标志来校正第二操作数的有效符号

RRC A

XRL A,@R1

MOV C,ACC.7

ASN: MOV 1EH,C ; 将第二操作数的有效符号存入位 1EH 中

XRL A,@R0 ; 与第一操作数的符号比较

RLC A

MOV F0,C ; 保存比较结果

LCALL MVR0 ; 将[R0]传送到第一工作区中

LCALL AS1 ; 在工作寄存器中完成代数运算

MOV0: INC R0 ; 将结果传回到[R0]中的子程序入口

INC R0

MOV A,R4 ; 传回尾数的低字节

MOV @R0,A

DEC R0

MOV A,R3 ; 传回尾数的高字节

MOV @R0,A

DEC R0

MOV A,R2 ; 取结果的阶码

MOV C,1FH ; 取结果的数符

MOV ACC.7,C ; 拼入阶码中

MOV @R0,A

CLR ACC.7 ; 不考虑数符

CLR OV ; 清除溢出标志

CJNE A,#3FH,MV01; 阶码是否上溢?

SETB OV ; 设立溢出标志

MV01: MOV A,@R0 ; 取出带数符的阶码

RET

MVR0: MOV A,@R0 ; 将[R0]传送到第一工作区中的子程序

MOV C,ACC.7 ; 将数符保存在位 1FH 中

MOV 1FH,C

MOV C,ACC.6 ; 将阶码扩充为 8bit 补码

MOV ACC.7,C

MOV R2,A ; 存放在 R2 中

INC R0

MOV A,@R0 ; 将尾数高字节存放在 R3 中

MOV R3,A

INC R0

MOV A,@R0 ; 将尾数低字节存放在 R4 中

MOV R4,A

```

DEC R0 ; 恢复数据指针
DEC R0
RET
MVR1: MOV A,@R1 ; 将[R1]传送到第二工作区中的子程序
MOV C,ACC.7 ; 将数符保存在位 1EH 中
MOV 1EH,C
MOV C,ACC.6 ; 将阶码扩充为 8bit 补码
MOV ACC.7,C
MOV R5,A ; 存放在 R5 中
INC R1
MOV A,@R1 ; 将尾数高字节存放在 R6 中
MOV R6,A
INC R1
MOV A,@R1 ; 将尾数低字节存放在 R7 中
MOV R7,A
DEC R1 ; 恢复数据指针
DEC R1
RET
AS1: MOV A,R6 ; 读取第二操作数尾数高字节
ORL A,R7
JZ AS2 ; 第二操作数为零, 不必运算
MOV A,R3 ; 读取第一操作数尾数高字节
ORL A,R4
JNZ EQ1
MOV A,R6 ; 第一操作数为零, 结果以第二操作数为准
MOV R3,A
MOV A,R7
MOV R4,A
MOV A,R5
MOV R2,A
MOV C,1EH
MOV 1FH,C
AS2: RET
EQ1: MOV A,R2 ; 对阶, 比较两个操作数的阶码
XRL A,R5
JZ AS4 ; 阶码相同, 对阶结束
JB ACC.7,EQ3 ; 阶符互异
MOV A,R2 ; 阶符相同, 比较大小
CLR C
SUBB A,R5
JC EQ4
EQ2: CLR C ; 第二操作数右规一次
MOV A,R6 ; 尾数缩小一半
RRC A
MOV R6,A

```



```

MOV A,R7
RRC A
MOV R7,A
INC R5 ; 阶码加一
ORL A,R6 ; 尾数为零否?
JNZ EQ1 ; 尾数不为零, 继续对阶
MOV A,R2 ; 尾数为零, 提前结束对阶
MOV R5,A
SJMP AS4
EQ3: MOV A,R2 ; 判断第一操作数阶符
JNB ACC.7,EQ2; 如为正, 右规第二操作数
EQ4: CLR C
LCALL RR1 ; 第一操作数右规一次
ORL A,R3 ; 尾数为零否?
JNZ EQ1 ; 不为零, 继续对阶
MOV A,R5 ; 尾数为零, 提前结束对阶
MOV R2,A
AS4: JB F0,AS5 ; 尾数加减判断
MOV A,R4 ; 尾数相加
ADD A,R7
MOV R4,A
MOV A,R3
ADDC A,R6
MOV R3,A
JNC AS2
LJMP RR1 ; 有进位, 右规一次
AS5: CLR C ; 比较绝对值大小
MOV A,R4
SUBB A,R7
MOV B,A
MOV A,R3
SUBB A,R6
JC AS6
MOV R4,B ; 第一尾数减第二尾数
MOV R3,A
LJMP RLN ; 结果规格化
AS6: CPL 1FH ; 结果的符号与第一操作数相反
CLR C ; 结果的绝对值为第二尾数减第一尾数
MOV A,R7
SUBB A,R4
MOV R4,A
MOV A,R6
SUBB A,R3
MOV R3,A
RLN: MOV A,R3 ; 浮点数规格化

```

```

ORL A,R4 ; 尾数为零否?
JNZ RLN1
MOV R2,#0C1H; 阶码取最小值
RET
RLN1: MOV A,R3
JB ACC.7,RLN2; 尾数最高位为一否?
CLR C ; 不为1,左规一次
LCALL RL1
SJMP RLN ; 继续判断
RLN2: CLR OV ; 规格化结束
RET
RL1: MOV A,R4 ; 第一操作数左规一次
RLC A ; 尾数扩大一倍
MOV R4,A
MOV A,R3
RLC A
MOV R3,A
DEC R2 ; 阶码减一
CJNE R2,#0C0H,RL1E; 阶码下溢否?
CLR A
MOV R3,A ; 阶码下溢,操作数以零计
MOV R4,A
MOV R2,#0C1H
RL1E: CLR OV
RET
RR1: MOV A,R3 ; 第一操作数右规一次
RRC A ; 尾数缩小一半
MOV R3,A
MOV A,R4
RRC A
MOV R4,A
INC R2 ; 阶码加一
CLR OV ; 清溢出标志
CJNE R2,#40H,RR1E; 阶码上溢否?
MOV R2,#3FH ; 阶码溢出
SETB OV
RR1E: RET

```

4. 浮点数乘法

标 号: FMUL

入口条件: 被乘数在[R0]中, 乘数在[R1]中。

出口信息: OV=0时, 积仍在[R0]中, OV=1时, 溢出。

影响资源: PSW、A、B、R2~R7、位1EH、1FH堆栈需求: 6字节

FMUL: LCALL MVR0 ; 将[R0]传送到第一工作区中

```

MOV A,@R0
XRL A,@R1 ; 比较两个操作数的符号
RLC A
MOV 1FH,C ; 保存积的符号
LCALL MUL0 ; 计算积的绝对值
LJMP MOV0 ; 将结果传回到[R0]中
MUL0: LCALL MVR1 ; 将[R1]传送到第二工作区中
MUL1: MOV A,R3 ; 第一尾数为零否?
ORL A,R4
JZ MUL6
MOV A,R6 ; 第二尾数为零否?
ORL A,R7
JZ MUL5
MOV A,R7 ; 计算  $R3R4 \times R6R7 \rightarrow R3R4$ 
MOV B,R4
MUL AB
MOV A,B
XCH A,R7
MOV B,R3
MUL AB
ADD A,R7
MOV R7,A
CLR A
ADDC A,B
XCH A,R4
MOV B,R6
MUL AB
ADD A,R7
MOV R7,A
MOV A,B
ADDC A,R4
MOV R4,A
CLR A
RLC A
XCH A,R3
MOV B,R6
MUL AB
ADD A,R4
MOV R4,A
MOV A,B
ADDC A,R3
MOV R3,A
JB ACC.7,MUL2; 积为规格化数否?
MOV A,R7 ; 左规一次
RLC A

```

```

MOV R7,A
LCALL RL1
MUL2: MOV A,R7
JNB ACC.7,MUL3
INC R4
MOV A,R4
JNZ MUL3
INC R3
MOV A,R3
JNZ MUL3
MOV R3,#80H
INC R2
MUL3: MOV A,R2 ; 求积的阶码
ADD A,R5
MD: MOV R2,A ; 阶码溢出判断
JB ACC.7,MUL4
JNB ACC.6,MUL6
MOV R2,#3FH ; 阶码上溢, 设立标志
SETB OV
RET
MUL4: JB ACC.6,MUL6
MUL5: CLR A ; 结果清零 (因子为零或阶码下溢)
MOV R3,A
MOV R4,A
MOV R2,#41H
MUL6: CLR OV
RET

```

5. 浮点数除法

标 号: FDIV

入口条件: 被除数在[R0]中, 除数在[R1]中。

出口信息: OV=0 时, 商仍在[R0]中, OV=1 时, 溢出。

影响资源: PSW、A、B、R2~R7、位 1EH、1FH 堆栈需求: 5 字节

```

FDIV: INC R0
MOV A,@R0
INC R0
ORL A,@R0
DEC R0
DEC R0
JNZ DIV1
MOV @R0,#41H; 被除数为零, 不必运算
CLR OV
RET
DIV1: INC R1

```

```

MOV A,@R1
INC R1
ORL A,@R1
DEC R1
DEC R1
JNZ DIV2
SETB OV ; 除数为零, 溢出
RET
DIV2: LCALL MVR0 ; 将[R0]传送到第一工作区中
MOV A,@R0
XRL A,@R1 ; 比较两个操作数的符号
RLC A
MOV 1FH,C ; 保存结果的符号
LCALL MVR1 ; 将[R1]传送到第二工作区中
LCALL DIV3 ; 调用工作区浮点除法
LJMP MOV0 ; 回传结果
DIV3: CLR C ; 比较尾数的大小
MOV A,R4
SUBB A,R7
MOV A,R3
SUBB A,R6
JC DIV4
LCALL RR1 ; 被除数右规一次
SJMP DIV3
DIV4: CLR A ; 借用 R0R1R2 作工作寄存器
XCH A,R0 ; 清零并保护之
PUSH ACC
CLR A
XCH A,R1
PUSH ACC
MOV A,R2
PUSH ACC
MOV B,#10H ; 除法运算, R3R4 / R6R7 → R0R1
DIV5: CLR C
MOV A,R1
RLC A
MOV R1,A
MOV A,R0
RLC A
MOV R0,A
MOV A,R4
RLC A
MOV R4,A
XCH A,R3
RLC A

```

```

XCH A,R3
MOV F0,C
CLR C
SUBB A,R7
MOV R2,A
MOV A,R3
SUBB A,R6
ANL C,/F0
JC DIV6
MOV R3,A
MOV A,R2
MOV R4,A
INC R1
DIV6: DJNZ B,DIV5
MOV A,R6 ; 四舍五入
CLR C
RRC A
SUBB A,R3
CLR A
ADDC A,R1 ; 将结果存回 R3R4
MOV R4,A
CLR A
ADDC A,R0
MOV R3,A
POP ACC ; 恢复 R0R1R2
MOV R2,A
POP ACC
MOV R1,A
POP ACC
MOV R0,A
MOV A,R2 ; 计算商的阶码
CLR C
SUBB A,R5
LCALL MD ; 阶码检验
LJMP RLN ; 规格化

```

6. 浮点数清零

标 号: FCLR

入口条件: 操作数在[R0]中。

出口信息: 操作数被清零。

影响资源: A 堆栈需求: 2 字节

```

FCLR: INC R0
INC R0
CLR A

```

```

MOV @R0,A
DEC R0
MOV @R0,A
DEC R0
MOV @R0,#41H
RET

```

7. 浮点数判零

标 号: FZER

入口条件: 操作数在[R0]中。

出口信息: 若累加器 A 为零, 则操作数[R0]为零, 否则不为零。

影响资源: A 堆栈需求: 2 字节

```

FZER: INC R0
INC R0
MOV A,@R0
DEC R0
ORL A,@R0
DEC R0
JNZ ZERO
MOV @R0,#41H
ZERO: RET

```

8. 浮点数传送

标 号: FMOV

入口条件: 源操作数在[R1]中, 目标地址为[R0]。

出口信息: [R0]=[R1], [R1]不变。

影响资源: A 堆栈需求: 2 字节

```

FMov: INC R0
INC R0
INC R1
INC R1
MOV A,@R1
MOV @R0,A
DEC R0
DEC R1
MOV A,@R1
MOV @R0,A
DEC R0
DEC R1
MOV A,@R1
MOV @R0,A
RET

```

9. 浮点数压栈

标 号: FPUS

入口条件: 操作数在[R0]中。

出口信息: 操作数压入栈顶。

影响资源: A、R2、R3 堆栈需求: 5 字节

FPUS: POP ACC ; 将返回地址保存在 R2R3 中

MOV R2,A

POP ACC

MOV R3,A

MOV A,@R0 ; 将操作数压入堆栈

PUSH ACC

INC R0

MOV A,@R0

PUSH ACC

INC R0

MOV A,@R0

PUSH ACC

DEC R0

DEC R0

MOV A,R3 ; 将返回地址压入堆栈

PUSH ACC

MOV A,R2

PUSH ACC

RET ; 返回主程序

10. 浮点数出栈

标 号: FPOP

入口条件: 操作数处于栈顶。

出口信息: 操作数弹至[R0]中。

影响资源: A、R2、R3 堆栈需求: 2 字节

FPOP: POP ACC ; 将返回地址保存在 R2R3 中

MOV R2,A

POP ACC

MOV R3,A

INC R0

INC R0

POP ACC ; 将操作数弹出堆栈, 传送到[R0]中

MOV @R0,A

DEC R0

POP ACC

MOV @R0,A

DEC R0

POP ACC

MOV @R0,A
 MOV A,R3 ; 将返回地址压入堆栈
 PUSH ACC
 MOV A,R2
 PUSH ACC
 RET ; 返回主程序

11. 浮点数代数值比较（不影响待比较操作数）

标 号: FCMP

入口条件: 待比较操作数分别在[R0]和[R1]中。

出口信息: 若 CY=1, 则[R0] < [R1], 若 CY=0 且 A=0 则 [R0] = [R1], 否则[R0] > [R1]。

影响资源: A、B、PSW 堆栈需求: 2 字节

FCMP: MOV A,@R0 ; 数符比较
 XRL A,@R1
 JNB ACC.7,CMP2
 MOV A,@R0 ; 两数异号, 以[R0]数符为准
 RLC A
 MOV A,#0FFH
 RET
 CMP2: MOV A,@R1 ; 两数同号, 准备比较阶码
 MOV C,ACC.6
 MOV ACC.7,C
 MOV B,A
 MOV A,@R0
 MOV C,ACC.7
 MOV F0,C ; 保存[R0]的数符
 MOV C,ACC.6
 MOV ACC.7,C
 CLR C ; 比较阶码
 SUBB A,B
 JZ CMP6
 RLC A ; 取阶码之差的符号
 JNB F0,CMP5
 CPL C ; [R0]为负时, 结果取反
 CMP5: MOV A,#0FFH ; 两数不相等
 RET
 CMP6: INC R0 ; 阶码相同时, 准备比较尾数
 INC R0
 INC R1
 INC R1
 CLR C
 MOV A,@R0
 SUBB A,@R1
 MOV B,A ; 保存部分差

```

DEC R0
DEC R1
MOV A,@R0
SUBB A,@R1
DEC R0
DEC R1
ORL A,B ; 生成是否相等信息
JZ CMP7
JNB F0,CMP7
CPL C ; [R0]为负时, 结果取反
CMP7: RET

```

12. 浮点绝对值函数

标 号: FABS

入口条件: 操作数在[R0]中。

出口信息: 结果仍在[R0]中。

影响资源: A 堆栈需求: 2 字节

```

FABS: MOV A,@R0 ; 读取操作数的阶码
CLR ACC.7 ; 清除数符
MOV @R0,A ; 回传阶码
RET

```

13. 浮点符号函数

标 号: FSGN

入口条件: 操作数在[R0]中。

出口信息: 累加器 A=1 时为正数, A=0FFH 时为负数, A=0 时为零。

影响资源: PSW、A 堆栈需求: 2 字节

```

FSGN: INC R0 ; 读尾数
MOV A,@R0
INC R0
ORL A,@R0
DEC R0
DEC R0
JNZ SGN
RET ; 尾数为零, 结束
SGN: MOV A,@R0 ; 读取操作数的阶码
RLC A ; 取数符
MOV A,#1 ; 按正数初始化
JNC SGN1 ; 是正数, 结束
MOV A,#0FFH ; 是负数, 改变标志
SGN1: RET

```

14. 浮点取整函数

标 号: FINT

入口条件：操作数在[R0]中。

出口信息：结果仍在[R0]中。

影响资源：PSW、A、R2、R3、R4、位 1FH 堆栈需求：6 字节

FINT: LCALL MVR0 ; 将[R0]传送到第一工作区中

LCALL INT ; 在工作寄存器中完成取整运算

LJMP MOV0 ; 将结果传回到[R0]中

INT: MOV A,R3

ORL A,R4

JNZ INTA

CLR 1FH ; 尾数为零, 阶码也清零, 结束取整

MOV R2,#41H

RET

INTA: MOV A,R2

JZ INTB ; 阶码为零否?

JB ACC.7,INTB; 阶符为负否?

CLR C

SUBB A,#10H ; 阶码小于 16 否?

JC INTD

RET ; 阶码大于 16, 已经是整数

INTB: CLR A ; 绝对值小于一, 取整后正数为零, 负数为负一

MOV R4,A

MOV C,1FH

RRC A

MOV R3,A

RL A

MOV R2,A

JNZ INTC

MOV R2,#41H

INTC: RET

INTD: CLR F0 ; 舍尾标志初始化

INTE: CLR C

LCALL RRR1 ; 右规一次

ORL C,F0 ; 记忆舍尾情况

MOV F0,C

CJNE R2,#10H,INTE: 阶码达到 16 (尾数完全为整数) 否?

JNB F0,INTF ; 舍去部分为零否?

JNB 1FH,INTF; 操作数为正数否?

INC R4 ; 对于带小数的负数, 向下取整

MOV A,R4

JNZ INTF

INC R3

INTF: LJMP RLN ; 将结果规格化

15. 浮点倒数函数

标 号: FRCP

入口条件: 操作数在[R0]中。

出口信息: OV=0 时, 结果仍在[R0]中, OV=1 时, 溢出。

影响资源: PSW、A、B、R2~R7、位 1EH、1FH 堆栈需求: 5 字节

```
FRCP: MOV A,@R0
MOV C,ACC.7
MOV 1FH,C ; 保存数符
MOV C,ACC.6 ; 绝对值传送到第二工作区
MOV ACC.7,C
MOV R5,A
INC R0
MOV A,@R0
MOV R6,A
INC R0
MOV A,@R0
MOV R7,A
DEC R0
DEC R0
ORL A,R6
JNZ RCP
SETB OV ; 零不能求倒数, 设立溢出标志
RET
RCP: MOV A,R6
JB ACC.7,RCP2; 操作数格式化否?
CLR C ; 格式化之
MOV A,R7
RLC A
MOV R7,A
MOV A,R6
RLC A
MOV R6,A
DEC R5
SJMP RCP
RCP2: MOV R2,#1 ; 将数值 1.00 传送到第一工作区
MOV R3,#80H
MOV R4,#0
LCALL DIV3 ; 调用工作区浮点除法, 求得倒数
LJMP MOV0 ; 回传结果
```

16. 浮点数平方

标 号: FSQU

入口条件: 操作数在[R0]中。

出口信息：OV=0 时，平方值仍然在[R0]中，OV=1 时溢出。

影响资源：PSW、A、B、R2~R7、位 1EH、1FH 堆栈需求：9 字节

FSQU: MOV A,R0 ; 将操作数

XCH A,R1 ; 同时作为乘数

PUSH ACC ; 保存 R1 指针

LCALL FMUL ; 进行乘法运算

POP ACC

MOV R1,A ; 恢复 R1 指针

RET

17. 浮点数开平方（快速逼近算法）

标 号：FSQR

入口条件：操作数在[R0]中。

出口信息：OV=0 时，平方根仍在[R0]中，OV=1 时，负数开平方出错。

影响资源：PSW、A、B、R2~R7 堆栈需求：2 字节

FSQR: MOV A,@R0

JNB ACC.7,SQR

SETB OV ; 负数开平方，出错

RET

SQR: INC R0

INC R0

MOV A,@R0

DEC R0

ORL A,@R0

DEC R0

JNZ SQ

MOV @R0,#41H; 尾数为零，不必运算

CLR OV

RET

SQ: MOV A,@R0

MOV C,ACC.6 ; 将阶码扩展成 8bit 补码

MOV ACC.7,C

INC A ; 加一

CLR C

RRC A ; 除二

MOV @R0,A ; 得到平方根的阶码，回存之

INC R0 ; 指向被开方数尾数的高字节

JC SQR0 ; 原被开方数的阶码是奇数吗？

MOV A,@R0 ; 是奇数，尾数右规一次

RRC A

MOV @R0,A

INC R0

MOV A,@R0

RRC A

```

MOV @R0,A
DEC R0
SQR0: MOV A,@R0
JZ SQR9 : 尾数为零, 不必运算
MOV R2,A : 将尾数传送到 R2R3 中
INC R0
MOV A,@R0
MOV R3,A
MOV A,R2 : 快速开方, 参阅定点子程序说明
ADD A,#57H
JC SQR2
ADD A,#45H
JC SQR1
ADD A,#24H
MOV B,#0E3H
MOV R4,#80H
SJMP SQR3
SQR1: MOV B,#0B2H
MOV R4,#0A0H
SJMP SQR3
SQR2: MOV B,#8DH
MOV R4,#0D0H
SQR3: MUL AB
MOV A,B
ADD A,R4
MOV R4,A
MOV B,A
MUL AB
XCH A,R3
CLR C
SUBB A,R3
MOV R3,A
MOV A,B
XCH A,R2
SUBB A,R2
MOV R2,A
SQR4: SETB C
MOV A,R4
RLC A
MOV R6,A
CLR A
RLC A
MOV R5,A
MOV A,R3
SUBB A,R6

```

```

MOV B,A
MOV A,R2
SUBB A,R5
JC SQR5
INC R4
MOV R2,A
MOV R3,B
SJMP SQR4
SQR5: MOV A,R4
XCH A,R2
RRC A
MOV F0,C
MOV A,R3
MOV R5,A
MOV R4,#8
SQR6: CLR C
MOV A,R3
RLC A
MOV R3,A
CLR C
MOV A,R5
SUBB A,R2
JB F0,SQR7
JC SQR8
SQR7: MOV R5,A
INC R3
SQR8: CLR C
MOV A,R5
RLC A
MOV R5,A
MOV F0,C
DJNZ R4,SQR6
MOV A,R3 ; 将平方根的尾数回传到[R0]中
MOV @R0,A
DEC R0
MOV A,R2
MOV @R0,A
SQR9: DEC R0 ; 数据指针回归原位
CLR OV ; 开方结果有效
RET

```

18. 浮点数多项式计算

标 号: FPLN

入口条件: 自变量在[R0]中, 多项式系数在调用指令之后, 以 40H 结束。

出口信息: OV=0 时, 结果仍在[R0]中, OV=1 时, 溢出。

影响资源: DPTR、PSW、A、B、R2~R7、位 1EH、1FH 堆栈需求: 4 字节

FPLN: POP DPH ; 取出多项式系数存放地址

POP DPL

XCH A,R0 ; R0、R1 交换角色, 自变量在[R1]中

XCH A,R1

XCH A,R0

CLR A ; 清第一工作区

MOV R2,A

MOV R3,A

MOV R4,A

CLR 1FH

PLN1: CLR A ; 读取一个系数, 并装入第二工作区

MOVC A,@A+DPTR

MOV C,ACC.7

MOV 1EH,C

MOV C,ACC.6

MOV ACC.7,C

MOV R5,A

INC DPTR

CLR A

MOVC A,@A+DPTR

MOV R6,A

INC DPTR

CLR A

MOVC A,@A+DPTR

MOV R7,A

INC DPTR ; 指向下一个系数

MOV C,1EH ; 比较两个数符

RRC A

XRL A,23H

RLC A

MOV F0,C ; 保存比较结果

LCALL AS1 ; 进行代数加法运算

CLR A ; 读取下一个系数的第一个字节

MOVC A,@A+DPTR

CJNE A,#40H,PLN2; 是结束标志吗?

XCH A,R0 ; 运算结束, 恢复 R0、R1 原来的角色

XCH A,R1

XCH A,R0

LCALL MOV0 ; 将结果回传到[R0]中

CLR A

INC DPTR

JMP @A+DPTR ; 返回主程序

PLN2: MOV A,@R1 ; 比较自变量和中间结果的符号
 XRL A,23H
 RLC A
 MOV 1FH,C ; 保存比较结果
 LCALL MUL0 ; 进行乘法运算
 SJMP PLN1 ; 继续下一项运算

19. 以 10 为底的浮点对数函数

标 号: FLOG

入口条件: 操作数在[R0]中。

出口信息: OV=0 时, 结果仍在[R0]中, OV=1 时, 负数或零求对数出错。

影响资源: DPTR、PSW、A、B、R2~R7、位 1EH、1FH 堆栈需求: 9 字节

FLOG: LCALL FLN ; 先以 e 为底求对数
 JNB OV,LOG
 RET ; 如溢出则停止计算
 LOG: MOV R5,#0FFH; 系数 0.43430(1/Ln10)
 MOV R6,#0DEH
 MOV R7,#5CH
 LCALL MUL1 ; 通过相乘来换底
 LJMP MOV0 ; 传回结果

20. 以 e 为底的浮点对数函数

标 号: FLN

入口条件: 操作数在[R0]中。

出口信息: OV=0 时, 结果仍在[R0]中, OV=1 时, 负数或零求对数出错。

影响资源: DPTR、PSW、A、B、R2~R7、位 1EH、1FH 堆栈需求: 7 字节

FLN: LCALL MVR0 ; 将[R0]传送到第一工作区
 JB 1FH,LNOV; 负数或零求对数, 出错
 MOV A,R3
 ORL A,R4
 JNZ LN0
 LNOV: SETB OV
 RET
 LN0: CLR C
 LCALL RL1 ; 左规一次
 CLR A
 XCH A,R2 ; 保存原阶码, 清零工作区的阶码
 PUSH ACC
 LCALL RLN ; 规格化
 LCALL MOV0 ; 回传
 LCALL FPLN ; 用多项式计算尾数的对数
 DB 7BH,0F4H,30H; 0.029808
 DB 0FEH,85H,13H; -0.12996
 DB 7FH,91H,51H; 0.28382

```

DB 0FFH,0FAH,0BAH; -0.4897
DB 0,0FFH,0CAH; 0.99918
DB 70H,0C0H,0;  $1.1442 \times 10^{-5}$ 
DB 40H ; 结束
POP ACC ; 取出原阶码
JNZ LN1
RET ; 如为零, 则结束
LN1: CLR 1EH ; 清第二区数符
MOV C,ACC.7
MOV F0,C ; 保存阶符
JNC LN2
CPL A ; 当阶码为负时, 求其绝对值
INC A
LN2: MOV R2,A ; 阶码的绝对值乘以 0.69315
MOV B,#72H
MUL AB
XCH A,R2
MOV R7,B
MOV B,#0B1H
MUL AB
ADD A,R7
MOV R7,A ; 乘积的尾数在 R6R7R2 中
CLR A
ADDC A,B
MOV R6,A
MOV R5,#8 ; 乘积的阶码初始化 (整数部分为一字节)
LN3: JB ACC.7,LN4; 乘积格式化
MOV A,R2
RLC A
MOV R2,A
MOV A,R7
RLC A
MOV R7,A
MOV A,R6
RLC A
MOV R6,A
DEC R5
SJMP LN3
LN4: MOV C,F0 ; 取出阶符, 作为乘积的数符
MOV ACC.7,C
LJMP ASN ; 与尾数的对数合并, 得原操作数的对数

```

21. 以 10 为底的浮点指数函数

标 号: FE10

入口条件：操作数在[R0]中。

出口信息：OV=0时，结果仍在[R0]中，OV=1时，溢出。

影响资源：DPTR、PSW、A、B、R2~R7、位 1EH、1FH 堆栈需求：6 字节

FE10: MOV R5,#2 ; 加权系数为 3.3219(Log210)

MOV R6,#0D4H

MOV R7,#9AH

SJMP EXP ; 先进行加权运算，后以 2 为底统一求幂

22. 以 e 为底的浮点指数函数

标 号：FEXP

入口条件：操作数在[R0]中。

出口信息：OV=0时，结果仍在[R0]中，OV=1时，溢出。

影响资源：DPTR、PSW、A、B、R2~R7、位 1EH、1FH 堆栈需求：6 字节

FEXP: MOV R5,#1 ; 加权系数为 1.44272(Lng2e)

MOV R6,#0B8H

MOV R7,#0ABH

EXP: CLR 1EH ; 加权系数为正数

LCALL MVR0 ; 将[R0]传送到第一工作区

LCALL MUL1 ; 进行加权运算

SJMPE20 ; 以 2 为底统一求幂

23. 以 2 为底的浮点指数函数

标 号：FE2

入口条件：操作数在[R0]中。

出口信息：OV=0时，结果仍在[R0]中，OV=1时，溢出。

影响资源：DPTR、PSW、A、B、R2~R7、位 1EH、1FH 堆栈需求：6 字节

FE2: LCALL MVR0 ; 将[R0]传送到第一工作区

E20: MOV A,R3

ORL A,R4

JZ EXP1 ; 尾数为零

MOV A,R2

JB ACC.7,EXP2; 阶符为负?

SETB C

SUBB A,#6 ; 阶码大于 6 否?

JC EXP2

JB 1FH,EXP0; 数符为负否?

MOV @R0,#3FH; 正指数过大，幂溢出

INC R0

MOV @R0,#0FFH

INC R0

MOV @R0,#0FFH

DEC R0

DEC R0

```

SETB OV
RET
EXP0: MOV @R0,#41H: 负指数过大, 幂下溢, 清零处理
CLR A
INC R0
MOV @R0,A
INC R0
MOV @R0,A
DEC R0
DEC R0
CLR OV
RET
EXP1: MOV @R0,#1 : 指数为零, 幂为 1.00
INC R0
MOV @R0,#80H
INC R0
MOV @R0,#0
DEC R0
DEC R0
CLR OV
RET
EXP2: MOV A,R2 : 将指数复制到第二工作区
MOV R5,A
MOV A,R3
MOV R6,A
MOV A,R4
MOV R7,A
MOV C,1FH
MOV 1EH,C
LCALL INT ; 对第一区取整
MOV A,R3
JZ EXP4
EXP3: CLR C : 使尾数高字节 R3 对应一个字节整数
RRC A
INC R2
CJNE R2,#8,EXP3
EXP4: MOV R3,A
JNB 1FH,EXP5
CPL A ; 并用补码表示
INC A
EXP5: PUSH ACC ; 暂时保存之
LCALL RLN ; 重新规格化
CPL 1FH
SETB F0
LCALL AS1 : 求指数的小数部分

```

LCALL MOV0 ; 回传指数的小数部分
 LCALL FPLN ; 通过多项式计算指数的小数部分的幂
 DB 77H,0B1H,0C9H; 1.3564×10^{-3}
 DB 7AH,0A1H,68H; 9.8514×10^{-3}
 DB 7CH,0E3H,4FH; 0.055495
 DB 7EH,0F5H,0E7H; 0.24014
 DB 0,0B1H,72H; 0.69315
 DB 1,80H,0 ; 1.00000
 DB 40H ; 结束
 POP ACC ; 取出指数的整数部分
 ADD A,R2 ; 按补码加到幂的阶码上
 MOV R2,A
 CLR 1FH ; 幂的符号为正
 LJMP MOV0 ; 将幂传回[R0]中

24. 双字节十六进制定点数转换成格式化浮点数

标 号: DTOF

入口条件: 双字节定点数的绝对值在[R0]中, 数符在位 1FH 中, 整数部分的位数在 A 中。

出口信息: 转换成格式化浮点数在[R0]中(三字节)。

影响资源: PSW、A、R2、R3、R4、位 1FH 堆栈需求: 6 字节

DTOF: MOV R2,A ; 按整数的位数初始化阶码
 MOV A,@R0 ; 将定点数作尾数
 MOV R3,A
 INC R0
 MOV A,@R0
 MOV R4,A
 DEC R0
 LCALL RLN ; 进行规格化
 LJMP MOV0 ; 传送结果到[R0]中

25. 格式化浮点数转换成双字节定点数

标 号: FTOD

入口条件: 格式化浮点操作数在[R0]中。

出口信息: OV=1 时溢出, OV=0 时转换成功: 定点数的绝对值在[R0]中(双字节), 数符在位 1FH 中, F0=1 时为整数, CY=1 时为一字节整数一字节小数, 否则为纯小数。

影响资源: PSW、A、B、R2、R3、R4、位 1FH 堆栈需求: 6 字节

FTOD: LCALL MVR0 ; 将[R0]传送到第一工作区
 MOV A,R2
 JZ FTD4 ; 阶码为零, 纯小数
 JB ACC.7,FTD4; 阶码为负, 纯小数
 SETB C
 SUBB A,#10H
 JC FTD1

```

SETB OV ; 阶码大于 16, 溢出
RET
FTD1: SETB C
MOV A,R2
SUBB A,#8 ; 阶码大于 8 否?
JC FTD3
FTD2: MOV B,#10H ; 阶码大于 8, 按双字节整数转换
LCALL FTD8
SETB F0 ; 设立双字节整数标志
CLR C
CLR OV
RET
FTD3: MOV B,#8 ; 按一字节整数一字节小数转换
LCALL FTD8
SETB C ; 设立一字节整数一字节小数标志
CLR F0
CLR OV
RET
FTD4: MOV B,#0 ; 按纯小数转换
LCALL FTD8
CLR OV ; 设立纯小数标志
CLR F0
CLR C
RET
FTD8: MOV A,R2 ; 按规定的整数位数进行右规
CJNE A,B,FTD9
MOV A,R3 ; 将双字节结果传送到[R0]中
MOV @R0,A
INC R0
MOV A,R4
MOV @R0,A
DEC R0
RET
FTD9: CLR C
LCALL RR1 ; 右规一次
SJMP FTD8

```

26. 浮点 BCD 码转换成格式化浮点数

标 号: BTOF

入口条件: 浮点 BCD 码操作数在[R0]中。

出口信息: 转换成的格式化浮点数仍在[R0]中。

影响资源: PSW、A、B、R2~R7、位 1DH~1FH 堆栈需求: 6 字节

BTOF: INC R0 ; 判断是否为零。

INC R0

```

MOV A,@R0
MOV R7,A
DEC R0
MOV A,@R0
MOV R6,A
DEC R0
ORL A,R7
JNZ BTF0
MOV @R0,#41H; 为零, 转换结束。
RET
BTF0: MOV A,@R0
MOV C,ACC.7
MOV 1DH,C ; 保存数符。
CLR 1FH ; 以绝对值进行转换。
MOV C,ACC.6 ; 扩充阶码为八位。
MOV ACC.7,C
MOV @R0,A
JNC BTF1
ADD A,#19 ; 是否小于 1E-19?
JC BTF2
MOV @R0,#41H; 小于 1E-19 时以 0 计。
INC R0
MOV @R0,#0
INC R0
MOV @R0,#0
DEC R0
DEC R0
RET
BTF1: SUBB A,#19
JC BTF2
MOV A,#3FH ; 大于 1E19 时封顶。
MOV C,1DH
MOV ACC.7,C
MOV @R0,A
INC R0
MOV @R0,#0FFH
INC R0
MOV @R0,#0FFH
DEC R0
DEC R0
RET
BTF2: CLR A ; 准备将 BCD 码尾数转换成十六进制浮点数。
MOV R4,A
MOV R3,A
MOV R2,#10H ; 至少两个字节。

```

```

BTF3: MOV A,R7
ADD A,R7
DA A
MOV R7,A
MOV A,R6
ADDC A,R6
DA A
MOV R6,A
MOV A,R4
RLC A
MOV R4,A
MOV A,R3
RLC A
MOV R3,A
DEC R2
JNB ACC.7,BTF3: 直到尾数规格化。
MOV A,R6 ; 四舍五入。
ADD A,#0B0H
CLR A
ADDC A,R4
MOV R4,A
CLR A
ADDC A,R3
MOV R3,A
JNC BTF4
MOV R3,#80H
INC R2
BTF4: MOV DPTR,#BTFL: 准备查表得到十进制阶码对应的浮点数。
MOV A,@R0
ADD A,#19 ; 计算表格偏移量。
MOV B,#3
MUL AB
ADD A,DPL
MOV DPL,A
JNC BTF5
INC DPH
BTF5: CLR A ; 查表。
MOVC A,@A+DPTR
MOV C,ACC.6
MOV ACC.7,C
MOV R5,A
MOV A,#1
MOVC A,@A+DPTR
MOV R6,A
MOV A,#2

```


MOVC A,@A+DPTR
 MOV R7,A
 LCALL MUL1 ; 将阶码对应的浮点数和尾数对应的浮点数相乘。
 MOV C,1DH ; 取出数符。
 MOV 1FH,C
 LJMP MOV0 ; 传送转换结果。

27. 格式化浮点数转换成浮点 BCD 码

标 号: FTOB

入口条件: 格式化浮点操作数在[R0]中。

出口信息: 转换成的浮点 BCD 码仍在[R0]中。

影响资源: PSW、A、B、R2~R7、位 1DH~1FH 堆栈需求: 6 字节

```

FTOB: INC R0
MOV A,@R0
INC R0
ORL A,@R0
DEC R0
DEC R0
JNZ FTB0
MOV @R0,#41H
RET
FTB0: MOV A,@R0
MOV C,ACC.7
MOV 1DH,C
CLR ACC.7
MOV @R0,A
LCALL MVR0
MOV DPTR,#BFL0; 绝对值大于或等于 1 时的查表起点。
MOV B,#0 ; 十的 0 次幂。
MOV A,R2
JNB ACC.7,FTB1
MOV DPTR,#BTFL; 绝对值小于 1E-6 时的查表起点。
MOV B,#0EDH ; 十的-19 次幂。
ADD A,#16
JNC FTB1
MOV DPTR,#BFLN; 绝对值大于或等于 1E-6 时的查表起点。
MOV B,#0FAH ; 十的-6 次幂。
FTB1: CLR A ; 查表, 找到一个比待转换浮点数大的整数幂。
MOVC A,@A+DPTR
MOV C,ACC.6
MOV ACC.7,C
MOV R5,A
MOV A,#1
MOVC A,@A+DPTR
  
```

```

MOV R6,A
MOV A,#2
MOVC A,@A+DPTR
MOV R7,A
MOV A,R5 ; 和待转换浮点数比较。
CLR C
SUBB A,R2
JB ACC.7,FTB2; 差为负数。
JNZ FTB3
MOV A,R6
CLR C
SUBB A,R3
JC FTB2
JNZ FTB3
MOV A,R7
CLR C
SUBB A,R4
JC FTB2
JNZ FTB3
MOV R5,B ; 正好是表格中的数。
INC R5 ; 幂加一。
MOV R6,#10H ; 尾数为 0.1000。
MOV R7,#0
SJMP FTB6 ; 传送转换结果。
FTB2: INC DPTR ; 准备表格下一项。
INC DPTR
INC DPTR
INC B ; 幂加一。
SJMP FTB1 ;
FTB3: PUSH B ; 保存幂值。
LCALL DIV3 ; 相除, 得到一个二进制浮点数的纯小数。
FTB4: MOV A,R2 ; 取阶码。
JZ FTB5 ; 为零吗?
CLR C
LCALL RR1 ; 右规。
SJMP FTB4
FTB5: POP ACC ; 取出幂值。
MOV R5,A ; 作为十进制浮点数的阶码。
LCALL HB2 ; 转换尾数的十分位和百分位。
MOV R6,A
LCALL HB2 ; 转换尾数的千分位和万分位。
MOV R7,A
MOV A,R3 ; 四舍五入。
RLC A
CLR A

```

```

ADDC A,R7
DA A
MOV R7,A
CLR A
ADDC A,R6
DA A
MOV R6,A
JNC FTB6
MOV R6,#10H
INC R5
FTB6: INC R0 ; 存放转换结果。
INC R0
MOV A,R7
MOV @R0,A
DEC R0
MOV A,R6
MOV @R0,A
DEC R0
MOV A,R5
MOV C,1DH ; 取出数符。
MOV ACC.7,C
MOV @R0,A
RET
HB2: MOV A,R4 ; 尾数扩大 100 倍。
MOV B,#100
MUL AB
MOV R4,A
MOV A,B
XCH A,R3
MOV B,#100
MUL AB
ADD A,R3
MOV R3,A
JNC HB21
INC B
HB21: MOV A,B ; 将整数部分转换成 BCD 码。
MOV B,#10
DIV AB
SWAP A
ORL A,B
RET
BTFL: DB 41H,0ECH,1EH ; 1.0000E-19
DB 45H,93H,93H ; 1.0000E-18
DB 48H,0B8H,78H ; 1.0000E-17
DB 4BH,0E6H,96H ; 1.0000E-16

```

DB 4FH,90H,1DH ; 1.0000E-15
 DB 52H,0B4H,25H ; 1.0000E-14
 DB 55H,0E1H,2EH ; 1.0000E-13
 DB 59H,8CH,0BDH ; 1.0000E-12
 DB 5CH,0AFH,0ECH ; 1.0000E-11
 DB 5FH,0DBH,0E7H ; 1.0000E-10
 DB 63H,89H,70H ; 1.0000E-9
 DB 66H,0ABH,0CCH ; 1.0000E-8
 DB 69H,0D6H,0C0H ; 1.0000E-7
 BFLN: DB 6DH,86H,38H ; 1.0000E-6
 DB 70H,0A7H,0C6H ; 1.0000E-5
 DB 73H,0D1H,0B7H ; 1.0000E-4
 DB 77H,83H,12H ; 1.0000E-3
 DB 7AH,0A3H,0D7H ; 1.0000E-2
 DB 7DH,0CCH,0CDH ; 1.0000E-1
 BFL0: DB 1,80H,00H ; 1.0000
 DB 4,0A0H,00H ; 1.0000E1
 DB 7,0C8H,00H ; 1.0000E2
 DB 0AH,0FAH,00H ; 1.0000E3
 DB 0EH,9CH,40H ; 1.0000E4
 DB 11H,0C3H,50H ; 1.0000E5
 DB 14H,0F4H,24H ; 1.0000E6
 DB 18H,98H,97H ; 1.0000E7
 DB 1BH,0BEH,0BCH ; 1.0000E8
 DB 1EH,0EEH,6BH ; 1.0000E9
 DB 22H,95H,03H ; 1.0000E10
 DB 25H,0BAH,44H ; 1.0000E11
 DB 28H,0E8H,0D5H ; 1.0000E12
 DB 2CH,91H,85H ; 1.0000E13
 DB 2FH,0B5H,0E6H ; 1.0000E14
 DB 32H,0E3H,60H ; 1.0000E15
 DB 36H,8EH,1CH ; 1.0000E16
 DB 39H,31H,0A3H ; 1.0000E17
 DB 3CH,0DEH,0BH ; 1.0000E18
 DB 40H,8AH,0C7H ; 1.0000E19

28. 浮点余弦函数

标 号: FCOS

入口条件: 操作数在[R0]中。

出口信息: 结果仍在[R0]中。

影响资源: DPTR、PSW、A、B、R2~R7、位 1DH~1FH 堆栈需求: 6 字节

FCOS: LCALL FABS ; $\cos(-X) = \cos X$

MOV R5,#1 ; 常数 1.5708 ($\pi / 2$)

MOV R6,#0C9H

```

MOV R7,#10H
CLR 1EH
LCALL MVR0
CLR F0
LCALL AS1 :  $x + (\pi/2)$ 
LCALL MOV0 : 保存结果, 接着运行下面的 FSIN 程序

```

29. 浮点正弦函数

标 号: FSIN

入口条件: 操作数在[R0]中。

出口信息: 结果仍在[R0]中。

影响资源: DPTR、PSW、A、B、R2~R7、位 1DH~1FH 堆栈需求: 6 字节

```

FSIN: MOV A,@R0
MOV C,ACC.7
MOV 1DH,C : 保存自变量的符号
CLR ACC.7 : 统一按正数计算
MOV @R0,A
LCALL MVR0 : 将[R0]传送到第一工作区
MOV R5,#0 : 系数 0.636627(2/π)
MOV R6,#0A2H
MOV R7,#0FAH
CLR 1EH
LCALL MUL1 : 相乘, 自变量按(π/2)规一化
MOV A,R2 : 将结果复制到第二区
MOV R5,A
MOV A,R3
MOV R6,A
MOV A,R4
MOV R7,A
LCALL INT : 第一区取整, 获得象限信息
MOV A,R2
JZ SIN2
SIN1: CLR C : 将浮点象限数转换成定点象限数
LCALL RR1
CJNE R2,#10H,SIN1
MOV A,R4
JNB ACC.1,SIN2
CPL 1DH : 对于第三、四象限, 结果取反
SIN2: JB ACC.0,SIN3
CPL 1FH : 对于第一、三象限, 直接求规一化的小数
SJMP SIN4
SIN3: MOV A,R4 : 对于第二、四象限, 准备求其补数
INC A
MOV R4,A

```

```

JNZ SIN4
INC R3
SIN4: LCALL RLN ; 规格化
SETB F0
LCALL AS1 ; 求自变量归一化等效值
LCALL MOV0 ; 回传
LCALL FPLN ; 用多项式计算正弦值
DB 7DH,93H,28H: 0.07185
DB 41H,0,0 ; 0
DB 80H,0A4H,64H: -0.64215
DB 41H,0,0 ; 0
DB 1.0C9H,2: 1.5704
DB 41H,0,0 : 0
DB 40H ; 结束
MOV A,@R0 ; 结果的绝对值超过 1.00 吗?
JZ SIN5
JB ACC.6,SIN5
INC R0 ; 绝对值按 1.00 封顶
MOV @R0,#80H
INC R0
MOV @R0,#0
DEC R0
DEC R0
MOV A,#1
SIN5: MOV C,1DH ; 将数符拼入结果中
MOV ACC.7,C
MOV @R0,A
RET

```

30. 浮点反正切函数

标 号: FATN

入口条件: 操作数在[R0]中。

出口信息: 结果仍在[R0]中。

影响资源: DPTR、PSW、A、B、R2~R7、位 1CH~1FH 堆栈需求: 7 字节

```

FATN: MOV A,@R0
MOV C,ACC.7
MOV 1DH,C ; 保存自变量数符
CLR ACC.7 ; 自变量取绝对值
MOV @R0,A
CLR 1CH ; 清求余运算标志
JB ACC.6,ATN1: 自变量为纯小数否?
JZ ATN1
SETB 1CH ; 置位求余运算标志
LCALL FRCP ; 通过倒数运算, 转换成纯小数

```

ATN1: LCALL FPLN ; 通过多项式运算, 计算反正切函数值
 DB 0FCH,0E4H,91H; -0.055802
 DB 7FH,8FH,37H; 0.27922
 DB 0FFH,0EDH,0E0H; -0.46460
 DB 7BH,0E8H,77H; 0.028377
 DB 0,0FFH,68H; 0.9977
 DB 72H,85H,0ECH; 3.1930×10^{-5}
 DB 40H ; 结束
 JNB 1CH,ATN2; 需要求余运算否?
 CPL 1FH ; 准备运算标志
 MOV C,1FH
 MOV F0,C ; 常数 $1.5708(\pi/2)$
 MOV R5,#1
 MOV R6,#0C9H
 MOV R7,#10H
 LCALL AS1 ; 求余运算
 LCALL MOV0 ; 回传
 ATN2: MOV A,@R0 ; 拼入结果的数符
 MOV C,1DH
 MOV ACC.7,C
 MOV @R0,A
 RET

31. 浮点弧度数转换成浮点度数

标 号: RTOD

入口条件: 浮点弧度数在[R0]中。

出口信息: 转换成的浮点度数仍在[R0]中。

影响资源: PSW、A、B、R2~R7、位 1EH、1FH 堆栈需求: 6 字节

RTOD: MOV R5,#6 ; 系数 $(180/\pi)$ 传送到第二工作区
 MOV R6,#0E5H
 MOV R7,#2FH
 SJMP DR ; 通过乘法进行转换

32. 浮点度数转换成浮点弧度数

标 号: DTOR

入口条件: 浮点度数在[R0]中。

出口信息: 转换成的浮点弧度数仍在[R0]中。

影响资源: PSW、A、B、R2~R7、位 1EH、1FH 堆栈需求: 6 字节

DTOR: MOV R5,#0FBH; 系数 $(\pi/180)$ 传送到第二工作区
 MOV R6,#8EH
 MOV R7,#0FAH
 DR: LCALL MVR0 ; 将[R0]传送到第一工作区
 CLR 1EH ; 系数为正
 LCALL MUL1 ; 通过乘法进行转换

LJMP MOV0 : 结果传送到[R0]中

END

附录 C 单片机 C51 编程几个有用的模块

单片机系统中常用到时钟中断、通讯及键盘扫描等模块。这些模块使用前后台系统模型。为达到最大的灵活性，需要在用户工程中定义 Config.h 文件，在其中定义各模块可选参数的设置，而不是直接更改源代码。这些可选内容大部分为宏定义，如果不定义宏相应的功能在编译时被屏蔽，不会增加代码长度。具体可选内容见各模块中的说明。在 Config.h 文件中还要包含一个单片机硬件的资源头文件。各模块使用了定义在 Common.h 中的一些数据类型。如：BIT(bit) BYTE(unsigned char)等，具体请参见源程序。

1. 时钟模块

在单片机软件设计中，时钟是重要资源，为了充分利用时钟资源，故设计本时钟模块。本模块使用定时器 0，在完成用户指定功能的同时，还能够自动处理一些其他模块中与时钟相关的信息。时钟模块由声明文件 Timer.h 以及实现文件 Timer.c 组成。用户应该在 Config.h 中定义宏 TIMER_RELOAD 来设定定时器 0 的重装载初值。推荐的定时器 0 的中断时间大于 1ms。

在程序的初始化阶段调用时钟模块的初始化函数 InitTimerModule()之后，就可以使用时钟模块所以支持的各种功能。具体描述如下。

延时：当用户需要进行一定时间的延时，可以通过调用 Delay()来进行，参数为时钟中断的次数。如时钟中断周期为 1ms，想进行 100ms 的延时，则可以调用 Delay(100)。注意：如果延时的绝对时间小于时钟中断的周期，则不能够用本方法做到延时。

定时：当程序中需要使用定时功能时，如等待某外部事件，如果在一定时间内发生则继续执行，如果在这段时间内发生，则认为出现错误，转向错误处理机制。在此推荐一种编程模式，但用户可以用自己认为更合理的方式处理此类问题。

这里简单说明一下关于阻塞式函数及非阻塞式函数。简单说，阻塞式函数就是当检测完条件，如果不能完成则等待，如：

```
void CheckSomething()
{
    // gbitSuccessFlag is a global variable
    while(gbitSuccessFlag == FALSE)
    {
        // do nothing but waiting
    }
}
```

可以看到，当 bitSuccessFlag 没有被设置为 TRUE 时，函数保持等待状态不返回，这样就是阻塞式的函数。

另外一种情况：


```

BIT CheckSomething()
{
    if(gbitSuccessFlag == TRUE)
    {
        // ...
        return TRUE;
    }
    return FALSE;
}

```

在这里，如果所检测的事件完成，函数进行检测之后，立刻返回，通过返回值报告完成情况；如果没有完成，则等待调用者分配再次执行的机会。这样的函数就是非阻塞函数。

在应用定时功能时，首先要将检测函数定义成非阻塞函数。如上面的第二个版本的 **CheckSomething**。

然后下面模式：

```

    BIT bitDone = FALSE;
ResetClock(); // clear timer interrupt times counter
while(GetClock() < MAX_WAITINGTIME)
{
    if(CheckSomething() == TRUE)
    {
        bitDone = TRUE;
        break;
    }
}
if(bitDone == FALSE)
{
    // process time out
}

```

或者简单写成：

```

BIT bitDone = FALSE;
ResetClock();
while(GetClock() < MAX_WAITINGTIME && (bitDone = CheckSomething()));
if(bitDone == FALSE)
{
    // ...
}

```

软件看门狗：实现具有局限性的看门狗功能。在程序中合适的地方加入对软件看门狗的复位函数 **ResetWatchDog()**，在 **Config.h** 中加入宏 **TIMER_WATCHDOGTIMEOUT**。当程序运行时，如果在发生 **TIMER_WATCHDOGTIMEOUT** 次时钟中断之内没有复位软件看门狗，则系统复位。

注意：如果没有加入 **TIMER_WATCHDOGTIMEOUT** 宏，程序中的 **ResetWatchDog** 没有任何用处，不用删除。如果系统不能实现时钟中断，则软件看门狗也同时失去功能。

目前版本的时钟模块的复位功能并不是完全复位，主要表现在当复位之后，系统将不再

响应任何中断。所以软件看门狗只是一个程序的调试功能，不应该将它用于正式工作的程序，此时应该使用硬件看门狗。

用户自定义任务：如果想在时钟中断内执行一些耗时较短的任务，可以定义回调函数 `OnTimerInterrupt`。函数原形为：`void OnTimerInterrupt()`；

如果想在发生时钟中断时执行一些功能，而这些功能又耗时相对较长，不合适放在中断响应函数内部，则可以在程序中的主循环中的任意地方添加：`ImpTimerService()`，同时提供原形为 `void OnTimerEvent()` 的回调函数。具体的程序如下所示：

```
void main()
{
    Initialize();
    while(TRUE)
    {
        // ... working
        ImpTimerService();
        // ... working
    }
}

void OnTimerEvent()
{
    // do some task
}
```

对通讯模块提供支持：如通讯中的各种超时等，见通讯模块中的详细说明。

对键盘扫描模块提供支持：可以自动调用键盘扫描模块，见键盘扫描模块中的详细说明。

对程序调试提供支持：在程序开发过程中，有时为了判断程序是不是在工作，常用利用单片机系统的某一空闲引脚通过一个限流电阻接一个发光二极管，在程序中间隔固定时间交替控制发光管的明暗。实现这个功能只要在 `Config.h` 文件中定义 `TIMER_FLASHLED` 宏，如：`#define TIMER_FLASHLED P1_0`

则当时钟中断发生 256 次之后，改变发光管的状态。

2. 通讯模块

串口资源作为单片机与外界通信的常用手段，通讯模块提供了完全缓冲的串口通讯底层机制，适用于长度不大的数据包的发送及接收。如果处理关键数据，需要用户自己提供纠错协议。通讯模块由声明文件 `SComm.h` 及实现文件 `SComm.c` 组成。

初始化：调用函数 `InitSCommModule()` 来初始化通讯模块：

`void InitSCommModule(BYTE byTimerReload, BIT bitTurbo)`

参数说明：

`byTimerReload`: 定时器 1 的重装载初始值。

`bitTurbo`: 当此参数为 `TRUE` 时，串行通讯在定时器 1 的溢出速率基础上加倍。为 `FALSE` 时，串行通讯速率为定时器 1 的溢出速率。

缓冲区：模块使用了由宏 `SCOMM_SENDBUFSIZE`、`SCOMM_RECEBUFSIZE` 及

SCOMM_PKGBUFSIZE 所指定长度的三个缓冲区, 分别为发送、接收及数据包(用于处理接收到的数据)缓冲区(如果没有使用异步接收功能, 则不需要使用数据包缓冲区)。

在缺省时, 这三个宏都被定义为 10, 但用户可以自己按照系统的 RAM 资源占用情况在 Config.h 中重定义缓冲区的大小。需要注意的是, 如果缓冲的长度不够, 当发送或接收长数据包的时候可能会发生问题, 关于数据缓冲区的最小值的设置可以参考下面的说明。

注意: 需要尽快取出接收缓冲区中的数据, 否则当缓冲区满之后, 新的数据将被简单的丢掉。

字节级服务函数: 在 Config.h 文件中定义了宏 SCOMM_DriverInterface(如: #define SCOMM_DriverInterface), 则可以使用字节级服务函数, 即通讯模块的底层函数。共有两个函数可以使用:

void SendByte(BYTE byData);

发送一个字节, 如果当前缓冲区满, 则等待。参数 byData 为要发送的数据。

BYTE ReceByte();

接收一个字节, 如果当前缓冲区中没有数据, 则此函数阻塞, 直到接收到数据为止。接收到数据通过返回值返回。

可以通过调用 IsSendBufEmpty() IsSendBufFull() IsReceBufEmpty() IsReceBufFull()宏来判断缓冲区的空或满, 以防系统阻塞。

不推荐直接使用这一级的服务函数, 应该使用高层次上的服务函数或者在这一级服务函数的基础上构造自己的通讯函数。

数据包级服务函数: 在 Config.h 文件中定义宏 SCOMM_PackageInterface (如: #define SCOMM_PackageInterface)则可以使用数据包级服务函数。共有两个函数可以使用:

void SendPackage(BYTE* pbyData, BYTE byLen);

发送数据包, 参数 pbyData 为将要发送的数据包缓冲区(数组)的指针, byLen 为将要发送的数据包的长度。

当没有定义 SCOMM_DriverInterface 时, 数据被完全缓冲。即不能够发送长度超过发送缓冲区长度的数据包。当定义了 SCOMM_DriverInterface 时, 采用单字节发送, 这时不限制需要发送的数据的长度。

BYTE RecePackage(BYTE* pbyData, BYTE byLen);

接收数据包, 参数 pbyData 为存放将要接收的数据的缓冲区, byLen 为缓冲区长度。返回值为接收到的字节数, 当模块的接收缓冲区为空时, 函数非阻塞, 立即返回, 返回值为零。

同步发送接收服务函数

比如在一个串行总线多机通讯系统中, 主机需要定时循检各从机的状态, 往往是发一个包含从机地址及指令的数据包给从机, 之后等待一定的时间, 从机需要在这段时间之内给主机一个应答, 如果没有这个应答, 则认为从机工作状态出错, 转去进行相应的处理。在这个模型里, 主机不能够不进行等待而给另一台从机发送指令, 也不能够不管从机在很久没有应答的情况下继续等待。还有一种情况, 比如当使用 485 总线进行通信时, 如果是两条通讯线则系统只能工作在半双工模式下, 总线在同一时间内只能工作在发送或接收, 为了防止发送和接收相互干扰, 这时的通讯常常需要使用同步发送和接收。

当在 Config.h 文件中定义宏 SCOMM_SyncInterface 后, 则可以使用通讯模块提供同步

发送接收函数:

```
void SendPackage(BYTE* pbyData, BYTE byLen);
```

发送数据包, 参数 `pbyData` 为将要改善的数据包的缓冲区指针, `byLen` 为将要发送的数据包的长度。

这个函数可以保证等待一个完整的数据包完全发送出去之后, 它才返回, 在这段时间内, 它会阻塞运行。

`BYTE SyncRecePackage(BYTE* pbyBuf, BYTE byBufLen, WORD wTimeout, BYTE byParam);` 接收数据包。返回值为接收到的数据包长度。参数 `pbyBuf` 为将要接收数据包的缓冲区的指针, `byBufLen` 为提供的缓冲区的长度, `wTimeout` 为通信超时值, 如果在发生了由 `wTimeout` 所指定次数的时钟中断而还没有接收到或没有接收到完整的数据包时, 函数返回零, 最后一个参数 `byParam` 的含义见后面的解释。

异步发送接收服务函数

在一个简单的系统或多机通讯系统中的从机上, 一般情况下不需要复杂的停等的工作模式, 而且往往单片机需要对硬件进行控制和检测, 不允许长时间的停下来检测通讯, 但又要求当需要通讯时需要尽快的反应速度, 这时就需要使用异步发送和接收服务函数。

使用异步发送和接收服务函数需要在 `Config.h` 文件中定义 `SCOMM_AsyncInterface` 宏。

同样提供两个服务函数:

```
void SendPackage(BYTE* pbyData, BYTE byLen);
```

发送数据包, 参数 `pbyData` 为将要改善的数据包的缓冲区指针, `byLen` 为将要发送的数据包的长度。

这里的函数的接口与同步发送和接收的服务函数相同。关于这里的细节, 见后面对同步和异步服务函数的说明。

```
void AsyncRecePackage(BYTE byParam);
```

接收数据包, 参数 `byParam` 的意义见后面的描述。

使用异步通讯需要用户定义一个回调函数, 原型如下:

```
void OnRecePackage(BYTE* pbyData, BYTE byBufLen);
```

当异步接收服务函数接收到数据包之后, 调用 `OnRecePackage` 回调函数, 在 `pbyData` 指定的缓冲区中存放数据包, `byBufLen` 为数据包的长度。

在 `Config.h` 文件中定义宏 `SCOMM_TIMEOUT` 可以设定异步接收的超时值, 当开始接收数据包, 但没有收完数据而发生了 `SCOMM_TIMEOUT` 次时钟中断后, 认为接收超时, 将已接收到的数据删除。

同步和异步通讯服务函数

有些情况下, 比如一个通讯系统中, 由一台计算机通过串口控制主机, 主机通过串口连接很多从机, 主机的串口采用分时复用, 在这样的模型中, 主机和控制计算机之间的通讯可以使用, 异步通讯方式, 而主机与从机可以使用同步通讯方式。而同步和异步的发送函数接口是相同的, 在这样的情况下, 发送都是同步的。在这样的模型中, 当使用不同的接收函数之前, 需要注意清除接收缓冲区中的内容, 通讯模块提供函数: `ClearReceBuffer` 来做到这一点, 此函数原型如下:

```
void ClearReceBuffer();
```

通讯过程中, 数据包往往是有固定的格式的, 这种格式需要根据用户所使用的协议的不

同而不同。同步和异步接收服务函数支持从接收到的数据中识别出一定格式的数据包。

举例说明：目前使用的协议决定数据包的格式为固定的包头 0xff，固定的长度 4 个字节。其它的细节在这里不重要，所以忽略掉。

为了能够使用 `SyncRecePackage` 或 `AsyncRecePackage` 函数从接收到的数据中识别出如上格式的数据包，有两种方法：

第一种办法是在 `Config.h` 文件中定义宏 `SCOMM_SimplePackageFormat`，说明数据包为一种简单格式，比如上面的协议。

之后还要定义两个宏分别用来识别数据包头和数据包尾，两个宏分别是：

`IsPackageHeader(x)`和 `IsPackageTailor(x, y, z)`

接收函数(`SyncRecePackage` 和 `AsyncRecePackage`)在没有开始接收数据包(准确的说是还没有从接收到的数据包中找到包头的时候)，会对接收到的每一个字节的数据调用 `IsPackageHeader` 宏，将相应的数据作为参数，如果 `IsPackageHeader` 宏的结果为 `TRUE`，则认为找到了数据包头，否则继续对下一个字节进行判断。

上面的协议对应的 `IsPackageHeader` 宏可以写为：

```
#define IsPackageHeader(x) ((x) == 0xff)
```

当接收到包头之后，接收函数会对接下来的每一个字节数据调用 `IsPackagTailor` 宏来判断是不是已经接收完数据包，三个参数分别为：

x：当前判断的数据。

y：从包头开始到当前被判断的数据止的计数值，即当前已经接收到的字节数。

z：用户在调用 `SyncRecePackage` 或 `AsyncRecePackage` 时指定的 `byParam` 参数。

与 `IsPackageHeader` 相似，如果宏 `IsPackageTailor` 的运算结果为 `TRUE`，则认为接收到完整的数据包，则调用相应的回调函数(对于异步接收函数)或返回(对于同步接收函数)。如果运算结果为 `FALSE` 则继续判断下一个字节的数据。

上面的协议对应的 `IsPackageTailor` 宏可以写为：

```
#define IsPackageTailor(x, y, z) ((y) >= (z))
```

当然，用户也可以将 `IsPackageHeader` 和 `IsPackageTailor` 定义成为函数，通过 `BIT` 类型的返回值来向调用者提供与相应宏相同的信息。

另一种办法需要在 `Config.h` 文件中定义宏 `SCOMM_ComplexPackageFormat`。(需要注意的是，不能够同时定义 `SCOMM_SimplePackageFormat` 和 `SCOMM_ComplexPackageFormat` 宏，否则会造成严重的不可预见性错误。

这时需要提供回调函数 `QueryPackageFormat`，原形如下：

```
BYTE QueryPackageFormat(BYTE byData, BYTE byCount, BYTE byParam);
```

函数中三个参数的含义与使用简单数据包格式时判断数据包尾的宏的参数相同。

函数通过返回值来通知作为调用者的接收函数对接收到的数据如何处理，但目前这种方法仅为需要处理复杂数据包格式时的一种可选方法，但不推荐。用户如果想使用这种方法可以自己更改接收函数中相应的

```
#ifdef SCOMM_ComplexPackageFormat
```

```
#endif // SCOMM_ComplexPackageFormat
```

预编译指令之间的内容。

例如指定 `QueryPackageFormat` 的返回值的含义：

0: 继续找数据包头或继续找数据包尾。

1: 找到数据包头。

2: 找到数据包尾

3: 数据包出错, 需要抛弃。

然后更改源代码来实现上面的协议。

注意: 当用户需要使用字符串的时候, 可以利用简单的包装函数将字符串转换为字节数组。所以没有必要提供专用的字符串处理函数。

3. 键盘扫描模块

键盘扫描模块有两种工作方式, 一种为自动的由时钟模块调用, 另一种是由程序员自行调用。

(1) 由时钟模块自动调用的方式

将时钟模块实现文件(Timer.h)及键盘扫描模块的实现文件(KBScan.c)包含进工程, 在 Config.h 文件中添加 `TIMER_KBSCANDELAY` 宏。时钟模块自动对时钟中断进行计数, 当达到 `TIMER_KBSCANDELAY` 宏所定义的值后, 自动调用键盘扫描模块中的函数 `KBScanProcess()` 进行键盘扫描, 也就是说, 这个宏的值可以决定按键消抖动的时间。

用户应该提供两个回调函数 `OnKBScan()` 及 `OnKeyPressed()`。在函数 `OnKBScan` 中进行键盘扫描, 并返回扫描码。扫描码的类型缺省为 `BYTE`, 当键盘规模较大时, `BYTE` 不能够完全包含键盘信息时, 可在 Config.h 文件中重定义宏 `KBVALUE`, 如下:

```
#define KBVALUE      WORD
```

这样, 就可以使用 16 位的键盘扫描码, 如果此时还达不到要求, 可以将键盘扫描码定义成一个结构, 但这样做将会增加代码量及消耗更多的 RAM 资源, 故不推荐。

扫描模块调用 `OnKBScan` 取得扫描码, 并调用用户可以重定义的宏 `IsNoKeyPressed` 来判断是否有键按下, 缺省的 `IsNoKeyPressed` 实现如下:

```
#define IsNoKeyPressed(x)    ((x) == 0x00)
```

即认为 `OnKBScan` 返回 0 扫描码时为没有键按下, 如果扫描函数返回其它非零扫描码做为无键按下的扫描码时, 可以在 Config.h 文件中重定义 `IsNoKeyPressed` 宏的实现。

8 位键盘扫描码(缺省值)时, 相应的扫描函数为:

```
BYTE OnKBScan()
```

当扫描模块经过软件消抖动之后, 发现有键按下, 就会调用另一个回调函数 `OnKeyPressed`。函数的声明应该如下:

```
void OnKeyPressed(BYTE byKBValue, BYTE byState)
```

其中的参数 `byKBValue` 的类型为 `BYTE`, 此为缺省值, 如果使用其它类型的扫描码, 就将此参数变为相应类型。这个值由 `OnKBScan` 返回。另一个参数 `byState` 在通常情况下为零。但当用户在 Config.h 中定义宏 `KBSCAN_BRUSHCOUNT`, 同时键盘上的某键被按住不放时, 扫描模块对它自己的调用(注意这里和 `TIMER_KBSCANDELAY` 宏不同, `TIMER_KBSCANDELAY` 是时钟中断足够的次数后调用扫描模块, 而 `KBSCAN_BRUSHCOUNT` 为扫描模块自身的被调用次数)进行计数, 当达到 `KBSCAN_BRUSHCOUNT` 时, 扫描模块调用 `OnKeyPressed`, 此时第一个参数的含义不变, 而 `byState` 变成 1, 同时计数器复位, 又经过一段时间后, 用值为 3 的 `byState` 调用 `OnKeyPressed`。这样就可以很方便的实现多功能键或者检测某键的长时间被按下。

(2) 由用户自行调用

由用户自行在程序中调用扫描模块, 而不是由时钟中断自行调用。其他与方式 1 相同。

注意:

① 函数 `KBScanProcess` 为非阻塞函数, 它将在很快的时间内返回, 等待再次分配给它执行的机会。

② 函数 `KBScanProcess` 是在时钟中断外部运行的, 它的过程可以被任何中断打断, 但不影响系统运行。

③ `byState` 的最大值为 250, 之后被复位为零。

4. 应用举例

现在来举例说明上述几个模块的使用方法。

(1) 硬件环境描述

为了控制一盏灯, 需要单片机提供一个做控制功能的开关量, 这里不描述外部接口电路, 只说明当单片机的 `P10` 脚为高电平时, 灯灭, 当 `P10` 脚为低电平时, 灯亮。

可以通过计算机由串口发送命令来控制, 或通过一个按键(push button 不是自锁式的按键)来手动控制(按键接在 `P11` 脚上, 当键没有按下时, `P11` 电平为高, 键按下时, 引脚电平被接低), 当使用按键手动控制的时候, 需要给计算机发送通知。

设定串口通讯指令如下:

数据包由 `0xff` 做包头, 4 个字节长, 第二个字节为命令代码, 第三个字节为数据, 最后一个字节为校验位。

命令和数据代码有如下组合:

(计算机发给单片机)

`0x10 0x01`: 计算机控制灯亮(数据位是非零值即可)。

`0x10 0x00`: 计算机控制灯灭。

(单片机发给计算机)

`0x11 0x01`: 单片机正常执行控制指令, 返回(数据位是非零值即可)。

`0x11 0x00`: 单片机不能够正常执行控制指令, 或控制指令错(不明含义的数据包或校验错等)。

`0x12 0x01`: 手动控制灯亮(数据位是非零值即可)。

`0x12 0x00`: 手动控制灯灭。

(2) 建立工程

在硬盘上建立文件夹 `Projects`, 在 `Projects` 下建立 `Common` 文件夹及 `Example` 文件夹。将各模块的头文件及实现文件拷贝到 `Common` 文件夹下(推荐使用这样的文件组织结构, 其他工程也可以建立在 `Projects` 下, 各工程共享 `Common` 文件夹中的代码)。

启动 KeilC 的 IDE, 在 `Example` 下建立新工程, 将各模块的实现文件包含进工程。

在 `Example` 文件夹下建立 `Output` 文件夹, 更改工程设置, 将 `Output` 作为输出文件和 `List` 文件的输出文件夹(推荐使用这样的结构, 当保存工程文件时, 可以简单地删除 `Output` 文件夹中的内容而不会误删有用的工程文件)。

建立工程配置头文件 `Config.h` 及工程主文件 `Example.c`, 并将 `Exmaple.c` 文件加入工程。

(3) 输入代码

代码的具体编写过程略。下面是最后的 `Config.h` 文件及 `Example.c` 文件。

```

//
// File: Config.h
//
#ifndef _CONFIG_H_
#define _CONFIG_H_
#include <Atmel/At89x52.h>           // 使用 AT89C52 做控制
#include "../Common/Common.h"       // 使用自定义的数据类型
#define TIMER_RELOAD      922      // 11.0592MHz 晶振, 1ms 中断周期
#define TIMER_KBSCANDELAY  40      // 40ms 重检测按键状态, 即 40ms 消抖
#define SCOMM_AsyncInterface // 使用异步通讯服务
#define IsPackageHeader(x)  ((x) == 0xff) // 判断包头是不是 0xff
#define IsPackageTailor(x, y, z) ((y) <= (z)) // 判断包的长度是不是足够
#endif // _CONFIG_H_

//
// File: Example.c
//
#include <Atmel/At89x52.h>
#include "../Common/Common.h"
#include "../Common/Timer.h"
#include "../Common/Scomm.h"
#include "../Common/KBScan.h"

BIT gbitLampState = 1;           // 灯的状态, 缺省为 off

static void Initialize()
{
    InitTimerModule();           // 初始化时钟模块
    InitScommModule(0xfd, TRUE); // 初始化通讯模块, 11.0592MHz 晶振,
                                // 波特率为 19200
    EA = 1;                     // 开中断
}

void main()
{
    Initialize();               // 初始化
    while(TRUE)                // 主循环
    {
        ImpTimerService();      // 实现时钟中断服务, 如键盘扫描
        AsyncRecePackage(4);     // 接收 4 个字节长的数据包
    }
}

// 在中断外部响应时钟中断事件
void OnTimerEvent()

```



```

{
    // do nothing
}

// 控制外部灯
static void TriggerLamp(BIT bEnable)
{
    P10 = ~bEnable;          // 需要反相控制
}

// 键扫描回调函数
BYTE KBScan()
{
    BIT b;
    P11 = 1;                 // 读之前拉高引脚电平
    b = P11;                 // 读入引脚状态
    return ~b;               // 数据反相做扫描码
}

// 计算校验和
static BYTE CalcChecksum(BYTE* pbyBuf, BYTE byLen)
{
    BYTE by, bySum = 0;
    for(by = 0; by < byLen; by++)
        bySum += pbyBuf[by];
    return 0 - bySum;
}

// 接收到键盘消息回调函数
void OnKeyPressed(BYTE byValue, BYTE byState)
{
    BYTE by[4];
    if(byState == 0)
    {
        switch(byValue)
        {
            case 0x01:
                gbitLampState = ~gbitLampState; // 灯状态取反
                TriggerLamp(gbitLampState); // 执行控制
                by[0] = 0xff;                // 构造数据包
                by[1] = 0x12;
                by[2] = (BYTE)gbitLampState;
                by[3] = CalcChecksum(by, 3); // 求校验和
                SendPackage(by, 4);          // 发送数据包
                break;
        }
    }
}

```

```

        // 处理其他扫描码
        default:
            break;
    }
}

// 接收到数据包回调函数
void OnRecePackage(BYTE* pbyBuf, BYTE byBufLen)
{
    BYTE by[4];
    by[0] = 0xff;
    by[1] = 0x11;
    if(byBufLen != 4 || pbyBuf[3] != CalcChecksum(pbyBuf, 3))
    {
        by[2] = 0;
        by[3] = CalcChecksum(by, 3);
        SendPackage(by, 4);          // 处理长度或校验和不正确
    }

    switch(pbyBuf[1])
    {
    case 0x10:
        gbitLampState = (BIT)pbyBuf[2];
        TriggerLamp(gbitLampState);
        by[2] = 1;
        by[3] = CalcChecksum(by, 3);
        SendPackage(by, 4);          // 发送成功执行通知
        break;

    default:
        // 不知道的命令
        by[2] = 0;
        by[3] = CalcChecksum(by, 3);
        SendPackage(by, 4);          // 发送没有成功执行通知
        break;
    }
}

```

附录 D 头文件 W77E58.h

```

/*--BYTE Registers-----*/
sfr P0  = 0x80;
sfr P1  = 0x90;
sfr P2  = 0xA0;

```

```

sfr P3 = 0xB0;
#define p0 P0
#define p1 P1
#define p2 P2
#define p3 P3
sfr PSW = 0xD0;
sfr ACC = 0xE0;
sfr B = 0xF0;
sfr SP = 0x81;
sfr DPL = 0x82;
sfr DPH = 0x83;
sfr PCON = 0x87; //PCON.7(SMOD)波特率加倍, PCON.1(PD)掉电方式, PCON.0(IDL)冻结方式
                //PCON.6(SMOD0)帧错检测允许, PCON.3
(GF1)PCON.2(GF0)
sfr TCON = 0x88; //定时控制寄存器
sfr TMOD = 0x89; // "gate, c/t, m1, m0" x2 定时器方式 GATE=1 时只有 intx=1 时才可以开放定时器 x;
                //c/t=1 时计数方式, =0 时定时方式。m1m0=00 时 13 位计数,
                //01 时 16 位=10 时自装入 8 位,
                //11 时定时器 0 分两个, 定时器 1 停止。
sfr TL0 = 0x8A;
sfr TL1 = 0x8B;
sfr TH0 = 0x8C;
sfr TH1 = 0x8D;
sfr IE = 0xA8;
sfr IP = 0xB8; //中断优先级
sfr SCON = 0x98; //串口控制与状态
sfr SBUF = 0x99;

/*--W77E58 Extensions-----*/
sfr T2CON = 0xC8;
sfr T2MOD = 0xC9; //HC5,HC4,HC3,HC2,T2CR,-,T2OE,DCEN
                //hc5-hc2=1 时外中断 5-2 标志硬件自动清除。
                //t2cr=1 捕获完成时自动复位
                //t2oe=1 定时器 2 输出允许
                //dcen=1 减计数允许, 结合外部输入 t2ex 使用, 16 位自装入模式
sfr RCAP2L = 0xCA; //重装的预置数, 捕获的输出
sfr RCAP2H = 0xCB;
sfr TL2 = 0xCC;
sfr TH2 = 0xCD;

sfr DPL1 = 0x84;
sfr DPH1 = 0x85;
sfr DPS = 0x86;

```

```

sfr CKCON = 0x8e; //wd1,wd0,t2m,t1m,t0m,md2,md1,md0
//wd1wd0:看门狗计数
00=2^17,01=2^20,10=2^23,11=2^26
//t2m,t1m,t0m 等于 1 时时钟 4 分频
//md2,md1,md0 MOVX 执行机器周期 000=2,
001=3, 。 。 。 111=9
sfr EXIF = 0x91; //ie5,ie4,ie3,ie2,XT/RG,RGMD,RGSL,-
//ie5,ie3=1 外中断下跳变中断标志
//ie4,ie3=2 外中断上跳变中断标志
sfr P4 = 0xa5; //低四位有效
sfr SADDR = 0xa9; //从机串口 0 地址
sfr SADDR1 = 0xaa; //从机串口 1 地址
sfr SADEN = 0xb9; //串口 0 地址屏蔽, 等于 0 时所有地址都会引起中断
sfr SADEN1 = 0xba; //串口 1 地址屏蔽, 等于 0 时所有地址都会引起中断
sfr SCON1 = 0xc0;
sfr SBUF1 = 0xc1;
sfr ROMMAP = 0xc2; //ROMMAP.7 为等待信号使能, 用 movx 指令时, wait 脚为 p4.0
sfr PMR = 0xc4; //CD1,CD0,SWB,-,XTOFF,ALE-OFF,-DME0
//cd0cd1=0 时钟不变, =1-1/4, =2-1/64, =3=1/1024
//swb=1 强制 4 分频, 外中断或串口 1 中断唤醒
//aleoff=1 时 ale 信号终止, 外部内存访问时自动唤醒
//dme0=1 时内部 1kram 使能
sfr STATUS = 0xc5; //-,HIP,LIP,XTUP,SPTA1,SPRA1,SPTA0,SPRA0
//hip=1 正在处理高优先级中断
//lip=1 正在处理低优先级中断
//spta1 串口 0 正在发送数据
//spral 串口 0 正在接收数据
//spta0 串口 1 正在发送数据
//spra0 串口 1 正在接收数据
sfr TA = 0xc7; //保护位
sfr WDCON = 0xd8; //SMOD_1,POR,-,WDIF,WTRF,EWT,RWT
//smod1 加倍串口 1 的波特率,por 电源复位标志
//por 电源复位标志
//wdif 看门狗定时中断标志, 软清
//wtrf 看门狗定时器复位标志, 看门狗引起复位后, 该位置 1
//ewt 允许看门狗定时器自动复位
//rwt 复位看门狗定时器, 如果看门狗溢出还没有被复位计数器, 将会
引起中断, 再过 512 周期将复位
sfr EIE = 0xe8; //-, -, -, EWDI,EX5,EX4,EX3,EX2
//ewdi=1, 允许看门狗中断?
//ex5-ex2; 外部中断允许
sfr EIP = 0xf8; //-, -, -, pwdi, px5,px4,px3,px2
//pwdi=1 看门狗中断优先
//px5-px2=1 外部中断优先

```

```

/*--BIT Registers-----*/
/* PSW      */
sbit CY  = 0xD7;
sbit AC  = 0xD6;
sbit F0  = 0xD5;
sbit RS1 = 0xD4;
sbit RS0 = 0xD3;
sbit OV  = 0xD2;
sbit P   = 0xD0;

/* TCON     */
sbit TF1 = 0x8F; //定时器 1 溢出标志, 自动清零
sbit TR1 = 0x8E; //timer1 运行控制位
sbit TF0 = 0x8D; //定时器 0 溢出标志, 自动清零
sbit TR0 = 0x8C; //timer0 运行控制位
sbit IE1 = 0x8B; //外中断 1 跳变中断请求标志, 自动清零
sbit IT1 = 0x8A; //中断 1 跳变检测使能
sbit IE0 = 0x89; //外中断 0 跳变中断请求标志, 自动清零
sbit IT0 = 0x88; //中断 0 跳变检测使能

/* IE       */
sbit EA  = 0xAF; //
sbit ES  = 0xAC;
sbit ET1 = 0xAB;
sbit EX1 = 0xAA;
sbit ET0 = 0xA9;
sbit EX0 = 0xA8;

/* IP       */
sbit PS  = 0xBC; //串口 0 中断优先设定
sbit PT1 = 0xBB; //定时器 1 中断优先
sbit PX1 = 0xBA; //外中断 1
sbit PT0 = 0xB9; //定时器 0 中断优先
sbit PX0 = 0xB8; //外中断 0

/* P3       */
sbit RD  = 0xB7;
sbit WR  = 0xB6;
sbit T1  = 0xB5;
sbit T0  = 0xB4;
sbit INT1 = 0xB3;
sbit INT0 = 0xB2;
sbit TXD = 0xB1;
sbit RXD = 0xB0;

```

```

/* SCON */
sbit SM0 = 0x9F; //SM0/FE 串口 0 方式 0 选择或帧错标志, 人工清零
sbit SM1 = 0x9E; //串口 1 工作方式选择 0-4/12tclk 同步, 1-10 位可设 bps 异步, 2-
64/32tclk 异步, 3-11 位可变 bps 异步
sbit SM2 = 0x9D; //允许方式 2 和 3 的多机通讯控制位。方式 2、3 中=1 时接收到的第 9 位为 0
时不启动接收中断 ri 标志
//方式 1 中, =1 时只有收到有效停止时才启
动 ri, 方式 0 中=1 将波特率提高 3 倍。
sbit REN = 0x9C; //允许接收标志
sbit TB8 = 0x9B; //方式 23 中要发送的第 9 位。
sbit RB8 = 0x9A; //方式 23 中接收到的第 9 位。
sbit TI = 0x99; //发送中断标志
sbit RI = 0x98; //接收中断标志

/*--W77E58 Extensions-----*/
/* PSW */
sbit F1 = 0xd1;

/* IE */
sbit ET2 = 0xAD; //定时器 2 允许
sbit ES1 = 0xae; //串口 1 优先

/* IP */
sbit PT2 = 0xBD; //定时器 2 优先
sbit PS1 = 0xbe; //串口 1 优先

/* P1 */
sbit T2 = 0x90; //计数器 2 输入端口
sbit T2EX = 0x91; //计数器捕获触发
sbit RXD1 = 0x92; //串口 1 入
sbit TXD1 = 0x93; //串口 2 出
sbit INT2 = 0x94; //外中断 2, 上跳变触发
sbit INT3 = 0x95; //外中断 3, 下跳变触发
sbit INT4 = 0x96; //外中断 4, 上跳变触发
sbit INT5 = 0x97; //外中断 5, 下跳变触发

/* T2CON */
sbit TF2 = 0xCF; //定时器 2 溢出标志, 软清除
sbit EXF2 = 0xCE; //定时器 2 外部标志, 当 exen2=1 且 t2ex 引脚负跳变引起捕获或重载时置
位, 软清。
sbit RCLK = 0xCD; //接收时钟标志, =1 时串口 0 用定时器 2 溢出做时钟。
sbit TCLK = 0xCC; //发送时钟标志, =1 时串口 0 用定时器 2 溢出做时钟。
sbit EXEN2 = 0xCB; //定时器 2 外部允许标志, =1 时若未用作波特率发生器, t2ex 脚的负跳
变引起捕获

```

```

sbit TR2 = 0xCA; //定时器 2 运行控制位
sbit C_T2 = 0xC9; //计数/定时选择, =1 时计数
sbit CP_RL = 0xC8; //捕获/重载标志, =1 时,外部允许时, vt2ex 负跳变发生捕获
//=0 时 溢出或外部允许时, vt2ex 负跳变发生重载

/* SCON */ //注释参考 SCON
sbit SM0_1 = 0xC7;
sbit SM1_1 = 0xC6;
sbit SM2_1 = 0xC5;
sbit REN_1 = 0xC4;
sbit TB8_1 = 0xC3;
sbit RB8_1 = 0xC2;
sbit TI_1 = 0xC1;
sbit RI_1 = 0xC0;

/* WDCON */
sbit SMOD_1 = 0xDF; //加倍串口 1 的波特率,pro 电源复位标志
sbit POR = 0xDE; //电源复位标志
sbit WDIF = 0xDB; //看门狗定时中断标志, 软清
sbit WTRF = 0xDA; //看门狗定时器复位标志, 看门狗引起复位后, 该位置 1
sbit EWT = 0xD9; //允许看门狗定时器自动复位
sbit RWT = 0xD8; //复位看门狗定时器, 如果看门狗溢出还没有被复位计数器, 将会引起中断, 再过 512 周期将复位

/* EIE */
sbit EWDI = 0xec; //允许看门狗中断
sbit EX5 = 0xeb; //ex5-ex2: 外部中断允许
sbit EX4 = 0xea;
sbit EX3 = 0xe9;
sbit EX2 = 0xe8;

/* EIP */
sbit PWD1 = 0xfc; //pmdi=1 看门狗中断优先
sbit PX5 = 0xfb; //px5-px2=1 外部中断优先
sbit PX4 = 0xfa;
sbit PX3 = 0xf9;
sbit PX2 = 0xf8;

```